

TRƯỜNG ĐẠI HỌC XÂY DỰNG HÀ NỘI
KHOA CÔNG NGHỆ THÔNG TIN

-----□□&□□-----



BÁO CÁO ĐỒ ÁN MÔN HỌC
THIẾT KẾ KIẾN TRÚC PHẦN MỀM
ĐỀ TÀI:
HỆ THỐNG CUNG CẤP DỊCH VỤ ĐĂNG TIN
BẤT ĐỘNG SẢN

Giảng viên hướng dẫn: GV TS. Phạm Hữu Tùng

Danh sách sinh viên:

1. Nguyễn Thị Ngọc Ánh – 0001467 - 67PM1
2. Bùi Thái Bảo – 0001567 - 67PM1
3. Đặng Thị Thùy Linh – 1500267 - 67PM1
4. Tạ Văn Long – 0060467 - 67PM1
5. Nguyễn Hoàng Việt – 0292767 - 67PM1

Nhóm: 5 – 67PM1

LỜI CẢM ƠN

Sau quá trình nghiên cứu, phân tích và triển khai đề tài một cách nghiêm túc và có hệ thống, nhóm chúng em đã hoàn thành đồ án theo đúng mục tiêu, phạm vi và yêu cầu đã được đề ra. Kết quả đạt được là sự tổng hợp của tinh thần trách nhiệm, ý thức học tập nghiêm túc và sự phối hợp chặt chẽ giữa các thành viên trong nhóm. Đồng thời, đồ án chỉ có thể được hoàn thiện nhờ sự hướng dẫn, chỉ bảo tận tình và những ý kiến đóng góp có giá trị chuyên môn sâu sắc của thầy/cô giảng dạy bộ môn Công nghệ phần mềm – Khoa Công nghệ thông tin.

Nhóm chúng em xin bày tỏ lòng biết ơn chân thành và sâu sắc tới thầy/cô đã luôn theo sát quá trình thực hiện đồ án, kịp thời định hướng phương pháp nghiên cứu, điều chỉnh nội dung chuyên môn và hỗ trợ giải quyết các khó khăn phát sinh trong suốt quá trình học tập và triển khai đề tài. Những kiến thức chuyên ngành, kinh nghiệm thực tiễn và phương pháp làm việc khoa học mà thầy/cô truyền đạt đã góp phần quan trọng vào việc nâng cao chất lượng nội dung cũng như giá trị học thuật của đồ án.

Bên cạnh đó, nhóm chúng em xin gửi lời cảm ơn tới tập thể giảng viên Khoa Công nghệ thông tin đã trực tiếp giảng dạy và trang bị cho sinh viên nền tảng kiến thức cơ bản và chuyên sâu trong suốt quá trình học tập tại trường. Đây là tiền đề quan trọng để nhóm có thể vận dụng kiến thức lý thuyết vào thực tiễn và thực hiện đồ án một cách có hệ thống và hiệu quả.

Mặc dù nhóm đã nỗ lực nghiên cứu và triển khai đề tài với tinh thần trách nhiệm cao, tuy nhiên do hạn chế về thời gian thực hiện cũng như kinh nghiệm thực tiễn, đồ án không tránh khỏi những thiếu sót nhất định. Nhóm chúng em kính mong nhận được sự góp ý, nhận xét và chỉ dẫn quý báu từ quý thầy cô để tiếp tục hoàn thiện đồ án trong thời gian tới.

Nhóm chúng em xin kính chúc quý thầy cô sức khỏe, thành công trong sự nghiệp giảng dạy và nghiên cứu khoa học.

Chúng em xin trân trọng cảm ơn!

LỜI NÓI ĐẦU

Trong thời đại công nghệ 4.0, việc ứng dụng công nghệ thông tin vào các lĩnh vực kinh tế – xã hội ngày càng trở nên cấp thiết và mang lại những giá trị thiết thực. Đặc biệt trong thị trường bất động sản – một trong những lĩnh vực quan trọng của nền kinh tế Việt Nam – việc xây dựng các nền tảng giao dịch trực tuyến không chỉ giúp tối ưu hóa quá trình kết nối giữa người mua và người bán, mà còn góp phần minh bạch hóa thông tin, nâng cao hiệu quả và sự tin tưởng trong từng giao dịch.

Xuất phát từ nhu cầu đó, hệ thống **Propify – Hệ thống cung cấp dịch vụ đăng tin bất động sản** được xây dựng nhằm tạo ra một môi trường giao dịch bất động sản trực tuyến an toàn, hiệu quả và tương tác cao. Hệ thống cho phép người dùng đăng ký, tạo và quản lý tin đăng bán hoặc cho thuê bất động sản với đầy đủ hình ảnh và thông tin xác thực. Đồng thời, hệ thống hỗ trợ người tìm kiếm dễ dàng lọc và tiếp cận các bất động sản phù hợp thông qua bộ lọc thông minh đa điều kiện, đặt lịch hẹn xem nhà trực tiếp trên nền tảng và tương tác thời gian thực qua tính năng chat tích hợp.

Bên cạnh đó, hệ thống còn tích hợp cơ chế kiểm duyệt tin đăng chặt chẽ từ phía quản trị viên nhằm loại bỏ thông tin sai lệch và tin ảo, đồng thời cung cấp bộ công cụ quản trị toàn diện gồm dashboard theo dõi hoạt động, thống kê doanh thu, quản lý tài khoản người dùng và cấu hình các gói dịch vụ linh hoạt. Nhờ vậy, cả người đăng tin lẫn người tìm kiếm đều có trải nghiệm minh bạch, đáng tin cậy và thuận tiện hơn so với các kênh giao dịch truyền thống.

Với mục tiêu xây dựng một nền tảng toàn diện – dễ sử dụng – có khả năng mở rộng, hệ thống Propify hướng đến việc hỗ trợ đầy đủ vòng đời giao dịch bất động sản trực tuyến: từ đăng tin và tìm kiếm, nâng cấp gói dịch vụ và thanh toán, đến quản lý lịch hẹn và tương tác trực tiếp giữa các bên. Qua đó, dự án góp phần thúc đẩy quá trình chuyển đổi số trong lĩnh vực bất động sản Việt Nam, mang lại giá trị thực tiễn cho cả người dùng cá nhân lẫn các đơn vị môi giới chuyên nghiệp.

Trong quá trình thực hiện đồ án, nhóm đã cố gắng vận dụng các kiến thức đã học cùng với việc nghiên cứu, tìm hiểu các công nghệ mới để hoàn thành sản phẩm một cách tốt nhất. Mặc dù đã nỗ lực hết sức, song do hạn chế về kinh nghiệm và thời gian, báo cáo và hệ thống không tránh khỏi những thiếu sót nhất định. Nhóm rất mong nhận được sự góp ý, nhận xét từ thầy/cô và các bạn để hoàn thiện đề tài hơn trong các giai đoạn tiếp theo.

Hà Nội, tháng 6 năm 2025

Nhóm sinh viên thực hiện

MỤC LỤC

LỜI CẢM ƠN	2
LỜI NÓI ĐẦU.....	3
CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	7
1.1. Lý do chọn đề tài.....	9
1.2. Mục tiêu của đề tài	9
1.3. Phạm vi và giới hạn.....	10
1.3.1. Phạm vi triển khai (In-Scope)	10
1.3.2. Ngoài phạm vi (Out-of-Scope).....	10
1.4. Phương pháp thực hiện.....	10
CHƯƠNG 2. PHÂN TÍCH HỆ THỐNG.....	11
2.1. Mô tả bài toán.....	11
2.2. Yêu cầu chức năng	11
2.3. Yêu cầu phi chức năng.....	15
2.3.1. Hiệu năng.....	15
2.3.2. Bảo mật.....	15
2.3.3. Khả năng mở rộng và bảo trì	15
2.3.4. Khả dụng	15
2.4. Use Case Diagram tổng quát.....	15
2.5. Mô tả các Use Case chính	17
2.5.1 Người dùng - Đăng ký tài khoản	17
2.5.2. Người dùng - Đăng nhập	18
2.5.3. Người dùng - Quản lý tài khoản.....	19
2.5.4. Người dùng - Quản lý tin đăng.....	24
2.5.5. Người dùng - Quản lý lịch hẹn.....	40
2.5.6. Người dùng - Quản lý lịch sử giao dịch	47
2.5.7. Người dùng - Chat.....	48
2.5.8. Admin - Quản lý tài khoản	50
2.5.9. Admin - Quản lý tin đăng.....	52
2.5.10. Admin - Quản lý tiện ích.....	61
2.5.11. Admin - Quản lý tiện ích	62
2.6. Biểu đồ hoạt động (Các chức năng chính).....	65
2.6.1. Activity Diagram: Người dùng - Đăng ký tài khoản.....	65
2.6.2. Activity Diagram: Người dùng - Đăng nhập.....	66
2.6.3. Activity Diagram: Người dùng - Quên mật khẩu.....	67
2.6.4. Activity Diagram: Người dùng - Tạo tin đăng.....	68

2.6.5. Acitivity Diagram: Người dùng - Chinh sửa tin đăng.....	69
2.6.6. Acitivity Diagram: Người dùng - Đặt lịch hẹn	70
2.6.8. Acitivity Diagram: Người dùng - Nâng cấp gói tin	72
2.6.9. Acitivity Diagram: Người dùng - Thanh toán.....	73
2.6.10. Acitivity Diagram: Người dùng - Xác thực tin đăng	74
2.6.11. Acitivity Diagram: Admin – Duyệt tin đăng.....	75
2.6.12. Acitivity Diagram: Admin – Từ chối tin đăng	76
2.6.13. Acitivity Diagram: Admin – Khóa Tin đăng.....	77
2.6.14. Acitivity Diagram: Admin – Khóa/ Mở khóa tài khoản	78
2.6.15. . Acitivity Diagram: Admin - Xác thực tin đăng.....	80
2.7.1. Sequence Diagram: Người dùng - Đăng ký tài khoản	81
2.7.2. Sequence Diagram: Người dùng - Đăng nhập	82
2.7.4. Sequence Diagram: Người dùng - Tạo tin đăng.....	84
2.7.10. Sequence Diagram: Người dùng - Xác thực tin đăng	90
2.7.11. Sequence Diagram: Admin – Duyệt/ Từ chối tin đăng	91
CHƯƠNG 3. THIẾT KẾ HỆ THỐNG.....	94
3.1. Sơ đồ kiến trúc tổng thể hệ thống	94
3.2. Thiết kế lớp	96
3.2.1. Sơ đồ Lớp tổng thể (Class Diagram).....	96
3.2.2. Design Pattern đã sử dụng.....	100
3.2.3. Phân tích thiết kế theo từng chức năng	112
3.3. Thiết kế cơ sở dữ liệu (ERD, bảng, mối quan hệ)	195
3.3.1. Sơ đồ ERD.....	195
3.3.2. Danh sách bảng chính.....	196
3.3.3.Các mối quan hệ chính	197
3.3.4. Chi tiết bảng dữ liệu	200
3.4. Thiết kế giao diện người dùng (Mockup / Wireframe).....	219
3.4.1. Màn hình Đăng ký.....	219
3.4.2. Trang chủ và Tìm kiếm	220
3.4.3. Màn hình Tạo/sửa tin đăng.....	222
3.4.4. Màn hình quản lý lịch hẹn.....	224
3.4.5. Màn hình quản lý tin đăng – Danh sách tin đăng.....	224
3.4.6. Màn hình quản lý tin đăng – Danh sách xác thực tin đăng	225
3.4.7. Màn hình quản lý tài khoản	225
3.4.8. Màn hình Tin đăng yêu thích	226
3.4.9. Màn hình Tin đăng đã xem.....	227

3.4.10. Màn hình Nâng cấp gói tin	227
3.4.11. Màn hình Lịch sử giao dịch.....	228
CHƯƠNG 4. CÀI ĐẶT VÀ TRIỂN KHAI HỆ THỐNG	229
4.1. Môi trường triển khai	229
4.2. Cài đặt hệ thống	229
4.2.1. Yêu cầu hệ thống.....	229
4.2.2. Các bước cài đặt chương trình.....	229
CHƯƠNG 5. KẾT QUẢ VÀ ĐÁNH GIÁ	230
5.1. Kết quả thử nghiệm hệ thống	230
5.1.1. Kết quả kiểm thử theo module	230
5.1.2. Kịch bản kiểm thử tích hợp End-to-End	232
5.2. Đánh giá hiệu quả hệ thống.....	234
5.2.1. Đánh giá theo mục tiêu đề ra	234
5.2.2. Điểm mạnh của hệ thống.....	234
5.2.3. Hạn chế và hướng phát triển.....	235

DANH MỤC HÌNH ẢNH

Hình 1: Usecase Người dùng.....	16
Hình 2: Usecase Admin	16
Hình 3: Usecase Người dùng - Quản lý tài khoản.....	20
Hình 4: Usecase Người dùng - Quản lý tin đăng.....	25
Hình 5: Usecase Người dùng - Quản lý lịch hẹn.....	40
Hình 6: Usecase Người dùng - Quản lý lịch sử giao dịch	47
Hình 7: Usecase Admin - Quản lý tài khoản	50
Hình 8: Usecase Admin - Quản lý tin đăng.....	52
Hình 9: Usecase Admin - Quản lý tiện ích	61
Hình 10: Usecase Admin - Quản lý tiện ích.....	62
Hình 11: Acitivity Diagram: Người dùng - Đăng ký tài khoản.....	65
Hình 12: Acitivity Diagram: Người dùng - Đăng nhập.....	66
Hình 13: Acitivity Diagram: Người dùng - Quên mật khẩu.....	67
Hình 14: Acitivity Diagram: Người dùng - Tạo tin đăng	69
Hình 15: Acitivity Diagram: Người dùng - Chỉnh sửa tin đăng.....	69
Hình 16: Acitivity Diagram: Người dùng - Đặt lịch hẹn.....	70
Hình 17: Acitivity Diagram: Người dùng - Xác nhận/Từ chối lịch hẹn.....	71
Hình 18: Acitivity Diagram: Người dùng - Nâng cấp gói tin.....	72
Hình 19: Acitivity Diagram: Người dùng - Thanh toán	73
Hình 20: Acitivity Diagram: Người dùng - Xác thực tin đăng.....	74
Hình 21: Acitivity Diagram: Admin – Duyệt tin đăng	75
Hình 22: Acitivity Diagram: Admin – Từ chối tin đăng	76
Hình 23: Acitivity Diagram: Admin – Khóa Tin đăng.....	77
Hình 24: Acitivity Diagram: Admin – Khóa tài khoản	78
Hình 25; Acitivity Diagram: Admin –Mở khóa tài khoản.....	79
Hình 26: Acitivity Diagram: Admin - Xác thực tin đăng.....	80
Hình 27: Sequence Diagram: Người dùng - Đăng ký tài khoản.....	81
Hình 28: Sequence Diagram: Người dùng - Đăng nhập.....	82
Hình 29:Sequence Diagram: Người dùng - Quên mật khẩu.....	83
Hình 30: Sequence Diagram: Người dùng - Tạo tin đăng.....	84
Hình 31:Sequence Diagram: Người dùng - Chỉnh sửa tin đăng.....	85
Hình 32:Sequence Diagram: Người dùng - Đặt lịch hẹn.....	86
Hình 33:Sequence Diagram: Người dùng - Xác nhận/Từ chối lịch hẹn.....	87
Hình 34: Sequence Diagram: Người dùng - Nâng cấp gói tin.....	88
Hình 35:Sequence Diagram: Người dùng - Thanh toán	89
Hình 36:Sequence Diagram: Người dùng - Xác thực tin đăng.....	90
Hình 37: Sequence Diagram: Admin – Duyệt/ Từ chối tin đăng	91
Hình 38:Sequence Diagram: Admin – Khóa tài khoản	92
Hình 39: Sequence Diagram: Admin - Xác thực tin đăng.....	93
Hình 40: Sơ đồ kiến trúc tổng thể hệ thống.....	94
Hình 41: Sơ đồ tổng thể class	96
Hình 42: Sơ đồ ERD	195
Hình 43:Màn hình trang Đăng ký	219
Hình 44: Màn hình trang nhập OTP	220
Hình 45: Màn hình Trang chủ.....	221
Hình 46: Màn hình trang Mua bán/Cho thuê.....	222

Hình 47: Màn hình trang Đăng tin	223
Hình 48: Màn hình trang Quản lý lịch hẹn	224
Hình 49: Màn hình trang Quản lý tin đăng – Danh sách tin đăng	225
Hình 50: Màn hình trang Quản lý tin đăng – Danh sách xác thực tin đăng	225
Hình 51: Màn hình trang Quản lý tài khoản	226
Hình 52: Màn hình trang Tin đăng yêu thích.....	226
Hình 53: Màn hình trang Tin đã xem	227
Hình 54: Màn hình trang Nâng cấp gói tin	228
Hình 55: Màn hình trang Quản lý lịch sử giao dịch	228

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1. Lý do chọn đề tài

Thị trường bất động sản Việt Nam đang phát triển nhanh chóng, kéo theo nhu cầu ngày càng cao về các nền tảng trực tuyến hỗ trợ giao dịch mua bán và cho thuê bất động sản. Tuy nhiên, các hệ thống hiện có đang đối mặt với nhiều hạn chế nghiêm trọng ảnh hưởng đến trải nghiệm của cả người đăng tin lẫn người tìm kiếm.

Về phía người đăng tin, tình trạng bão hòa thông tin khiến các bài đăng dễ dàng bị trôi đi giữa hàng nghìn tin khác, gây khó khăn trong việc tiếp cận khách hàng tiềm năng. Người bán thiếu công cụ hiệu quả để nâng cao vị trí hiển thị hay xác minh uy tín của tin đăng.

Về phía người tìm kiếm, việc lọc thông tin chất lượng từ "biển" dữ liệu không đồng nhất là bài toán khó. Ngoài ra, tình trạng tin ảo và thông tin sai lệch gây mất lòng tin, trong khi các rào cản trong việc kết nối trực tiếp giữa hai bên cũng là điểm yếu rõ ràng của các hệ thống hiện tại.

Từ thực tiễn đó, nhóm đề xuất xây dựng nền tảng Propify – một hệ thống đăng tin bất động sản toàn diện, giải quyết trực tiếp những vấn đề nêu trên thông qua bộ lọc thông minh, cơ chế kiểm duyệt tin đăng, gói dịch vụ linh hoạt và tính năng tương tác thời gian thực.

1.2. Mục tiêu của đề tài

Đề tài hướng đến các mục tiêu cụ thể:

Đối với người dùng (khách hàng):

- Xây dựng hệ thống đăng ký, đăng nhập (bao gồm bên thứ ba Google OAuth2) và quản lý tài khoản cá nhân đầy đủ.
- Cho phép tạo, chỉnh sửa, xóa và quản lý tin đăng bất động sản dễ dàng với quy trình kiểm duyệt rõ ràng.
- Cung cấp hệ thống tìm kiếm thông minh và bộ lọc đa điều kiện để người mua tiếp cận bất động sản phù hợp nhanh chóng.
- Hỗ trợ tương tác giữa người dùng: chat thời gian thực, lưu tin yêu thích và đặt lịch hẹn xem nhà.
- Tích hợp thanh toán và nâng cấp gói tin để tăng hiệu quả hiển thị tin đăng.

Đối với quản trị viên (Admin):

- Cung cấp dashboard tổng quan để theo dõi hoạt động hệ thống.
- Quản lý tin đăng: duyệt, từ chối, khóa và xác thực.
- Quản lý tài khoản người dùng: tìm kiếm, xem chi tiết, khóa/mở khóa.
- Quản lý gói dịch vụ và tiện ích hệ thống; theo dõi doanh thu và xuất báo cáo.

1.3. Phạm vi và giới hạn

1.3.1. Phạm vi triển khai (In-Scope)

- Xác thực nâng cao: đăng nhập bên thứ ba (Google OAuth2), luồng quên/đổi mật khẩu với OTP qua email.
- Quản lý tin đăng chuyên sâu: gỡ tin đăng, bộ lọc đa điều kiện, xác thực tin đăng chính chủ.
- Hệ thống giao dịch: nâng cấp gói tin, thanh toán, quản lý lịch sử giao dịch.
- Hệ thống tương tác: chat thời gian thực và quy trình quản lý lịch hẹn (đặt/sửa/hủy/xác nhận/từ chối).
- Quản trị vận hành: phê duyệt/từ chối/khóa tin đăng, quản lý vòng đời các gói dịch vụ.

1.3.2. Ngoài phạm vi (Out-of-Scope)

- Thanh toán online thật (chỉ triển khai thanh toán giả lập/mock).
- Xác minh số điện thoại qua SMS OTP.

1.4. Phương pháp thực hiện

Nhóm triển khai dự án theo phương pháp Agile Scrum với các Sprint 2 tuần, kết hợp các kỹ thuật phân tích và thiết kế hệ thống chuẩn:

- Phân tích yêu cầu: Thu thập từ tài liệu đặc tả nghiệp vụ (SRS), xây dựng Use Case Diagram và Activity Diagram mô tả luồng nghiệp vụ.
- Thiết kế hệ thống: Thiết kế cơ sở dữ liệu (ERD), kiến trúc 3-layer (Controller – Service – Repository), API RESTful.
- Lập trình: Frontend Vue.js, Backend Laravel, tích hợp các dịch vụ bên thứ ba (Cloudinary, Google Auth, VnPay).
- Kiểm thử: Kiểm thử từng module, kiểm thử tích hợp và kiểm thử hệ thống theo kịch bản thực tế.
- Theo dõi tiến độ: Sử dụng Jira Scrum Board để quản lý task và backlog.

CHƯƠNG 2. PHÂN TÍCH HỆ THỐNG

2.1. Mô tả bài toán

Hệ thống Propify được xây dựng nhằm giải quyết bài toán kết nối hiệu quả giữa người mua/thuê và người bán/cho thuê bất động sản trên một nền tảng trực tuyến minh bạch, an toàn và tương tác cao.

Hệ thống phải đảm bảo quy trình đầy đủ từ đăng ký tài khoản, tạo và kiểm duyệt tin đăng, tìm kiếm và lọc bất động sản, tương tác giữa các bên, đến thanh toán gói dịch vụ và quản lý lịch hẹn. Quản trị viên đóng vai trò trung gian đảm bảo chất lượng nội dung và vận hành hệ thống.

2.2. Yêu cầu chức năng

Tên chức năng	Tác nhân	Mô tả tóm tắt
Đăng ký tài khoản	Người dùng	Tạo tài khoản mới trên hệ thống.
Đăng nhập tài khoản	Người dùng	Đăng nhập hệ thống bằng mật khẩu hoặc Google...
Xem thông tin tài khoản	Người dùng	Xem thông tin profile cá nhân.
Chỉnh sửa thông tin cá nhân	Người dùng	Thay đổi họ tên, số điện thoại, ảnh đại diện...
Đổi mật khẩu	Người dùng	Thay đổi mật khẩu bảo mật hiện tại.
Quên mật khẩu	Người dùng	Lấy lại quyền truy cập qua Email hoặc SMS.
Tạo tin đăng	Người dùng	Nhập thông tin, hình ảnh để đăng tin căn hộ mới.
Xem danh sách tin đăng	Người dùng	Hiển thị các tin đăng dưới dạng lưới hoặc danh sách.
Map view Danh sách tin đăng	Người dùng	Hiển thị vị trí các căn hộ trực quan trên bản đồ.
Xem chi tiết tin đăng	Người dùng	Hiển thị đầy đủ thông tin, hình ảnh và liên hệ của tin đăng.
Chỉnh sửa tin đăng	Người dùng	Cập nhật lại nội dung, giá cả hoặc hình ảnh tin đã đăng.

Tên chức năng	Tác nhân	Mô tả tóm tắt
Gỡ tin đăng	Người dùng	Ẩn hoặc dừng hiển thị tin đăng sau khi giao dịch xong.
Tìm kiếm tin đăng	Người dùng	Tra cứu nhanh tin đăng theo từ khóa, địa điểm, mã tin.
Bộ lọc tin đăng / Lọc tin đăng	Người dùng	Lọc nâng cao theo khoảng giá, diện tích.
Yêu thích tin đăng	Người dùng	Lưu lại các bài đăng quan tâm để xem lại sau.
Xác thực tin đăng	Người dùng	Gửi giấy tờ, yêu cầu xác thực tin đăng chính chủ.
Cảnh báo tin đăng	Người dùng	Báo cáo các tin đăng sai sự thật, lừa đảo hoặc trùng lặp.
Chat với người đăng	Người dùng	Nhắn tin trực tiếp trao đổi giữa khách hàng và chủ tin đăng.
Đặt lịch hẹn	Người dùng	Gửi yêu cầu ngày, giờ đến xem căn hộ thực tế.
Xem thông tin lịch hẹn	Người dùng	Theo dõi trạng thái và thời gian các lịch hẹn.
Xác nhận lịch hẹn	Người dùng	Chủ tin đăng chấp nhận yêu cầu hẹn xem nhà của khách.
Từ chối lịch hẹn	Người dùng	Chủ tin đăng từ chối lịch hẹn và đề xuất thời gian khác.
Hủy lịch hẹn	Người dùng	Một trong hai bên chủ động hủy lịch hẹn đã xác nhận.
Nâng cấp gói tin	Người dùng	Mua gói tin nổi bật để đẩy bài viết lên vị trí ưu tiên.
Thanh toán	Người dùng	Thanh toán chi phí dịch vụ qua ngân hàng.
Tìm kiếm lịch sử giao dịch	Người dùng	Tra cứu lại các hóa đơn, lệnh nạp tiền/thanh toán.

Tên chức năng	Tác nhân	Mô tả tóm tắt
Lọc lịch sử giao dịch	Người dùng	Phân loại giao dịch theo thời gian, trạng thái hoặc loại dịch vụ.
Xem chi tiết lịch sử giao dịch	Người dùng	Xem chi tiết số tiền, nội dung và mã tham chiếu giao dịch.
Xem thông tin tài khoản	Admin	Xem thông tin danh sách các tài khoản trong hệ thống.
Tìm kiếm tài khoản	Admin	Tra cứu tài khoản theo tên, email, số điện thoại.
Lọc tài khoản	Admin	Phân loại tài khoản theo trạng thái hoặc ngày tạo.
Xem chi tiết tài khoản	Admin	Xem lịch sử hoạt động, tin đăng và giao dịch của tài khoản.
Khóa tài khoản	Admin	Vô hiệu hóa tài khoản vi phạm điều khoản.
Mở khóa tài khoản	Admin	Khôi phục hoạt động cho tài khoản sau khi xử lý vi phạm.
Xem danh sách tin đăng	Admin	Quản lý toàn bộ danh sách bài đăng trên hệ thống.
Xem chi tiết tin đăng	Admin	Kiểm duyệt nội dung chi tiết của một tin đăng cụ thể.
Tìm kiếm tin đăng	Admin	Tìm kiếm bài đăng phục vụ công tác quản lý, kiểm duyệt.
Bộ lọc tin đăng / Lọc tin đăng	Admin	Lọc tin theo trạng thái chờ duyệt, đã duyệt hoặc bị khóa.
Xác thực tin đăng	Admin	Kiểm tra giấy tờ và gắn nhãn xác thực cho bài đăng.

Tên chức năng	Tác nhân	Mô tả tóm tắt
Duyệt tin đăng	Admin	Phê duyệt cho phép hiển thị tin đăng mới hoặc tin chỉnh sửa.
Từ chối tin đăng	Admin	Từ chối tin đăng không hợp lệ và gửi thông báo lý do.
Khóa tin đăng	Admin	Khóa hiển thị tin đăng vi phạm quy chế hoặc bị báo cáo.
Mở khóa tin đăng	Admin	Cho phép hiển thị lại tin đăng sau khi đã sửa đúng quy định.
Xem chi tiết thông tin gói tin	Admin	Quản lý, cấu hình và theo dõi lượng sử dụng các gói tin.
Thêm tiện ích	Admin	Tạo mới các tiện ích cho căn hộ (Bể bơi, Gym, Điều hòa...).
Sửa tiện ích	Admin	Cập nhật tên, icon hoặc mô tả của tiện ích có sẵn.
Cài đặt hiển thị tiện ích	Admin	Thiết lập thứ tự ưu tiên hiển thị tiện ích ngoài giao diện.
Xem danh sách lịch sử giao dịch	Admin	Hiển thị danh sách toàn bộ các giao dịch nạp tiền, đối soát trên hệ thống.
Tìm kiếm lịch sử giao dịch	Admin	Tìm nhanh giao dịch theo mã GD, tên, email, sđt khách hàng.
Bộ lọc lịch sử giao dịch	Admin	Lọc nâng cao theo trạng thái, gói tin, ngày tháng.
Xuất báo cáo lịch sử giao dịch	Admin	Trích xuất toàn bộ hoặc danh sách giao dịch đã lọc ra file CSV.
Xem doanh thu	Admin	Theo dõi dòng tiền nạp và doanh thu từ việc bán gói tin.

Tên chức năng	Tác nhân	Mô tả tóm tắt
Xuất báo cáo	Admin	Trích xuất file Excel/PDF về doanh thu, tăng trưởng hệ thống.

2.3. Yêu cầu phi chức năng

2.3.1. Hiệu năng

- Hệ thống phải phản hồi các API trong vòng dưới 2 giây với tải bình thường.
- Sử dụng Redis cache để tối ưu tốc độ truy vấn dữ liệu thường xuyên.
- Elasticsearch hỗ trợ tìm kiếm nhanh và chính xác với lượng tin đăng lớn.

2.3.2. Bảo mật

- Xác thực JWT token với thời hạn 24 giờ; token bị vô hiệu sau khi đổi mật khẩu.
- Mật khẩu được mã hóa bằng bcrypt trước khi lưu vào cơ sở dữ liệu.
- Rate limiting giới hạn số lần đăng nhập sai và gửi OTP để chống tấn công brute-force.
- Không trả về thông tin nhạy cảm (mật khẩu, token) trong API response.

2.3.3. Khả năng mở rộng và bảo trì

- Kiến trúc 3-layer (Controller – Service – Repository) đảm bảo tách biệt logic nghiệp vụ.
- Code được tổ chức theo chuẩn PSR (PHP) và chuẩn Vue.js Composition API.
- Hỗ trợ horizontal scaling thông qua thiết kế stateless API.

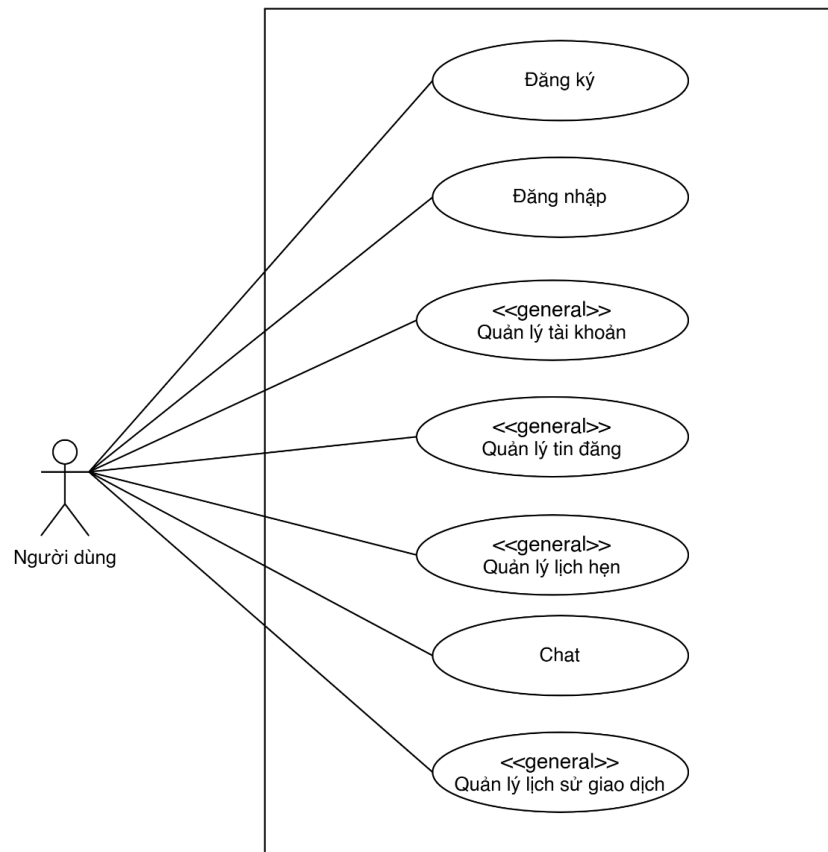
2.3.4. Khả dụng

- Hệ thống phải đạt uptime tối thiểu 99% trong giờ hoạt động.
- Xử lý graceful degradation khi các dịch vụ bên thứ ba (Cloudinary, Google) gặp sự cố.

2.4. Use Case Diagram tổng quát

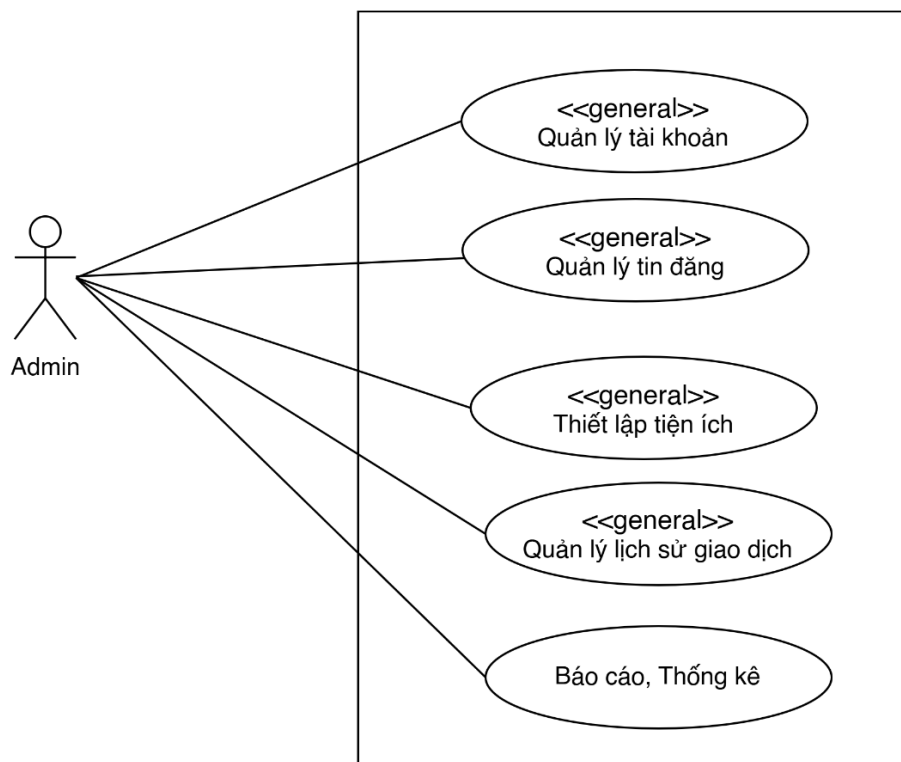
Hệ thống có hai actor chính:

- Người dùng (User): cá nhân hoặc môi giới có nhu cầu đăng tin hoặc tìm kiếm bất động sản.



Hình 1: Usecase Người dùng

- Quản trị viên (Admin): người quản lý hệ thống, thực hiện kiểm duyệt và vận hành.



Hình 2: Usecase Admin

2.5. Mô tả các Use Case chính

2.5.1 Người dùng - Đăng ký tài khoản

UC ID	UC - 01			
UC Name	Đăng ký tài khoản			
Actors	Người dùng			
Description				
Preconditions	1. Người dùng chưa đăng nhập			
Postconditions	- Tài khoản được tạo trong hệ thống - Trạng thái = ACTIVE (nếu xác thực thành công) - JWT token được cấp			
Main Flow		STT	Thực hiện	Hành động
		1	Người dùng	Chọn “Đăng ký”
		2	Hệ thống	Hiển thị form đăng ký
		3	Người dùng	Nhập họ tên, email, mật khẩu
		4	Người dùng	Chọn “Gửi”
		5	Hệ thống	Validate dữ liệu
		6	Hệ thống	Kiểm tra email tồn tại
		7	Hệ thống	Tạo tài khoản (status = PENDING)
		8	Hệ thống	Sinh OTP và gửi email
		9	Hệ thống	Yêu cầu nhập OTP
		10	Người dùng	Nhập OTP
		11	Hệ thống	Kiểm tra OTP
		12	Hệ thống	Cập nhật status = ACTIVE
		13	Hệ thống	Tạo JWT token
		14	Hệ thống	Trả về kết quả thành công
Alternative Flows	A1: Dữ liệu không hợp lệ → Hiển thị lỗi từng field → Quay lại bước 3 A2: Email đã tồn tại → Thông báo lỗi → Không tạo tài khoản A3: OTP sai → Hiển thị lỗi → Cho nhập lại OTP			

	A4: OTP hết hạn → Yêu cầu gửi lại OTP
Exception Flow	- Lỗi gửi email → retry - Timeout OTP → yêu cầu resend - Lỗi DB → rollback - Lỗi server → trả ERR-500

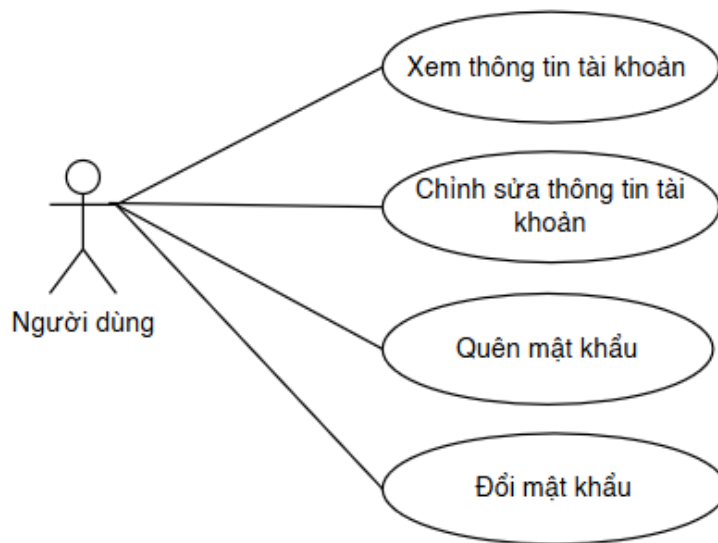
2.5.2. Người dùng - Đăng nhập

UC ID	UC - 02																																						
UC Name	Đăng nhập tài khoản																																						
Actors	Người dùng																																						
Description	Cho phép người dùng đăng nhập bằng email/SĐT và mật khẩu																																						
Preconditions	Người dùng chưa đăng nhập																																						
Postconditions	- Người dùng đăng nhập thành công - Token được cấp																																						
Main Flow		<table><tr><th>STT</th><th>Thực hiện</th><th>Hành động</th></tr><tr><td>1</td><td>Người dùng</td><td>Mở màn hình đăng nhập</td></tr><tr><td>2</td><td>Người dùng</td><td>Nhập email/SĐT và mật khẩu</td></tr><tr><td>3</td><td>Người dùng</td><td>Nhấn “Đăng nhập”</td></tr><tr><td>4</td><td>Hệ thống</td><td>Validate dữ liệu</td></tr><tr><td>5</td><td>Hệ thống</td><td>Gửi request login</td></tr><tr><td>6</td><td>Hệ thống</td><td>Kiểm tra tài khoản</td></tr><tr><td>7</td><td>Hệ thống</td><td>So sánh mật khẩu</td></tr><tr><td>8</td><td>Hệ thống</td><td>Tạo JWT token</td></tr><tr><td>9</td><td>Hệ thống</td><td>Trả token</td></tr><tr><td>10</td><td>Hệ thống</td><td>Lưu session</td></tr><tr><td>11</td><td>Hệ thống</td><td>Redirect trang chủ</td></tr></table>	STT	Thực hiện	Hành động	1	Người dùng	Mở màn hình đăng nhập	2	Người dùng	Nhập email/SĐT và mật khẩu	3	Người dùng	Nhấn “Đăng nhập”	4	Hệ thống	Validate dữ liệu	5	Hệ thống	Gửi request login	6	Hệ thống	Kiểm tra tài khoản	7	Hệ thống	So sánh mật khẩu	8	Hệ thống	Tạo JWT token	9	Hệ thống	Trả token	10	Hệ thống	Lưu session	11	Hệ thống	Redirect trang chủ	
STT	Thực hiện	Hành động																																					
1	Người dùng	Mở màn hình đăng nhập																																					
2	Người dùng	Nhập email/SĐT và mật khẩu																																					
3	Người dùng	Nhấn “Đăng nhập”																																					
4	Hệ thống	Validate dữ liệu																																					
5	Hệ thống	Gửi request login																																					
6	Hệ thống	Kiểm tra tài khoản																																					
7	Hệ thống	So sánh mật khẩu																																					
8	Hệ thống	Tạo JWT token																																					
9	Hệ thống	Trả token																																					
10	Hệ thống	Lưu session																																					
11	Hệ thống	Redirect trang chủ																																					
Alternative Flows	A1. Sai thông tin → Hiện thị lỗi → Quay lại bước 2 A2. Tài khoản bị khóa → Hiện thị thông báo → Dừng																																						
Exception Flow																																							

UC ID	UC - 03
UC Name	Đăng nhập bằng Google
Actors	Người dùng

Description	Cho phép đăng nhập bằng Google OAuth		
Preconditions	Người dùng có tài khoản Google		
Postconditions	- Đăng nhập thành công - Tạo tài khoản nếu chưa tồn tại		
Main Flow	STT	Thực hiện	Hành động
	1	Người dùng	Nhấn “Đăng nhập với Google”
	2	Hệ thống	Redirect đến Google
	3	Người dùng	Chọn tài khoản Google
	4	Google	Xác thực
	5	Google	Trả access token
	6	Hệ thống	Nhận token
	7	Hệ thống	Kiểm tra user tồn tại
	8	Hệ thống	Nếu chưa có → tạo mới
	9	Hệ thống	Tạo JWT
	10	Hệ thống	Login thành công
	11	Hệ thống	Redirect trang chủ
Alternative Flows	A1. User cancel → Dừng A2. Token không hợp lệ → Báo lỗi		
Exception Flow	- Lỗi OAuth → thông báo - Lỗi server → fail - Mất mạng → retry		

2.5.3. Người dùng - Quản lý tài khoản



Hình 3: Usecase Người dùng - Quản lý tài khoản

2.5.3.1. Xem thông tin tài khoản

UC ID	UC - 04			
UC Name	Xem thông tin tài khoản			
Actors	Người dùng			
Description	Cho phép người dùng xem thông tin tài khoản cá nhân của mình.			
Preconditions	☐ Người dùng đã đăng nhập ☐ Có token hợp lệ			
Postconditions	Thông tin tài khoản được hiển thị			
Main Flow		STT	Thực hiện	Hành động
		1	Người dùng	Chọn “Thông tin tài khoản”
		2	Hệ thống	Kiểm tra token
		3	Hệ thống	Gửi request lấy thông tin user
		4	Hệ thống	Query DB
		5	Hệ thống	Trả dữ liệu
		6	Hệ thống	Hiển thị thông tin
		7	Người dùng	Xem thông tin
Alternative Flows	A1. Token không hợp lệ - Tại bước 2 → Redirect login A2. Không có dữ liệu - Tại bước 5 → Hiển thị empty			

Exception Flow	- Lỗi server → hiển thị lỗi - Mất mạng → retry - Timeout → thông báo
----------------	--

2.5.3.2. *Chỉnh sửa thông tin tài khoản*

UC ID	UC - 05																																									
UC Name	Chỉnh sửa thông tin cá nhân																																									
Actors	Người dùng																																									
Description	Cho phép người dùng chỉnh sửa và cập nhật thông tin cá nhân.																																									
Preconditions	- Người dùng đã đăng nhập - Có token hợp lệ																																									
Postconditions	Thông tin được cập nhật thành công																																									
Main Flow		<table><tr><th>STT</th><th>Thực hiện</th><th>Hành động</th></tr><tr><td>1</td><td>Người dùng</td><td>Chọn “Chỉnh sửa thông tin”</td></tr><tr><td>2</td><td>Hệ thống</td><td>Load dữ liệu hiện tại</td></tr><tr><td>3</td><td>Hệ thống</td><td>Hiển thị form</td></tr><tr><td>4</td><td>Người dùng</td><td>Chỉnh sửa thông tin</td></tr><tr><td>5</td><td>Người dùng</td><td>Upload avatar (nếu có)</td></tr><tr><td>6</td><td>Người dùng</td><td>Nhấn “Lưu”</td></tr><tr><td>7</td><td>Hệ thống</td><td>Validate dữ liệu</td></tr><tr><td>8</td><td>Hệ thống</td><td>Upload avatar</td></tr><tr><td>9</td><td>Hệ thống</td><td>Gửi request update</td></tr><tr><td>10</td><td>Hệ thống</td><td>Update DB</td></tr><tr><td>11</td><td>Hệ thống</td><td>Trả kết quả</td></tr><tr><td>12</td><td>Hệ thống</td><td>Hiển thị thông báo thành công</td></tr></table>	STT	Thực hiện	Hành động	1	Người dùng	Chọn “Chỉnh sửa thông tin”	2	Hệ thống	Load dữ liệu hiện tại	3	Hệ thống	Hiển thị form	4	Người dùng	Chỉnh sửa thông tin	5	Người dùng	Upload avatar (nếu có)	6	Người dùng	Nhấn “Lưu”	7	Hệ thống	Validate dữ liệu	8	Hệ thống	Upload avatar	9	Hệ thống	Gửi request update	10	Hệ thống	Update DB	11	Hệ thống	Trả kết quả	12	Hệ thống	Hiển thị thông báo thành công	
STT	Thực hiện	Hành động																																								
1	Người dùng	Chọn “Chỉnh sửa thông tin”																																								
2	Hệ thống	Load dữ liệu hiện tại																																								
3	Hệ thống	Hiển thị form																																								
4	Người dùng	Chỉnh sửa thông tin																																								
5	Người dùng	Upload avatar (nếu có)																																								
6	Người dùng	Nhấn “Lưu”																																								
7	Hệ thống	Validate dữ liệu																																								
8	Hệ thống	Upload avatar																																								
9	Hệ thống	Gửi request update																																								
10	Hệ thống	Update DB																																								
11	Hệ thống	Trả kết quả																																								
12	Hệ thống	Hiển thị thông báo thành công																																								
Alternative Flows	A1. Dữ liệu không hợp lệ → Hiển thị lỗi từng field → Quay lại bước 4 A2. Không thay đổi dữ liệu → Hiển thị “Không có thay đổi” → Không gọi API																																									
Exception Flow	- Upload lỗi → retry - Lỗi server → fail - Mất mạng → retry																																									

2.5.3.3. *Đổi mật khẩu*

UC ID	UC - 06
-------	---------

UC Name	Đổi mật khẩu		
Actors	Người dùng		
Description	Cho phép người dùng thay đổi mật khẩu tài khoản của mình.		
Preconditions	Người dùng đã đăng nhập		
Postconditions	Mật khẩu được cập nhật		
Main Flow		STT	Thực hiện Hành động
		1	Người dùng Mở màn hình “Đổi mật khẩu”
		2	Người dùng Nhập mật khẩu hiện tại
		3	Người dùng Nhập mật khẩu mới
		4	Người dùng Nhập xác nhận mật khẩu
		5	Người dùng Nhấn “Xác nhận”
		6	Hệ thống Validate dữ liệu
		7	Hệ thống Kiểm tra mật khẩu hiện tại
		8	Hệ thống Validate mật khẩu mới
		9	Hệ thống Hash mật khẩu
		10	Hệ thống Update DB
		11	Hệ thống Invalidate token
		12	Hệ thống Trả kết quả thành công
		13	Hệ thống Redirect về login
Alternative Flows	A1. Sai mật khẩu hiện tại → Hiển thị lỗi → Quay lại bước 2 A2. Mật khẩu mới không hợp lệ → Hiển thị lỗi → Quay lại bước 3 A3. Xác nhận không khớp → Hiển thị lỗi → Quay lại bước 4		
Exception Flow	- Lỗi server → thông báo - Mất mạng → retry - Timeout → fail		

2.5.3.4. Quên mật khẩu

UC ID	UC - 07
-------	---------

UC Name	Quên mật khẩu																																																																	
Actors	Người dùng																																																																	
Description	Cho phép người dùng khôi phục mật khẩu thông qua OTP email.																																																																	
Preconditions	• Người dùng chưa đăng nhập • Có email hợp lệ																																																																	
Postconditions	• Mật khẩu được cập nhật • Người dùng có thể đăng nhập lại																																																																	
Main Flow	<table><tr><th>ST T</th><th>Thực hiện</th><th>Hành động</th></tr><tr><td>1</td><td>Người dùng</td><td>Chọn “Quên mật khẩu”</td></tr><tr><td>2</td><td>Hệ thống</td><td>Hiển thị form nhập email</td></tr><tr><td>3</td><td>Người dùng</td><td>Nhập email</td></tr><tr><td>4</td><td>Người dùng</td><td>Nhấn “Gửi OTP”</td></tr><tr><td>5</td><td>Hệ thống</td><td>Kiểm tra email</td></tr><tr><td>6</td><td>Hệ thống</td><td>Sinh OTP</td></tr><tr><td>7</td><td>Hệ thống</td><td>Gửi OTP qua email</td></tr><tr><td>8</td><td>Hệ thống</td><td>Hiển thị form nhập OTP</td></tr><tr><td>9</td><td>Người dùng</td><td>Nhập OTP</td></tr><tr><td>10</td><td>Người dùng</td><td>Nhấn “Xác nhận”</td></tr><tr><td>11</td><td>Hệ thống</td><td>Validate OTP</td></tr><tr><td>12</td><td>Hệ thống</td><td>Hiển thị form mật khẩu mới</td></tr><tr><td>13</td><td>Người dùng</td><td>Nhập mật khẩu mới</td></tr><tr><td>14</td><td>Người dùng</td><td>Nhập xác nhận mật khẩu</td></tr><tr><td>15</td><td>Người dùng</td><td>Nhấn “Đổi mật khẩu”</td></tr><tr><td>16</td><td>Hệ thống</td><td>Validate mật khẩu</td></tr><tr><td>17</td><td>Hệ thống</td><td>Hash password</td></tr><tr><td>18</td><td>Hệ thống</td><td>Update DB</td></tr><tr><td>19</td><td>Hệ thống</td><td>Thông báo thành công</td></tr><tr><td>20</td><td>Hệ thống</td><td>Redirect login</td></tr></table>			ST T	Thực hiện	Hành động	1	Người dùng	Chọn “Quên mật khẩu”	2	Hệ thống	Hiển thị form nhập email	3	Người dùng	Nhập email	4	Người dùng	Nhấn “Gửi OTP”	5	Hệ thống	Kiểm tra email	6	Hệ thống	Sinh OTP	7	Hệ thống	Gửi OTP qua email	8	Hệ thống	Hiển thị form nhập OTP	9	Người dùng	Nhập OTP	10	Người dùng	Nhấn “Xác nhận”	11	Hệ thống	Validate OTP	12	Hệ thống	Hiển thị form mật khẩu mới	13	Người dùng	Nhập mật khẩu mới	14	Người dùng	Nhập xác nhận mật khẩu	15	Người dùng	Nhấn “Đổi mật khẩu”	16	Hệ thống	Validate mật khẩu	17	Hệ thống	Hash password	18	Hệ thống	Update DB	19	Hệ thống	Thông báo thành công	20	Hệ thống	Redirect login
ST T	Thực hiện	Hành động																																																																
1	Người dùng	Chọn “Quên mật khẩu”																																																																
2	Hệ thống	Hiển thị form nhập email																																																																
3	Người dùng	Nhập email																																																																
4	Người dùng	Nhấn “Gửi OTP”																																																																
5	Hệ thống	Kiểm tra email																																																																
6	Hệ thống	Sinh OTP																																																																
7	Hệ thống	Gửi OTP qua email																																																																
8	Hệ thống	Hiển thị form nhập OTP																																																																
9	Người dùng	Nhập OTP																																																																
10	Người dùng	Nhấn “Xác nhận”																																																																
11	Hệ thống	Validate OTP																																																																
12	Hệ thống	Hiển thị form mật khẩu mới																																																																
13	Người dùng	Nhập mật khẩu mới																																																																
14	Người dùng	Nhập xác nhận mật khẩu																																																																
15	Người dùng	Nhấn “Đổi mật khẩu”																																																																
16	Hệ thống	Validate mật khẩu																																																																
17	Hệ thống	Hash password																																																																
18	Hệ thống	Update DB																																																																
19	Hệ thống	Thông báo thành công																																																																
20	Hệ thống	Redirect login																																																																
Alternative Flows	A1. Email không tồn tại → Hiển thị lỗi → Dừng flow A2. OTP sai → Hiển thị lỗi → Nhập lại A3. OTP hết hạn → Yêu cầu gửi lại OTP																																																																	

	A4. Mật khẩu không hợp lệ → Hiện thị lỗi → Nhập lại
Exception Flow	• Lỗi gửi email → retry • Lỗi server → fail • Mất mạng → retry

2.5.4. Người dùng - Quản lý tin đăng



Hình 4: Usecase Người dùng - Quản lý tin đăng

2.5.4.1. Tạo tin đăng

UC ID	UC - 08
UC Name	Tạo tin đăng
Actors	Người dùng

Description	Cho phép người dùng tạo và gửi tin đăng bất động sản lên hệ thống để được kiểm duyệt và hiển thị.		
Preconditions	• Người dùng đã đăng nhập vào hệ thống • Người dùng đã cập nhật số điện thoại hợp lệ		
Postconditions	• Tin đăng được lưu vào hệ thống • Trạng thái tin đăng là “Chờ duyệt” • Media (ảnh/video) được lưu và liên kết với tin đăng		
Main Flow		ST T	Thực hiện Hành động
	1	Người dùng	Chọn chức năng “Đăng tin”
	2	Hệ thống	Hiển thị form nhập thông tin tin đăng
	3	Người dùng	Nhập các thông tin cần thiết (tiêu đề, giá, diện tích, địa chỉ, mô tả,...)
	4	Người dùng	Tải lên hình ảnh và/hoặc video
	5	Người dùng	Nhấn nút “Đăng tin”
	6	Hệ thống	Kiểm tra tính hợp lệ của dữ liệu
	7	Hệ thống	Upload media và nhận URL
	8	Hệ thống	Tính điểm chất lượng tin đăng
	9	Hệ thống	Lưu tin đăng vào cơ sở dữ liệu với trạng thái “Chờ duyệt”
	10	Hệ thống	Thông báo đăng tin thành công
Alternative Flows	A1. Chưa có số điện thoại Tại bước 1 hoặc 2, nếu phát hiện người dùng chưa có số điện thoại → Hệ thống yêu cầu cập nhật → Dừng use case A2. Thiếu hoặc sai dữ liệu Tại bước 6, nếu dữ liệu không hợp lệ → Hệ thống hiển thị lỗi (thiếu thông tin, sai định dạng,...) → Quay lại bước 3 để nhập lại A3. Upload media thất bại Tại bước 7, nếu upload lỗi → Hệ thống thông báo lỗi upload → Người dùng thực hiện upload lại		

	A4. Lỗi hệ thống Tại bất kỳ bước nào nếu xảy ra lỗi hệ thống → Hệ thống hiển thị thông báo lỗi chung → Use case kết thúc
Exception Flow	- Mất kết nối mạng khi gửi dữ liệu → thông báo lỗi, cho phép gửi lại - Timeout khi upload → yêu cầu upload lại - Lỗi server → thông báo thất bại

2.5.4.2. Xem tin đăng

UC ID	UC - 09																																									
UC Name	Xem danh sách tin đăng																																									
Actors	Người dùng																																									
Description	Cho phép người dùng xem và quản lý danh sách tin đăng của mình.																																									
Preconditions	Người dùng đã đăng nhập																																									
Postconditions	Danh sách tin đăng được hiển thị																																									
Main Flow		<table><tr><th>ST T</th><th>Thực hiện</th><th>Hành động</th></tr><tr><td>1</td><td>Người dùng</td><td>Truy cập “Danh sách tin đăng”</td></tr><tr><td>2</td><td>Hệ thống</td><td>Gửi request lấy danh sách</td></tr><tr><td>3</td><td>Hệ thống</td><td>Query DB theo userId</td></tr><tr><td>4</td><td>Hệ thống</td><td>Trả dữ liệu</td></tr><tr><td>5</td><td>Hệ thống</td><td>Hiển thị bảng</td></tr><tr><td>6</td><td>Người dùng</td><td>Nhập từ khóa tìm kiếm</td></tr><tr><td>7</td><td>Hệ thống</td><td>Gọi API search</td></tr><tr><td>8</td><td>Hệ thống</td><td>Hiển thị kết quả</td></tr><tr><td>9</td><td>Người dùng</td><td>Chọn filter</td></tr><tr><td>10</td><td>Hệ thống</td><td>Reload dữ liệu</td></tr><tr><td>11</td><td>Người dùng</td><td>Chuyển trang</td></tr><tr><td>12</td><td>Hệ thống</td><td>Load page mới</td></tr></table>	ST T	Thực hiện	Hành động	1	Người dùng	Truy cập “Danh sách tin đăng”	2	Hệ thống	Gửi request lấy danh sách	3	Hệ thống	Query DB theo userId	4	Hệ thống	Trả dữ liệu	5	Hệ thống	Hiển thị bảng	6	Người dùng	Nhập từ khóa tìm kiếm	7	Hệ thống	Gọi API search	8	Hệ thống	Hiển thị kết quả	9	Người dùng	Chọn filter	10	Hệ thống	Reload dữ liệu	11	Người dùng	Chuyển trang	12	Hệ thống	Load page mới	
ST T	Thực hiện	Hành động																																								
1	Người dùng	Truy cập “Danh sách tin đăng”																																								
2	Hệ thống	Gửi request lấy danh sách																																								
3	Hệ thống	Query DB theo userId																																								
4	Hệ thống	Trả dữ liệu																																								
5	Hệ thống	Hiển thị bảng																																								
6	Người dùng	Nhập từ khóa tìm kiếm																																								
7	Hệ thống	Gọi API search																																								
8	Hệ thống	Hiển thị kết quả																																								
9	Người dùng	Chọn filter																																								
10	Hệ thống	Reload dữ liệu																																								
11	Người dùng	Chuyển trang																																								
12	Hệ thống	Load page mới																																								
Alternative Flows	A1. Không có dữ liệu Tại bước 4 → Hiển thị “Không có tin đăng” A2. Không có kết quả tìm kiếm																																									

	Sau bước 7 → Hiển thị empty state
Exception Flow	- API lỗi → hiển thị lỗi - Timeout → retry

2.5.4.3. Xem chi tiết tin đăng

UC ID	UC - 10			
UC Name	Xem chi tiết tin đăng			
Actors	Người dùng			
Description	Cho phép người dùng xem thông tin chi tiết của một tin đăng bất động sản.			
Preconditions	- Tin đăng tồn tại trong hệ thống - Tin có trạng thái hợp lệ (Đã duyệt)			
Postconditions	- Thông tin tin đăng được hiển thị - View count được cập nhật			
Main Flow		STT	Thực hiện	Hành động
		1	User	Click vào tin đăng
		2	System	Nhận PostID
		3	System	Gọi API lấy dữ liệu
		4	System	Kiểm tra trạng thái tin
		5	System	Nếu hợp lệ → trả dữ liệu
		6	System	Hiển thị thông tin cơ bản
		7	System	Hiển thị hình ảnh/video
		8	System	Hiển thị mô tả
		9	System	Hiển thị thông tin liên hệ
		10	System	Hiển thị tiện ích
		11	System	Tăng view count
		12	System	Hiển thị tin liên quan
Alternative Flows	A1: Tin không tồn tại → Hiển thị thông báo “Tin không tồn tại” A2: Tin chưa duyệt → Không hiển thị nội dung A3: Không có media → Hiển thị ảnh mặc định			

Exception Flow	• Lỗi API → hiển thị lỗi • Timeout → retry • Network lỗi → thông báo
----------------	--

2.5.4.4. *Chỉnh sửa tin đăng*

UC ID	UC - 11																																															
UC Name	Chỉnh sửa tin đăng																																															
Actors	Người dùng																																															
Description	Cho phép người dùng chỉnh sửa tin đăng đã tạo và gửi lại để kiểm duyệt																																															
Preconditions	• Người dùng đã đăng nhập • Tin đăng tồn tại • Người dùng là chủ tin																																															
Postconditions	• Tin đăng được cập nhật • Media được cập nhật • Trạng thái = “Chờ duyệt”																																															
Main Flow		<table><tr><th>STT</th><th>Thực hiện</th><th>Hành động</th></tr><tr><td>1</td><td>Người dùng</td><td>Chọn “Sửa” tin đăng</td></tr><tr><td>2</td><td>Hệ thống</td><td>Kiểm tra quyền</td></tr><tr><td>3</td><td>Hệ thống</td><td>Lấy dữ liệu tin đăng</td></tr><tr><td>4</td><td>Hệ thống</td><td>Hiển thị form với dữ liệu cũ</td></tr><tr><td>5</td><td>Người dùng</td><td>Chỉnh sửa thông tin</td></tr><tr><td>6</td><td>Người dùng</td><td>Thêm/xóa media</td></tr><tr><td>7</td><td>Người dùng</td><td>Nhấn “Cập nhật”</td></tr><tr><td>8</td><td>Hệ thống</td><td>Validate dữ liệu</td></tr><tr><td>9</td><td>Hệ thống</td><td>Upload media mới</td></tr><tr><td>10</td><td>Hệ thống</td><td>Nhận URL</td></tr><tr><td>11</td><td>Hệ thống</td><td>Tính lại quality score</td></tr><tr><td>12</td><td>Hệ thống</td><td>Update DB</td></tr><tr><td>13</td><td>Hệ thống</td><td>Set trạng thái “Chờ duyệt”</td></tr><tr><td>14</td><td>Hệ thống</td><td>Trả thông báo thành công</td></tr></table>	STT	Thực hiện	Hành động	1	Người dùng	Chọn “Sửa” tin đăng	2	Hệ thống	Kiểm tra quyền	3	Hệ thống	Lấy dữ liệu tin đăng	4	Hệ thống	Hiển thị form với dữ liệu cũ	5	Người dùng	Chỉnh sửa thông tin	6	Người dùng	Thêm/xóa media	7	Người dùng	Nhấn “Cập nhật”	8	Hệ thống	Validate dữ liệu	9	Hệ thống	Upload media mới	10	Hệ thống	Nhận URL	11	Hệ thống	Tính lại quality score	12	Hệ thống	Update DB	13	Hệ thống	Set trạng thái “Chờ duyệt”	14	Hệ thống	Trả thông báo thành công	
STT	Thực hiện	Hành động																																														
1	Người dùng	Chọn “Sửa” tin đăng																																														
2	Hệ thống	Kiểm tra quyền																																														
3	Hệ thống	Lấy dữ liệu tin đăng																																														
4	Hệ thống	Hiển thị form với dữ liệu cũ																																														
5	Người dùng	Chỉnh sửa thông tin																																														
6	Người dùng	Thêm/xóa media																																														
7	Người dùng	Nhấn “Cập nhật”																																														
8	Hệ thống	Validate dữ liệu																																														
9	Hệ thống	Upload media mới																																														
10	Hệ thống	Nhận URL																																														
11	Hệ thống	Tính lại quality score																																														
12	Hệ thống	Update DB																																														
13	Hệ thống	Set trạng thái “Chờ duyệt”																																														
14	Hệ thống	Trả thông báo thành công																																														
Alternative Flows	A1. Không có quyền																																															

	- Tại bước 2 → Hiện thị: “Không có quyền” → Dừng A2. Dữ liệu không hợp lệ - Tại bước 8 → Hiện thị lỗi từng field → Quay lại bước 5 A3. Upload lỗi - Tại bước 9 → Hiện thị lỗi → Cho phép upload lại
Exception Flow	- Mất mạng → thông báo + retry - Timeout upload → yêu cầu upload lại - Lỗi server → thông báo hệ thống

2.5.4.5. Gỡ tin đăng

UC ID	UC - 11		
UC Name	Gỡ tin đăng		
Actors	Người dùng		
Description	Cho phép người dùng gỡ tin đăng của mình để ngừng hiển thị trên hệ thống.		
Preconditions	- Người dùng đã đăng nhập - Tin đăng tồn tại - Người dùng là chủ tin		
Postconditions	- Trạng thái tin = “Đã gỡ” - Tin không còn hiển thị công khai		
Main Flow	ST T	Thực hiện	Hành động
	1	Người dùng	Chọn “Gỡ tin”
	2	Hệ thống	Gửi request kiểm tra trạng thái
	3	Hệ thống	Lấy trạng thái tin từ DB
	4	Hệ thống	Kiểm tra trạng thái
	5	Hệ thống	Hiện thị popup xác nhận
	6	Người dùng	Nhấn “Xác nhận”
	7	Hệ thống	Gửi request gỡ tin

		8	Hệ thống	Cập nhật trạng thái = Đã gỡ
		9	Hệ thống	Lưu log
		10	Hệ thống	Trả thông báo thành công
		11	Hệ thống	Refresh danh sách
Alternative Flows	A1. Trạng thái không hợp lệ - Tại bước 4 → Nếu trạng thái ≠ “Đang đăng” → Hiện thị: “Không cho phép gỡ” → Dừng A2. Người dùng hủy thao tác - Tại bước 5 → User chọn “Hủy” → Kết thúc			
Exception Flow	- Lỗi DB → hiển thị lỗi hệ thống - Mất mạng → retry - Timeout → thông báo thất bại			

2.5.4.6. Yêu thích tin đăng

UC ID	UC - 12		
UC Name	Thêm tin đăng vào yêu thích		
Actors	Người dùng		
Description	Cho phép người dùng lưu một tin đăng vào danh sách yêu thích.		
Preconditions	- Người dùng đã đăng nhập. - Tin đăng tồn tại trên hệ thống. - Tin đăng chưa nằm trong danh sách yêu thích của người dùng.		
Postconditions	- Tin đăng được thêm vào danh sách yêu thích. - Biểu tượng trái tim chuyển sang trạng thái đã chọn.		
Main Flow		STT	Thực hiện
			Hành động
		1	Người dùng
		2	Hệ thống
		3	Hệ thống
		4	Hệ thống

		5	Hệ thống	Cập nhật trạng thái biểu tượng trái tim.
		6	Hệ thống	Hiển thị thông báo thêm yêu thích thành công.
Alternative Flows	AF-01 (Bỏ yêu thích) Tại bước 1, nếu tin đăng đã nằm trong danh sách yêu thích và người dùng tiếp tục nhấn biểu tượng trái tim, hệ thống thực hiện bỏ yêu thích và cập nhật trạng thái biểu tượng.			
Exception Flow	EF-01 (Chưa đăng nhập) Tại bước 2, nếu người dùng chưa đăng nhập, hệ thống hiển thị thông báo TB_01 và chuyển tới màn hình đăng nhập. EF-02 (Tin đăng không tồn tại) Tại bước 3, nếu tin đăng không tồn tại hoặc đã bị xóa, hệ thống hiển thị thông báo. EF-03 (Lỗi hệ thống) Tại bước 4, nếu hệ thống không thể lưu dữ liệu do lỗi mạng hoặc lỗi máy chủ, hệ thống hiển thị thông báo TB_03 và cho phép người dùng thực hiện lại thao tác.			

2.5.4.7. Nâng cấp tin đăng

UC ID	UC - 13			
UC Name	Nâng cấp gói tin			
Actors	Người dùng			
Description	Cho phép người dùng nâng cấp gói hiển thị cho tin đăng đang ở trạng thái Đang duyệt.			
Preconditions	- Người dùng đã đăng nhập. - Người dùng là chủ sở hữu tin đăng. - Tin đăng tồn tại. - Tin đăng ở trạng thái Đang duyệt.			
Postconditions	- Gói tin được cập nhật. - Giao dịch được lưu. - Người dùng nhận được thông báo thanh toán thành công.			
Main Flow		STT	Thực hiện	Hành động
		1	Người dùng	Truy cập chi tiết tin đăng.
		2	Người dùng	Chọn chức năng Nâng cấp gói tin.

		3	Hệ thống	Hiện thị gói hiện tại và danh sách gói nâng cấp.
		4	Người dùng	Chọn gói tin mong muốn.
		5	Người dùng	Chọn thời hạn sử dụng.
		6	Hệ thống	Tính toán số tiền cần thanh toán.
		7	Hệ thống	Hiện thị màn hình xác nhận thanh toán.
		8	Người dùng	Nhấn Thanh toán.
		9	Hệ thống	Thực hiện giao dịch thanh toán.
		10	Hệ thống	Nhận kết quả từ cổng thanh toán.
		11	Hệ thống	Cập nhật gói tin.
		12	Hệ thống	Lưu lịch sử giao dịch.
		13	Hệ thống	Gửi thông báo thanh toán thành công.
		14	Hệ thống	Hiện thị thông báo thành công.
Alternative Flows	AF-01 (Hủy nâng cấp) Tại bước 7, nếu người dùng nhấn nút Hủy hoặc quay lại, hệ thống đóng màn hình xác nhận và không thực hiện thanh toán. AF-02 (Thay đổi gói tin) Tại bước 4, nếu người dùng chọn gói khác, hệ thống cập nhật lại thông tin quyền lợi và số tiền tương ứng. AF-03 (Thay đổi thời hạn) Tại bước 5, nếu người dùng thay đổi thời hạn sử dụng, hệ thống tính lại số tiền thanh toán.			
Exception Flow	EF-01 (Tin không đủ điều kiện nâng cấp) Tại bước 3, nếu tin đăng không ở trạng thái Đang duyệt, hệ thống hiển thị thông. EF-02 (Không phải chủ tin) Tại bước 3, nếu người dùng không phải chủ sở hữu tin đăng, hệ thống hiển thị thông báo TB_02. EF-03 (Thanh toán thất bại) Tại bước 10, nếu giao dịch bị từ chối hoặc thất bại, hệ thống hiển thị thông báo TB_03.			

	EF-04 (Hủy thanh toán) Tại bước 9, nếu người dùng hủy giao dịch trên cổng thanh toán, hệ thống hiển thị thông báo TB_04. EF-05 (Lỗi kết nối cổng thanh toán) Tại bước 9, nếu hệ thống không kết nối được cổng thanh toán, hiển thị thông báo TB_05. EF-06 (Lỗi cập nhật gói tin) Tại bước 11, nếu hệ thống không cập nhật được gói tin sau khi thanh toán thành công, hiển thị thông báo TB_06.
--	--

2.5.4.8. Xác thực tin đăng

UC ID	UC - 14			
UC Name	Xác thực tin đăng			
Actors	Người dùng			
Description	Cho phép người dùng gửi hồ sơ xác thực quyền sở hữu bất động sản cho tin đăng thuộc nhu cầu Mua bán.			
Preconditions	1. Người dùng đã đăng nhập. 2. Người dùng là chủ sở hữu tin đăng. 3. Tin đăng tồn tại trên hệ thống. 4. Tin đăng thuộc nhu cầu Mua bán. 5. Tin đăng chưa có hồ sơ xác thực ở trạng thái Chờ duyệt.			
Postconditions	• Hồ sơ xác thực được tạo thành công. • Trạng thái xác thực chuyển sang Chờ duyệt. • Hồ sơ được lưu vào hệ thống.			
Main Flow		STT	Thực hiện	Hành động
		1	Người dùng	Truy cập màn hình Xác thực tin đăng.
		2	Hệ thống	Hiển thị giao diện tải hồ sơ xác thực.
		3	Người dùng	Tải lên ảnh CCCD/CMND mặt trước.
		4	Người dùng	Tải lên ảnh CCCD/CMND mặt sau.
		5	Người dùng	Tải lên ảnh giấy tờ pháp lý.
		6	Người dùng	Chọn hoặc bỏ chọn tùy chọn Công khai thông tin.
		7	Người dùng	Nhấn nút Đăng lại tin.

	8	Hệ thống	Kiểm tra quyền sở hữu tin đăng.
	9	Hệ thống	Kiểm tra nhu cầu tin đăng là Mua bán.
	10	Hệ thống	Kiểm tra dữ liệu bắt buộc và tính hợp lệ của tệp tải lên.
	11	Hệ thống	Lưu hồ sơ xác thực.
	12	Hệ thống	Cập nhật trạng thái xác thực thành Chờ duyệt.
	13	Hệ thống	Hiển thị thông báo gửi yêu cầu xác thực thành công.
Alternative Flows	AF-01 (Xóa ảnh đã tải lên) Tại bước 3, bước 4 hoặc bước 5, nếu người dùng chọn xóa ảnh đã tải lên, hệ thống loại bỏ tệp khỏi phiên làm việc hiện tại và cho phép tải lên lại. AF-02 (Hủy xác thực) Tại bước 7, nếu người dùng rời khỏi màn hình hoặc đóng trình duyệt trước khi gửi xác thực, hệ thống không lưu hồ sơ xác thực. AF-03 (Thay đổi tùy chọn công khai thông tin) Tại bước 6, nếu người dùng thay đổi trạng thái checkbox Công khai thông tin, hệ thống cập nhật lựa chọn trước khi gửi hồ sơ xác thực.		
Exception Flow	EF-01 (Thiếu CCCD/CMND mặt trước) Tại bước 10, nếu chưa tải lên ảnh CCCD/CMND mặt trước, hệ thống hiển thị thông báo, EF-03 (Thiếu CCCD/CMND mặt sau) Tại bước 10, nếu chưa tải lên ảnh CCCD/CMND mặt sau, hệ thống hiển thị thông báo. EF-04 (Thiếu giấy tờ pháp lý) Tại bước 10, nếu chưa tải lên giấy tờ pháp lý, hệ thống hiển thị thông báo. EF-05 (Định dạng tệp không hợp lệ) Tại bước 10, nếu tệp tải lên không thuộc định dạng JPG, JPEG hoặc PNG, hệ thống hiển thị thông báo. EF-06 (Dung lượng ảnh vượt giới hạn) Tại bước 10, nếu ảnh tải lên có dung lượng lớn hơn 30MB, hệ thống hiển thị thông báo. EF-07 (Mất kết nối)		

	Tại bước 11, nếu hệ thống không thể lưu dữ liệu do lỗi mạng hoặc lỗi máy chủ, hệ thống hiển thị thông báo và cho phép người dùng thực hiện lại thao tác.
--	--

2.5.4.9. Báo cáo tin đăng

UC ID	UC - 15		
UC Name	Báo cáo tin đăng		
Actors	Người dùng		
Description	Người dùng gửi phản hồi về sai phạm của tin đăng để hệ thống kiểm tra lại nội dung.		
Preconditions	Người dùng đang ở màn hình chi tiết tin đăng; Người dùng đã đăng nhập hệ thống.		
Postconditions	Tin đăng bị gắn cờ cảnh báo trong quản trị.		
Main Flow	STT	Thực hiện	Hành động
	1	Hệ thống	Hiển thị "Báo cáo tin đăng" tại mỗi chi tiết căn hộ
	2	Người dùng	Chọn một trong các lý do vi phạm có sẵn (Ví dụ: Giá không đúng thực tế, Địa chỉ sai...).
	4	Người dùng	Nhấn nút "Gửi phản ánh".
	5	Hệ thống	Hiển thị popup Nhập lý do, hình ảnh
	6	Người dùng	Nhập lý do xác nhận cảnh báo => Chọn xác nhận
	7	Hệ thống	Kiểm tra tính hợp lệ của dữ liệu .
	8	Hệ thống	Lưu báo cáo vào DB, hiển thị thông báo thành công và đóng form.
Alternative Flows	AF-01 (Hủy báo cáo): Tại bước 2, nếu người dùng nhấn nút "X" hoặc click ra ngoài form, hệ thống đóng giao diện báo cáo và không lưu dữ liệu. AF-06 (Hủy xác nhận): Tại bước 6, nếu người dùng nhấn nút "X" hoặc Hủy, hệ thống đóng giao diện báo cáo và không lưu dữ liệu.		
Exception Flow	EF-01 (Mất kết nối):		

	Tại bước 6, nếu hệ thống không thể lưu dữ liệu do lỗi mạng, hiển thị thông báo lỗi và cho phép người dùng thử lại.
--	--

2.5.4.10. Tìm kiếm tin đăng

UC ID	UC - 16																																
UC Name	Tìm kiếm tin đăng																																
Actors	Người dùng																																
Description	Cho phép người dùng tìm kiếm tin đăng bất động sản bằng từ khóa																																
Preconditions	- Hệ thống hoạt động bình thường. - Có dữ liệu tin đăng trên hệ thống.																																
Postconditions	- Danh sách kết quả tìm kiếm được hiển thị. - Lịch sử tìm kiếm được lưu (nếu người dùng đã đăng nhập).																																
Main Flow		<table><tr><th>STT</th><th>Thực hiện</th><th>Hành động</th></tr><tr><td>1</td><td>Người dùng</td><td>Truy cập trang chủ hoặc trang tìm kiếm.</td></tr><tr><td>2</td><td>Người dùng</td><td>Nhập từ khóa.</td></tr><tr><td>3</td><td>Người dùng</td><td>Nhấn nút Tìm kiếm.</td></tr><tr><td>4</td><td>Hệ thống</td><td>Kiểm tra dữ liệu tìm kiếm.</td></tr><tr><td>5</td><td>Hệ thống</td><td>Thực hiện truy vấn dữ liệu theo điều kiện tìm kiếm.</td></tr><tr><td>6</td><td>Hệ thống</td><td>Loại bỏ các tin không đủ điều kiện hiển thị.</td></tr><tr><td>7</td><td>Hệ thống</td><td>Sắp xếp kết quả theo tiêu chí mặc định hoặc tiêu chí người dùng lựa chọn.</td></tr><tr><td>8</td><td>Hệ thống</td><td>Hiển thị danh sách kết quả tìm kiếm.</td></tr><tr><td>9</td><td>Hệ thống</td><td>Lưu lịch sử tìm kiếm (nếu người dùng đã đăng nhập).</td></tr></table>	STT	Thực hiện	Hành động	1	Người dùng	Truy cập trang chủ hoặc trang tìm kiếm.	2	Người dùng	Nhập từ khóa.	3	Người dùng	Nhấn nút Tìm kiếm.	4	Hệ thống	Kiểm tra dữ liệu tìm kiếm.	5	Hệ thống	Thực hiện truy vấn dữ liệu theo điều kiện tìm kiếm.	6	Hệ thống	Loại bỏ các tin không đủ điều kiện hiển thị.	7	Hệ thống	Sắp xếp kết quả theo tiêu chí mặc định hoặc tiêu chí người dùng lựa chọn.	8	Hệ thống	Hiển thị danh sách kết quả tìm kiếm.	9	Hệ thống	Lưu lịch sử tìm kiếm (nếu người dùng đã đăng nhập).	
STT	Thực hiện	Hành động																															
1	Người dùng	Truy cập trang chủ hoặc trang tìm kiếm.																															
2	Người dùng	Nhập từ khóa.																															
3	Người dùng	Nhấn nút Tìm kiếm.																															
4	Hệ thống	Kiểm tra dữ liệu tìm kiếm.																															
5	Hệ thống	Thực hiện truy vấn dữ liệu theo điều kiện tìm kiếm.																															
6	Hệ thống	Loại bỏ các tin không đủ điều kiện hiển thị.																															
7	Hệ thống	Sắp xếp kết quả theo tiêu chí mặc định hoặc tiêu chí người dùng lựa chọn.																															
8	Hệ thống	Hiển thị danh sách kết quả tìm kiếm.																															
9	Hệ thống	Lưu lịch sử tìm kiếm (nếu người dùng đã đăng nhập).																															
Alternative Flows	AF-01 (Tìm kiếm bằng từ khóa) Tại bước 2, nếu người dùng chỉ nhập từ khóa tìm kiếm, hệ thống thực hiện tìm kiếm theo từ khóa mà không áp dụng các bộ lọc khác. AF-02 (Sắp xếp kết quả)																																

	Tại bước 8, nếu người dùng thay đổi tiêu chí sắp xếp, hệ thống cập nhật lại danh sách kết quả theo tiêu chí được chọn. AF-03 (Chuyển trang) Tại bước 8, nếu người dùng chọn trang tiếp theo hoặc trang trước, hệ thống tải dữ liệu tương ứng với trang được chọn.
Exception Flow	EF-01 (Không tìm thấy dữ liệu) Tại bước 8, nếu không có tin đăng nào phù hợp với điều kiện tìm kiếm, hệ thống hiển thị thông báo. EF-02 (Từ khóa không hợp lệ) Tại bước 4, nếu từ khóa tìm kiếm vượt quá độ dài cho phép theo cấu hình hệ thống, hệ thống hiển thị thông báo. EF-03 (Lỗi tải dữ liệu) Tại bước 5, nếu hệ thống không thể truy xuất dữ liệu, hiển thị thông báo và cho phép người dùng thực hiện tìm kiếm lại. EF-04 (Mất kết nối) Tại bước 5, nếu xảy ra lỗi kết nối mạng hoặc máy chủ, hệ thống hiển thị thông báo.

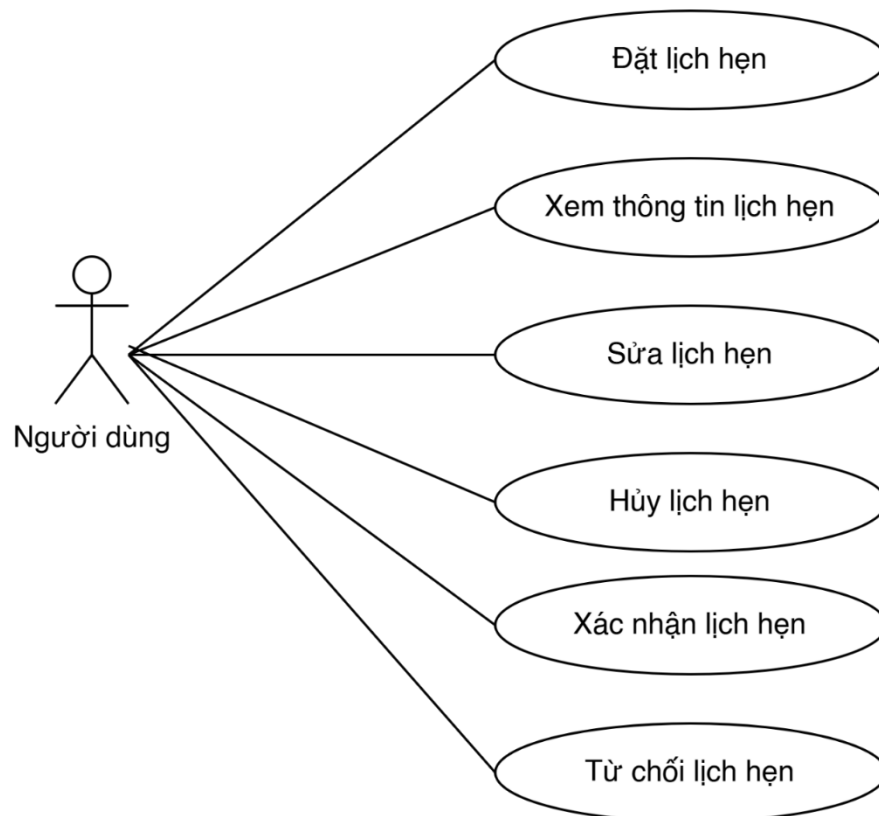
2.5.4.11. Bộ lọc tin đăng

UC ID	UC - 17		
UC Name	Lọc tin đăng		
Actors	Khách vãng lai, Người dùng		
Description	Cho phép người dùng lọc danh sách tin đăng theo người đăng, khoảng giá và diện tích.		
Preconditions	Hệ thống có dữ liệu tin đăng. Người dùng đang ở màn hình danh sách tin đăng.		
Postconditions	Danh sách tin đăng được cập nhật theo điều kiện lọc.		
Main Flow	STT	Thực hiện	Hành động
	1	Người dùng	Truy cập trang danh sách tin đăng.
	2	Hệ thống	Hiển thị bộ lọc tìm kiếm.
	3	Người dùng	Chọn điều kiện Người đăng.
	4	Người dùng	Chọn điều kiện Khoảng giá hoặc nhập giá trị tùy chỉnh.
	5	Người dùng	Chọn điều kiện Diện tích hoặc nhập giá trị tùy chỉnh.
	6	Người dùng	Nhấn nút Áp dụng bộ lọc.

		7	Hệ thống	Kiểm tra tính hợp lệ của dữ liệu lọc.
		8	Hệ thống	Thực hiện lọc dữ liệu.
		9	Hệ thống	Hiển thị danh sách kết quả phù hợp.
		10	Hệ thống	Hiển thị số lượng kết quả tìm được.
Alternative Flows	AF-01 (Lọc theo người đăng) Tại bước 3, nếu người dùng chọn Chủ nhà, hệ thống chỉ hiển thị các tin đăng được đăng bởi Chủ nhà. AF-02 (Lọc theo môi giới) Tại bước 3, nếu người dùng chọn Môi giới, hệ thống chỉ hiển thị các tin đăng được đăng bởi Môi giới. AF-03 (Lọc theo khoảng giá có sẵn) Tại bước 4, nếu người dùng chọn một khoảng giá có sẵn, hệ thống áp dụng khoảng giá đó để lọc dữ liệu. AF-04 (Lọc theo giá tùy chỉnh) Tại bước 4, nếu người dùng nhập giá trị Từ và Đến, hệ thống sử dụng khoảng giá tùy chỉnh để lọc dữ liệu. AF-05 (Lọc theo diện tích có sẵn) Tại bước 5, nếu người dùng chọn một khoảng diện tích có sẵn, hệ thống áp dụng khoảng diện tích đó để lọc dữ liệu. AF-06 (Lọc theo diện tích tùy chỉnh) Tại bước 5, nếu người dùng nhập giá trị Từ và Đến, hệ thống sử dụng khoảng diện tích tùy chỉnh để lọc dữ liệu. AF-07 (Xóa bộ lọc) Tại bước 6, nếu người dùng chọn Đặt lại bộ lọc, hệ thống đưa toàn bộ điều kiện lọc về giá trị mặc định và hiển thị lại danh sách ban đầu.			
Exception Flow	EF-01 (Khoảng giá không hợp lệ) Tại bước 7, nếu giá trị Từ lớn hơn giá trị Đến, hệ thống hiển thị thông báo. EF-02 (Diện tích không hợp lệ) Tại bước 7, nếu diện tích Từ lớn hơn diện tích Đến, hệ thống hiển thị thông báo. EF-03 (Giá trị âm) Tại bước 7, nếu giá trị nhập nhỏ hơn hoặc bằng 0, hệ thống tự động nhảy về 0. EF-04 (Không tìm thấy dữ liệu)			

	Tại bước 9, nếu không có tin đăng phù hợp điều kiện lọc, hệ thống hiển thị thông báo. EF-05 (Lỗi tải dữ liệu) Tại bước 8, nếu hệ thống không thể truy xuất dữ liệu, hiển thị thông báo và cho phép người dùng thực hiện lại thao tác.
--	---

2.5.5. Người dùng - Quản lý lịch hẹn



Hình 5: Usecase Người dùng - Quản lý lịch hẹn

2.5.5.1. Đặt lịch hẹn

UC ID	UC - 18
UC Name	Đặt lịch hẹn
Actors	Người dùng
Description	
Preconditions	1. Người dùng đã đăng nhập. 2. Căn hộ ở trạng thái "Đang đăng".
Postconditions	1. Yêu cầu đặt lịch được lưu thành công.

	2. Hệ thống hiển thị thông báo thành công và gửi thông báo cho các bên.		
Main Flow	ST T	Thực hiện	Hành động
	1	Người dùng	Nhấn button "Đặt lịch xem nhà".
	2	Hệ thống	Người dùng kiểm tra đăng nhập và căn hộ có phải của người dùng không
	3	Hệ thống	Kiểm tra lịch hẹn chưa hoàn thành của người dùng tại căn hộ này
	4a	Hệ thống	Hiển thị popup đặt lịch với các thông tin mặc định
	5a	Người dùng	Chọn Ngày và Khung giờ xem.
	6a	Người dùng	Nhập/chỉnh sửa thông tin liên hệ và ghi chú.
	7a	Người dùng	Nhấn button "Đặt lịch ngay".
	8a	Hệ thống	Validate dữ liệu và lưu vào cơ sở dữ liệu
	9a	Hệ thống	Gửi thông báo và đóng popup
Alternative Flows	AF-01: Đặt lịch gấp (Time-out rút ngắn) Điều kiện: Thời gian từ lúc đặt đến lúc hẹn thực tế ($T_{\text{hẹn}} - T_{\text{đặt}}$) nhỏ hơn 6 tiếng. - Hệ thống thực hiện các bước từ 1 đến 6 của Luồng chính. - Tại bước 7, hệ thống tính toán thời hạn xác nhận mới: $\text{Time-out} = (T_{\text{hẹn}} - T_{\text{đặt}}) - 1 \text{ giờ}$. - Hệ thống gửi thông báo cho Người dùng kèm thời hạn xác nhận rút ngắn để đảm bảo khách không phải chờ đợi quá sát giờ hẹn.		
Exception Flow	EF -01: Bước 3b Thực hiện cảnh báo (Bạn đã có lịch hẹn với căn hộ này. Vui lòng xem lại.) EF -02: Bước 2 Thực hiện cảnh báo (Bạn không thể đặt lịch căn hộ của chính mình.) EF -03: Lỗi kết nối hoặc tải trang không thành công. Hệ thống hiển thị thông báo: Có lỗi xảy ra, vui lòng thử lại sau!		

2.5.5.2. Xem lịch hẹn

UC ID	UC - 19			
UC Name	Xem danh sách lịch hẹn			
Actors	Người dùng			
Description				
Preconditions	Người dùng đã đăng nhập thành công vào hệ thống			
Postconditions	1. Danh sách lịch hẹn hiển thị chính xác theo tab phân hệ và bộ lọc trạng thái được chọn; 2. Số lượng bản ghi trên các nút bộ lọc được cập nhật đồng bộ.			
Main Flow		ST T	Thực hiện	Hành động
		1	Người dùng	Nhấn tab "Quản lý lịch hẹn".
		2	Hệ thống	Kiểm tra trạng thái đăng nhập của người dùng. Hệ thống lấy thông tin định danh Account_ID của phiên đăng nhập hiện tại để làm tham số truy vấn dữ liệu.
		3	Hệ thống	Kiểm tra lịch hẹn chưa hoàn thành của người dùng tại căn hộ này
		4a	Hệ thống	Hiển thị popup đặt lịch với các thông tin mặc định
		5a	Người dùng	Chọn Ngày và Khung giờ xem.
		6a	Người dùng	Nhập/chỉnh sửa thông tin liên hệ và ghi chú.
		7a	Người dùng	Nhấn button "Đặt lịch ngay".
		8a	Hệ thống	Validate dữ liệu và lưu vào cơ sở dữ liệu
		9a	Hệ thống	Gửi thông báo và đóng popup
Alternative Flows	AF-01: Đặt lịch gấp (Time-out rút ngắn) Điều kiện: Thời gian từ lúc đặt đến lúc hẹn thực tế (\$T_{\{hẹn\}} - T_{\{đặt\}}\$) nhỏ hơn 6 tiếng. - Hệ thống thực hiện các bước từ 1 đến 6 của Luồng chính.			

	2. Tại bước 7, hệ thống tính toán thời hạn xác nhận mới: $\\$Time-out = (T_{\{hẹn\}} - T_{\{đặt\}}) - 1 \text{ giờ.}$ 3. Hệ thống gửi thông báo cho Người dùng kèm thời hạn xác nhận rút ngắn để đảm bảo khách không phải chờ đợi quá sát giờ hẹn.
Exception Flow	EF -01: Bước 3b Thực hiện cảnh báo (Bạn đã có lịch hẹn với căn hộ này. Vui lòng xem lại.) EF -02: Bước 2 Thực hiện cảnh báo (Bạn không thể đặt lịch căn hộ của chính mình.) EF -03: Lỗi kết nối hoặc tải trang không thành công. Hệ thống hiển thị thông báo: "Có lỗi xảy ra, vui lòng thử lại sau! "

2.5.5.3. Chấp nhận lịch hẹn

UC ID	UC - 20			
UC Name	Chấp nhận lịch hẹn			
Actors	Người dùng (Chủ nhà)			
Description	Người dùng chấp nhận yêu cầu xem nhà của khách hàng.			
Preconditions	1. Lịch hẹn ở trạng thái "Chờ xác nhận" 2. Trong thời hạn Time-out.			
Postconditions	1. Trạng thái chuyển sang "Đã xác nhận" 2. Gửi thông báo cho khách.			
Main Flow		ST T	Thực hiện	Hành động
		1	Người dùng	Truy cập tab "Lịch của khách"
		2	Người dùng	Nhấn nút "Xác nhận" trên thẻ lịch hẹn tương ứng.
		3	Hệ thống	Hệ thống kiểm tra trạng thái và time - out
		4	Hệ thống	Truy vấn và hiển thị danh sách các thẻ lịch hẹn (Cards).
Alternative Flows	AF-01: Chuyển đổi sang xem lịch của khách hàng đặt 1. Người dùng nhấn chọn tab "Lịch của khách". 2. Hệ thống thực hiện thay đổi tham số truy vấn: tìm kiếm các bản ghi lịch hẹn thuộc các căn hộ mà Account_ID này đang làm chủ sở hữu (Host).			

	- Hệ thống tính toán lại số lượng bản ghi cho các nút bộ lọc của phân hệ mới. - Hệ thống render lại danh sách các thẻ lịch hẹn tương ứng của khách đặt. AF-02: Chuyển đổi số dòng hiển thị hoặc phân trang - Người dùng cuộn xuống cuối danh sách, chọn thay đổi số lượng dòng hiển thị (Ví dụ: từ 25 sang 50 dòng) hoặc nhấn nút chuyển sang "Trang kế tiếp". - Hệ thống thực hiện gửi request phân trang (page, limit) về server. - Hệ thống xóa danh sách cũ, tải và hiển thị tập dữ liệu mới lên màn hình.
Exception Flow	EF-01: Lỗi kết nối hệ thống (Network Error / Timeout) - Hệ thống chặn việc tải trang, hiển thị thông báo lỗi: <i>"Không thể tải danh sách lịch hẹn. Vui lòng thử lại sau!"</i>. - Giao diện hiển thị trạng thái trống (Empty State) kèm nút "Tải lại trang". EF-02: Không có dữ liệu lịch hẹn phù hợp - Hệ thống hiển thị hình ảnh minh họa trống kèm dòng thông báo: <i>"Không có lịch hẹn nào thuộc trạng thái này"</i>.

2.5.5.4. Từ chối lịch hẹn

UC ID	UC - 21		
UC Name	Từ chối lịch hẹn		
Actors	Người dùng		
Description	Người dùng không chấp nhận yêu cầu của khách và cung cấp lý do.		
Preconditions	Lịch hẹn ở trạng thái "Chờ xác nhận".		
Postconditions	Trạng thái chuyển sang "Từ chối".		
Main Flow	ST T	Thực hiện	Hành động
	1	Người dùng	Nhấn button " Từ chối " trên thẻ lịch hẹn.
	2	Hệ thống	Hiển thị Popup yêu cầu nhập/chọn lý do từ chối.
	3	Người dùng	Nhập lý do và nhấn xác nhận.

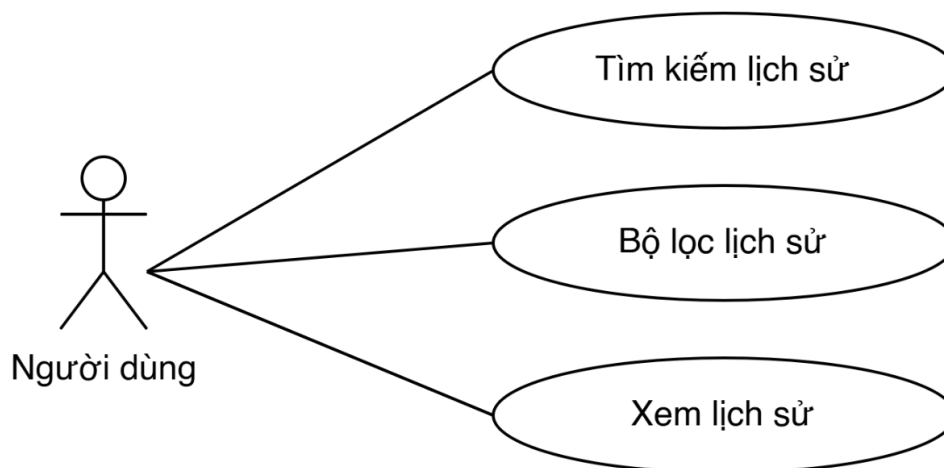
		4	Hệ thống	Cập nhật trạng thái thành "Từ chối" và lưu nội dung lý do.
		5	Hệ thống	Gửi thông báo từ chối cho Khách thuê.
Alternative Flows	N/A			
Exception Flow	N/A			

2.5.5.5. Hủy lịch hẹn

UC ID	UC - 22		
UC Name	Hủy lịch hẹn		
Actors	Người dùng		
Description	Người dùng hủy lịch đã xác nhận.		
Preconditions	Lịch hẹn ở trạng thái "Đã xác nhận", "Chờ xác nhận".		
Postconditions	Trạng thái chuyển sang "Chủ nhà hủy" hoặc "Hủy bởi khách".		
Main Flow	STT	Thực hiện	Hành động
	1	Người dùng	Tại danh sách lịch hẹn "Đã xác nhận", người dùng nhấn button "Hủy lịch" tương ứng.
	2	Hệ thống	Kiểm tra điều kiện thời gian hủy
	3	Hệ thống	Xác nhận điều kiện hợp lệ, hiển thị giao diện Popup yêu cầu người dùng nhập/chọn lý do hủy lịch.
	4	Người dùng	Nhập/chọn lý do và nhấn xác nhận nút "Hủy lịch" trên giao diện popup.
	5	Hệ thống	Kiểm tra tính bắt buộc của lý do hủy (không được để trống) theo quy tắc
	6	Hệ thống	Cập nhật dữ liệu vào DB: Đổi trạng thái lịch hẹn sang CANCELLED ("Đã Hủy"), ghi nhận cụ thể đối tượng thao tác ("Chủ nhà hủy" hoặc "Khách hủy") và lưu chuỗi văn bản lý do hủy.
	7	Hệ thống	Đóng popup, cập nhật giao diện bên ngoài danh sách và gửi

		thông báo (Push/Email) cho bên đối ứng còn lại biết lịch hẹn đã bị hủy.	
Alternative Flows	AF-01: Người dùng hủy lịch khi đang ở trạng thái "Chờ xử lý" 1. Tại bước 1 của luồng chính, cuộc hẹn đang ở trạng thái "Chờ xử lý". 2. Người dùng nhấn button "Từ chối" hoặc "Hủy yêu cầu" (tương đương hành động hủy lịch chủ động trước khi xác nhận). 3. Hệ thống bỏ qua bước 2 của luồng chính (không xét điều kiện $\\$ge\ 2\\$ giờ) và hiển thị trực tiếp Popup lý do hủy. 4. Thực hiện tiếp các bước 4, 5, 6, 7 của Luồng chính để đưa cuộc hẹn về trạng thái "Đã Hủy". AF-02: Người dùng hủy bỏ thao tác trên popup 1. Tại bước 3 hoặc bước 4 của luồng chính, người dùng nhấn nút "Quay lại" hoặc icon dấu "X" trên góc popup. 2. Hệ thống đóng popup, không lưu bất kỳ thay đổi nào và giữ nguyên trạng thái hiện tại của lịch hẹn.		
Exception Flow	EF-01: Vi phạm điều kiện thời gian hủy dưới 2 giờ (Đối với lịch đã xác nhận) 1. Tại bước 2 của Luồng chính, hệ thống kiểm tra và phát hiện khoảng cách thời gian từ lúc thực hiện đến lúc hẹn thực tế $\\$< 2\\$ giờ. 2. Hệ thống chặn không cho hiển thị popup nhập lý do, giữ nguyên trạng thái cuộc hẹn là "Đã xác nhận". 3. Hiển thị thông báo lỗi (TB_04): "<i>Bạn chỉ được hủy lịch trước giờ hẹn tối thiểu 2 giờ.</i>" EF-02: Bỏ trống trường lý do hủy lịch 1. Tại bước 4 của Luồng chính, người dùng để trống ô nhập nội dung lý do và nhấn nút xác nhận hủy. 2. Hệ thống phát hiện chuỗi dữ liệu rỗng. 3. Hệ thống ngăn chặn hành động lưu dữ liệu, giữ nguyên giao diện popup và hiển thị text cảnh báo đỏ: "<i>Vui lòng cung cấp lý do hủy lịch.</i>"		

2.5.6. Người dùng - Quản lý lịch sử giao dịch



Hình 6: Usecase Người dùng - Quản lý lịch sử giao dịch

2.5.6.1. Xem lịch sử giao dịch

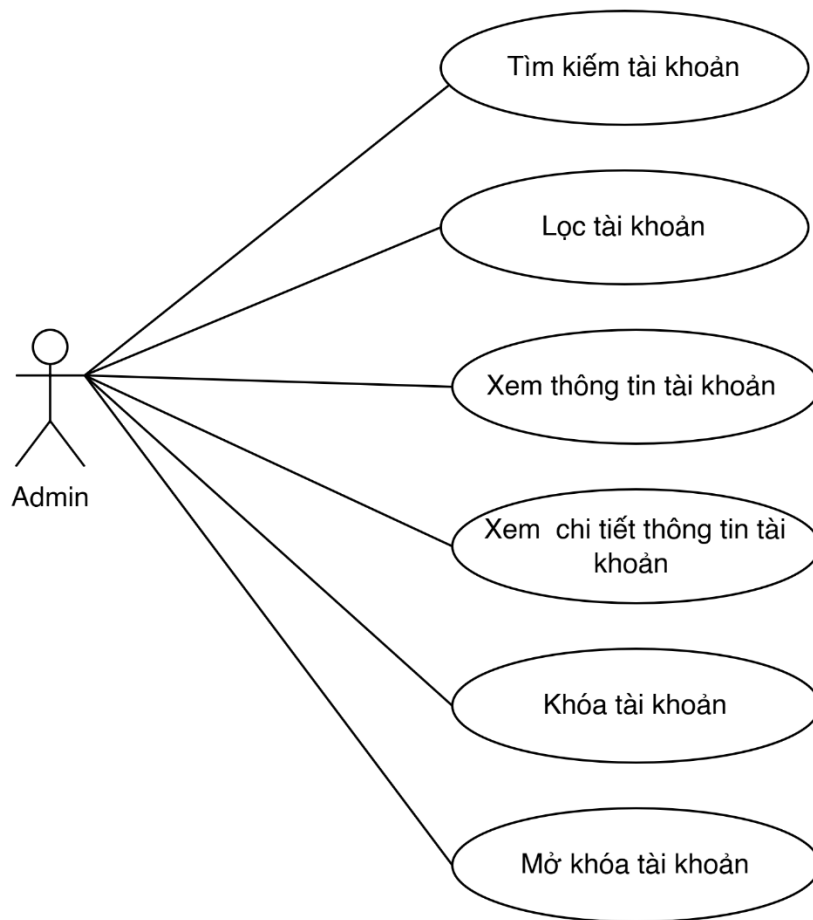
UC ID	UC - 23			
UC Name	Xem lịch sử và lọc giao dịch			
Actors	Người dùng hệ thống			
Description	Người dùng truy cập trang lịch sử để kiểm tra các giao dịch và lọc tìm kiếm bản ghi theo điều kiện cụ thể.			
Preconditions	Người dùng đã đăng nhập thành công vào tài khoản hệ thống.			
Postconditions	Hiển thị đúng danh sách giao dịch thỏa mãn điều kiện lọc.			
Main Flow	ST T	Thực hiện	Hành động	
	1	Người dùng	Nhấn vào liên kết "Lịch sử giao dịch" tại menu tài khoản cá nhân.	
	2	Hệ thống	Truy vấn cơ sở dữ liệu và tính toán hiển thị các khối tổng quan (Thành công, Chờ thanh toán, Tổng chi).	
	3	Hệ thống	Hiển thị bảng danh sách giao dịch mới nhất (mặc định sắp xếp thời gian giảm dần).	
	4	Người dùng	Nhập mã giao dịch vào thanh tìm kiếm và chọn trạng thái "Chờ thanh toán" từ bộ lọc nhanh.	
	5	Hệ thống	Thực hiện lọc dữ liệu real-time trên grid và trả về kết quả tương ứng thỏa mãn điều kiện.	

2.5.7. Người dùng - Chat

UC ID	UC - 24		
UC Name	Nhắn tin trực tuyến (Chat)		
Actors	Người dùng (Khách hàng/Chủ nhà)		
Description	Chức năng cho phép Khách thuê chủ động khởi tạo cuộc trò chuyện trực tuyến với Chủ tin đăng ngay từ màn hình chi tiết bất động sản để trao đổi thêm thông tin		
Preconditions	1. Khách thuê đã đăng nhập thành công vào tài khoản hệ thống. 2. Tin đăng của bất động sản đang ở trạng thái hoạt động ("Đang đăng").		
Postconditions	1. Cuộc hội thoại được khởi tạo thành công trong cơ sở dữ liệu và liên kết với phòng chat (Room_ID). 2. Tin nhắn được gửi và hiển thị tức thời trên giao diện của cả hai bên.		
Main Flow		ST T	Thực hiện Hành động
	1	Người dùng	Tại màn hình chi tiết tin đăng, khách thuê nhấn vào biểu tượng/nút " Nhắn tin " (Chat).
	2	Hệ thống	Kiểm tra trạng thái đăng nhập của Khách thuê. Nếu hợp lệ, hệ thống kiểm tra tiếp lịch sử trò chuyện cũ giữa Khách thuê và Chủ tin đăng.
	3	Hệ thống	Hệ thống tự động tạo một phòng chat mới (Room_ID) kết nối 2 tài khoản (hoặc mở lại phòng chat cũ nếu đã từng nhắn tin trước đó). Điều hướng Khách thuê vào màn hình cửa sổ Chat.
	4	Người dùng	Nhập nội dung văn bản vào ô nhập liệu và nhấn button " Gửi ".
	5	Hệ thống	Sử dụng giao thức kết nối Socket để đồng bộ và hiển thị tin nhắn ngay lập tức lên màn hình của cả Khách thuê lẫn Chủ tin đăng (Real-time), đồng thời bắn thông báo đẩy (Push Notification) cho Chủ tin đăng

Alternative Flows	AF-01: Chủ tin đăng phản hồi tin nhắn 1. Chủ tin đăng nhận được thông báo đầy, click vào thông báo để mở cửa sổ chat tương ứng. 2. Hệ thống tải toàn bộ lịch sử tin nhắn trong phòng chat (Room_ID). 3. Chủ tin đăng nhập nội dung văn bản phản hồi và nhấn "Gửi". 4. Hệ thống thực hiện tiếp các bước 6, 7 của Luồng chính để truyền tải tin nhắn ngược lại cho Khách thuê.
Exception Flow	EF-01: Người dùng chưa đăng nhập hệ thống 1. Tại bước 2 của luồng chính, hệ thống phát hiện Khách thuê chưa đăng nhập tài khoản. 2. Hệ thống chặn không điều hướng vào phòng chat, chuyển hướng người dùng đến màn hình Đăng nhập. EF-02: Tin nhắn gửi thất bại do mất kết nối mạng 1. Tại bước 6 của luồng chính, thiết bị của người dùng bị mất kết nối internet đột xuất khiến hệ thống không thể gửi gói tin lên server. 2. Hệ thống hiển thị icon dấu chấm than đỏ hoặc nhãn "Gửi thất bại" ngay cạnh bong bóng tin nhắn vừa nhập. 3. Cho phép người dùng nhấn vào icon đỏ để thực hiện gửi lại (Retry) tin nhắn đó khi mạng ổn định.

2.5.8. Admin - Quản lý tài khoản



Hình 7: Usecase Admin - Quản lý tài khoản

2.5.8.1. Khóa/Mở khóa

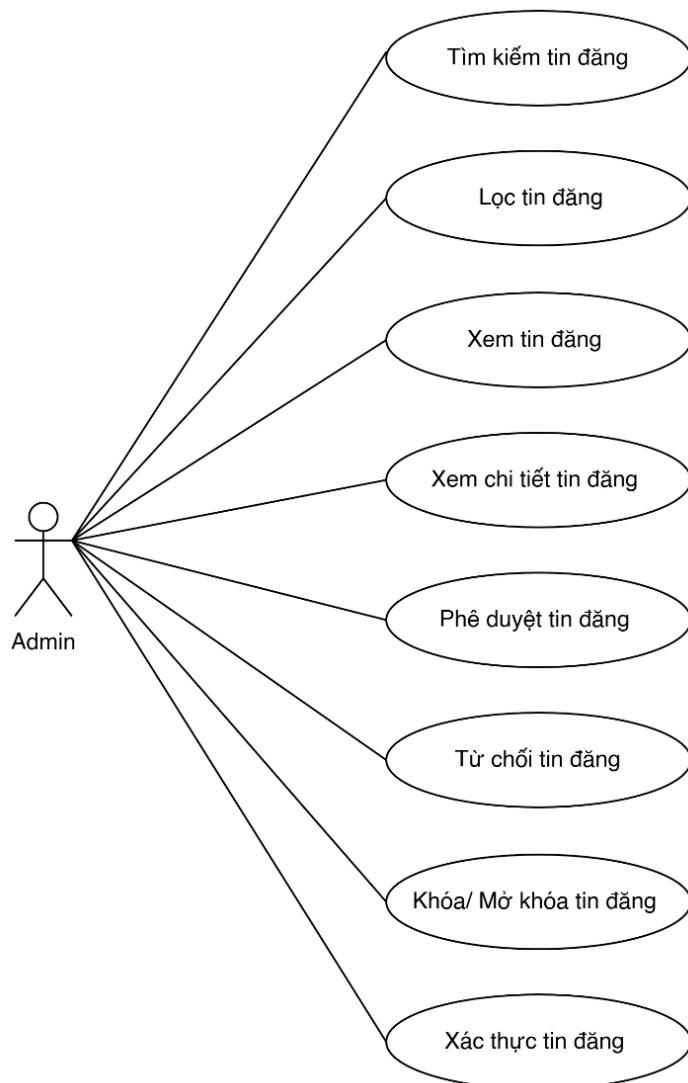
UC ID	UC - 25			
UC Name	Khóa tài khoản			
Actors	Admin			
Description	Admin thực hiện đình chỉ quyền truy cập và hoạt động của một tài khoản người dùng do phát hiện hành vi vi phạm quy chế.			
Preconditions	1. Admin đang ở màn hình danh sách hoặc màn hình chi tiết tài khoản. 2. Tài khoản mục tiêu đang ở trạng thái "Hoạt động".			
Postconditions	1. Trạng thái tài khoản đổi sang "Bị khóa" trong DB. 2. Hệ thống thu hồi phiên đăng nhập hiện tại của người dùng bị khóa và ẩn các tin đăng liên quan.			
Main Flow		ST T	Thực hiện	Hành động

		1	Admin	Nhấn chọn button " Khóa tài khoản ".
		2	Hệ thống	Hiển thị giao diện Popup "Xác nhận khóa tài khoản" chứa ô nhập văn bản lý do khóa.
		3	Admin	Nhập lý do khóa cụ thể vào ô text và nhấn button " Xác nhận khóa ".
		4	Hệ thống	Hệ thống: Đóng popup, hiển thị thông báo Toast thành công và cập nhật nhãn trạng thái trên danh sách thành "Đã khóa".
Alternative Flows	AF-01: Hủy bỏ thao tác mở khóa tài khoản - Tại bước 3, Admin nhấn nút "Quay lại" hoặc icon dấu "X" trên popup. - Hệ thống đóng popup, giữ nguyên trạng thái tài khoản là "Hoạt động" và không lưu dữ liệu.			
Exception Flow	N/A			

UC ID	UC - 26			
UC Name	Mở khóa tài khoản			
Actors	Admin			
Description	Admin thực hiện mở khóa của một tài khoản người dùng do nhận được khiếu nại từ tài khoản đã khóa.			
Preconditions	- Admin đang ở màn hình danh sách hoặc màn hình chi tiết tài khoản. - Tài khoản mục tiêu đang ở trạng thái "Đã khóa".			
Postconditions	1. Trạng thái tài khoản đổi sang "Hoạt động" trong DB.			
Main Flow		ST T	Thực hiện	Hành động
		1	Admin	Nhấn chọn button " Mở khóa tài khoản ".
		2	Hệ thống	Hiển thị giao diện Popup "Xác nhận mở khóa tài khoản".
		3	Admin	Nhập lý do khóa cụ thể vào ô text và nhấn button " Xác nhận khóa ".

		4	Hệ thống	Hệ thống: Đóng popup, hiển thị thông báo Toast thành công và cập nhật nhãn trạng thái trên danh sách thành "Hoạt động"
Alternative Flows	AF-01: Hủy bỏ thao tác khóa tài khoản - Tại bước 3, Admin nhấn nút "Quay lại" hoặc icon dấu "X" trên popup. - Hệ thống đóng popup, giữ nguyên trạng thái tài khoản là "Đã khóa" và không lưu dữ liệu.			
Exception Flow	N/A			

2.5.9. Admin - Quản lý tin đăng



Hình 8: Usecase Admin - Quản lý tin đăng

2.5.9.1. Xem danh sách tin đăng

UC ID	UC - 27																										
UC Name	Xem danh sách tin đăng																										
Actors	Admin																										
Description	Cho phép Admin xem danh sách toàn bộ tin đăng trên hệ thống.																										
Preconditions	1. Admin đã đăng nhập. 2. Admin có quyền truy cập chức năng Quản lý tin đăng.																										
Postconditions	Danh sách tin đăng được hiển thị thành công.																										
Main Flow		<table><tr><th>STT</th><th>Thực hiện</th><th>Hành động</th></tr><tr><td>1</td><td>Admin</td><td>Chọn menu Quản lý tin đăng.</td></tr><tr><td>2</td><td>Hệ thống</td><td>Kiểm tra quyền truy cập của Admin.</td></tr><tr><td>3</td><td>Hệ thống</td><td>Truy xuất danh sách tin đăng.</td></tr><tr><td>4</td><td>Hệ thống</td><td>Truy xuất thông tin gói tin, chủ nhà, trạng thái và thông tin duyệt tin.</td></tr><tr><td>5</td><td>Hệ thống</td><td>Hiển thị danh sách tin đăng theo thứ tự mặc định.</td></tr><tr><td>6</td><td>Hệ thống</td><td>Hiển thị phân trang dữ liệu.</td></tr><tr><td>7</td><td>Admin</td><td>Xem thông tin tin đăng trên danh sách.</td></tr></table>	STT	Thực hiện	Hành động	1	Admin	Chọn menu Quản lý tin đăng.	2	Hệ thống	Kiểm tra quyền truy cập của Admin.	3	Hệ thống	Truy xuất danh sách tin đăng.	4	Hệ thống	Truy xuất thông tin gói tin, chủ nhà, trạng thái và thông tin duyệt tin.	5	Hệ thống	Hiển thị danh sách tin đăng theo thứ tự mặc định.	6	Hệ thống	Hiển thị phân trang dữ liệu.	7	Admin	Xem thông tin tin đăng trên danh sách.	
STT	Thực hiện	Hành động																									
1	Admin	Chọn menu Quản lý tin đăng.																									
2	Hệ thống	Kiểm tra quyền truy cập của Admin.																									
3	Hệ thống	Truy xuất danh sách tin đăng.																									
4	Hệ thống	Truy xuất thông tin gói tin, chủ nhà, trạng thái và thông tin duyệt tin.																									
5	Hệ thống	Hiển thị danh sách tin đăng theo thứ tự mặc định.																									
6	Hệ thống	Hiển thị phân trang dữ liệu.																									
7	Admin	Xem thông tin tin đăng trên danh sách.																									
Alternative Flows	AF-01 (Chuyển trang) 1. Tại bước 6, nếu Admin chọn trang tiếp theo hoặc trang trước, hệ thống tải dữ liệu tương ứng với trang được chọn. AF-02 (Sắp xếp dữ liệu) 1. Tại bước 7, nếu Admin nhấn vào tiêu đề cột được hỗ trợ sắp xếp, hệ thống sắp xếp dữ liệu theo thứ tự tăng dần hoặc giảm dần. AF-03 (Làm mới danh sách) 1. Tại bước 7, nếu Admin tải lại trang hoặc nhấn nút làm mới, hệ thống tải lại dữ liệu mới nhất.																										
Exception Flow	EF-01 (Không có dữ liệu) 1. Tại bước 5, nếu hệ thống không có tin đăng nào, hiển thị thông báo. EF-02 (Không có quyền truy cập) 1. Tại bước 2, nếu người dùng không có quyền Admin, hệ thống từ chối truy cập và hiển thị thông báo. EF-03 (Lỗi tải dữ liệu)																										

	- Tại bước 3, nếu hệ thống không thể truy xuất dữ liệu, hiển thị thông báo TB_03. EF-04 (Mất kết nối) - Tại bước 3, nếu xảy ra lỗi mạng hoặc lỗi máy chủ, hệ thống hiển thị thông báo TB_04 và cho phép Admin thực hiện lại thao tác.
--	---

2.5.9.2. Xem chi tiết tin đăng

UC ID	UC - 28			
UC Name	Xem chi tiết tin đăng			
Actors	Admin			
Description	Cho phép Admin xem toàn bộ thông tin và thực hiện các tác vụ quản lý đối với một tin đăng.			
Preconditions	1. Admin đã đăng nhập. 2. Admin có quyền quản lý tin đăng. 3. Tin đăng tồn tại trên hệ thống.			
Postconditions	1. Thông tin chi tiết tin đăng được hiển thị. 2. Các thao tác của Admin được ghi nhận vào lịch sử.			
Main Flow		ST T	Thực hiện	Hành động
		1	Admin	Truy cập màn hình Quản lý tin đăng.
		2	Admin	Chọn một tin đăng cần xem.
		3	Hệ thống	Kiểm tra quyền truy cập.
		4	Hệ thống	Tải thông tin chi tiết tin đăng.
		5	Hệ thống	Hiển thị tab Thông tin.
		6	Hệ thống	Hiển thị tab Xác thực (nếu là tin Mua bán).
		7	Hệ thống	Hiển thị tab Cảnh báo.
		8	Hệ thống	Hiển thị tab Lịch sử tác vụ.
		9	Admin	Xem thông tin và thực hiện các tác vụ nghiệp vụ nếu cần.
Alternative Flows	NA			
Exception Flow	NA			

2.5.9.3. Từ chối tin đăng

UC ID	UC - 29				
UC Name	Từ chối tin đăng				
Actors	Admin				
Description	Cho phép Admin từ chối tin đăng không đáp ứng quy định của hệ thống.				
Preconditions	1. Admin đã đăng nhập. 2. Admin có quyền quản lý tin đăng. 3. Tin đăng tồn tại. 4. Tin đăng đang ở trạng thái Chờ duyệt.				
Postconditions	1. Tin đăng chuyển sang trạng thái Từ chối. 2. Lý do từ chối được lưu. 3. Người đăng nhận được thông báo.				
Main Flow		ST T	Thực hiện	Hành động	
		1	Admin	Mở màn hình Chi tiết tin đăng.	
		2	Admin	Nhấn nút Từ chối.	
		3	Hệ thống	Hiển thị popup Từ chối tin đăng.	
		4	Hệ thống	Hiển thị danh sách lý do từ chối có sẵn.	
		5	Admin	Chọn một hoặc nhiều lý do từ chối.	
		6	Admin	Nhập lý do khác (nếu cần).	
		7	Admin	Nhấn nút Xác nhận.	
		8	Hệ thống	Kiểm tra dữ liệu nhập.	
		9	Hệ thống	Cập nhật trạng thái tin thành Từ chối.	
		10	Hệ thống	Lưu lý do từ chối, người thực hiện và thời gian xử lý.	
		11	Hệ thống	Gửi thông báo cho người đăng tin.	
		12	Hệ thống	Hiển thị thông báo từ chối thành công.	
Alternative Flows	AF-01 (Chọn lý do có sẵn) 1. Tại bước 5, nếu Admin chọn một hoặc nhiều lý do có sẵn, hệ thống lưu các lý do đã chọn vào kết quả từ chối. AF-02 (Nhập lý do khác) 1. Tại bước 6, nếu Admin nhập nội dung vào trường "Lý do khác", hệ thống lưu nội dung này kèm theo các lý do đã chọn.				

	AF-03 (Chỉ nhập lý do khác) - Tại bước 6, nếu Admin không chọn lý do có sẵn nhưng nhập nội dung vào trường "Lý do khác", hệ thống vẫn cho phép thực hiện từ chối. AF-04 (Hủy từ chối) - Tại bước 7, nếu Admin nhấn nút Hủy hoặc biểu tượng "X", hệ thống đóng popup từ chối và không lưu dữ liệu.
Exception Flow	EF-01 (Chưa chọn hoặc nhập lý do) - Tại bước 8, nếu Admin không chọn lý do có sẵn và không nhập lý do khác, hệ thống hiển thị thông báo EF-02 (Tin không ở trạng thái Chờ duyệt) - Tại bước 9, nếu tin đăng không còn ở trạng thái Chờ duyệt, hệ thống hiển thị thông báo. EF-03 (Lý do khác vượt quá giới hạn) - Tại bước 8, nếu nội dung lý do khác vượt quá 500 ký tự, hệ thống hiển thị thông báo. EF-04 (Không tìm thấy tin đăng) - Tại bước 9, nếu hệ thống không tìm thấy tin đăng, hiển thị thông báo. EF-05 (Lỗi cập nhật dữ liệu) - Tại bước 9, nếu hệ thống không thể cập nhật trạng thái tin đăng, hiển thị thông báo và cho phép Admin thực hiện lại thao tác.

2.5.9.4. Duyệt tin đăng

UC ID	UC - 30
UC Name	Duyệt tin đăng
Actors	Admin
Description	Cho phép Admin phê duyệt tin đăng hợp lệ để hiển thị trên hệ thống.
Preconditions	- Admin đã đăng nhập. - Admin có quyền quản lý tin đăng. - Tin đăng tồn tại trên hệ thống. - Tin đăng đang ở trạng thái Chờ duyệt.
Postconditions	- Tin đăng chuyển sang trạng thái Đang đăng. - Hệ thống lưu thông tin duyệt. - Người đăng nhận được thông báo duyệt thành công.

Main Flow	ST T	Thực hiện	Hành động
	1	Admin	Mở màn hình Chi tiết tin đăng.
	2	Admin	Kiểm tra thông tin tin đăng.
	3	Admin	Nhấn nút Duyệt tin.
	4	Hệ thống	Kiểm tra trạng thái hiện tại của tin đăng.
	5	Hệ thống	Cập nhật trạng thái tin thành Đang đăng.
	6	Hệ thống	Lưu người duyệt và thời gian duyệt.
	7	Hệ thống	Ghi nhận lịch sử tác vụ.
	8	Hệ thống	Gửi thông báo cho người đăng tin.
	9	Hệ thống	Hiển thị thông báo duyệt thành công.
Alternative Flows	AF-01 (Đóng màn hình) 1. Tại bước 2, nếu Admin thoát khỏi màn hình chi tiết tin đăng, hệ thống không thực hiện thao tác duyệt.		
Exception Flow	EF-01 (Tin không ở trạng thái Chờ duyệt) 1. Tại bước 6, nếu tin đăng không còn ở trạng thái Chờ duyệt, hệ thống hiển thị thông báo. EF-02 (Không tìm thấy tin đăng) 1. Tại bước 6, nếu hệ thống không tìm thấy tin đăng, hiển thị thông báo. EF-03 (Không có quyền thực hiện) 1. Tại bước 6, nếu người dùng không có quyền Admin, hệ thống hiển thị thông báo. EF-04 (Lỗi cập nhật dữ liệu) 1. Tại bước 7, nếu hệ thống không thể cập nhật trạng thái tin đăng, hiển thị thông báo và cho phép Admin thực hiện lại thao tác. EF-05 (Mất kết nối hệ thống) 1. Tại bước 7, nếu xảy ra lỗi mạng hoặc lỗi máy chủ, hệ thống hiển thị thông báo.		

2.5.9.5 Khóa tin đăng

UC ID	UC - 31
UC Name	Khóa tin đăng
Actors	Admin

Description	Cho phép Admin khóa tin đang hoạt động trên hệ thống.		
Preconditions	- Admin đã đăng nhập. - Admin có quyền quản lý tin đăng. - Tin đăng tồn tại. - Tin đăng đang ở trạng thái Đang đăng.		
Postconditions	- Tin đăng chuyển sang trạng thái Khóa. - Lý do khóa được lưu. - Người đăng nhận được thông báo.		
Main Flow	ST T	Thực hiện	Hành động
	1	Admin	Mở màn hình Chi tiết tin đăng.
	2	Admin	Nhấn nút Khóa tin.
	3	Hệ thống	Hiện thị popup Khóa tin đăng.
	4	Hệ thống	Hiện thị danh sách lý do khóa tin.
	5	Admin	Chọn một hoặc nhiều lý do khóa.
	6	Admin	Nhập lý do khác (nếu có).
	7	Admin	Nhấn nút Xác nhận.
	8	Hệ thống	Kiểm tra dữ liệu nhập.
	9	Hệ thống	Kiểm tra trạng thái hiện tại của tin đăng.
	10	Hệ thống	Cập nhật trạng thái tin thành Khóa.
	11	Hệ thống	Lưu lý do khóa, người thực hiện và thời gian khóa.
	12	Hệ thống	Ghi nhận lịch sử tác vụ.
	13	Hệ thống	Gửi thông báo đến người đăng tin.
	14	Hệ thống	Hiện thị thông báo khóa tin thành công.
Alternative Flows	AF-01 (Hủy khóa tin) Tại bước 7, nếu Admin nhấn nút Hủy hoặc biểu tượng "X", hệ thống đóng popup và không lưu dữ liệu.		
Exception Flow	EF-02 (Tin không ở trạng thái Đang đăng) - Tại bước 9, nếu tin đăng không ở trạng thái Đang đăng, hệ thống hiện thị thông báo. EF-03 (Lý do khác vượt quá giới hạn) - Tại bước 8, nếu nội dung lý do khác vượt quá 500 ký tự, hệ thống hiện thị thông báo. EF-04 (Không tìm thấy tin đăng)		

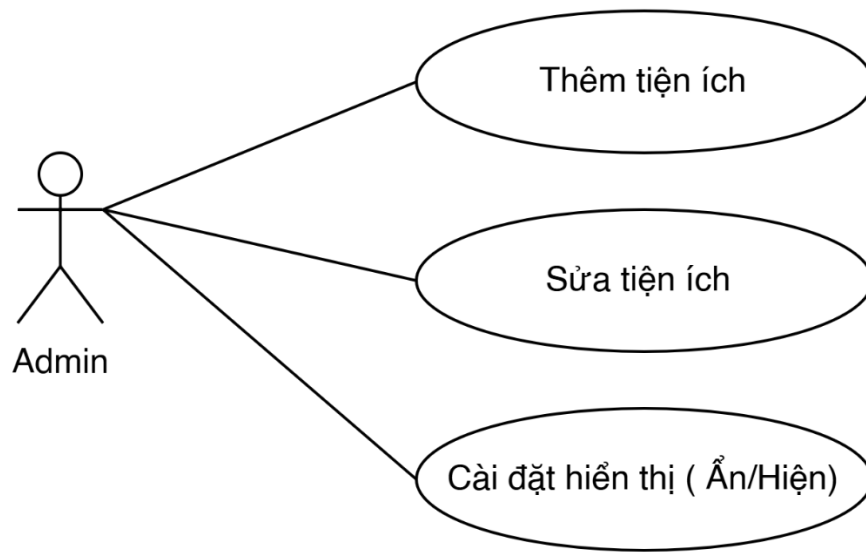
	1. Tại bước 9, nếu hệ thống không tìm thấy tin đăng, hiển thị thông báo. EF-05 (Lỗi cập nhật dữ liệu) 1. Tại bước 10, nếu hệ thống không thể cập nhật trạng thái tin đăng, hiển thị thông báo và cho phép thực hiện lại thao tác. EF-06 (Mất kết nối hệ thống) 1. Tại bước 10, nếu xảy ra lỗi mạng hoặc lỗi máy chủ, hệ thống hiển thị thông báo.
--	--

2.5.9.6. Xác thực tin đăng

UC ID	UC - 32			
UC Name	Xác thực tin đăng			
Actors	Admin			
Description	Cho phép Admin xác thực hồ sơ xác thực của tin đăng Mua bán.			
Preconditions	1. Admin đã đăng nhập hệ thống. 2. Admin có quyền quản lý tin đăng. 3. Tin đăng tồn tại. 4. Tin đăng thuộc nhu cầu Mua bán. 5. Trạng thái xác thực hiện tại là Chờ duyệt.			
Postconditions	1. Trạng thái xác thực chuyển thành Đã xác thực. 2. Thông tin xác thực được lưu. 3. Người đăng nhận được thông báo.			
Main Flow		S T T	Thực hiện	Hành động
		1	Admin	Mở màn hình Chi tiết tin đăng.
		2	Admin	Chọn tab Xác thực.
		3	Hệ thống	Hiển thị hồ sơ xác thực của tin đăng.
		4	Admin	Kiểm tra giấy tờ, hình ảnh và video xác thực.
		5	Admin	Nhấn nút Xác thực.
		6	Hệ thống	Kiểm tra điều kiện xác thực.
		7	Hệ thống	Cập nhật trạng thái xác thực thành Đã xác thực.
		8	Hệ thống	Lưu người xác thực và thời gian xác thực.
		9	Hệ thống	Ghi nhận lịch sử tác vụ.

		10	Hệ thống	Gửi thông báo đến người đăng tin.	
		11	Hệ thống	Hiển thị thông báo xác thực thành công.	
Alternative Flows	AF-01 (Xem hồ sơ xác thực) - Tại bước 3, Admin có thể xem toàn bộ giấy tờ, hình ảnh xác thực trước khi thực hiện xác thực. AF-02 (Thoát màn hình) - Tại bước 4, nếu Admin thoát khỏi màn hình chi tiết tin đăng, hệ thống không thực hiện xác thực và không lưu dữ liệu.				
Exception Flow	EF-01 (Tin đăng không thuộc nhu cầu Mua bán) - Tại bước 6, nếu tin đăng không thuộc nhu cầu Mua bán, hệ thống hiển thị thông báo. EF-02 (Hồ sơ không ở trạng thái Chờ duyệt) - Tại bước 6, nếu trạng thái xác thực không phải Chờ duyệt, hệ thống hiển thị thông báo. EF-03 (Không tìm thấy tin đăng) - Tại bước 6, nếu hệ thống không tìm thấy tin đăng, hiển thị thông báo. EF-04 (Không có quyền thực hiện) - Tại bước 6, nếu người dùng không có quyền Admin, hệ thống hiển thị thông báo . EF-05 (Lỗi cập nhật dữ liệu) - Tại bước 7, nếu hệ thống không thể cập nhật trạng thái xác thực, hiển thị thông báo và cho phép thực hiện lại thao tác. EF-06 (Mất kết nối hệ thống) - Tại bước 7, nếu xảy ra lỗi mạng hoặc lỗi máy chủ, hệ thống hiển thị thông báo .				

2.5.10. Admin - Quản lý tiện ích



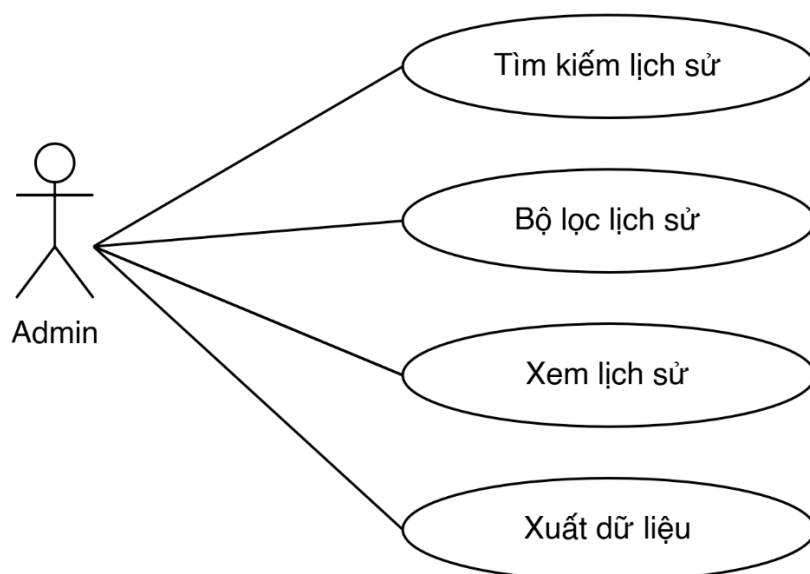
Hình 9: Usecase Admin - Quản lý tiện ích

2.5.10.1. Thêm mới/Sửa tiện ích

UC ID	UC - 33			
UC Name	Thêm mới/Sửa tiện ích			
Actors	Admin			
Description	Admin thực hiện tạo mới hoặc cập nhật thông tin tiện ích trong hệ thống.			
Preconditions	Admin đã đăng nhập vào hệ thống quản trị.			
Postconditions	Thông tin tiện ích được lưu và cập nhật trạng thái trên danh sách.			
Main Flow	ST	Thực hiện	Hành động	
	1	Admin	Nhấn button "+ Thêm tiện ích".	
	2	Hệ thống	Hiển thị popup "Thêm mới tiện ích" với các trường trống.	
	3	Admin	Nhập Tên tiện ích, Mô tả và chọn Trạng thái.	
	4	Admin	Nhấn button "Xác nhận" (Lưu).	
	5	Hệ thống	Validate dữ liệu (BR-01, BR-04). Lưu vào DB và hiển thị thông báo thành công (TB_01).	
	6	Hệ thống	Đóng popup và cập nhật lại bảng danh sách tiện ích.	

Alternative Flows	AF-01: Sửa tiện ích 1. Tại màn hình danh sách, Admin nhấn icon "Sửa" (ba chấm) tại dòng tiện ích tương ứng. 2. Hệ thống hiển thị popup "Sửa tiện ích" kèm dữ liệu cũ. 3. Admin thay đổi thông tin và nhấn "Xác nhận". Hệ thống thực hiện bước 5, 6 của Main Flow.
Exception Flow	EF – 01: Trùng tên tiện ích 1. Tại bước 5, nếu tên tiện ích đã tồn tại, hệ thống hiển thị thông báo lỗi và giữ nguyên popup để Admin sửa lại.

2.5.11. Admin - Quản lý tiện ích



Hình 10: Usecase Admin - Quản lý tiện ích

2.5.11.1. Xem lịch sử giao dịch

UC ID	UC - 34		
UC Name	Xem lịch sử và lọc giao dịch		
Actors	Admin		
Description	Người dùng truy cập trang lịch sử để kiểm tra các giao dịch và lọc tìm kiếm bản ghi theo điều kiện cụ thể.		
Preconditions	Người dùng đã đăng nhập thành công vào tài khoản hệ thống.		
Postconditions	Hiển thị đúng danh sách giao dịch thỏa mãn điều kiện lọc.		
Main Flow		ST T	Thực hiện Hành động

		1	Admin	Nhấn chọn mục "Lịch sử giao dịch" tại menu điều hướng bên trái.
		2	Hệ thống	Truy vấn cơ sở dữ liệu và tính toán hiển thị các khối tổng quan (Thành công, Chờ thanh toán, Tổng chi).
		3	Hệ thống	Hiển thị bảng danh sách giao dịch mới nhất (mặc định sắp xếp thời gian giảm dần).
		4	Admin	Nhập mã giao dịch vào thanh tìm kiếm và chọn trạng thái "Chờ thanh toán" từ bộ lọc nhanh.
		5	Hệ thống	Thực hiện lọc dữ liệu real-time trên grid và trả về kết quả tương ứng thỏa mãn điều kiện.

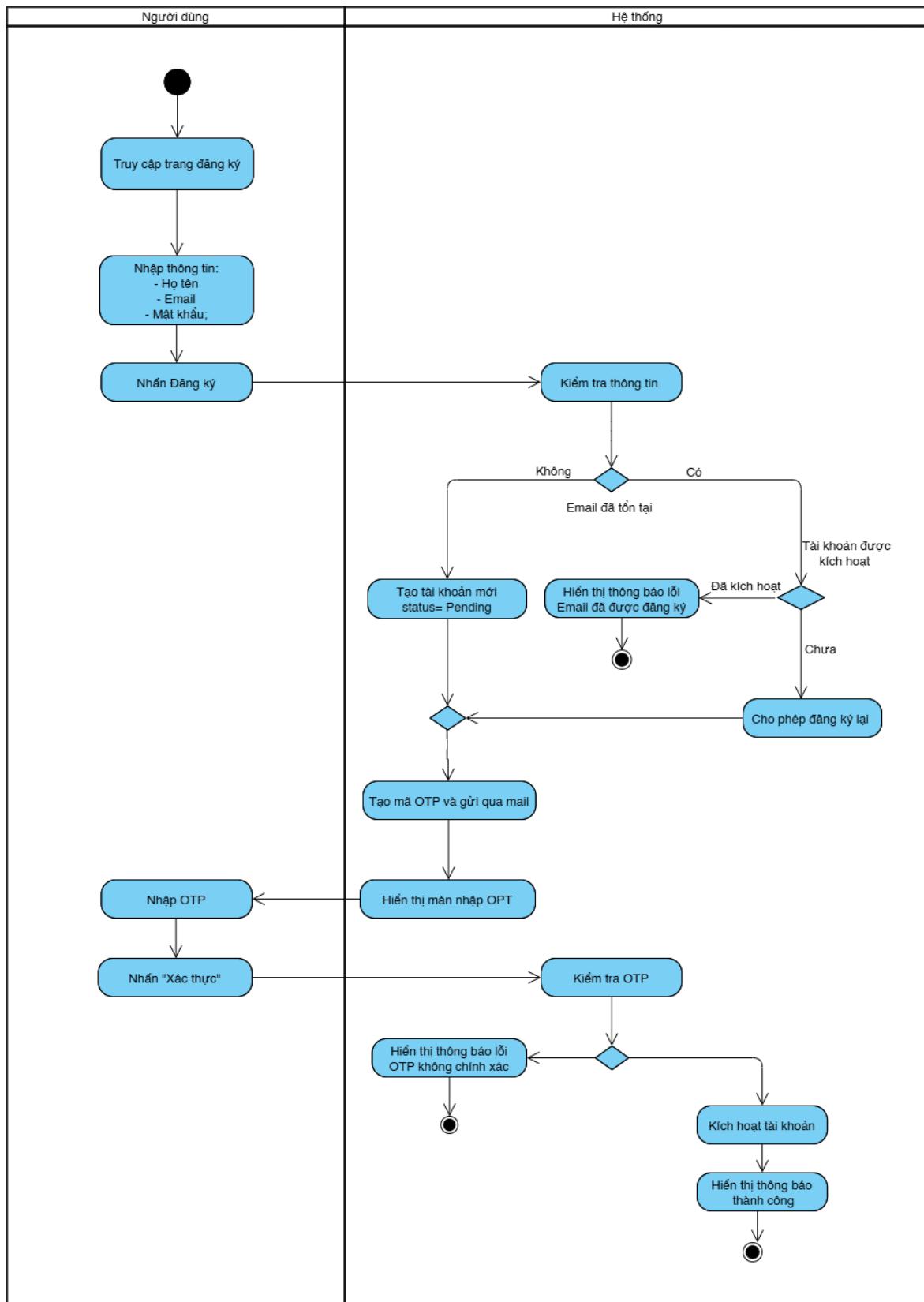
2.5.11.2. Xuất báo cáo

UC ID	UC - 35			
UC Name	Xuất báo cáo			
Actors	Admin			
Description	Cho phép Admin kết xuất toàn bộ hoặc một phần dữ liệu lịch sử giao dịch đang hiển thị dựa theo bộ lọc ra một tệp tin định dạng .csv để phục vụ công tác đối soát kế toán và lưu trữ nội bộ.			
Preconditions	Admin đang ở màn hình "Lịch sử giao dịch" và hệ thống đã tải xong dữ liệu bảng danh sách.			
Postconditions	Một tệp tin dạng file Excel/CSV chứa đúng và đủ các trường thông tin giao dịch được tải xuống thiết bị của Admin.			
Main Flow		STT	Thực hiện	Hành động
		1	Admin	Nhấn chọn button " Xuất báo cáo (CSV) "
		2	Hệ thống	Thực hiện quét toàn bộ các bản ghi giao dịch thỏa mãn điều kiện lọc hiện tại từ Database
		3	Hệ thống	Chuyển đổi cấu trúc dữ liệu thành định dạng CSV chuẩn với các tiêu đề cột:
		5	Hệ thống	Trả file về client để tự động kích hoạt tiến trình tải xuống (Download) của trình duyệt web,

				mở lại nút bấm và hiển thị Toast thông báo thành công	
Alternative Flows	AF-01: Xuất báo cáo phân mảnh theo bộ lọc cụ thể - Admin thực hiện thiết lập các điều kiện lọc (Ví dụ: Chỉ lọc các giao dịch "Thất bại" từ ngày 01/06/2026 đến ngày 25/06/2026). - Admin nhấn chọn nút "Xuất báo cáo (CSV)". - Hệ thống chỉ trích xuất dữ liệu của những bản ghi khớp với điều kiện thời gian và trạng thái đó để tạo file CSV tải xuống thiết bị.				
Exception Flow	EF-01: Lỗi hệ thống khi tệp dữ liệu quá lớn hoặc lỗi kết nối - Tại bước 3, số lượng dòng dữ liệu cần kết xuất vượt ngưỡng giới hạn xử lý của bộ nhớ hoặc kết nối mạng với DB bị ngắt giữa chừng. - Hiển thị thông báo lỗi hệ thống dạng Toast: <i>"Xuất báo cáo thất bại, vui lòng giới hạn khoảng thời gian lọc và thử lại!"</i>				

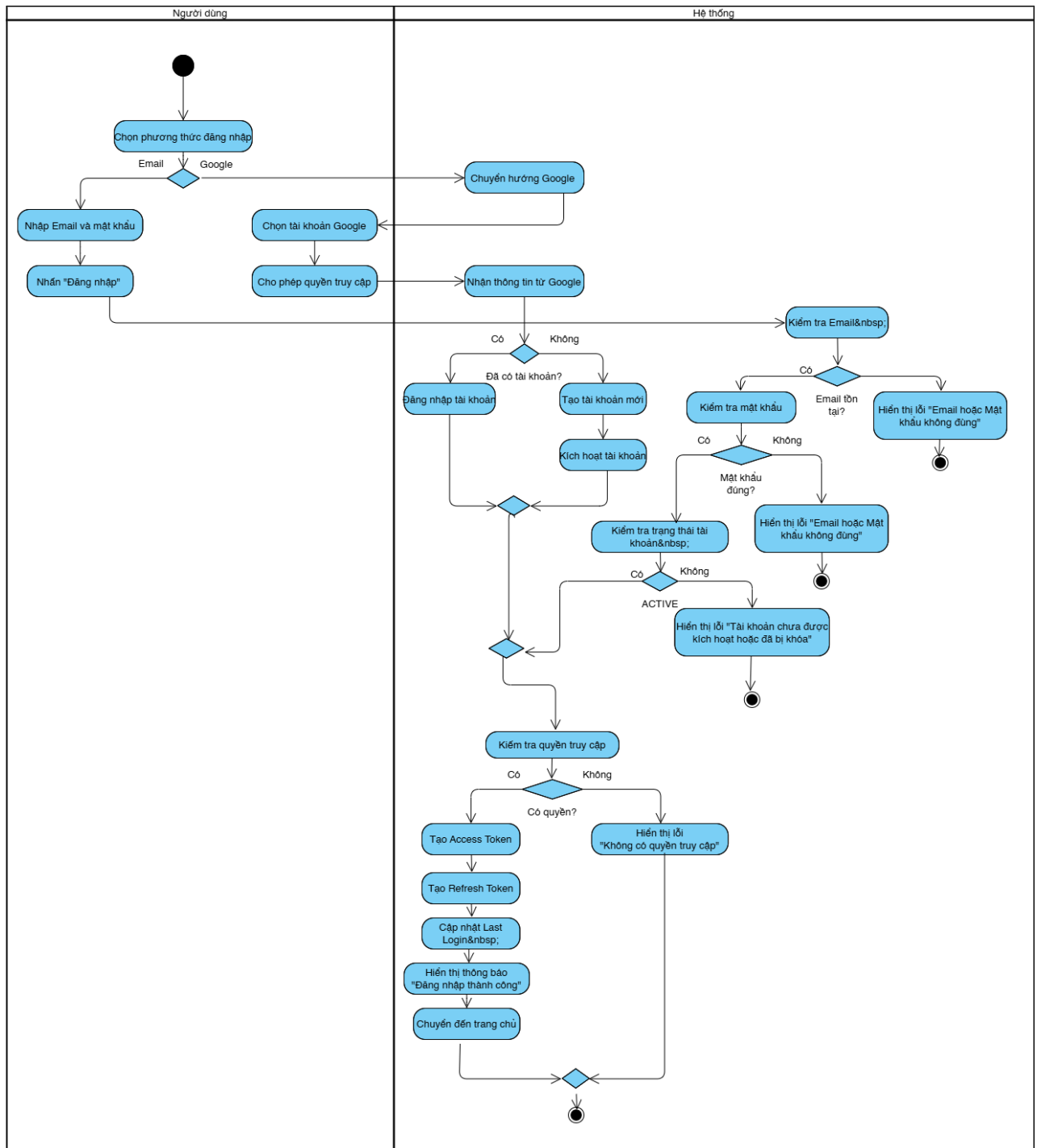
2.6. Biểu đồ hoạt động (Các chức năng chính)

2.6.1. Activity Diagram: Người dùng - Đăng ký tài khoản



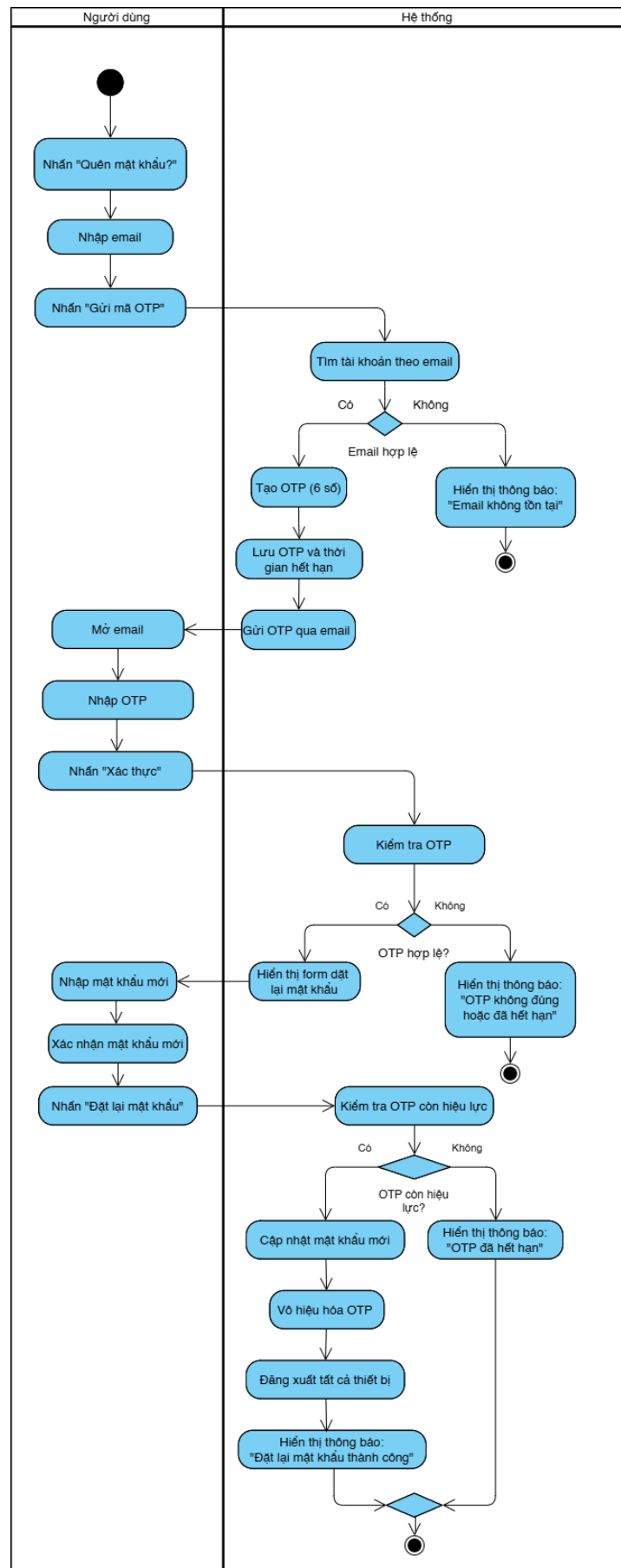
Hình 11: Activity Diagram: Người dùng - Đăng ký tài khoản

2.6.2. Activity Diagram: Người dùng - Đăng nhập



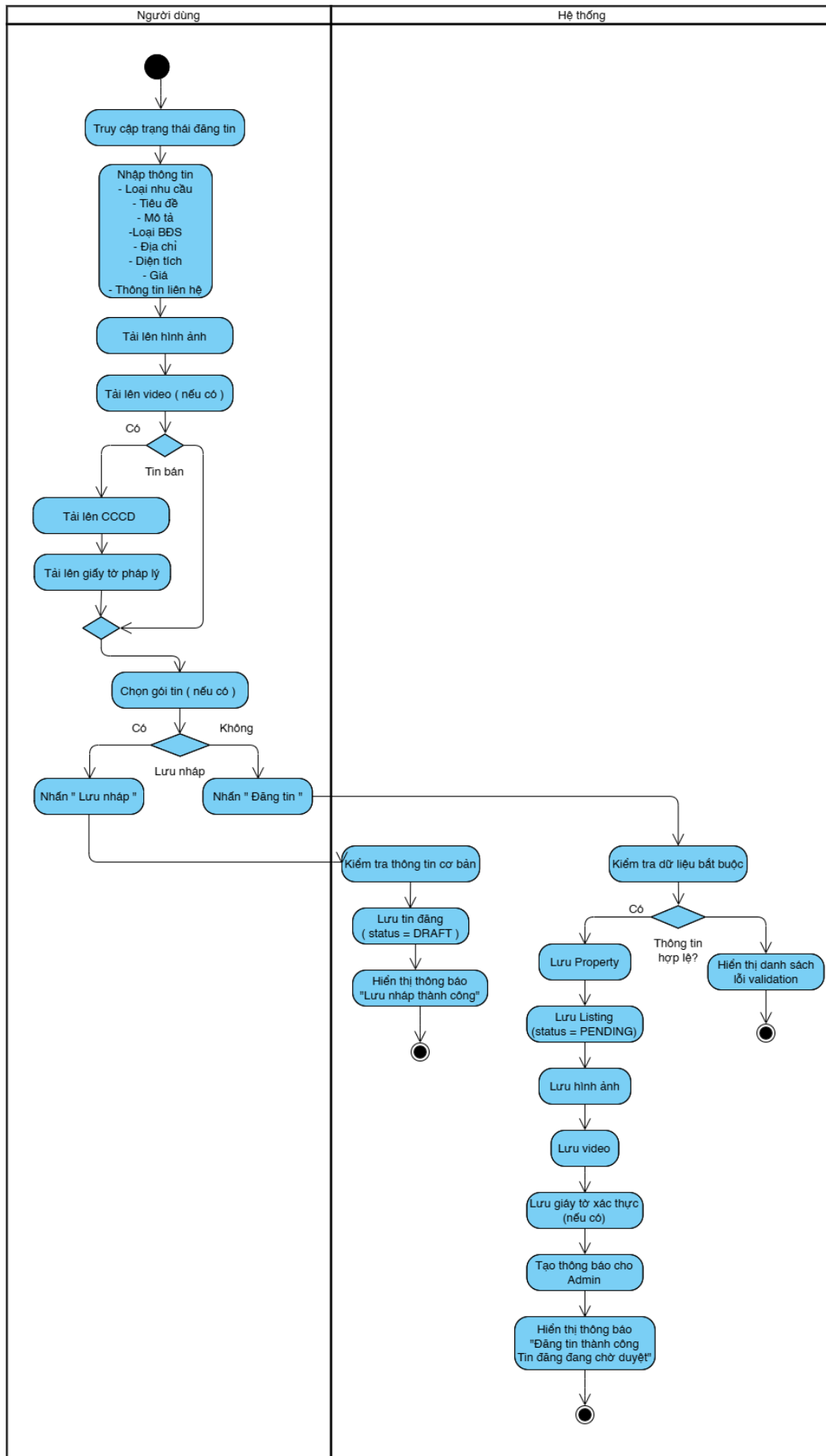
Hình 12: Activity Diagram: Người dùng - Đăng nhập

2.6.3. Activity Diagram: Người dùng - Quên mật khẩu



Hình 13: Activity Diagram: Người dùng - Quên mật khẩu

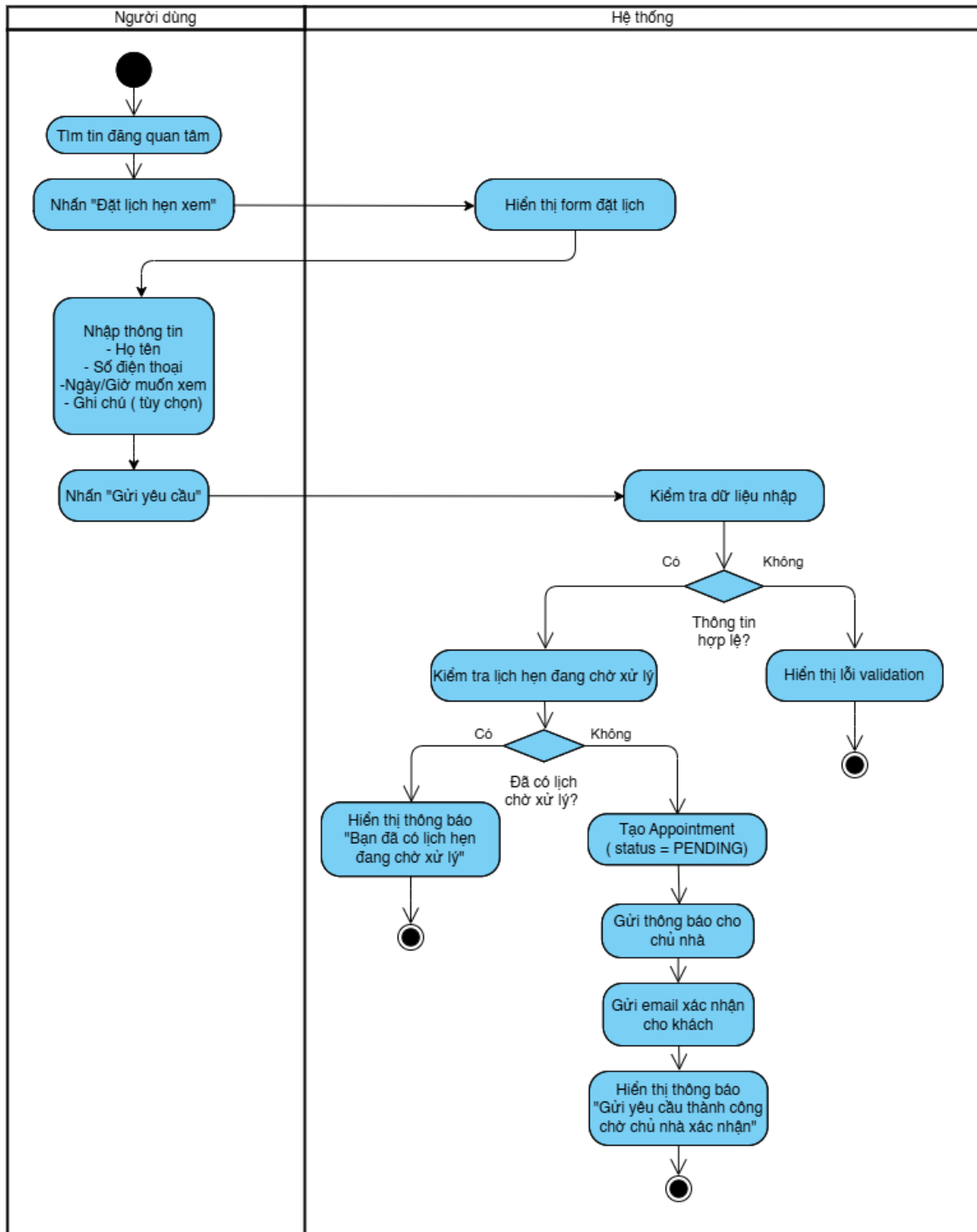
2.6.4. Activity Diagram: Người dùng - Tạo tin đăng



2.6.5. Activity Diagram: Người dùng - Chỉnh sửa tin đăng

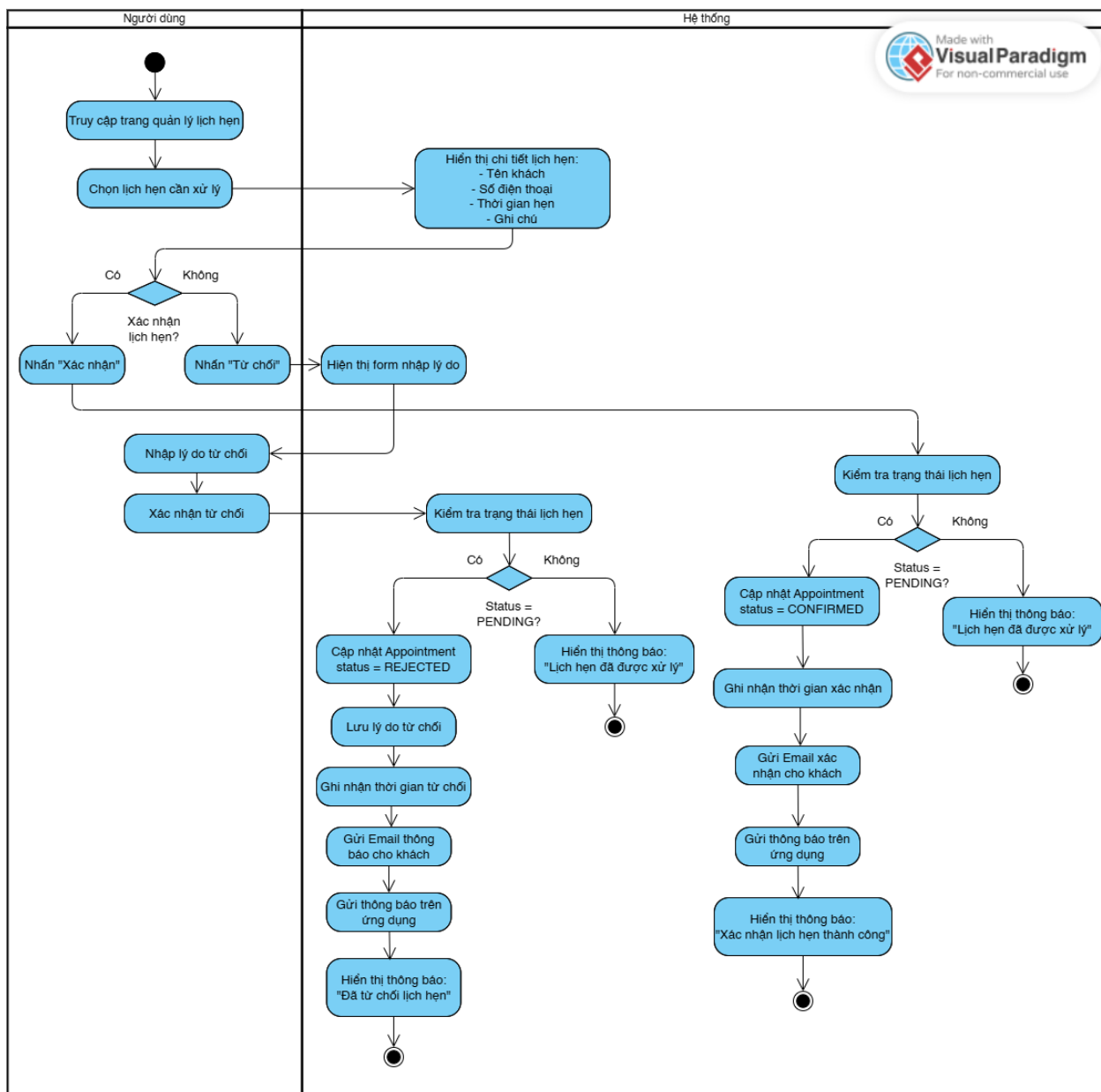


2.6.6. Activity Diagram: Người dùng - Đặt lịch hẹn



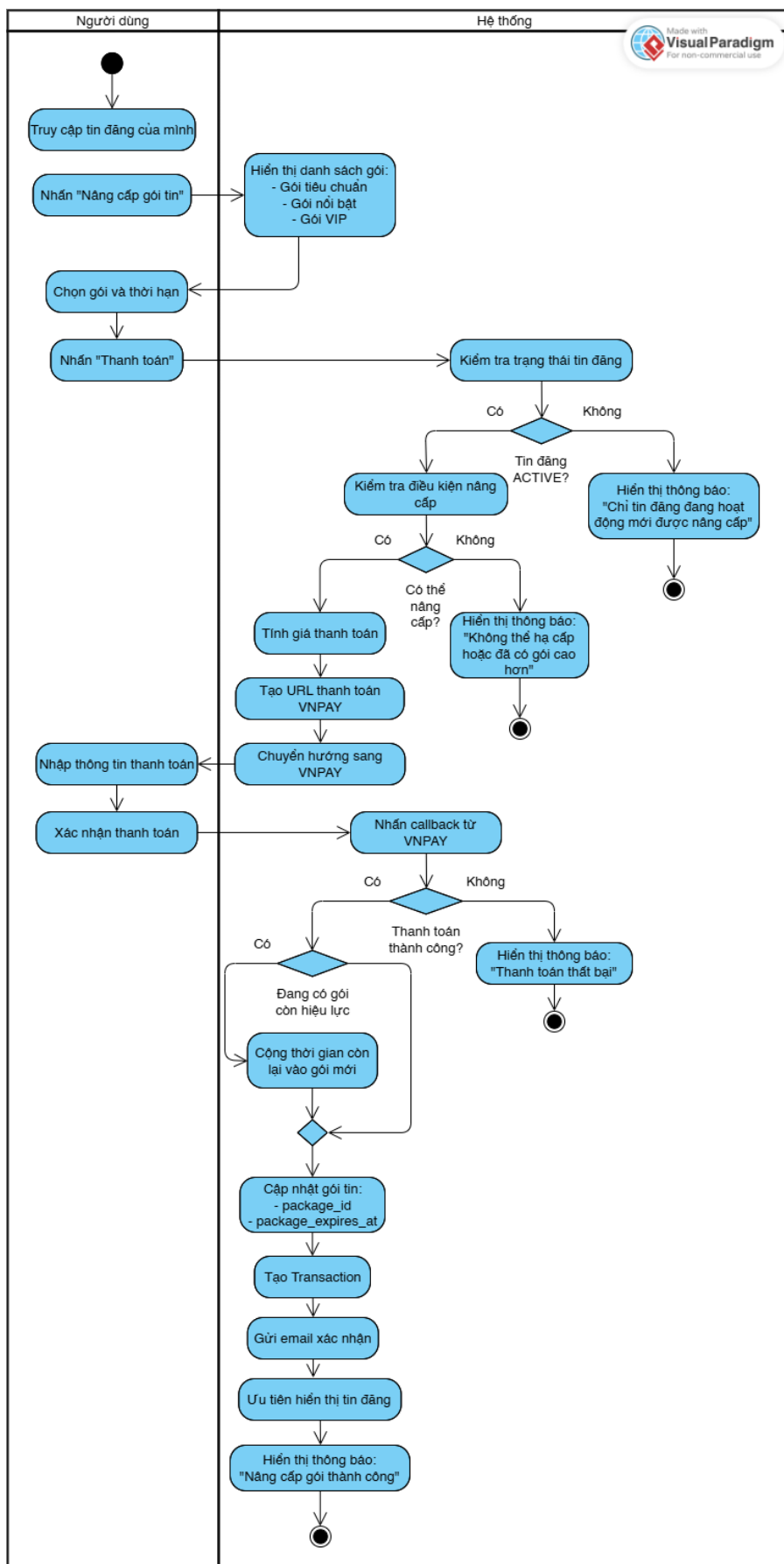
Hình 16: Activity Diagram: Người dùng - Đặt lịch hẹn

2.6.7. Activity Diagram: Người dùng - Xác nhận/Từ chối lịch hẹn



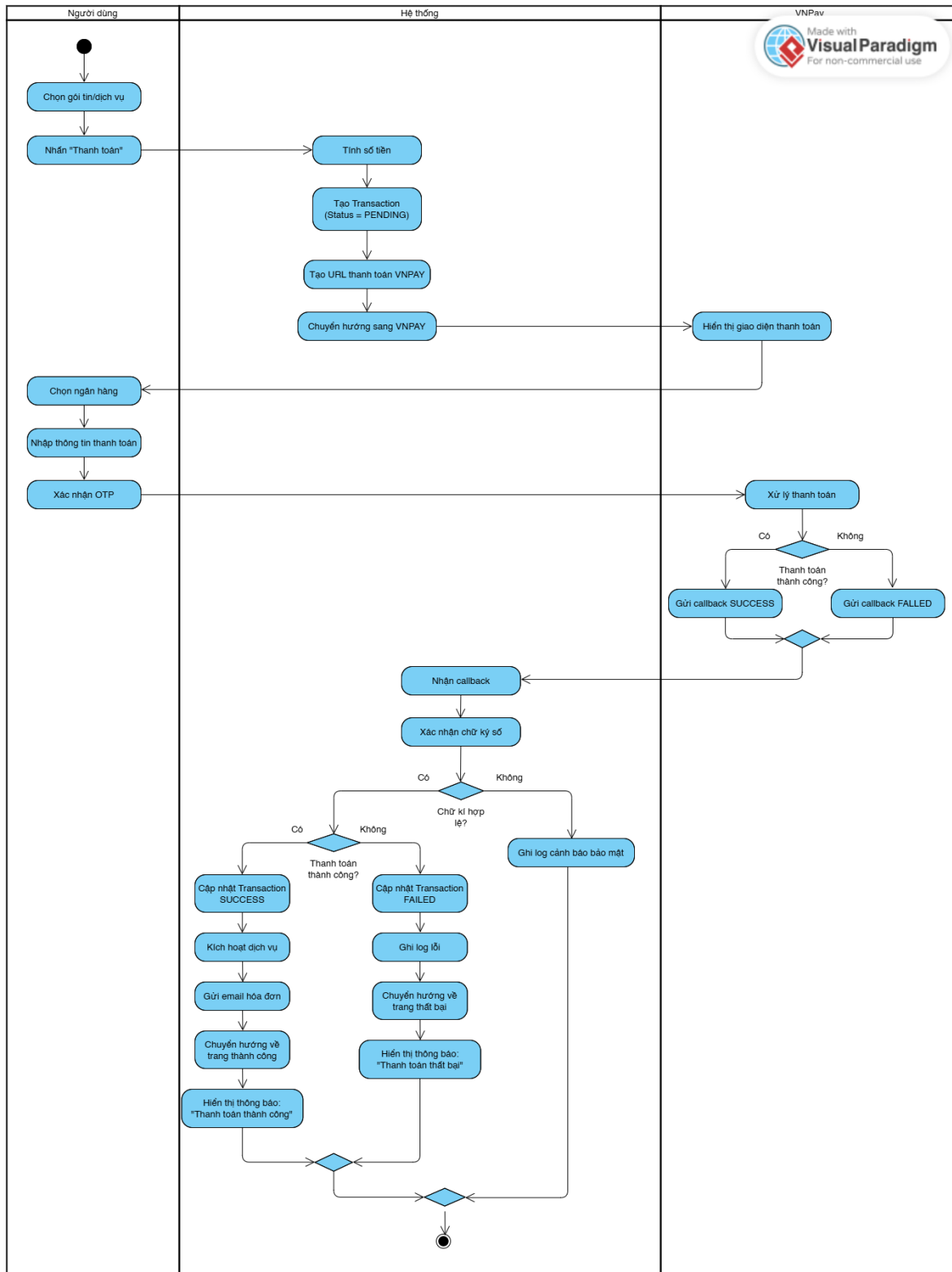
Hình 17: Activity Diagram: Người dùng - Xác nhận/Từ chối lịch hẹn

2.6.8. Activity Diagram: Người dùng - Nâng cấp gói tin



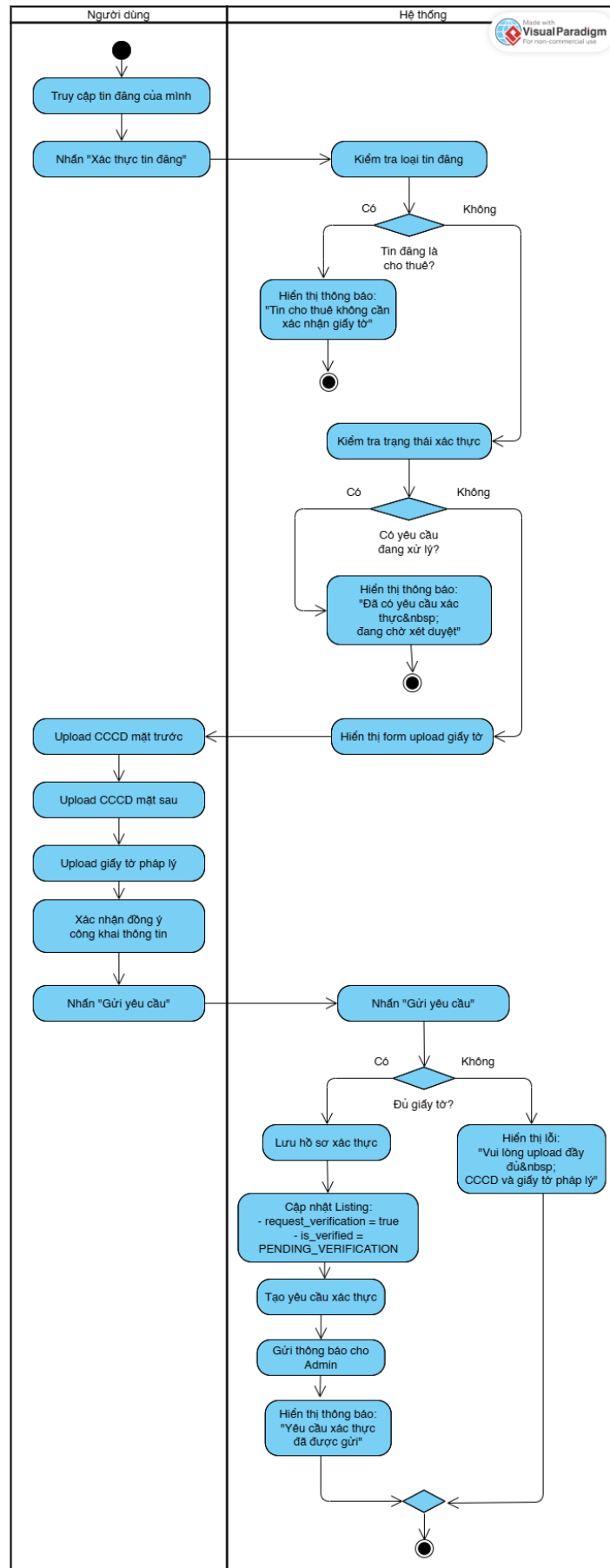
Hình 18: Activity Diagram: Người dùng - Nâng cấp gói tin

2.6.9. Activity Diagram: Người dùng - Thanh toán



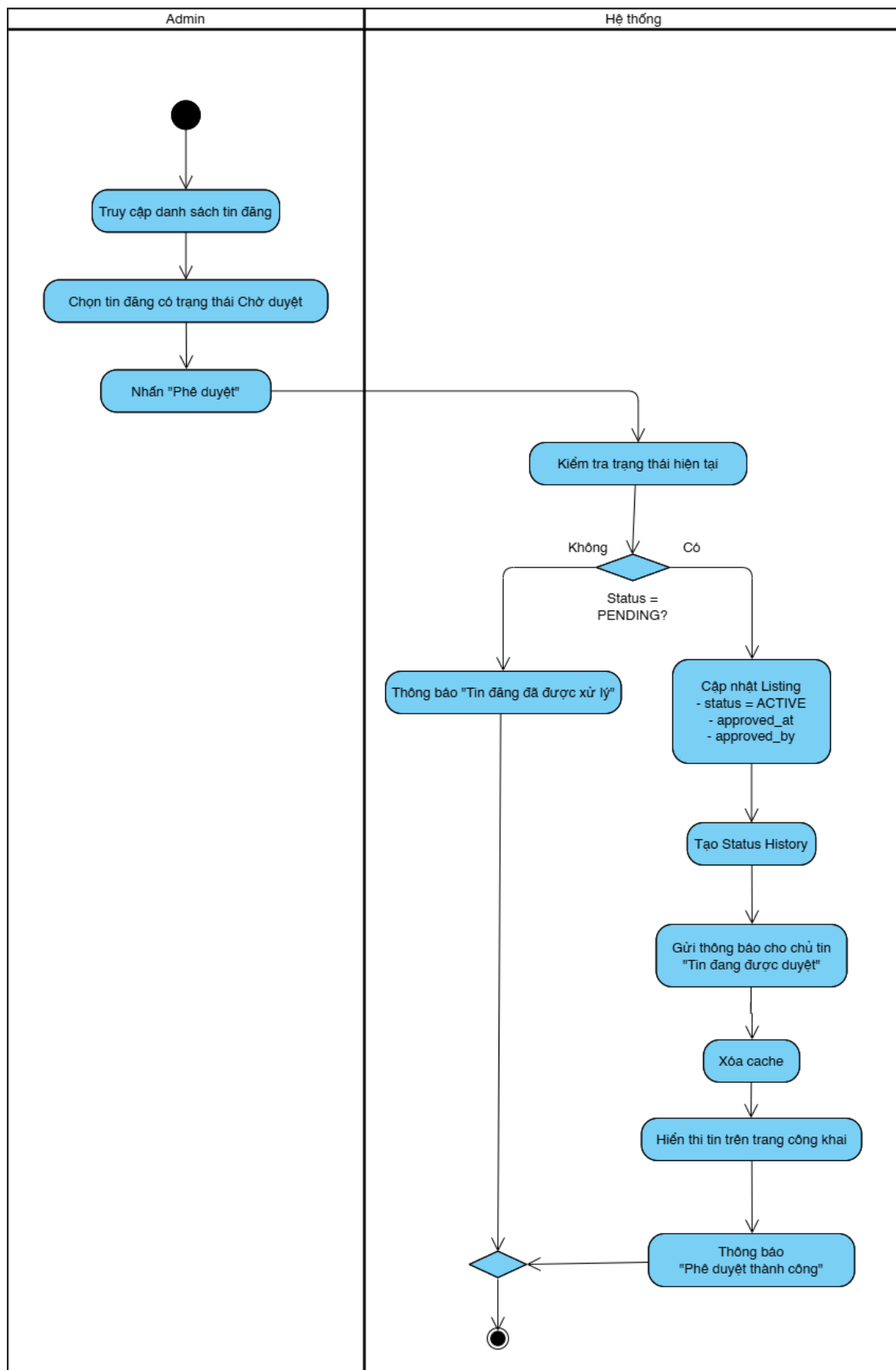
Hình 19: Activity Diagram: Người dùng - Thanh toán

2.6.10. Activity Diagram: Người dùng - Xác thực tin đăng



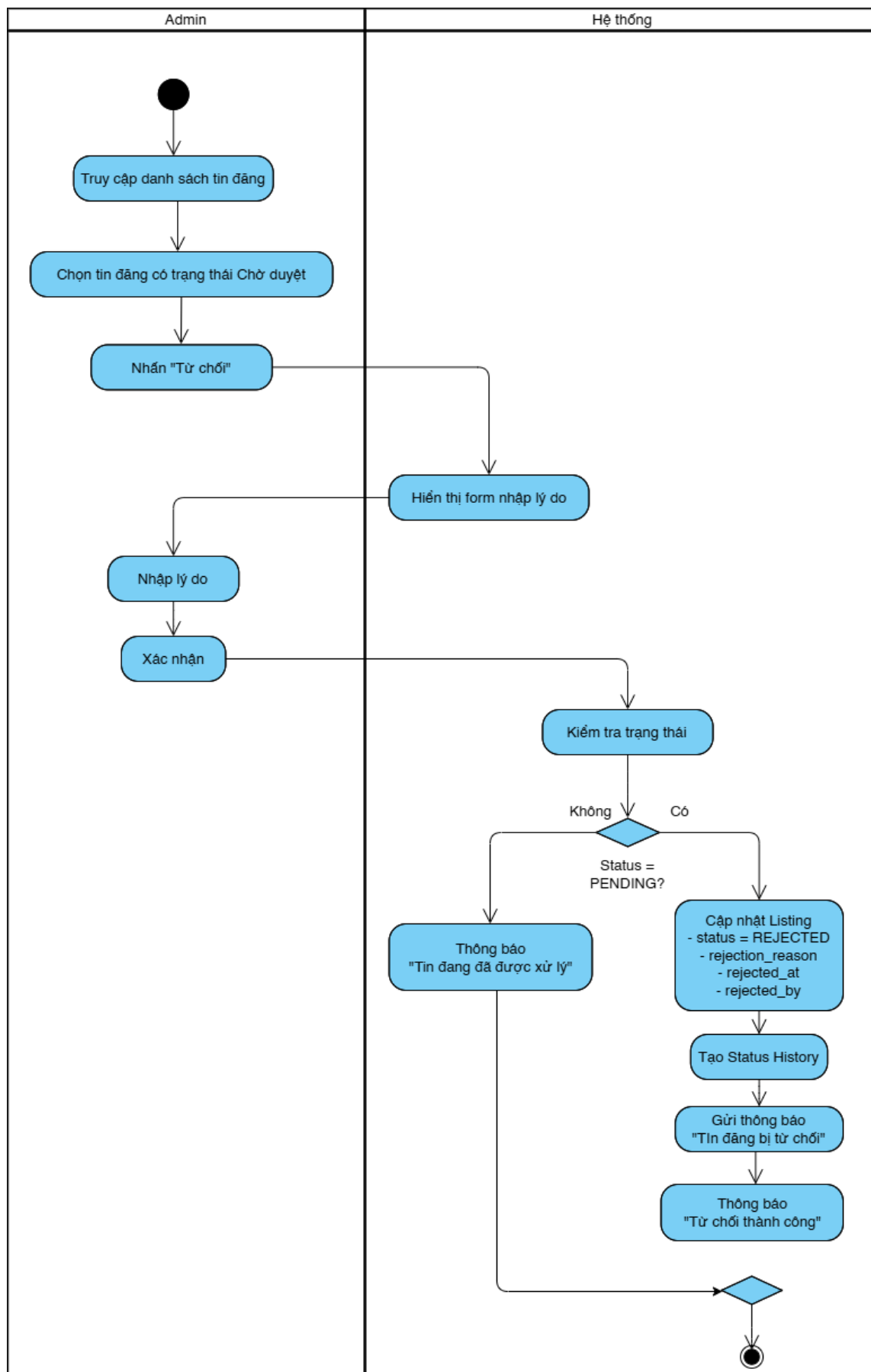
Hình 20: Activity Diagram: Người dùng - Xác thực tin đăng

2.6.11. Activity Diagram: Admin – Duyệt tin đăng



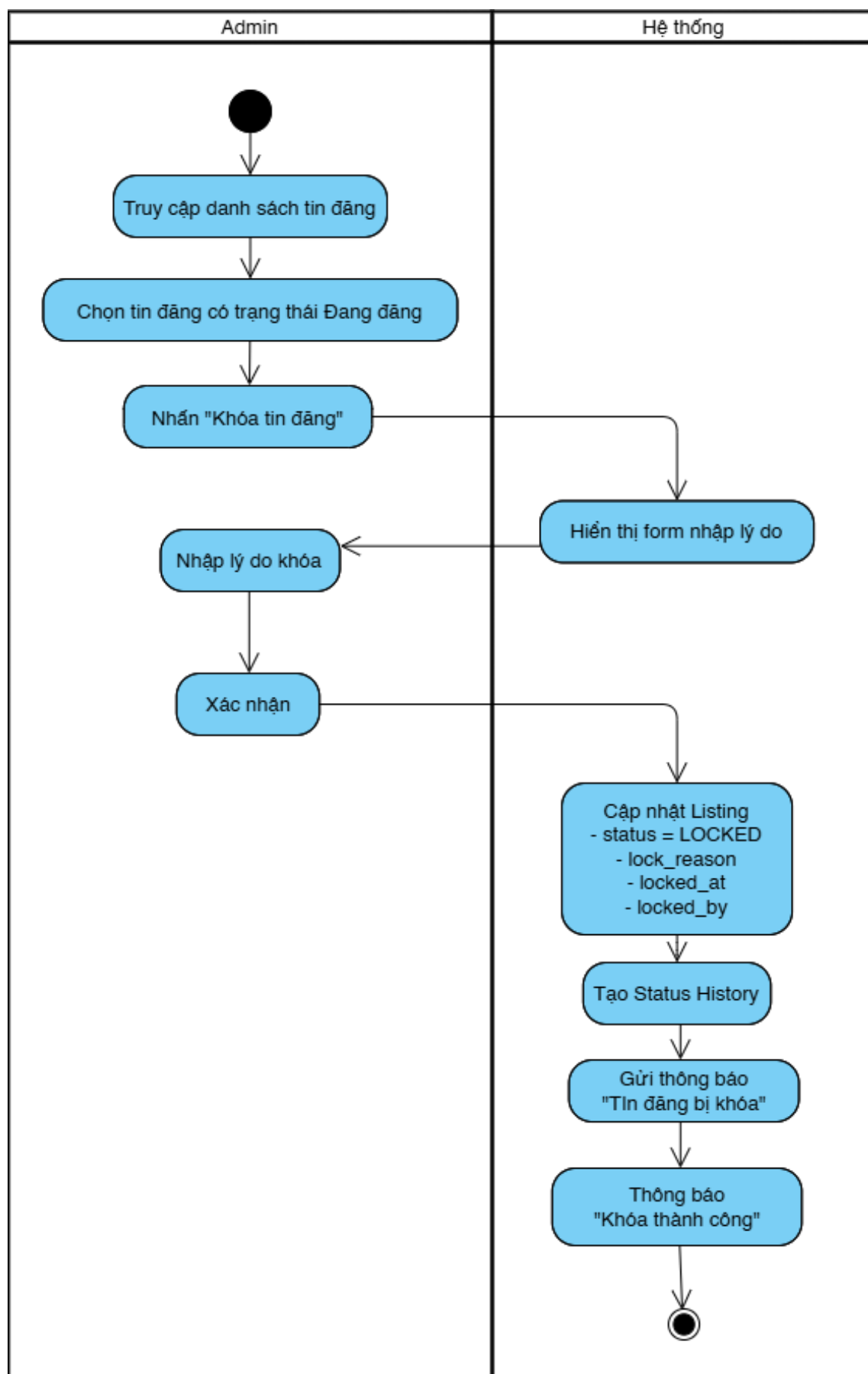
Hình 21: Activity Diagram: Admin – Duyệt tin đăng

2.6.12. Activity Diagram: Admin – Từ chối tin đăng



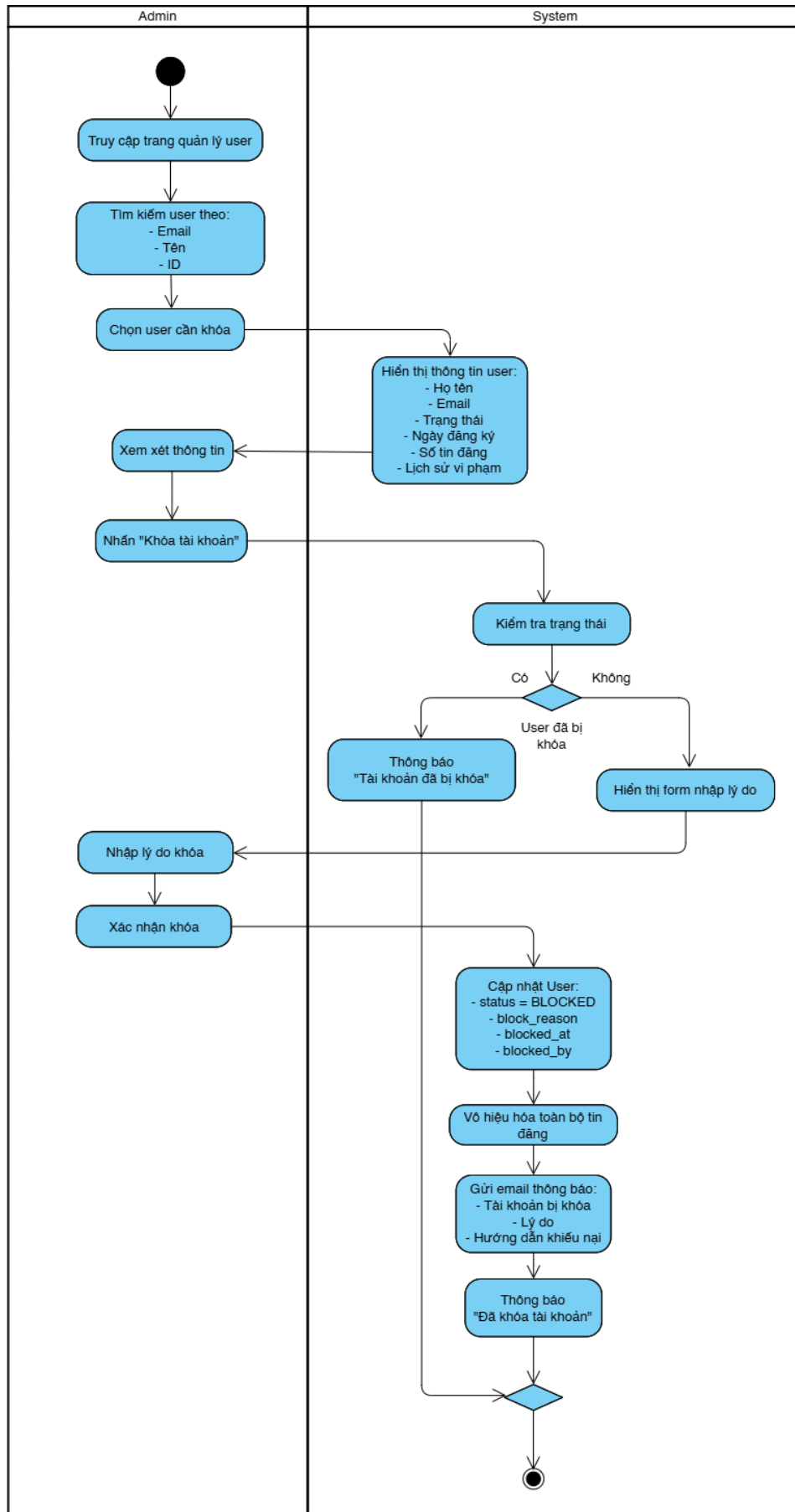
Hình 22: Activity Diagram: Admin – Từ chối tin đăng

2.6.13. Activity Diagram: Admin – Khóa Tin đăng

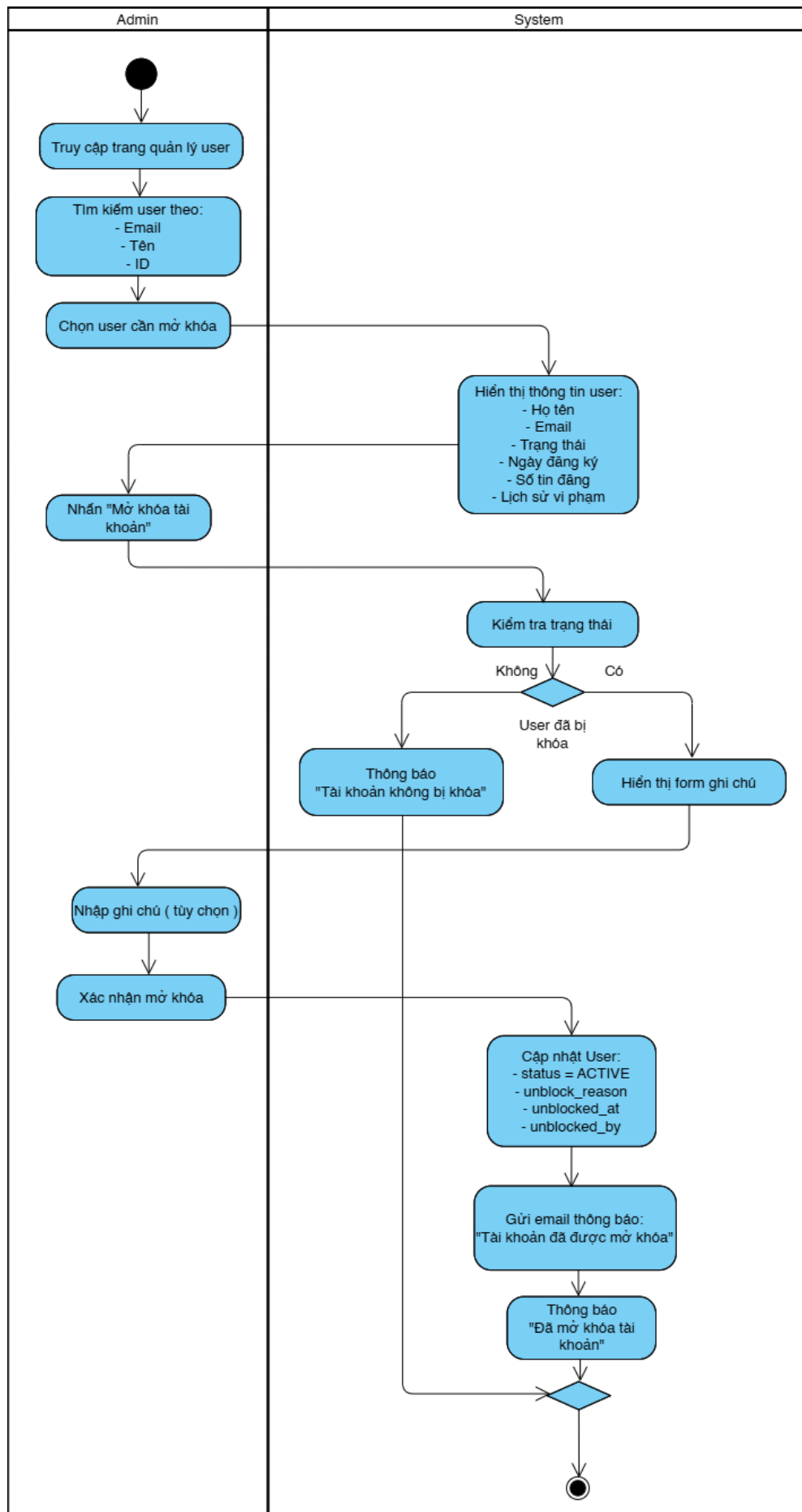


Hình 23: Activity Diagram: Admin – Khóa Tin đăng

2.6.14. Activity Diagram: Admin – Khóa/ Mở khóa tài khoản

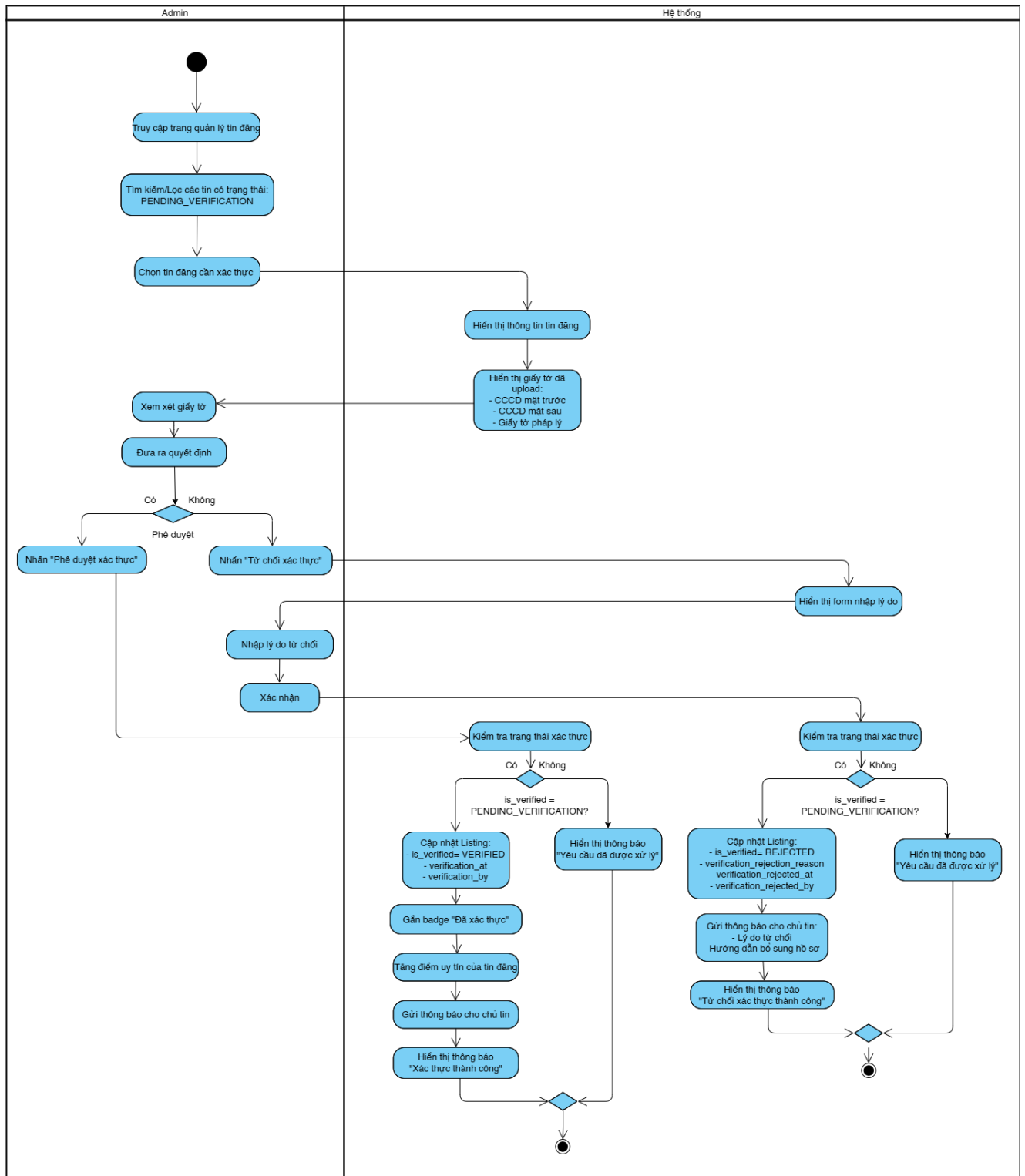


Hình 24: Activity Diagram: Admin – Khóa tài khoản



Hình 25; Activity Diagram: Admin –Mở khóa tài khoản

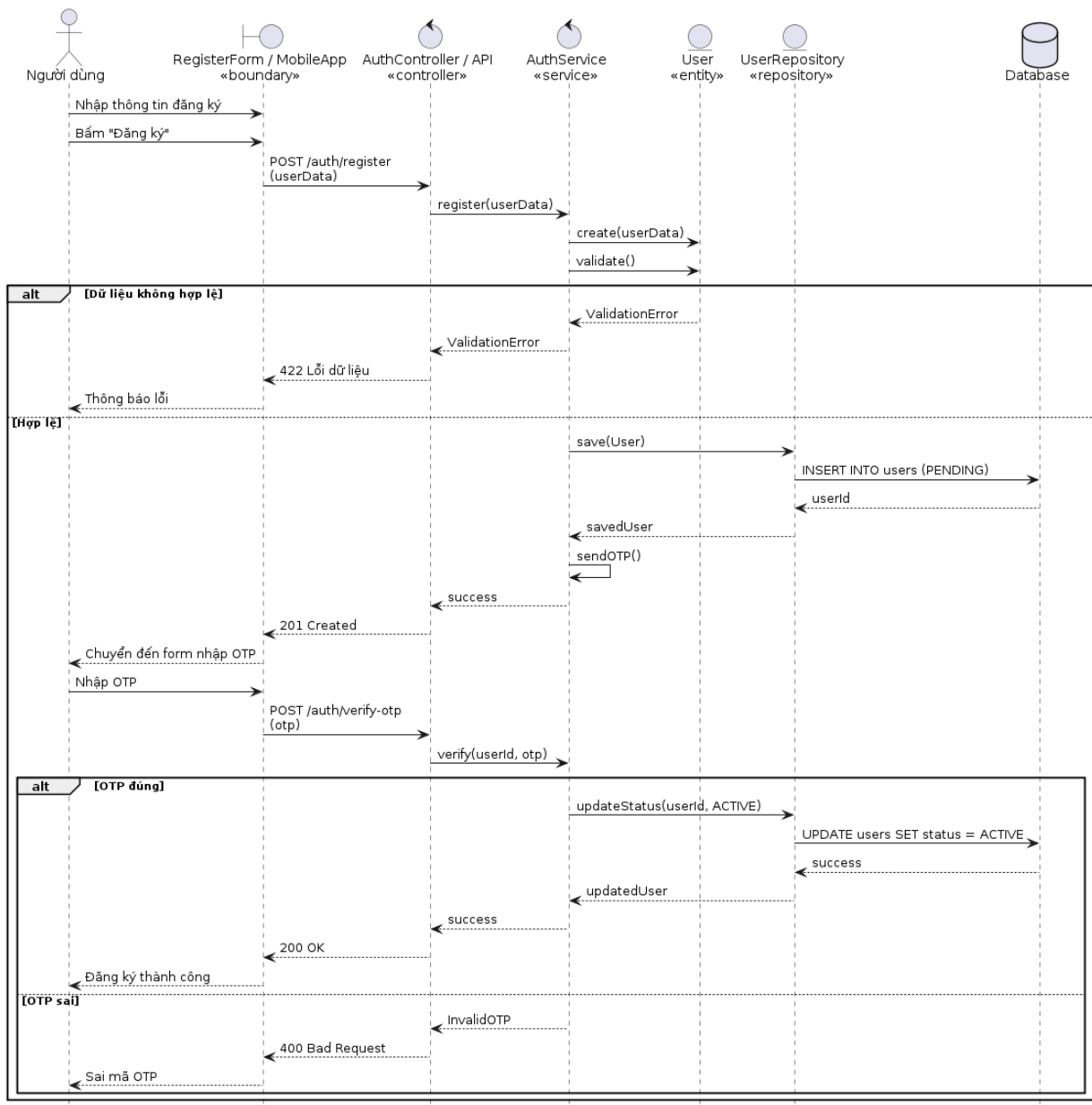
2.6.15. . Acitivity Diagram: Admin - Xác thực tin đăng



Hình 26: Acitivity Diagram: Admin - Xác thực tin đăng

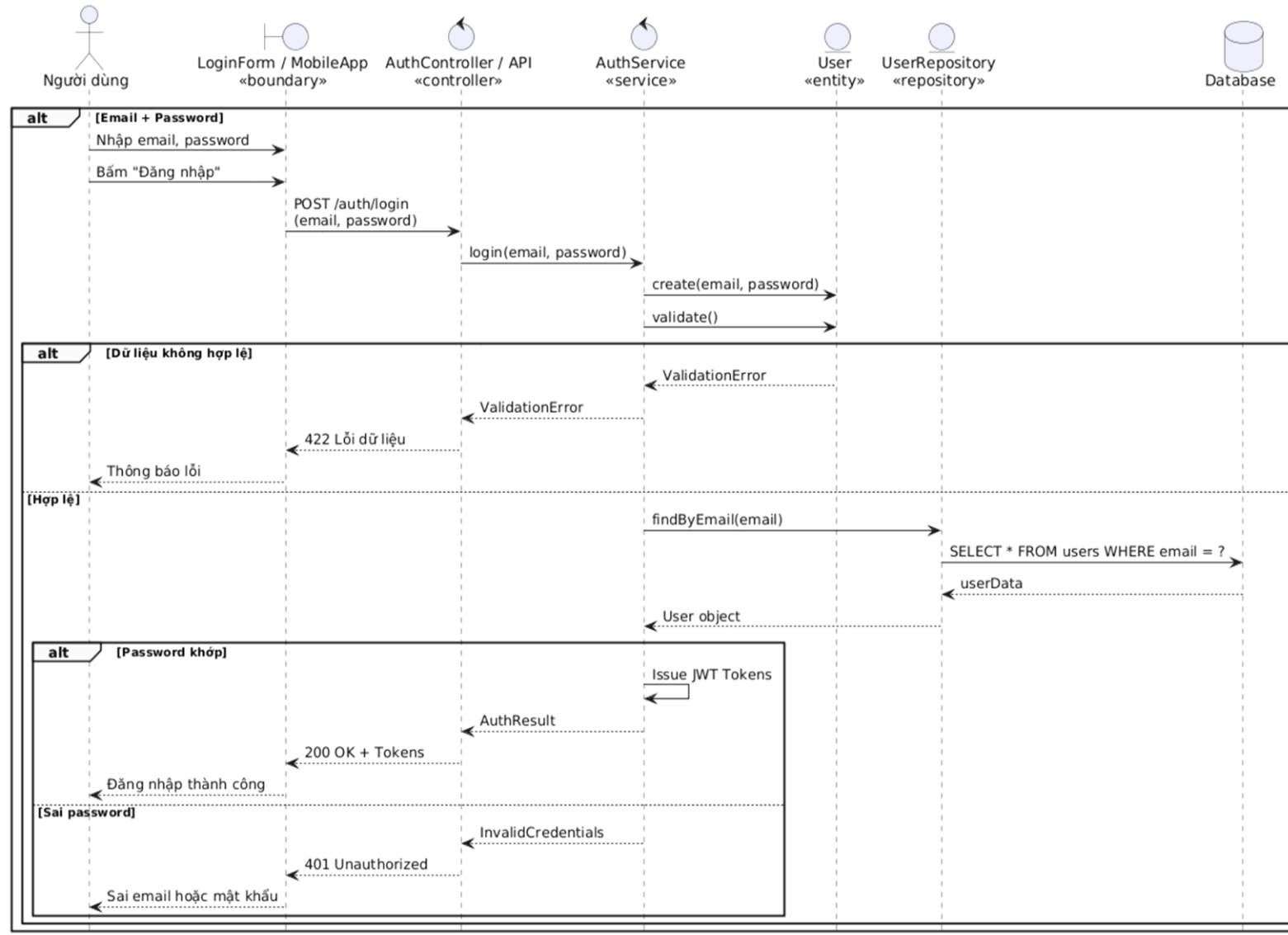
2.7. Biểu đồ tuần tự (Các chức năng chính)

2.7.1. Sequence Diagram: Người dùng - Đăng ký tài khoản



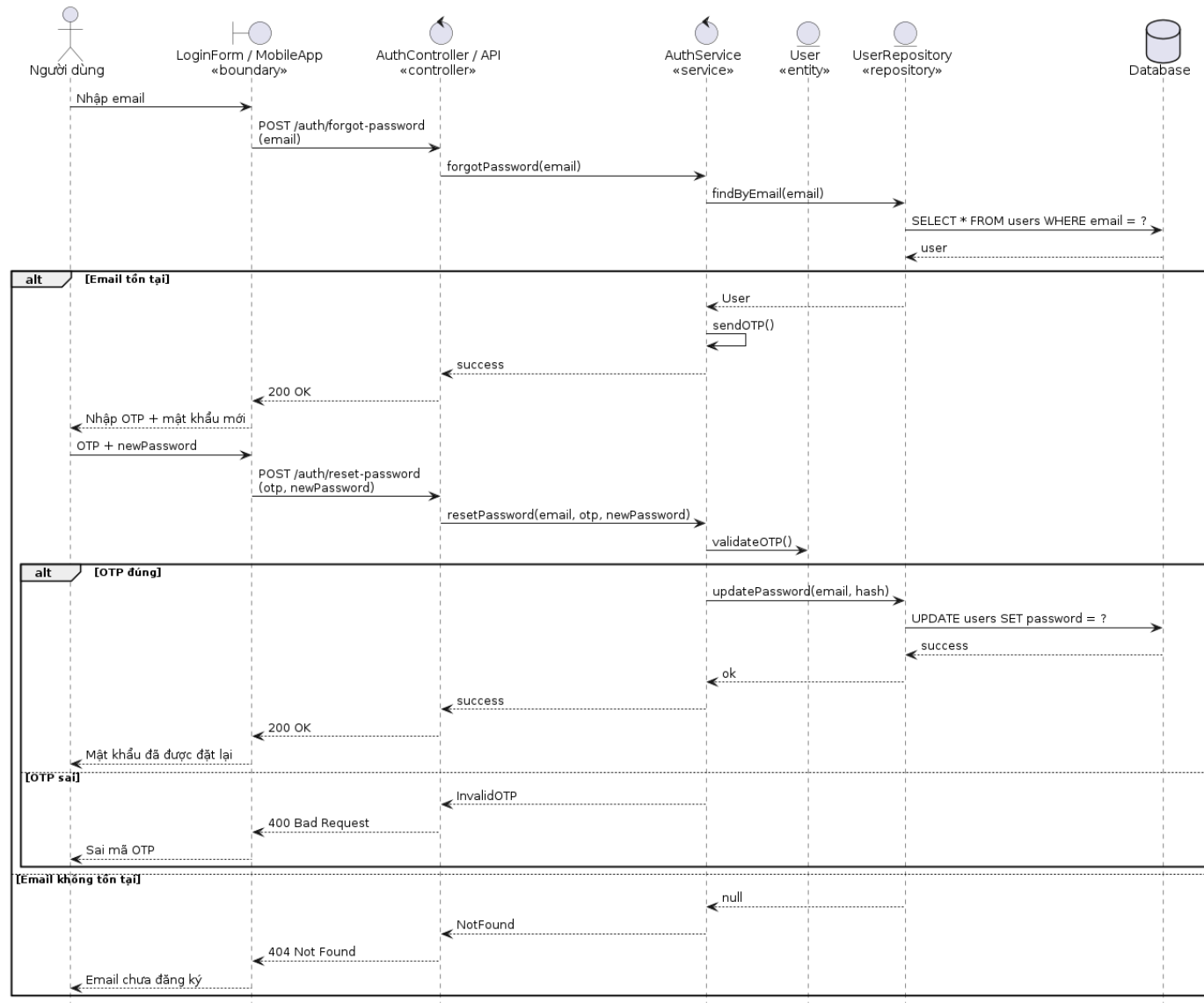
Hình 27: Sequence Diagram: Người dùng - Đăng ký tài khoản

2.7.2. Sequence Diagram: Người dùng - Đăng nhập



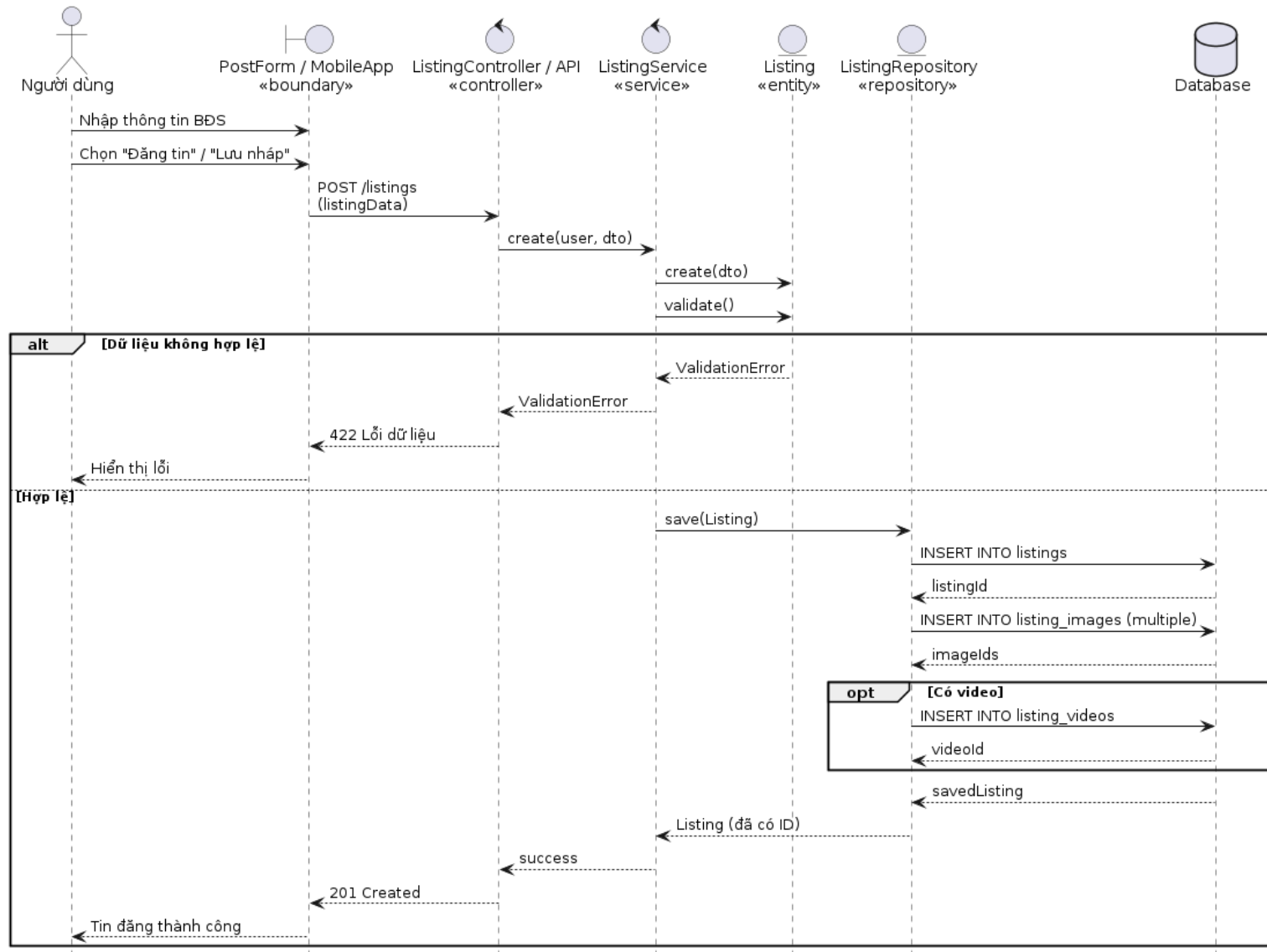
Hình 28: Sequence Diagram: Người dùng - Đăng nhập

2.7.3. Sequence Diagram: Người dùng - Quên mật khẩu



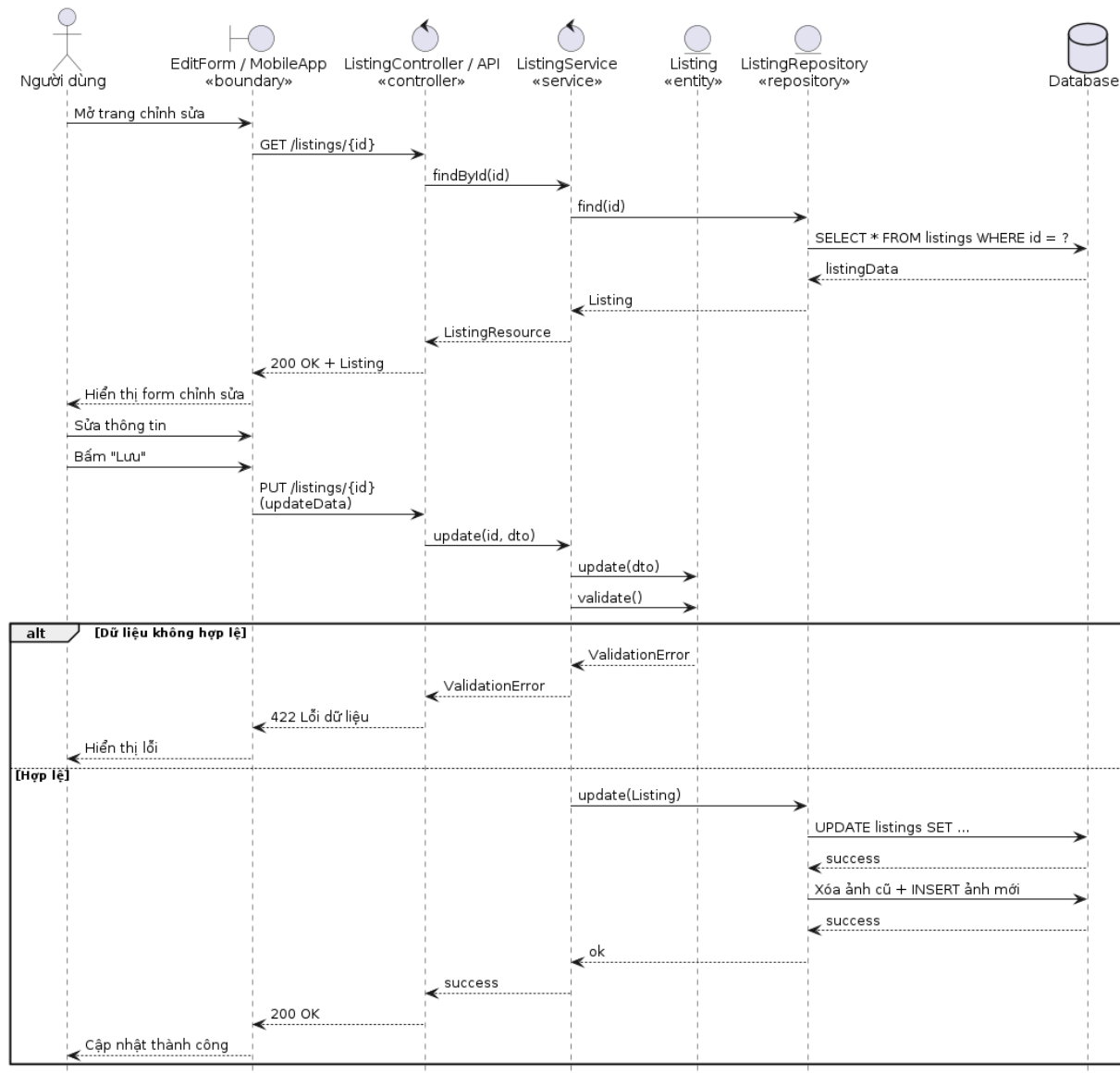
Hình 29: Sequence Diagram: Người dùng - Quên mật khẩu

2.7.4. Sequence Diagram: Người dùng - Tạo tin đăng



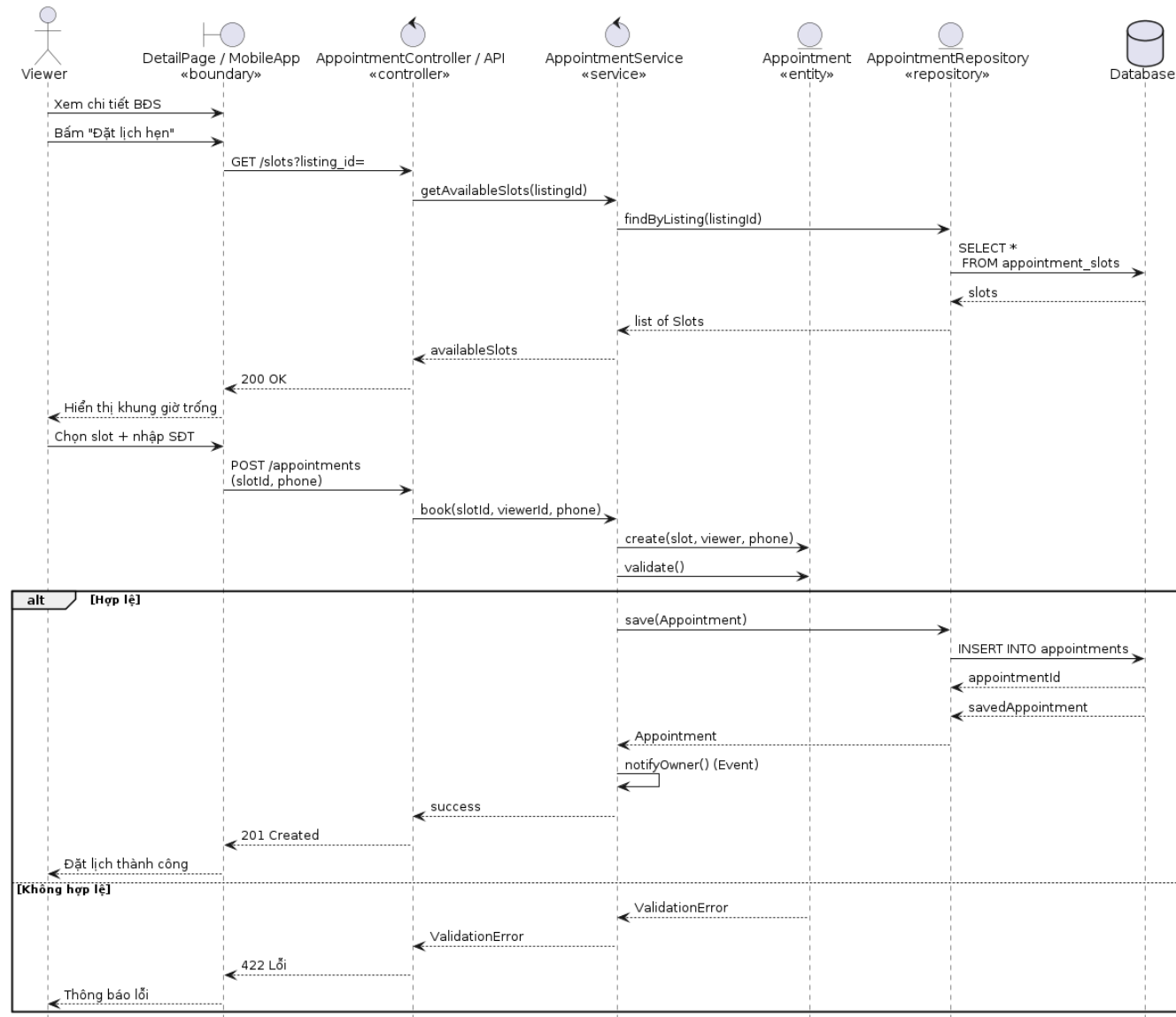
Hình 30: Sequence Diagram: Người dùng - Tạo tin đăng

2.7.5. Sequence Diagram: Người dùng - Chỉnh sửa tin đăng



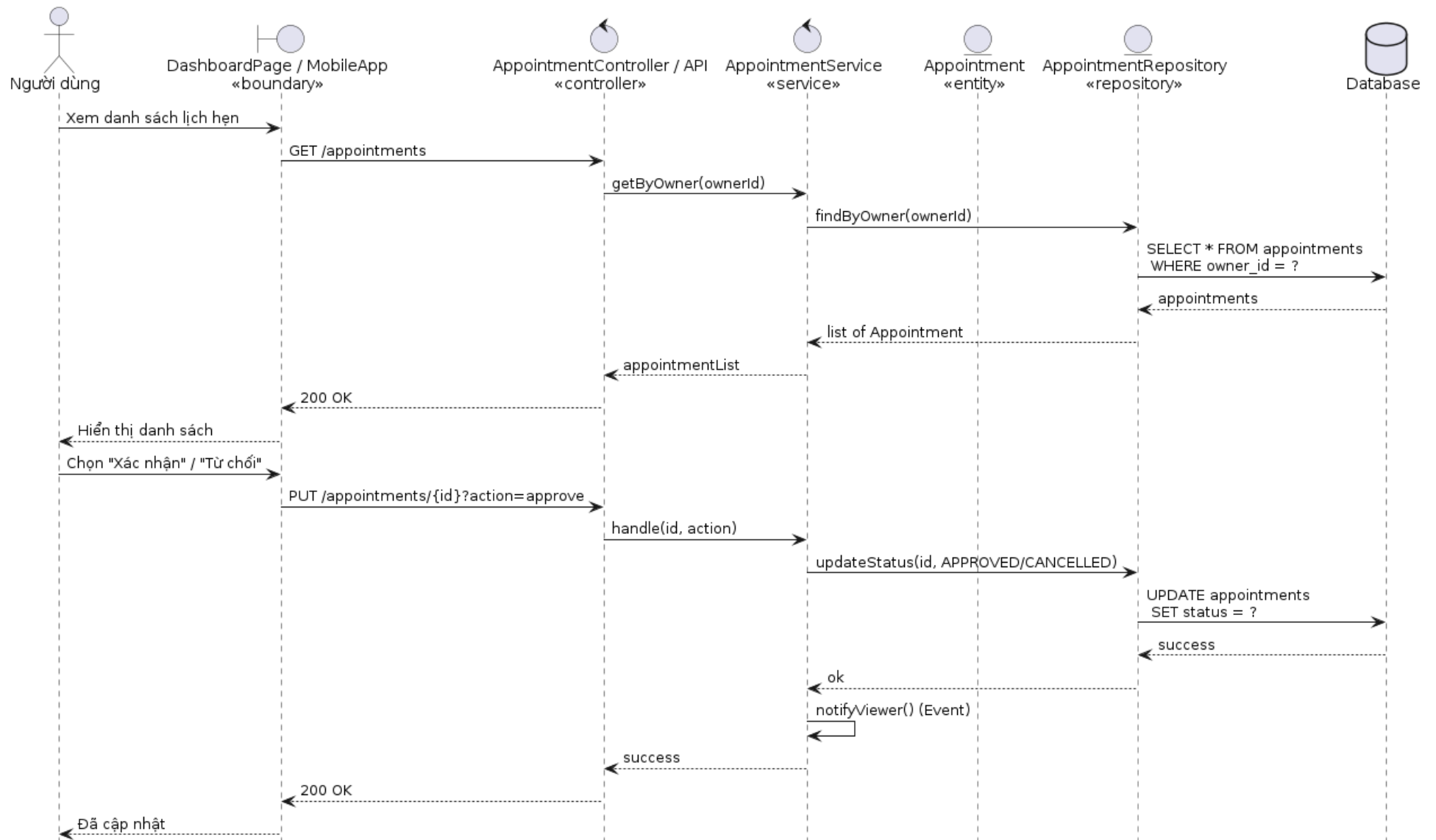
Hình 31: Sequence Diagram: Người dùng - Chỉnh sửa tin đăng

2.7.6. Sequence Diagram: Người dùng - Đặt lịch hẹn



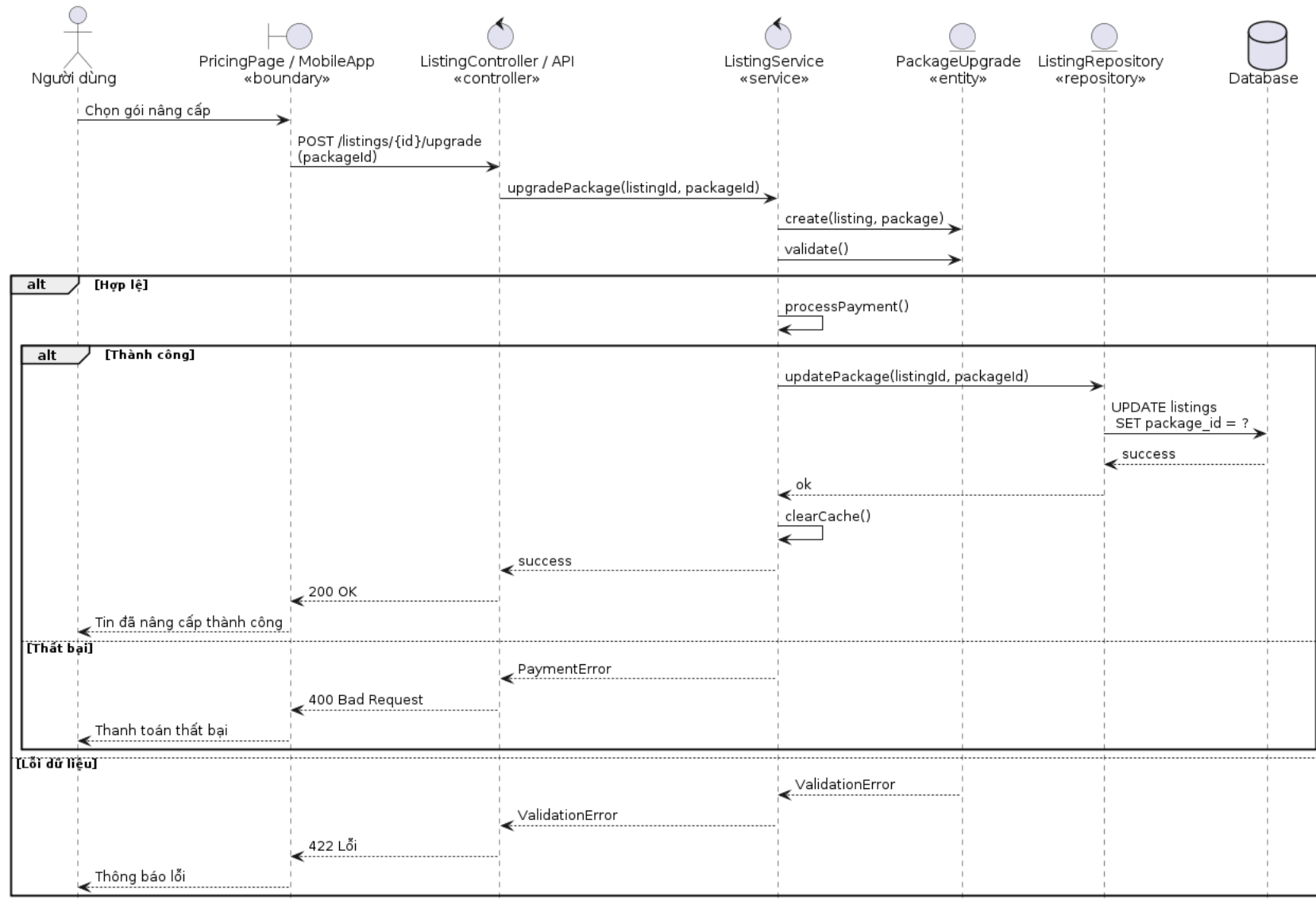
Hình 32: Sequence Diagram: Người dùng - Đặt lịch hẹn

2.7.7. Sequence Diagram: Người dùng - Xác nhận/Từ chối lịch hẹn



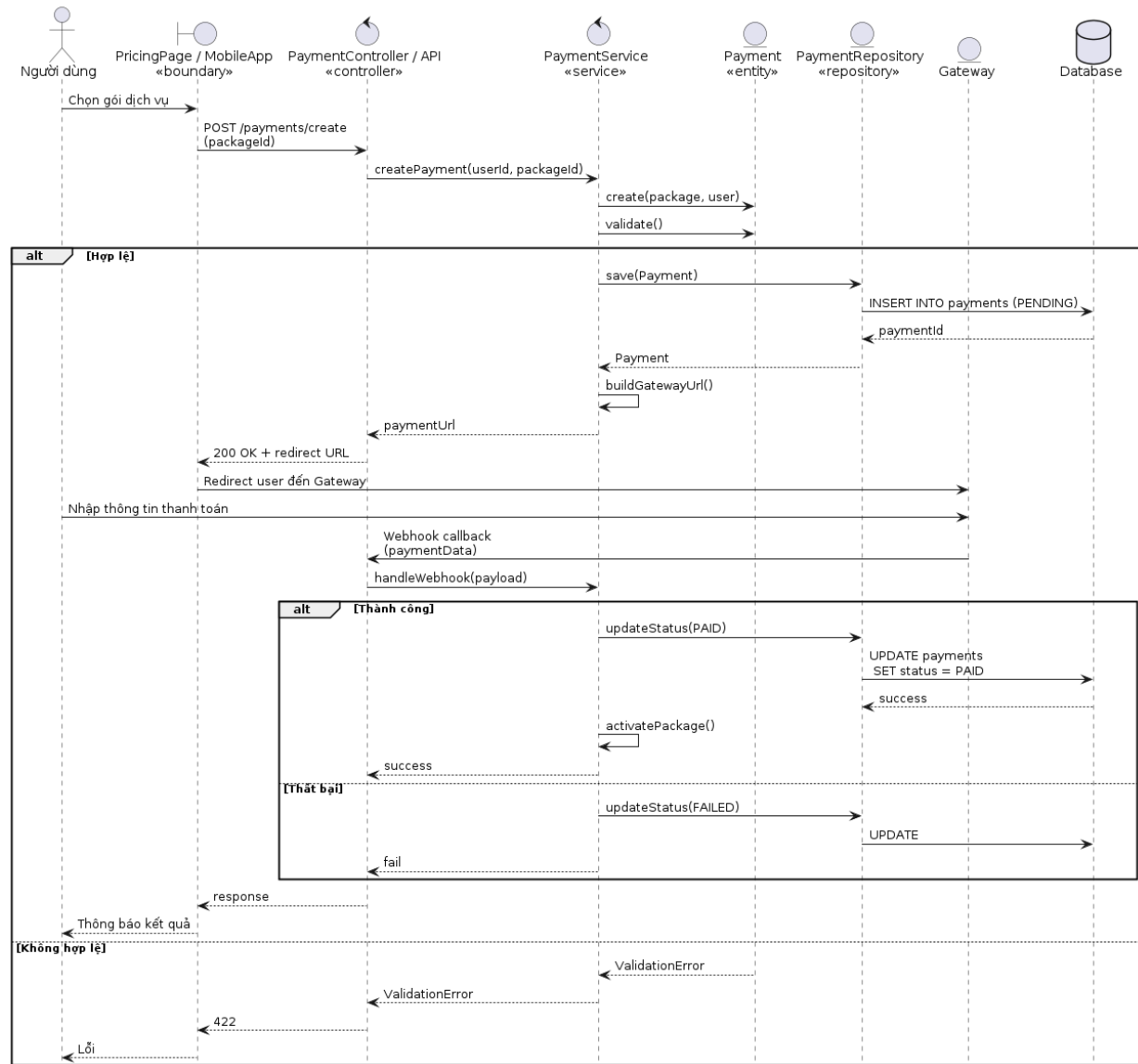
Hình 33: Sequence Diagram: Người dùng - Xác nhận/Từ chối lịch hẹn

2.7.8. Sequence Diagram: Người dùng - Nâng cấp gói tin



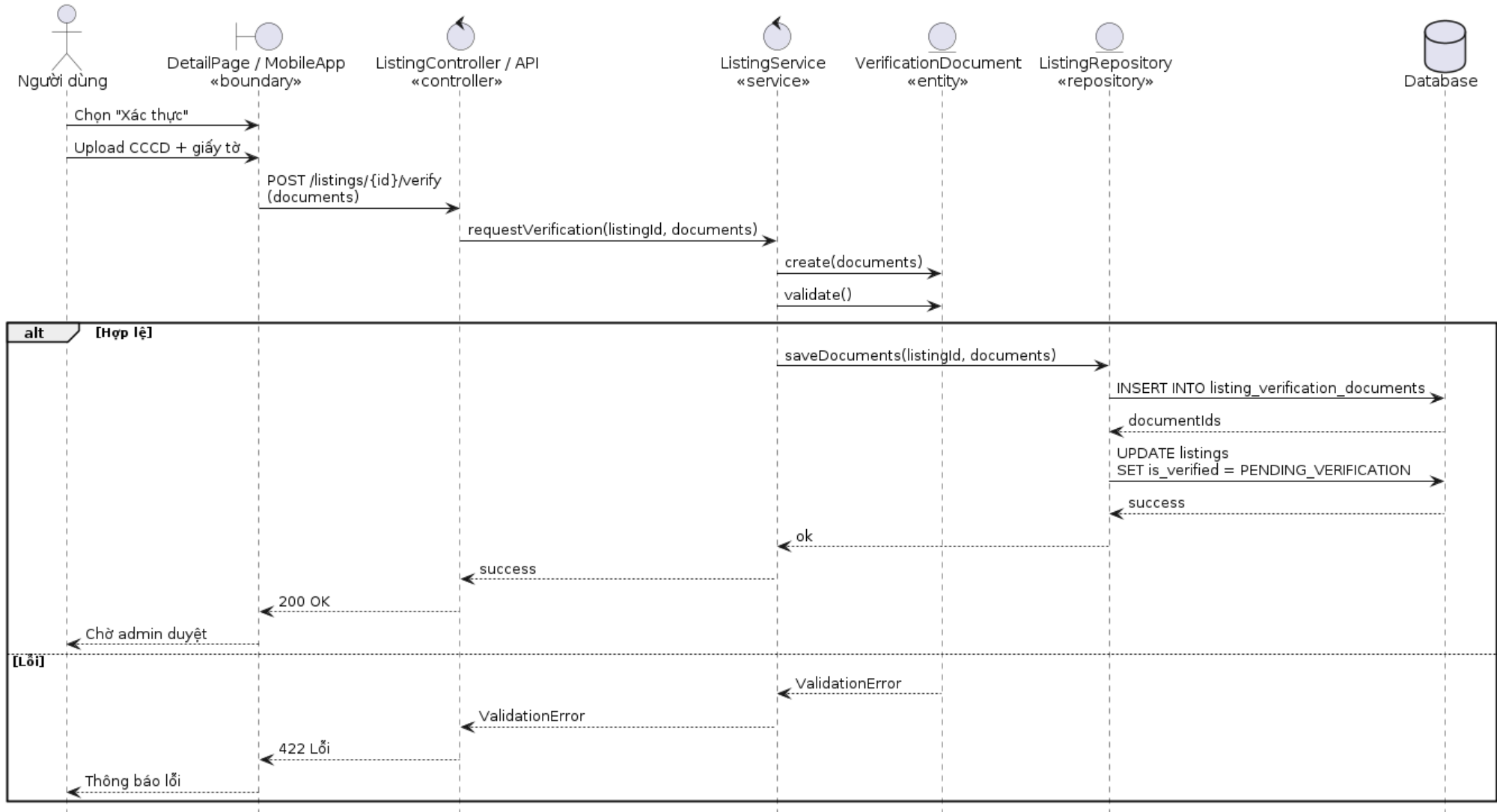
Hình 34: Sequence Diagram: Người dùng - Nâng cấp gói tin

2.7.9. Sequence Diagram: Người dùng - Thanh toán



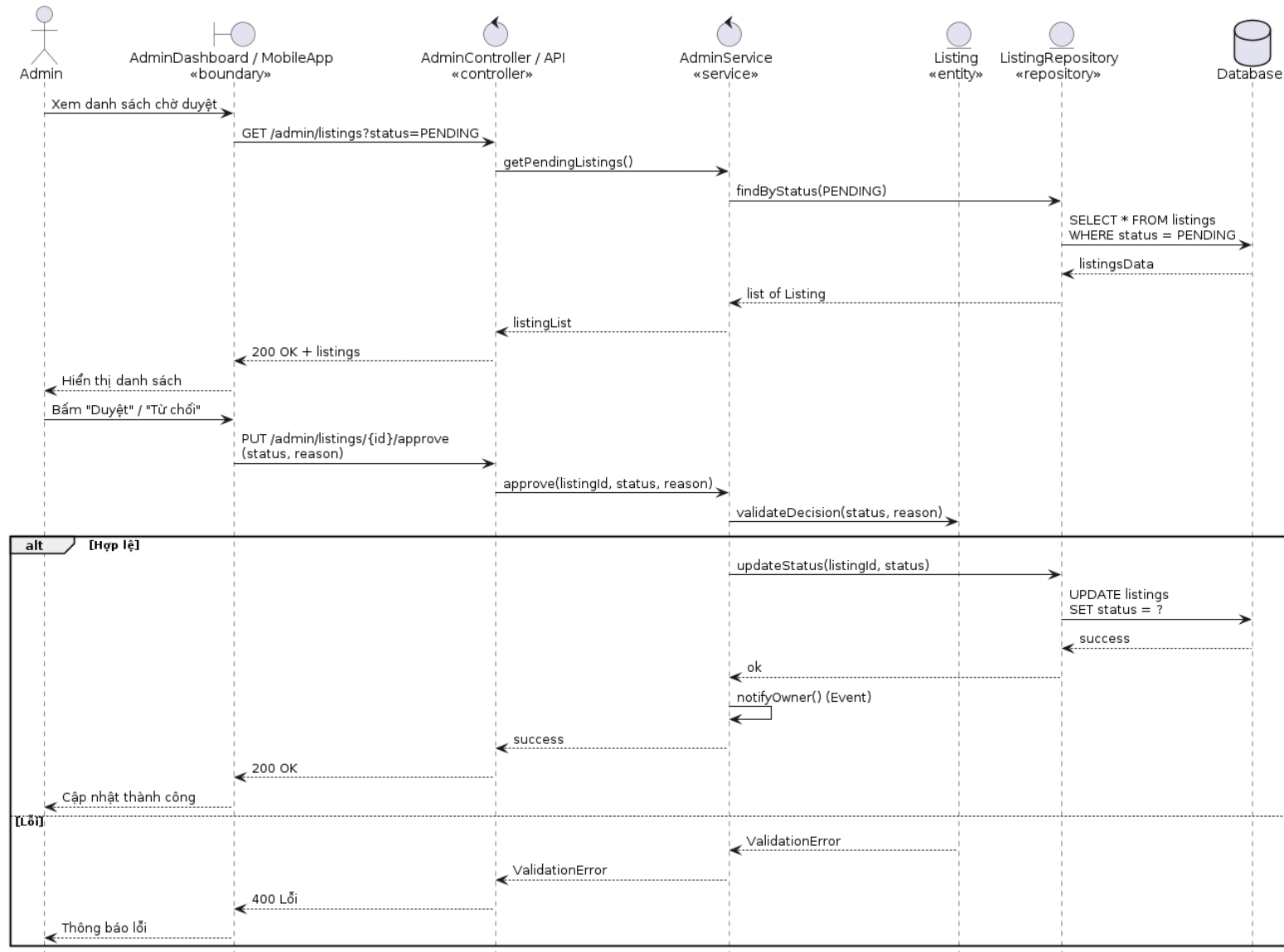
Hình 35: Sequence Diagram: Người dùng - Thanh toán

2.7.10. Sequence Diagram: Người dùng - Xác thực tin đăng



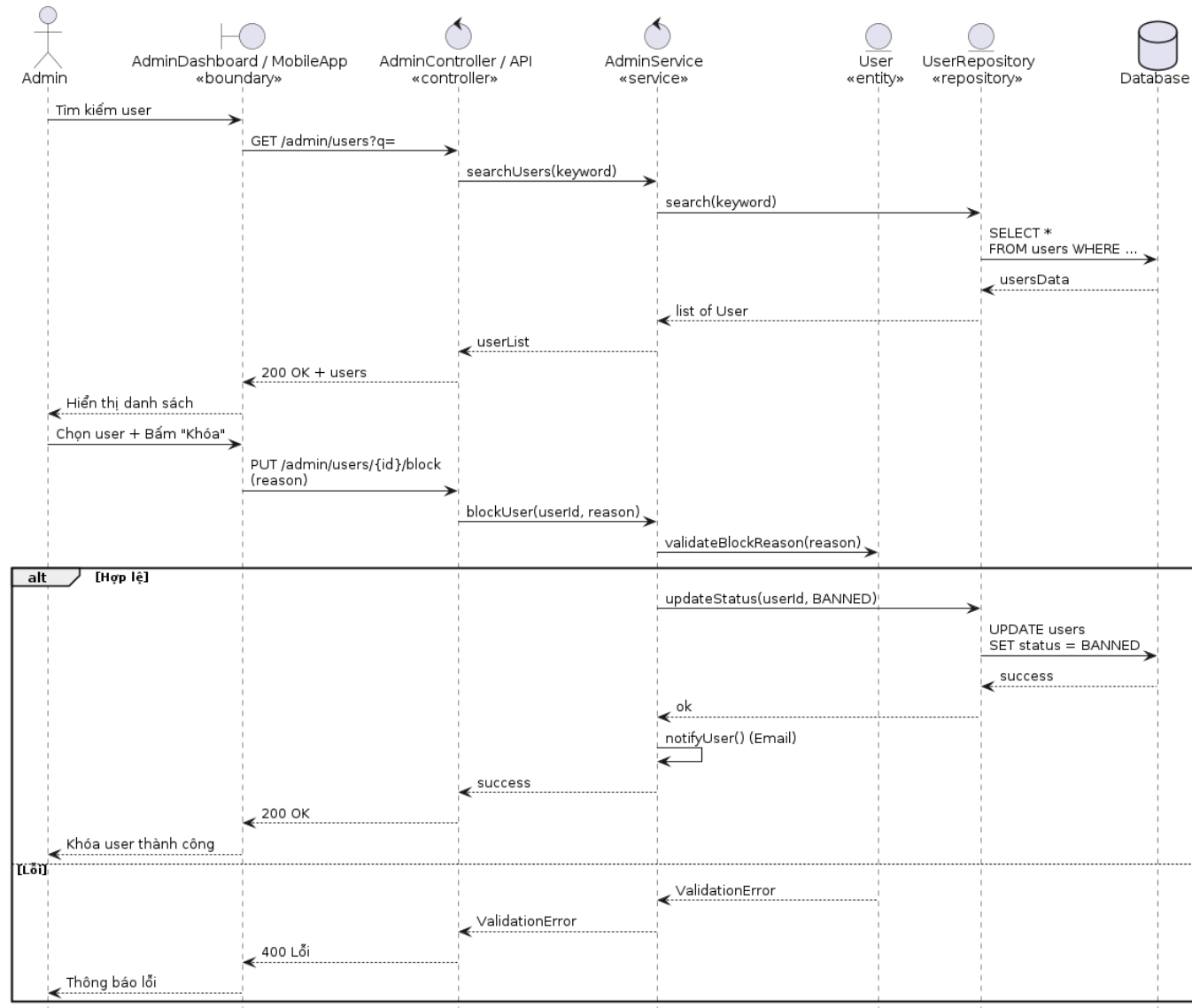
Hình 36: Sequence Diagram: Người dùng - Xác thực tin đăng

2.7.11. Sequence Diagram: Admin – Duyệt/ Từ chối tin đăng



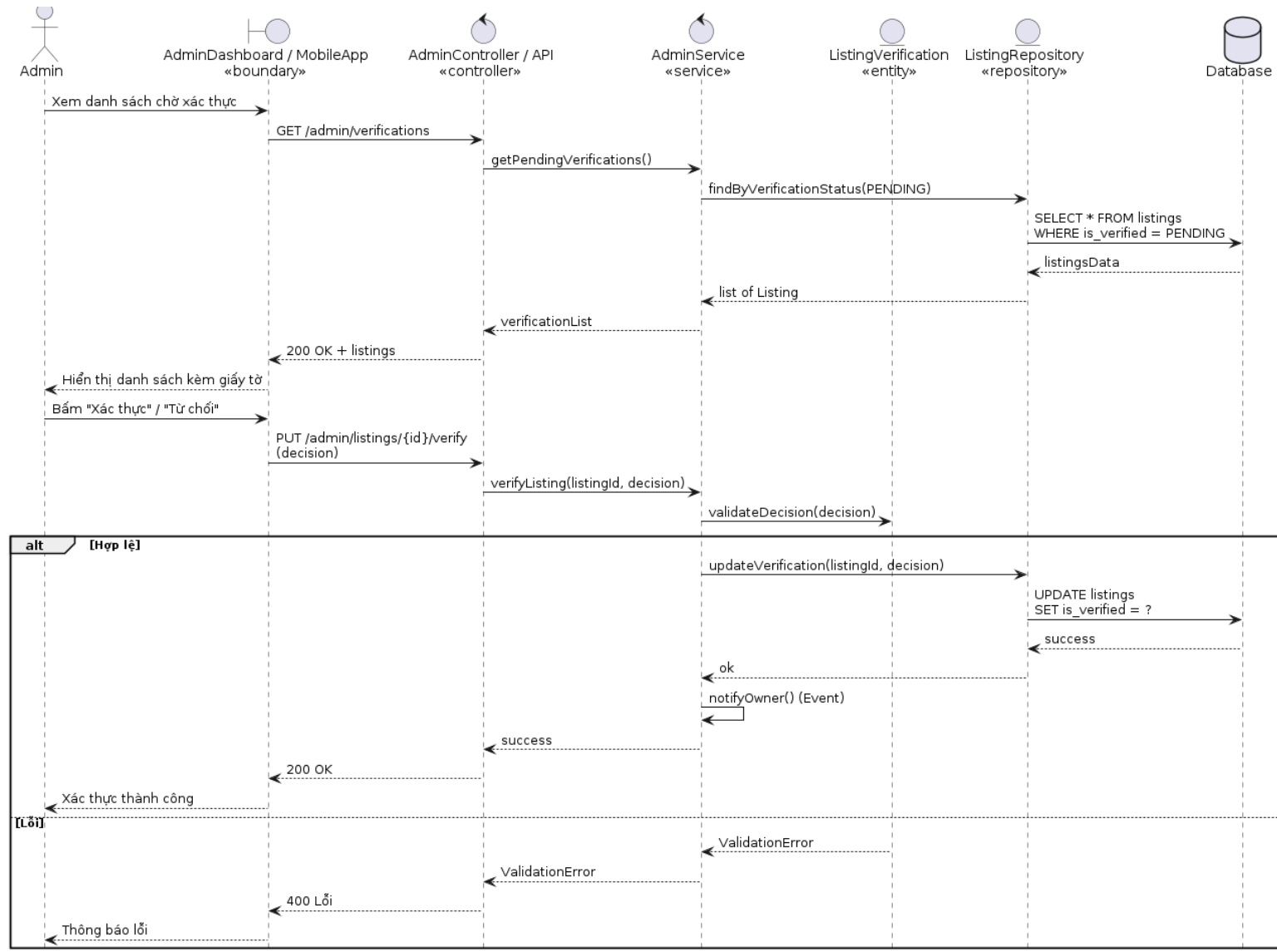
Hình 37: Sequence Diagram: Admin – Duyệt/ Từ chối tin đăng

2.7.12. Sequence Diagram: Admin – Khóa tài khoản



Hình 38: Sequence Diagram: Admin – Khóa tài khoản

2.7.13. Sequence Diagram: Admin - Xác thực tin đăng

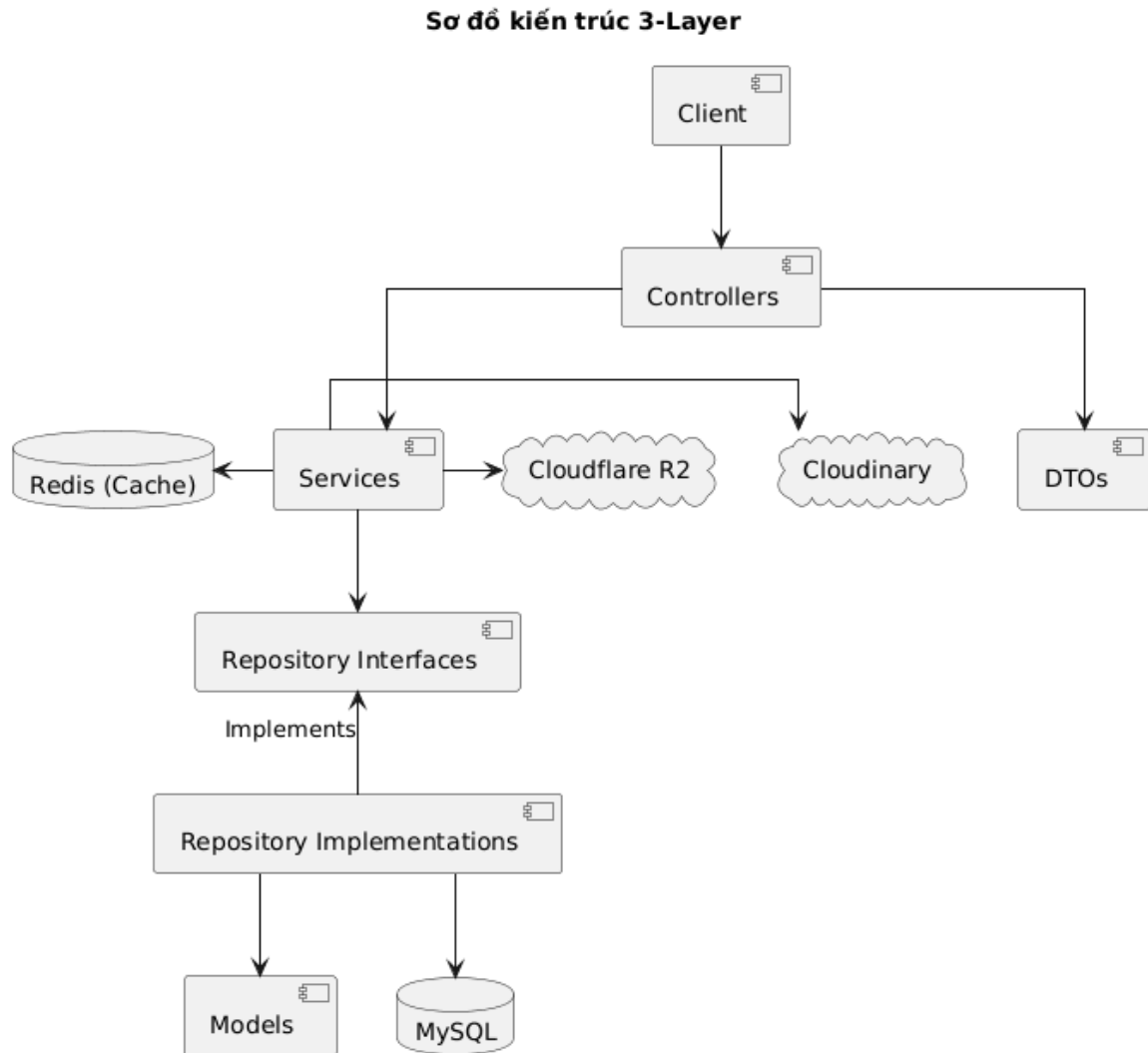


Hình 39: Sequence Diagram: Admin - Xác thực tin đăng

CHƯƠNG 3. THIẾT KẾ HỆ THỐNG

3.1. Sơ đồ kiến trúc tổng thể hệ thống

Hệ thống được xây dựng theo mô hình với kiến trúc phân lớp rõ ràng:



Hình 40: Sơ đồ kiến trúc tổng thể hệ thống

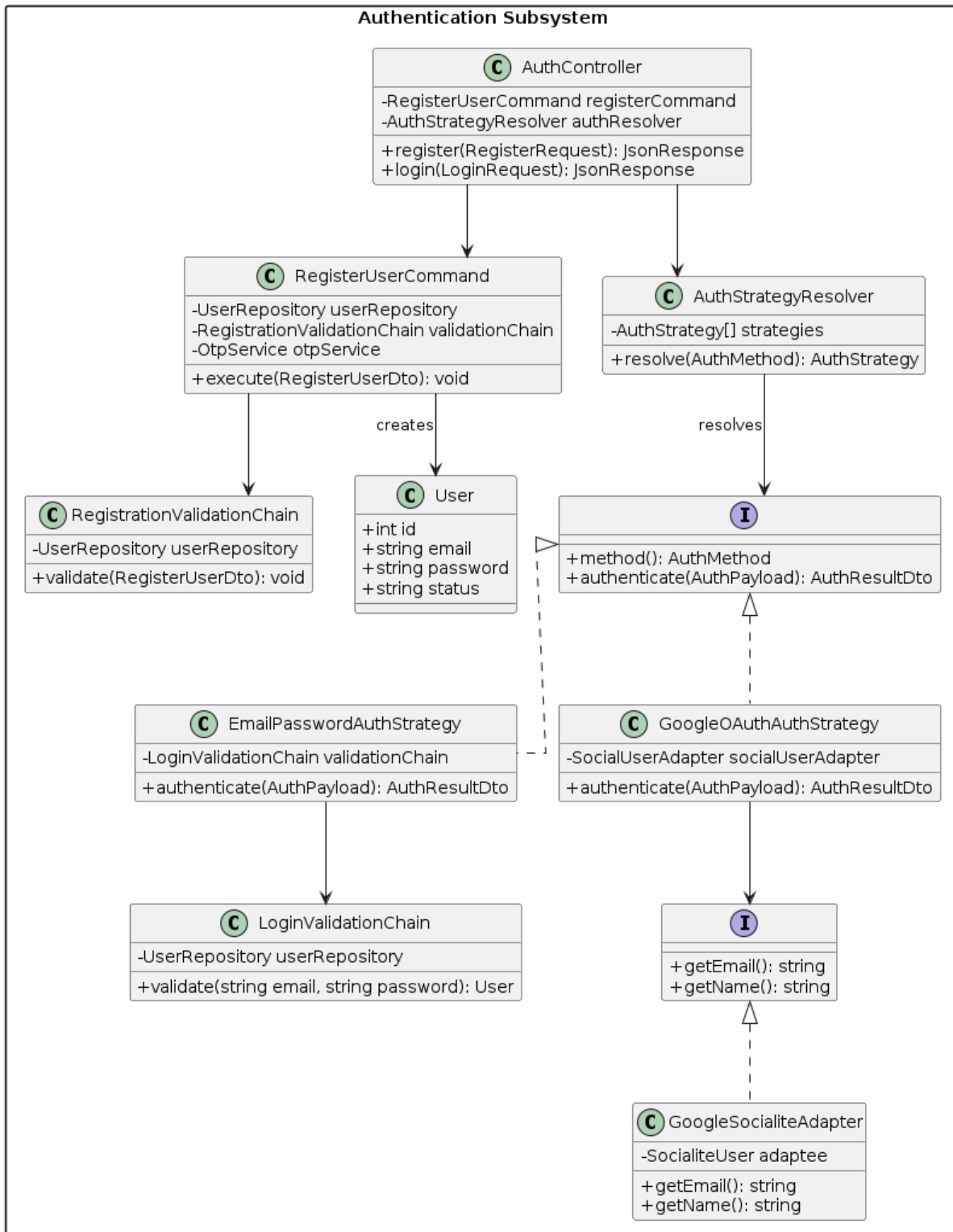
Lớp	Thành phần	Công nghệ	Mô tả
Presentation	PropifyFrontend	Vue.js	Giao diện người dùng – Tìm kiếm, đăng tin, quản lý lịch hẹn
Presentation	PropifyAdmin	Vue.js	Giao diện quản trị – Kiểm duyệt, báo cáo, cấu hình hệ thống

Presentation	Controllers	Laravel	Nhận request, validate qua FormRequest, trả response qua DTO
Application	Business Logic	Laravel	Xử lý nghiệp vụ: tính quality score, quản lý OTP, lịch hẹn...
Infrastructure	Data Access	Eloquent Repository	Truy vấn và cập nhật cơ sở dữ liệu qua Eloquent ORM
External System	Cache	Redis	Cache dữ liệu thường xuyên, quản lý session OTP, rate limit
External System	Storage	Cloudinary, CloudFlare R2	Lưu trữ và phân phối hình ảnh/video tin đăng
External System	Realtime	WebSocket/Socket.IO	Chat thời gian thực giữa người dùng
External System	Primary DB	MySQL	Lưu trữ toàn bộ dữ liệu nghiệp vụ

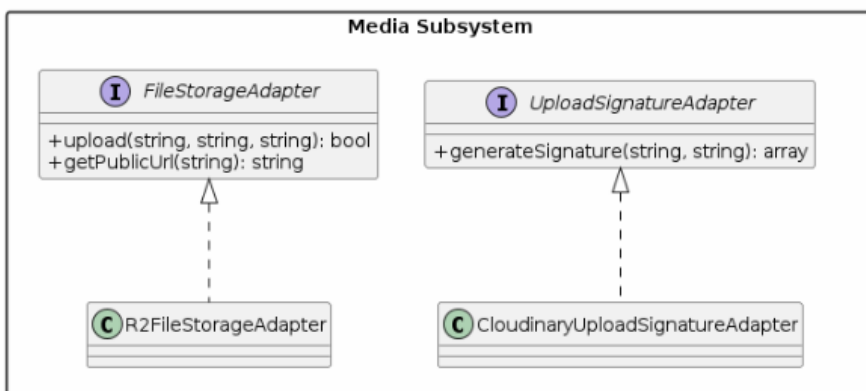
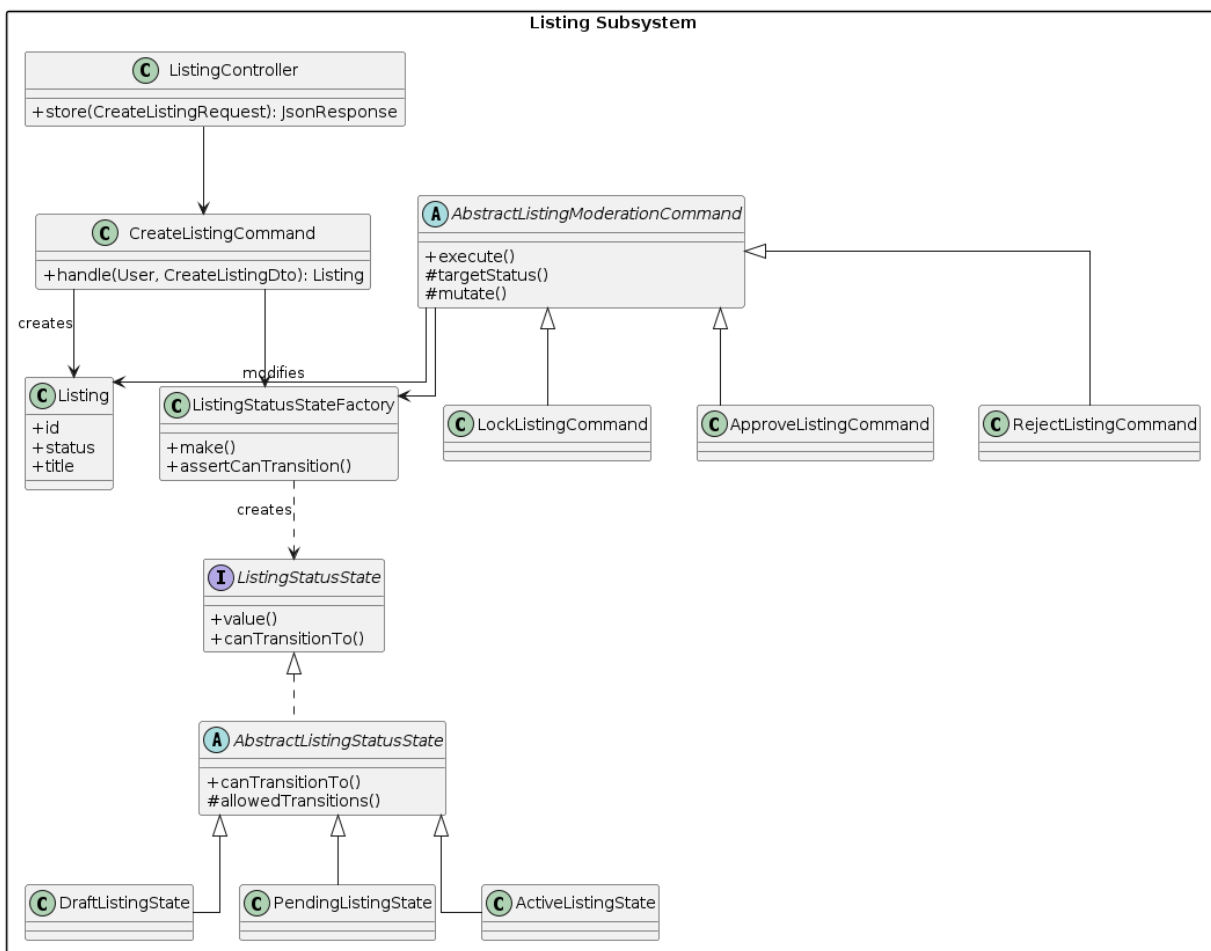
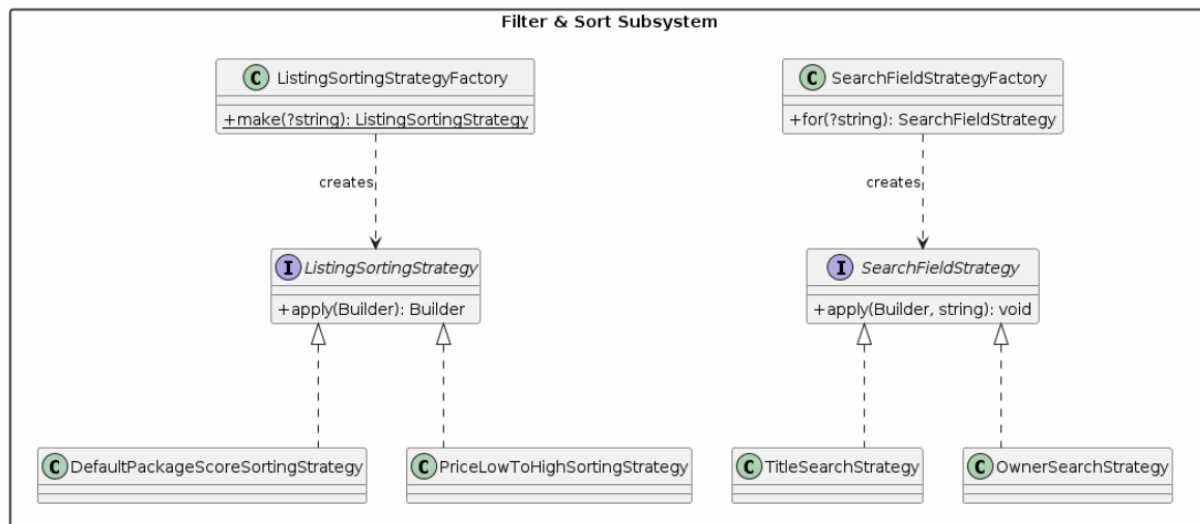
Luồng xử lý request: Client → HTTP/WebSocket → API Layer (Controllers → FormRequests → DTOs) → Service Layer → Repository Layer → Database/Infrastructure.

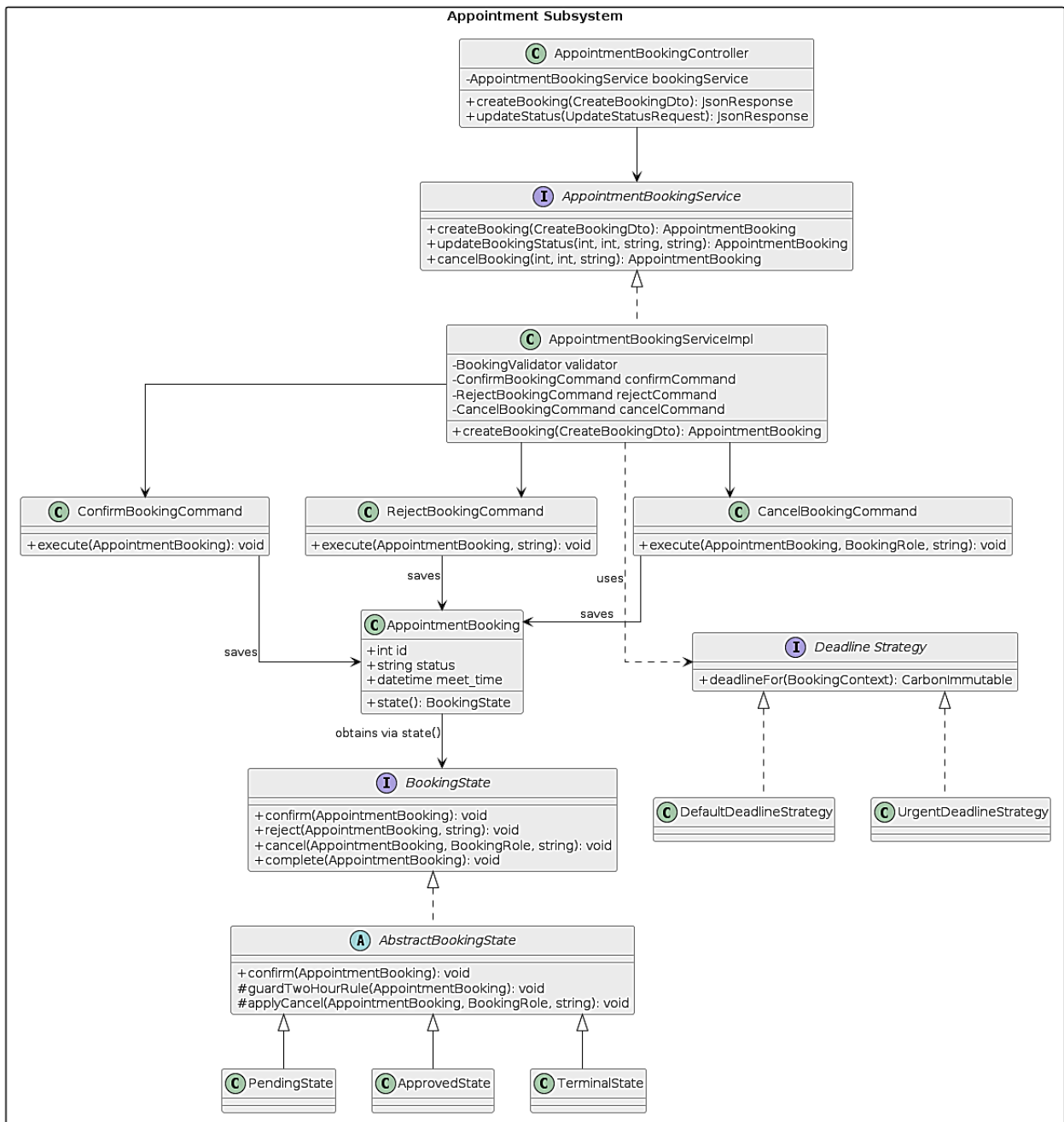
3.2. Thiết kế lớp

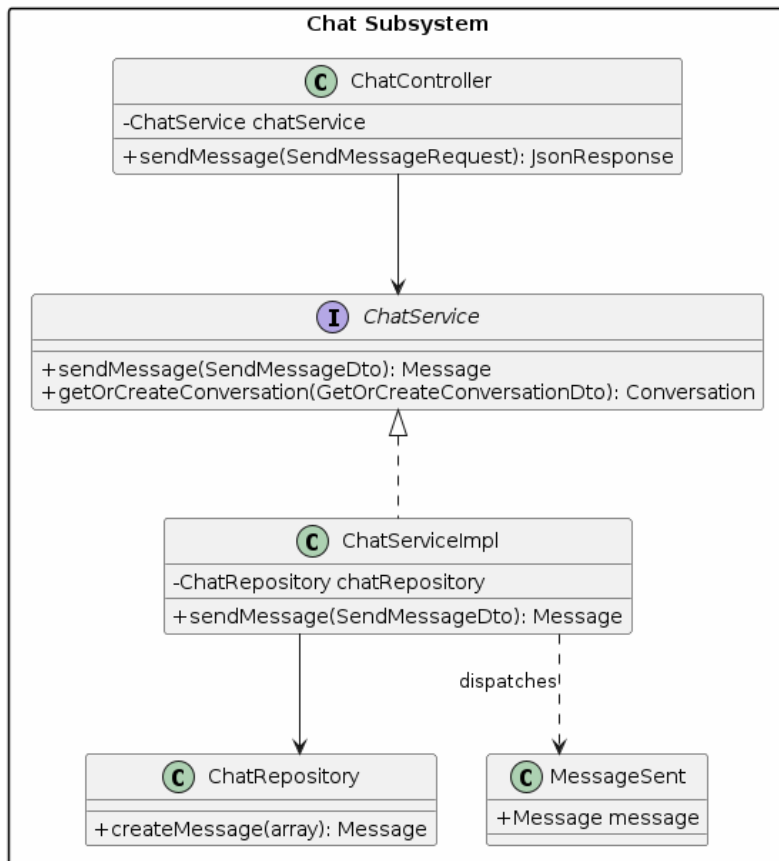
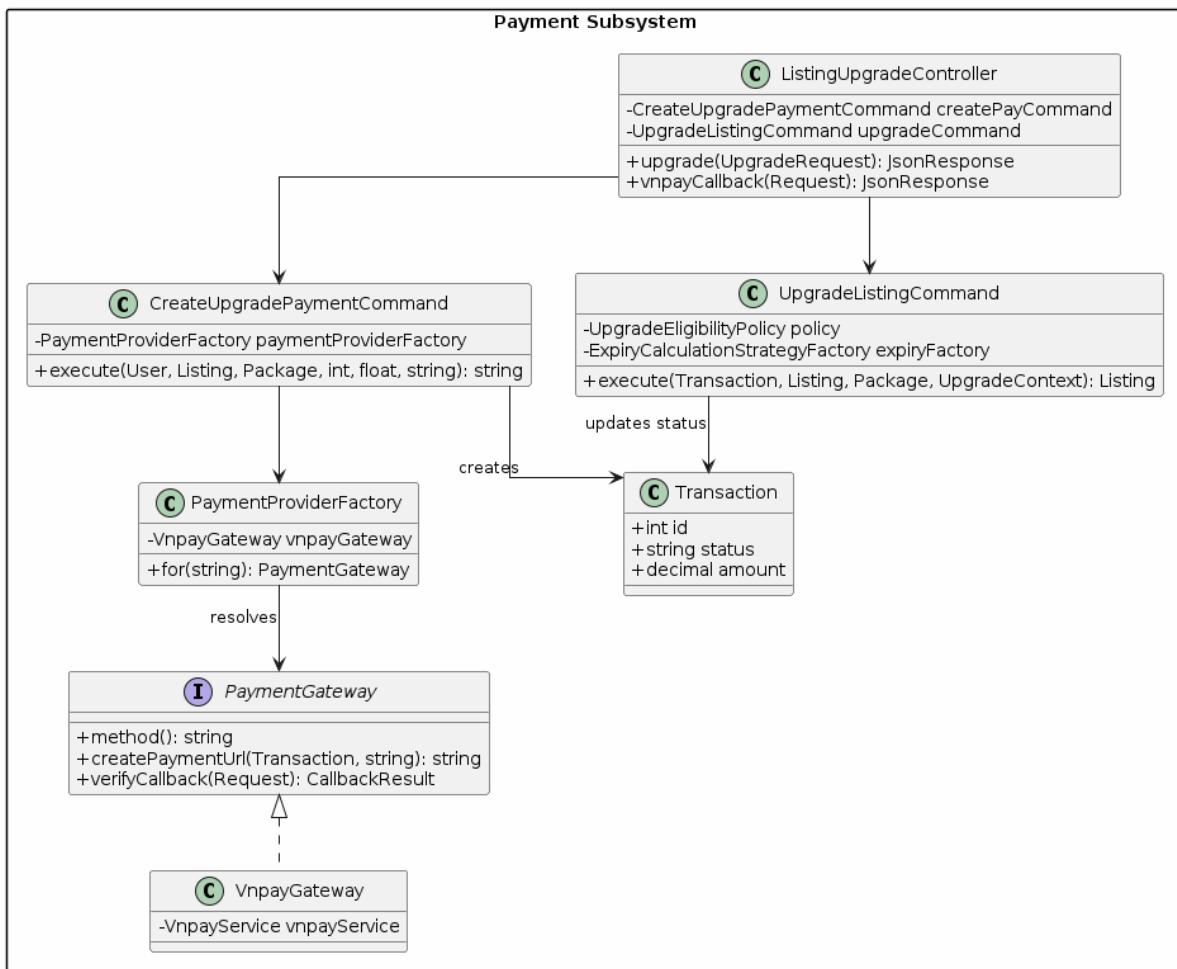
3.2.1. Sơ đồ Lớp tổng thể (Class Diagram)



Hình 41: Sơ đồ tổng thể class







3.2.2. Design Pattern đã sử dụng

3.2.2.1. Factory Pattern

a. Định nghĩa

Định nghĩa một interface để tạo đối tượng, nhưng để các lớp con quyết định lớp nào được khởi tạo. Factory Method cho phép một lớp trì hoãn việc khởi tạo cho các lớp con.

b. Áp dụng

Chức năng	
Đăng nhập	Khởi tạo đối tượng AuthStrategy tương ứng với phương thức đăng nhập.
Sắp xếp hiển thị	Sinh Strategy sắp xếp dựa trên tham số sortBy từ query string.
Lọc dữ liệu	Sinh Strategy tìm kiếm dựa trên tham số search_field.
Nâng cấp gói tin	Chọn lựa đối tượng tính hạn dùng phù hợp (mới mua hoặc gia hạn). Khởi tạo đối tượng áp dụng quyền lợi của gói tin.
Thanh toán	Trả về đối tượng Gateway thanh toán tương ứng (VNPAY, MOMO,...).
Xuất báo cáo (PDF, Excel)	Khởi tạo đúng Strategy theo loại báo cáo hoặc định dạng xuất dữ liệu do người dùng lựa chọn, giúp mã nguồn dễ mở rộng và bảo trì.

c. Ưu điểm cốt lõi của giải pháp

Ưu điểm	Giải thích
Ẩn logic khởi tạo	Client không tự new concrete class.
Dễ mở rộng loại mới	Thêm concrete class và cập nhật factory.
Giảm coupling	Service/controller phụ thuộc interface hoặc factory.
Chuẩn hóa lựa chọn object	Các tham số như sortBy, search_field, payment_method được xử lý tập trung.
Dễ kiểm soát fallback	Factory quyết định default strategy/gateway/state.

d. Bảng so sánh trực quan trước và sau refactoring

Tiêu chí	Trước Refactoring	Sau Refactoring
Khởi tạo strategy	Service/repository tự chọn class cụ thể	Factory trả về strategy phù hợp
Sắp xếp tin	sortBy dễ xử lý bằng if/else ở service	ListingSortingStrategyFactory xử lý
Tìm kiếm	search_field dễ map thủ công	SearchFieldStrategyFactory xử lý
Thanh toán	Payment method dễ map thủ công	PaymentProviderFactory chọn gateway
Tính hạn gói	Logic mua mới/gia hạn dễ trộn trong service	ExpiryCalculationStrategyFactory chọn strategy
Thêm loại mới	Sửa nhiều nơi gọi	Cập nhật factory

3.2.2.2. Command Pattern

a. Định nghĩa

Đóng gói một yêu cầu (request) thành một đối tượng độc lập, cho phép tham số hóa client với các yêu cầu khác nhau, xếp hàng (queue) hoặc ghi log các yêu cầu, và hỗ trợ các thao tác khôi phục(undo/redo).

b. Áp dụng

Chức năng	
Đăng ký tài khoản	Đóng gói toàn bộ quy trình tạo tài khoản (validate, hash mật khẩu, lưu DB, sinh OTP, phát event) khỏi Service.
Nâng cấp tin đăng	- Đóng gói quy trình sinh giao dịch PENDING và link thanh toán VNPAY. - Đóng gói quy trình nâng cấp gói tin sau khi thanh toán thành công (cập nhật trạng thái, tính hạn dùng, phát event).
Đặt lịch hẹn	- Đóng gói thao tác xác nhận lịch hẹn bởi chủ nhà (gọi State, lưu DB, phát event). - Đóng gói thao tác từ chối lịch hẹn kèm ghi chú lý do. - Đóng gói thao tác hủy lịch hẹn của cả khách và chủ nhà.

Tạo tin đăng	Đóng gói quy trình tạo tin đăng mới (lưu Property, hình ảnh, video, tài liệu pháp lý, tính điểm chất lượng, phát event).
--------------	--

c. Ưu điểm cốt lõi của giải pháp

Ưu điểm	Giải thích
Mỗi hành động là một class riêng	Tạo tin, cập nhật, lưu nháp, xác thực không bị trộn chung.
Service gọn hơn	ListingServiceImpl chỉ điều phối command thay vì xử lý mọi chi tiết.
Dễ mở rộng	Thêm hành động mới chỉ cần thêm command mới.
Dễ kiểm thử	Có thể test riêng từng command theo từng nghiệp vụ.
Tuân thủ SRP/OCP	Mỗi command có một trách nhiệm, mở rộng mà ít sửa code cũ.

d. Bảng so sánh trực quan trước và sau refactoring

Tiêu chí	Trước Refactoring	Sau Refactoring
Cấu trúc nghiệp vụ đăng tin	ListingServiceImpl xử lý nhiều logic trong các hàm lớn	Tách từng hành động thành command riêng
Tạo tin đăng	Logic tạo property, listing, media, giấy tờ, trạng thái nằm chung trong service	CreateListingCommand xử lý luồng tạo tin
Cập nhật tin đăng	Logic update bị trộn với logic create/draft/verification	UpdateListingCommand xử lý luồng cập nhật
Lưu nháp	Dễ phải viết nhánh điều kiện trong service	SaveDraftListingCommand điều phối luồng lưu nháp
Gửi/cập nhật xác thực	Dễ bị lẫn trong service chính	SubmitListingVerificationCommand xử lý riêng
Mở rộng hành động mới	Phải sửa service, dễ làm service phình to	Thêm command mới, service chỉ điều phối
Khả năng kiểm thử	Khó test từng hành động riêng vì logic nằm chung	Dễ unit test từng command độc lập
Tuân thủ SOLID	Vi phạm SRP/OCP do service ôm nhiều nghiệp vụ	Tốt hơn, mỗi command có một trách nhiệm rõ ràng

3.2.2.3. Strategy Pattern

a. Định nghĩa

Định nghĩa một họ các thuật toán, đóng gói từng thuật toán và làm cho chúng có thể hoán đổi cho nhau. Strategy cho phép thuật toán biến đổi độc lập với client sử dụng nó.

b. Áp dụng

Chức năng	
Đăng nhập	Cho phép chuyển đổi linh hoạt giữa các phương thức xác thực đăng nhập tại runtime.
Sắp xếp hiển thị tin	Đóng gói từng thuật toán ORDER BY riêng biệt, cho phép thêm/sửa cách sắp xếp mà không sửa Repository.
Lọc dữ liệu tìm kiếm	Đóng gói chiến lược tìm kiếm theo từng trường dữ liệu khác nhau
Nâng cấp tin - Tính hạn dùng	Chọn thuật toán tính ngày hết hạn gói tin (mới mua hay gia hạn).
Đặt lịch hẹn - Hạn chờ	Tính toán thời hạn xác nhận cuộc hẹn theo các quy tắc kinh doanh khác nhau.
Xem báo cáo và xuất báo cáo	Đóng gói từng thuật toán xử lý báo cáo riêng biệt, cho phép thêm hoặc thay đổi loại báo cáo (tháng, năm, quý, gói tin...) mà không cần sửa đổi Service hoặc Controller.

c. Ưu điểm cốt lõi của giải pháp

Ưu điểm	Giải thích
Thay đổi thuật toán linh hoạt	Có thể đổi cách sắp xếp/tìm kiếm bằng cách chọn strategy khác.
Loại bỏ if/else phức tạp	Không cần nhồi nhiều nhánh sort/search vào repository hoặc service.
Dễ mở rộng	Thêm cách sắp xếp hoặc trường tìm kiếm mới bằng cách tạo strategy mới.
Dễ kiểm thử	Có thể test từng strategy độc lập.
Tuân thủ OCP	Mở rộng thuật toán mới mà ít sửa code cũ.

d. Bảng so sánh trực quan trước và sau refactoring

Tiêu chí	Trước Refactoring	Sau Refactoring
Cách sắp xếp tin đăng	Logic ORDER BY có thể bị viết cứng trong repository hoặc dùng nhiều nhánh điều kiện	Mỗi kiểu sắp xếp là một ListingSortingStrategy riêng
Sắp xếp mặc định	Công thức ưu tiên gói tin, điểm chất lượng, thời gian để nằm lẫn trong query lớn	DefaultPackageScoreSortingStrategy đóng gói riêng công thức ranking
Sắp xếp theo giá/diện tích/ngày đăng	Dễ thêm nhiều if/else trong repository	Tách thành PriceLowToHighSortingStrategy, AreaHighToLowSortingStrategy, NewestListingSortingStrategy...
Cách tìm kiếm theo trường	Logic tìm theo tiêu đề, chủ tin, địa chỉ dễ gom trong một hàm lớn	Mỗi trường tìm kiếm là một SearchFieldStrategy riêng
Khi thêm thuật toán mới	Phải sửa logic cũ trong repository/service	Tạo strategy mới và cập nhật factory
Khả năng kiểm thử	Khó test riêng từng cách sort/search	Dễ test từng strategy độc lập
Khả năng đọc code	Query xử lý nhiều trách nhiệm, khó theo dõi	Tên class strategy thể hiện rõ mục đích xử lý
Tuân thủ SOLID	Chưa tốt, dễ vi phạm OCP khi thêm tiêu chí mới	Tốt hơn, mở rộng bằng class mới thay vì sửa logic cũ

3.2.2.4. Chain of Responsibility Pattern

a. Định nghĩa

Cho phép nhiều đối tượng có cơ hội xử lý một yêu cầu bằng cách xâu chuỗi chúng lại với nhau. Yêu cầu được truyền dọc theo chuỗi cho đến khi có một đối tượng xử lý nó.

b. Áp dụng

Chức năng	
Đăng ký tài khoản	Chuỗi kiểm tra tuần tự dữ liệu đăng ký: định dạng Email -> Password mạnh -> Email chưa tồn tại.

Đăng nhập	Chuỗi kiểm tra tuần tự dữ liệu đăng nhập: Email tồn tại -> User ACTIVE -> Đúng mật khẩu.
Kiểm tra điều kiện trước khi đăng tin	Chuỗi kiểm tra tuần tự trước khi cho đăng tin: kiểm tra số điện thoại đã xác thực -> kiểm tra giấy tờ xác thực nếu cần -> nếu lỗi thì dừng chuỗi và trả thông báo -> nếu hợp lệ thì cho phép tạo/cập nhật tin.

c. Ưu điểm cốt lõi của giải pháp

Ưu điểm	Giải thích
Mỗi rule một handler	Rule không bị dồn vào một hàm validate lớn.
Dễ thêm/bớt rule	Thêm handler mới và nối vào chain.
Dừng sớm khi lỗi	Handler có thể throw exception và dừng luồng.
Dễ đọc thứ tự xử lý	Nhìn pipeline/chain biết request đi qua những bước nào
Dễ test	Có thể test từng handler riêng.

d. Bảng so sánh trực quan trước và sau refactoring

Tiêu chí	Trước Refactoring	Sau Refactoring
Cách validate	Nhiều rule nằm trong một hàm lớn	Mỗi rule là một handler
Đăng ký	Email/password/email tồn tại dễ trộn chung	Tách thành chain kiểm tra tuần tự
Đăng nhập	Kiểm tra user, trạng thái, password dễ nằm trong một method	Tách thành handler theo từng bước
Đăng tin	Rule số điện thoại và giấy tờ xác thực dễ nằm trong command	UserPhoneVerifiedHandler , VerificationDocumentsHandler
Quên mật khẩu	Tìm user, gửi OTP, ghi log dễ trộn trong service	Tách thành các forgot password handler
Thêm rule mới	Phải sửa hàm validate cũ	Thêm handler mới
Bảo trì	Khó theo dõi khi rule tăng	Chain rõ ràng, dễ thay đổi thứ tự

3.2.2.5. Observer Pattern

a. Định nghĩa

Định nghĩa mối quan hệ một-nhiều giữa các đối tượng sao cho khi một đối tượng thay đổi trạng thái, tất cả các đối tượng phụ thuộc vào nó được thông báo và cập nhật tự động.

b. Áp dụng

Chức năng	
Chat	Phát tin nhắn realtime đến người nhận qua WebSocket.
Nâng cấp tin thành công	Thông báo cho người dùng vào mục thông báo
Đăng ký thành công	Gửi email chào mừng người dùng mới qua hàng đợi (Queue), không block API.
Xử lý sau khi lưu tin đăng	Sau khi tạo/cập nhật/gỡ/duyet tin, hệ thống phát event ListingSaved. Các listener độc lập xử lý hậu kỳ như xóa cache danh sách tin, ghi log, tạo thông báo cho người dùng.

c. Ưu điểm cốt lõi của giải pháp

Ưu điểm	Giải thích
Giảm coupling	Publisher không biết listener cụ thể.
Tách side effect	Gửi mail, log, cache, notification nằm ngoài nghiệp vụ chính.
Dễ mở rộng	Thêm listener mới mà ít sửa code cũ.
Hỗ trợ async	Listener có thể chuyển sang queue.
Tuân thủ OCP	Thêm phản ứng mới sau event mà không sửa publisher.

d. Bảng so sánh trực quan trước và sau refactoring

Ưu điểm	Giải thích
Giảm coupling	Publisher không biết listener cụ thể.
Tách side effect	Gửi mail, log, cache, notification nằm ngoài nghiệp vụ chính.
Dễ mở rộng	Thêm listener mới mà ít sửa code cũ.
Hỗ trợ async	Listener có thể chuyển sang queue.
Tuân thủ OCP	Thêm phản ứng mới sau event mà không sửa publisher.

Tiêu chí	Trước Refactoring	Sau Refactoring
Xử lý sau nghiệp vụ	Controller/service gọi trực tiếp mail, log, cache	Dispatch event, listener tự xử lý
Đăng ký user	Gửi welcome mail có thể nằm trong command/controller	UserRegistered phát event, listener gửi mail
Tạo/cập nhật tin	Xóa cache, log, notification dễ nằm trong command	ListingSaved kích hoạt các listener
Chat	Gửi tin nhắn và broadcast dễ gắn chặt	MessageSent được broadcast qua listener/event
Thêm side effect mới	Phải sửa publisher	Thêm listener mới
Coupling	Cao	Thấp

3.2.2.6. State Pattern

a. Định nghĩa

Cho phép một đối tượng thay đổi hành vi khi trạng thái nội tại của nó thay đổi. Đối tượng sẽ có vẻ như thay đổi lớp của nó.

b. Áp dụng

Chức năng	
Đặt lịch hẹn	Quản lý vòng đời lịch hẹn: PENDING -> APPROVED/COMPLETED/CANCELLED, đảm bảo mỗi trạng thái tự kiểm soát hành vi cho phép.
Tin đăng - Vòng đời	Quản lý chuyển đổi trạng thái tin đăng: DRAFT -> PENDING -> ACTIVE/LOCKED/REJECTED/UNLISTED, khóa chặt luồng hợp lệ.

c. Ưu điểm cốt lõi của giải pháp

Ưu điểm	Giải thích
Quản lý trạng thái rõ ràng	Mỗi state tự biết hành vi và transition hợp lệ.
Giảm if/else	Không rải điều kiện status ở nhiều nơi.
Giảm lỗi chuyển trạng thái	Transition được kiểm soát tập trung.
Dễ mở rộng workflow	Thêm state mới bằng class mới.
Dễ test	Test từng state độc lập.

d. Bảng so sánh trực quan trước và sau refactoring

Tiêu chí	Trước Refactoring	Sau Refactoring
Quản lý status	So sánh chuỗi/số trạng thái rải rác	State class đại diện từng trạng thái
Tin đăng	DRAFT/PENDING/ACTIVE/LOCKED/UNLISTED dễ set tùy ý	ListingStatusStateFactory kiểm soát transition
Lịch hẹn	PENDING/APPROVED/CANCELLED dễ dùng if/else	BookingState quản lý hành vi theo trạng thái
Giao dịch	PENDING/SUCCESS/SETTLED dễ bị xử lý phân tán	TransactionState kiểm soát vòng đời giao dịch
Thêm trạng thái mới	Phải sửa nhiều điều kiện	Thêm state class và cập nhật factory
Nguy cơ sai nghiệp vụ	Cao hơn	Thấp hơn do transition rõ ràng

3.2.2.7. Template Method Pattern

a. Định nghĩa

Xác định khung (skeleton) của một thuật toán trong một phương thức, để các lớp con định nghĩa lại các bước cụ thể mà không thay đổi cấu trúc thuật toán

b. Áp dụng

Chức năng	
Admin duyệt tin	Định nghĩa khung xử lý kiểm duyệt (Transaction, Lock DB, Check State, Save, Log History, Dispatch Event); các lớp con chỉ cần điền targetStatus()+mutate()

c. Ưu điểm cốt lõi của giải pháp

Ưu điểm	Giải thích
Tránh lặp quy trình	Các command duyệt/từ chối/khóa dùng chung khung xử lý.
Kiểm soát thứ tự bước	Template method đảm bảo bước nào chạy trước/sau.
Lớp con gọn	Lớp con chỉ override phần khác biệt.
Dễ bảo trì	Sửa quy trình chung ở abstract class.
Tăng nhất quán nghiệp vụ	Duyệt, từ chối, khóa đều qua cùng một pipeline xử lý.

d. Bảng so sánh trực quan trước và sau refactoring

Tiêu chí	Trước Refactoring	Sau Refactoring
Admin duyệt tin	Approve/reject/lock dễ lặp transaction, state check, log	AbstractListingModerationCommand định nghĩa khung chung
Thứ tự xử lý	Dễ lệch giữa các command	Template cố định thứ tự
Lớp con	Có thể chứa nhiều code lặp	Chỉ khai báo status đích và mutate riêng
Ghi lịch sử/event	Dễ thiếu ở một vài nhánh	Khung chung đảm bảo luôn ghi history/dispatch event
Bảo trì	Sửa nhiều command	Sửa abstract template khi thay đổi quy trình chung

3.2.2.8. Specification Pattern

a. Định nghĩa

Specification thông báo kiểm tra một đối tượng xem nó có thỏa mãn một tiêu chí nào đó không. Nó đóng gói một luật nghiệp vụ thành một đối tượng độc lập.

b. Áp dụng

Chức năng	
Nâng cấp tin đăng	Đóng gói các luật kiểm định nghiệp vụ (nâng cấp gói cao hơn hoặc gia hạn gói cùng loại) thành các lớp độc lập, dễ tái sử dụng.

c. Ưu điểm cốt lõi của giải pháp

Ưu điểm	Giải thích
Tách luật nghiệp vụ	Điều kiện nâng cấp/gia hạn không nằm rải rác trong service.
Dễ tái sử dụng	Cùng specification có thể dùng cho API, UI check, admin.
Dễ test	Test từng rule độc lập.
Dễ mở rộng rule	Thêm specification mới khi có chính sách mới.
Code tự mô tả	Tên class như CanUpgradeSpecification nói rõ rule đang kiểm tra.

d. Bảng so sánh trực quan trước và sau refactoring

Tiêu chí	Trước Refactoring	Sau Refactoring
Kiểm tra nâng cấp gói	Điều kiện có thể nằm trực tiếp trong service	CanUpgradeSpecification xử lý
Kiểm tra gia hạn	Dễ trộn với rule nâng cấp	CanRenewSpecification xử lý
Khi thêm rule mới	Sửa service hiện có	Tạo specification mới
Test rule	Khó tách rule khỏi service	Test từng specification độc lập
Bảo trì chính sách	Điều kiện phân tán	Rule nằm trong object riêng

3.2.2.9. Adapter Pattern

a. Định nghĩa

Chuyển đổi interface của một lớp thành interface khác mà client mong đợi. Adapter cho phép các lớp làm việc cùng nhau mà bình thường không thể do interface không tương thích.

b. Áp dụng

Chức năng	
Đăng nhập Google	Chuẩn hóa dữ liệu người dùng Google SDK về interface Domain thống nhất.
Thanh toán	Chuẩn hóa giao tiếp với cổng thanh toán VNPAY qua interface chung.
Lưu trữ Media(Cloudflare)	Chuẩn hóa thao tác upload và tạo URL truy cập file lên Cloudflare R2 (S3-compatible). Bọc S3Client để tạo Presigned URL bảo mật.
Upload chữ ký Media(Cloudinary)	Chuẩn hóa tạo chữ ký upload cho Frontend (signature, api_key, cloud_name, timestamp) để upload file trực tiếp lên Cloudinary CDN.

c. Ưu điểm cốt lõi của giải pháp

Ưu điểm	Giải thích
Giảm phụ thuộc provider	Code nghiệp vụ không phụ thuộc trực tiếp Cloudinary, R2, VNPAY, Google SDK.
Dễ thay thế dịch vụ ngoài	Có thể đổi provider bằng adapter mới.
Interface ổn định	Client gọi interface chung.
Dễ mock khi test	Mock adapter thay vì mock SDK phức tạp.
Tuân thủ DIP	Phụ thuộc abstraction thay vì concrete provider.

d. Bảng so sánh trực quan trước và sau refactoring

Tiêu chí	Trước Refactoring	Sau Refactoring
Upload Cloudinary	Controller/service phụ thuộc trực tiếp Cloudinary	Dùng UploadSignatureAdapter
Lưu trữ R2	Code dễ phụ thuộc S3/R2 client cụ thể	Dùng FileStorageAdapter
Thanh toán VNPAY	Logic nghiệp vụ dễ gắn với VnpayService	Dùng PaymentGateway và VnpayGateway
Google login	Domain dễ phụ thuộc object Socialite	GoogleSocialiteAdapter chuẩn hóa dữ liệu user
Chat realtime	Domain event dễ phụ thuộc cơ chế broadcast cụ thể	Reverb/Broadcast đóng vai trò adapter realtime

3.2.3. Phân tích thiết kế theo từng chức năng

3.2.3.1. Đăng nhập

- **Chức năng đã sử dụng các Design Patterns:** Strategy Pattern, Factory Method Pattern, Adapter Pattern

a. Strategy Pattern

- **Vấn đề cần giải quyết:**
 - Hệ thống hỗ trợ nhiều phương thức đăng nhập khác nhau: Email/Password truyền thống và Google OAuth liên kết bên thứ ba.
 - Nếu giải quyết bằng cách viết một hàm đăng nhập khổng lồ chứa câu lệnh switch-case hay if (method == 'GOOGLE'), hệ thống sẽ vi phạm nguyên tắc Open/Closed (OCP).
 - Mỗi lần thêm một cổng đăng nhập mới (như Facebook, Apple ID), ta lại phải sửa đổi và chạy lại regression test cho toàn bộ.
- Sơ đồ class



Thành phần	Lớp cụ thể trong dự án	Tác dụng
Context	Auth\AuthStrategyResolver	Quản lý danh sách các Strategy, nhận diện loại hình đăng nhập từ Client để cung cấp đối tượng Strategy xử lý tương thích.
Strategy	Auth\AuthStrategy	Định nghĩa phương thức authenticate(AuthPayload \$payload) chung để mọi cơ chế đăng nhập bắt buộc phải tuân theo.
ConcreteStrategy A	Auth\Strategies\EmailPasswordAuthStrategy	Chịu trách nhiệm xác thực thông tin đăng nhập truyền thống, gọi LoginValidationChain và sinh JWT Token.
ConcreteStrategy B	Auth\Strategies\GoogleOAuthAuthStrategy	Tiếp nhận access token hoặc auth code của Google, kiểm tra thông tin qua Google SDK, lưu hoặc cập nhật thông tin user và cấp JWT Token tương ứng.

- **Code triển khai**

AuthStrategy(Resolver)

```
final class AuthStrategyResolver    buith, 5/17/2026 8:03 PM • feat: implement modular authentication system w
{
    /**
     * @param AuthStrategy[] $strategies
     */
    2 usages  🧑 buith
    public function __construct(
        private readonly array $strategies,
    ) {}

    🧑 buith
    public function resolve(AuthMethod $method): AuthStrategy
    {
        foreach ($this->strategies as $strategy) {
            if ($strategy->method() === $method) {
                return $strategy;
            }
        }

        throw new LogicException( message: "No auth strategy registered for method [{ $method->value }].");
    }
}
```

AuthStrategy(Interface)

```
interface AuthStrategy
{    buith, 5/17/2026 8:03 PM • feat: implement modular authentication sys
    4 implementations  🧑 buith
    public function method(): AuthMethod;

    4 implementations  🧑 buith
    public function authenticate(AuthPayload $payload): AuthResultDto;
}
```

EmailPasswordAuthStrategy(Concrete)

```
final class EmailPasswordAuthStrategy implements AuthStrategy    buith, 5/17/2026 8:03 PM • feat: implement modular authentication system
{
    ⚙️ buith
    public function __construct(
        private readonly AuthFactory $authFactory,
        private readonly AuthTokenIssuer $tokenIssuer,
        private readonly LoginValidationChain $loginValidationChain,
    ) {}

    ⚙️ buith
    public function method(): AuthMethod
    {
        return AuthMethod::EmailPassword;
    }

    ⚙️ buith
    public function authenticate(AuthPayload $payload): AuthResultDto
```

GoogleOAuthAuthStrategy(Concrete)

```
final class GoogleOAuthAuthStrategy implements AuthStrategy
{
    1 usage    ⚙️ buith
    public function __construct(
        private readonly UserUpsertService $userUpsertService,
        private readonly AuthTokenIssuer $tokenIssuer,
    ) {}

    ⚙️ buith
    public function method(): AuthMethod
    {    buith, 5/17/2026 8:03 PM • feat: implement modular authentication system
        return AuthMethod::GoogleOAuth;
    }

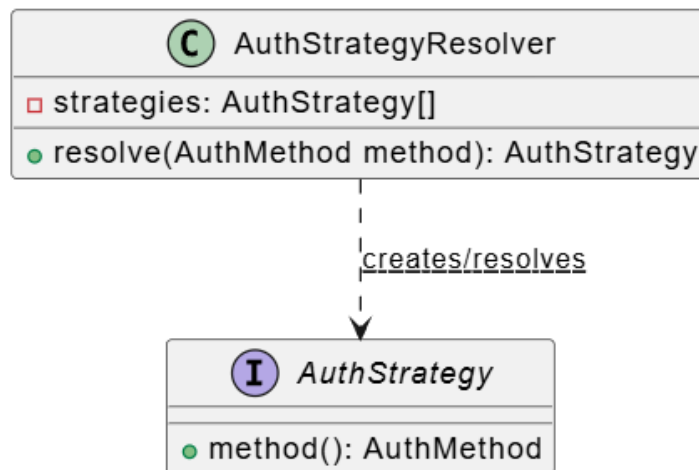
    ⚙️ buith
    public function authenticate(AuthPayload $payload): AuthResultDto
```

b. *Factory Method Pattern*

- **Vấn đề giải quyết**

- Làm thế nào để biết cách khởi tạo Strategy nào tại runtime mà không cần phải viết code cứng new GoogleOAuthAuthStrategy(...)?
- Client không nên biết chi tiết cấu tạo bên trong hoặc các tham số khởi dựng phức tạp của từng Strategy cụ thể.

- **Sơ đồ class**



Thành phần	Lớp cụ thể trong dự án	Tác dụng
Creator(Factory)	Auth\AuthStrategyResolver	Nhận vào danh sách các Strategy đã được đăng ký thông qua Container, triển khai hàm resolve(AuthMethod \$method) để tìm kiếm Strategy có type tương khớp và trả về đối tượng đó cho Controller sử dụng.
Strategy	Auth\AuthStrategy	Định nghĩa phương thức authenticate(AuthPayload \$payload) chung để mọi cơ chế đăng nhập bắt buộc phải tuân theo.

- **Code triển khai**

AuthStrategyResolver(Creator)

```
final class AuthStrategyResolver buith, 5/17/2026 8:03 PM • feat: implement modular authentication system W
{
    /**
     * @param AuthStrategy[] $strategies
     */
    2 usages  🐞 buith
    public function __construct(
        private readonly array $strategies,
    ) {}

    🐞 buith
    public function resolve(AuthMethod $method): AuthStrategy
    {
        foreach ($this->strategies as $strategy) {
            if ($strategy->method() === $method) {
                return $strategy;
            }
        }

        throw new LogicException( message: "No auth strategy registered for method [{ $method->value}].");
    }
}
```

AuthStrategy(Product)

```
interface AuthStrategy
{ buith, 5/17/2026 8:03 PM • feat: implement modular authentication sys
    4 implementations  🐞 buith
    public function method(): AuthMethod;

    4 implementations  🐞 buith
    public function authenticate(AuthPayload $payload): AuthResultDto;
}
```

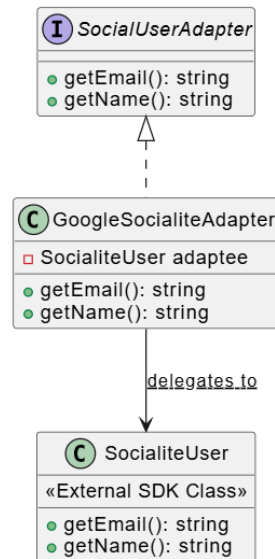
c. Adapter Pattern

- **Vấn đề giải quyết**

- Khi tích hợp đăng nhập qua Google OAuth, SDK bên thứ ba (như Laravel Socialite) trả về một đối tượng User thô có cấu trúc dữ liệu và tên phương thức (như getId(), user['emails'],...) hoàn toàn phụ thuộc vào nhà cung cấp SDK.

- Nếu ta sử dụng trực tiếp đối tượng thô này trong tầng nghiệp vụ lõi (Domain/Service), mã nguồn sẽ bị phụ thuộc chặt chẽ (tightly coupled) vào thư viện ngoài, khiến việc nâng cấp hoặc đổi nhà cung cấp SDK sau này trở nên cực kỳ rủi ro và tốn kém công sức sửa lỗi. Nếu muốn tích hợp thêm Facebook hay Apple thì phải viết thêm SDK riêng cho nó

- Sơ đồ class



Thành phần	Lớp cụ thể trong dự án	Tác dụng
Target Interface	Auth\SocialUserAdapter	Định nghĩa các phương thức chuẩn như getEmail(), getName(), getAvatar() để hệ thống sử dụng thống nhất.
Adapter	Auth\Adapters\GoogleSocialiteAdapter	Lớp bọc bên ngoài lớp của SDK Google để chuyển đổi interface.
Adaptee	Laravel\Socialite\Two\User (Từ SDK Socialite ngoài)	Đối tượng người dùng trả về trực tiếp từ SDK bên thứ ba.

- Code triển khai
SocialUserAdapter (Interface)

```

interface SocialUserAdapter {
    4 usages 2 implementations 2 people buith
    public function getProviderName(): string;

    3 usages 2 implementations 2 people buith
    public function getProviderId(): string;

    1 usage 2 implementations 2 people buith
    public function getName(): string;

    2 usages 2 implementations 2 people buith
    public function getEmail(): string;

    3 usages 2 implementations 2 people buith
    public function getAvatarUrl(): ?string;
}

```

GoogleSocialiteAdapter(Adapter)

```

final class GoogleSocialiteAdapter implements SocialUserAdapter
{
    1 usage new *
    public function __construct(private readonly SocialiteUser $socialiteUser) {}

    4 usages new *
    public function getProviderName(): string{return 'google'}

    3 usages new *
    public function getProviderId(): string{return $this->socialiteUser->getId();} You,

    1 usage new *
    public function getName(): string{return $this->socialiteUser->getName();}

    2 usages new *
    public function getEmail(): string{return $this->socialiteUser->getEmail();}

    3 usages new *
    public function getAvatarUrl(): ?string{return $this->socialiteUser->getAvatar();}

}

```

Socialite\Two\User(Adaptee)

```

interface User
{
    |
    | Get the unique identifier for the user.
    | Returns: string
    |
    | public function getId();
    |
    |
    | Get the nickname / username for the user.
    | Returns: null|string
    |
    | public function getNickname();
    |
    |
    | Get the full name of the user.
    | Returns: null|string
    |
    | public function getName();
    |
    |
    | Get the e-mail address of the user.
    | Returns: null|string
    |
    | public function getEmail();
    |
    |
    | Get the avatar / image URL for the user.
    | Returns: null|string
    |
    | public function getAvatar();
}

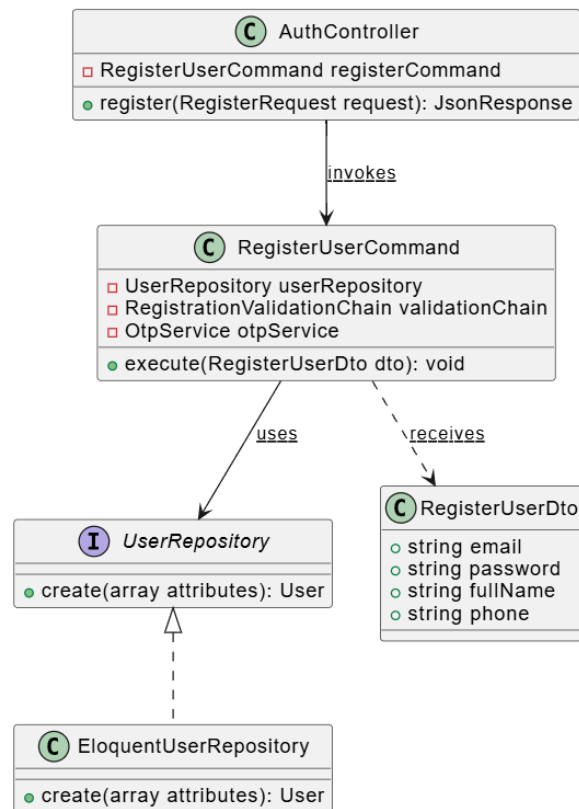
```

3.2.3.2. Đăng ký

- **Chức năng đã sử dụng các Design Patterns:** Command Pattern

a. Command Pattern

- Vấn đề cần giải quyết
 - Luồng đăng ký thành viên chứa rất nhiều bước nghiệp vụ liên tiếp và phức tạp: kiểm tra định dạng dữ liệu đầu vào, lưu thông tin tài khoản thô vào database ở trạng thái chờ kích hoạt, tạo mã kích hoạt OTP, gửi email OTP và phát domain event chào mừng.
 - Nếu viết trực tiếp toàn bộ logic này vào service, service sẽ bị phình to (Fat Controller), vi phạm Single Responsibility Principle (SRP) và khiến code cực kỳ khó tái sử dụng hoặc viết unit test độc lập.
- Sơ đồ class



Thành phần	Lớp cụ thể trong dự án	Tác dụng
Invoker	Auth\AuthController	Tiếp nhận HTTP Request từ client, thực hiện lọc dữ liệu đầu vào qua FormRequest, đóng gói vào RegisterUserDto và gọi phương thức execute() của RegisterUser Command
Command(Interface)	<i>(Không dùng do tận dụng DI & Class-based Command)</i>	Laravel Services/Commands thường viết trực tiếp dạng Concrete class.
ConcreteCommand	Laravel\Socialite\Two\User (Từ SDK Socialite ngoài)	Đóng gói quy trình tạo tài khoản mới. Có nhiệm vụ gọi Validator để kiểm tra điều kiện nghiệp vụ, tạo mật khẩu hash, gọi repository lưu dữ liệu, sinh mã OTP thông

		qua OtpService và phát đi event UserRegistered
Receiver	Repositories\UserRepository	Thực hiện hành động chèn bản ghi user mới vào cơ sở dữ liệu.

- Code triển khai

AuthServiceImpl (Invoker)

```

    buith
    public function register(RegisterUserDto $dto): void
    {
        $this->registerUserCommand->execute($dto);
    }

```

RegisterCommand(ConcreteCommand)

```

    public function execute(RegisterUserDto $dto): void
    {
        $this->validationChain->validate($dto);
        buith, 5/25/2026 11:20 AM • feat: implement comprehensive authentication service with registration, l
        DB::transaction(function () use ($dto) {
            $user = $this->userRepository->findByEmail($dto->email);

            if ($user) {
                $this->userRepository->update($user->id, [
                    'full_name' => $dto->fullName,
                    'password' => Hash::make($dto->password),
                ]);
                $user->refresh();
                Log::info( message: 'Existing pending user re-registered', ['user_id' => $user->id]);
            } else {
                $user = $this->userRepository->create([
                    'full_name' => $dto->fullName,
                    'email' => $dto->email,
                    'password' => Hash::make($dto->password),
                    'role' => UserRole::User->value,
                    'status' => UserStatus::Pending->value,
                ]);
                Log::info( message: 'New user registered (pending OTP)', ['user_id' => $user->id]);
            }

            $this->otpService->generate($user, context: OtpContext::REGISTER);
        });
    }

```

Receiver (Validation, UserRepo, otpService)

```
public function create(array $attributes): User
{
    return $this->model->create($attributes);
}
```

1 implementation  buith

```
public function update(int $id, array $attributes): User;
```

3.2.3.3. Nâng cấp tin đăng

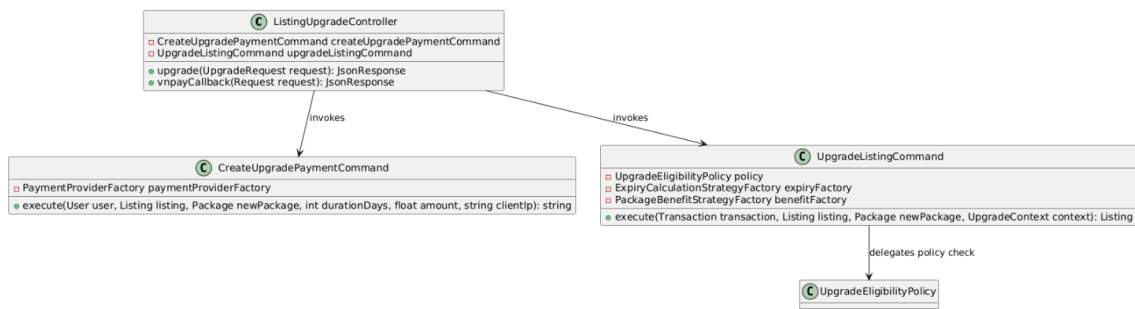
- **Chức năng đã sử dụng các Design Patterns:** Command Pattern, Strategy Pattern, Specification Pattern

a. Command Pattern

- **Vấn đề cần giải quyết:**

- Quy trình nâng cấp tin đăng bất động sản là một chuỗi hành động phức tạp liên quan đến tài chính: Xác định tính hợp lệ của tin đăng -> Khởi tạo giao dịch ở trạng thái PENDING -> Sinh đường dẫn thanh toán -> Nhận phản hồi thanh toán -> Cập nhật trạng thái giao dịch SUCCESS -> Thay đổi gói tin và tính hạn dùng mới cho tin đăng -> Áp dụng quyền lợi ưu tiên -> Phát event xóa bộ nhớ đệm.
- Nếu không được đóng gói chặt chẽ, việc rải rác các bước này trong Controller sẽ dễ dẫn đến mất mát dữ liệu hoặc không đồng bộ trạng thái giao dịch khi xảy ra lỗi mạng hoặc lỗi cơ sở dữ liệu giữa chừng.

- **Sơ đồ class**



Thành phần	Lớp cụ thể trong dự án	Tác dụng
------------	------------------------	----------

Invoker	Listing\ListingUpgradeController	Tiếp nhận callback thanh toán hoặc gửi yêu cầu nâng cấp tin.
ConcreteCommandA	Services\Listing\Upgrade\CreateUpgradePaymentCommand	Chịu trách nhiệm tạo bản ghi Transaction ở trạng thái PENDING, thiết lập thời gian hết hạn (15 phút), kích hoạt queue job tự động hủy nếu quá hạn và lấy URL thanh toán từ Payment Adapter để trả về cho người dùng
ConcreteCommandB	Services\Listing\Upgrade\UpgradeListingCommand	Được gọi khi cổng thanh toán VNPAY trả kết quả thành công. Command này thực thi kiểm tra tính hợp lệ qua Policy, tính ngày hết hạn của gói mới, cập nhật trạng thái Transaction sang SUCCESS, thay đổi package_id và package_expires_at của Tin đăng, đồng thời phát event ListingPackageUpgraded.
Receiver	App\Models\Listing App\Models\Transaction	Các model nhận lệnh thay đổi trạng thái trong CSDL.

- Code triển khai

ListingServiceImpl(Invoker)

```

public function createUpgradePayment(User $user, int $listingId, int $packageId, int $durationDays, string $clientId): string
{
    [$listing, $newPackage, $pricing, $context] = $this->prepareUpgrade($user, $listingId, $packageId, $durationDays);

    return $this->createUpgradePaymentCommand->execute(
        user: $user,
        listing: $listing,
        newPackage: $newPackage,
        durationDays: $durationDays,
        amount: (float) $pricing->price,
        clientId: $clientId,
    );
}

```


CreateUpgradePaymentCommand(Concrete)

```
final class CreateUpgradePaymentCommand buith, 5/25/2026 11:16 AM • feat: 1
{
    no usages  🐞 buith +1
    public function __construct(
        private readonly PaymentProviderFactory $paymentProviderFactory,
    ) {}

    🐞 buith +1
    public function execute(
        User $user,
        Listing $listing,
        Package $newPackage,
        int $durationDays,
        float $amount,
        string $clientIp
    ): string {
        $paymentExpiresAt = now()->addMinutes(15);
```

UpgradeListingCommand(Concrete)

```
final class UpgradeListingCommand buith, 5/25/2026 11:16 AM • feat: implement listing favorites, verification workflow, and pack
{
    🐞 buith
    public function __construct(
        private readonly UpgradeEligibilityPolicy $policy,
        private readonly ExpiryCalculationStrategyFactory $expiryFactory,
        private readonly PackageBenefitStrategyFactory $benefitFactory,
    ) {}

    🐞 buith
    public function execute(Transaction $transaction, Listing $listing, Package $newPackage, UpgradeContext $context): Listing
    {
        $this->policy->assertEligible($context);
        $expiresAt = $this->expiryFactory->make($context)->calculate($context);
        $benefitStrategy = $this->benefitFactory->make($newPackage);

        $updated = DB::transaction(function () use ($transaction, $listing, $newPackage, $expiresAt, $benefitStrategy, $context) {
            $transaction->update([
                'status' => 'SUCCESS',
                'transaction_date' => now(),
            ]);

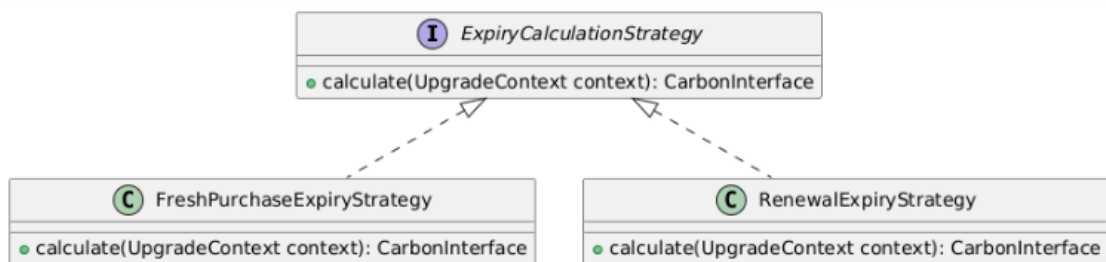
            $listing->update([
                'package_id' => $newPackage->id,
                'package_expires_at' => $expiresAt,
            ]);
        });
```

b. Strategy Pattern

- Vấn đề cần giải quyết:

- Hệ thống cho phép cả hai hành vi: Nâng cấp gói tin mới (thời hạn tính từ hôm nay) và Gia hạn gói tin trùng loại (ngày hết hạn mới phải được cộng dồn vào ngày hết hạn cũ nếu tin đăng chưa hết hạn).
- Nếu gom chung các thuật toán tính hạn dùng này vào một khối code, logic tính toán ngày tháng sẽ trở nên vô cùng rắc rối, nhiều nhánh rẽ phức tạp và rất khó kiểm soát các lỗi về mặt thời gian, dẫn đến xung đột quyền lợi của khách hàng trả phí.

- **Sơ đồ class**



Thành phần	Lớp cụ thể trong dự án	Ghi chú
Context	Services\Listing\Upgrade\UpgradeListingCommand	Lớp điều phối gọi Strategy để tính toán thời hạn.
Strategy(Interface)	Services\Listing\Upgrade\Expiry\ExpiryCalculationStrategy	Định nghĩa phương thức calculate(UpgradeContext \$context) trả về đối tượng thời gian CarbonInterface
ConcreteStrategyA	Services\Listing\Upgrade\Expiry\FreshPurchaseExpiryStrategy	Sử dụng thời gian hiện tại (\$context->now) làm mốc bắt đầu, cộng thêm số ngày đặt mua (durationDays)
ConcreteStrategy B	Services\Listing\Upgrade\Expiry\RenewalExpiryStrategy	Kiểm tra ngày hết hạn gói tin cũ. Nếu ngày cũ vẫn còn hiệu lực (chưa hết hạn), nó sẽ cộng dồn số ngày mua mới vào ngày cũ này. Nếu đã quá hạn,

		nó sẽ lấy ngày hiện tại làm mốc rồi cộng thêm.
--	--	--

- **Code triển khai**

UpgradeListingCommand

```
final class UpgradeListingCommand
{
    @buith
    public function __construct(
        private readonly UpgradeEligibilityPolicy $policy,
        private readonly ExpiryCalculationStrategyFactory $expiryFactory,
        private readonly PackageBenefitStrategyFactory $benefitFactory,
    ) {} buith, 5/25/2026 11:16 AM • feat: implement listing favorites, verification workflow, and package upgrade system

    @buith
    public function execute(Transaction $transaction, Listing $listing, Package $newPackage, UpgradeContext $context): Listing
    {
        $this->policy->assertEligible($context);
        $expiresAt = $this->expiryFactory->make($context)->calculate($context);
        $benefitStrategy = $this->benefitFactory->make($newPackage);

        $updated = DB::transaction(function () use ($transaction, $listing, $newPackage, $expiresAt, $benefitStrategy, $context) {
            $transaction->update([
                'status' => 'SUCCESS',
                'transaction_date' => now(),
            ]);
        });
    }
}
```

ExpiryCalculationStrategyFactory

```
interface ExpiryCalculationStrategy buith, 5/15/2026 4:04 PM • feat: i
{
    2 usages 2 implementations @buith
    public function calculate(UpgradeContext $context): CarbonInterface;
}
```

FreshPurchaseExpiryStrategy

```

final class FreshPurchaseExpiryStrategy implements ExpiryCalculationStrategy
{
    buith, 5/15/2026 4:04 PM • feat: implement modular listing upgrade system v
    3 usages  🧑 buith
    public function calculate(UpgradeContext $context): CarbonInterface
    {
        return $context->now->copy()->addDays($context->durationDays);
    }
}

```

RenewalExpiryStrategy

```

final class RenewalExpiryStrategy implements ExpiryCalculationStrategy
{
    3 usages  🧑 buith +1
    public function calculate(UpgradeContext $context): CarbonInterface
    {
        $baseTime = $context->listing->package_expires_at;

        if (! $baseTime || $baseTime->lessThanOrEqualTo($context->now)) {
            $baseTime = $context->now;
        }

        return $baseTime->copy()->addDays($context->durationDays);
    }
}

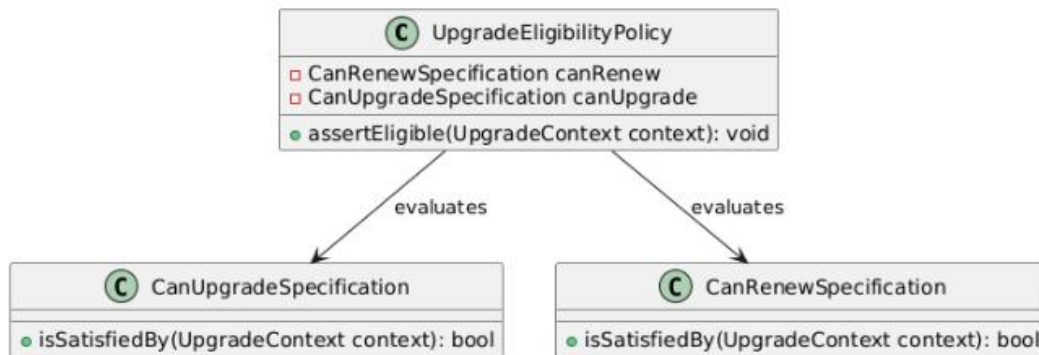
```

c. *Specification Pattern*

- **Vấn đề cần giải quyết:**

- Quy tắc kiểm định điều kiện nâng cấp/gia hạn gói tin của hệ thống rất phức tạp (ví dụ: Tin phải ACTIVE, người nâng cấp phải là chủ tin, gói mới phải đang hoạt động, và hoặc là nâng cấp lên gói có độ ưu tiên cao hơn, hoặc là gia hạn gói cùng loại).
- Nếu viết tất cả các điều kiện này bằng các câu lệnh if-else chồng chéo trong luồng thanh toán, mã nguồn sẽ trở nên cực kỳ khó đọc, không thể tái sử dụng các quy tắc kiểm định này ở nơi khác (như ẩn/hiện nút Nâng cấp ngoài giao diện), và khó thêm mới các luật kiểm duyệt khác trong tương lai.

- Sơ đồ class



Thành phần	Lớp cụ thể trong dự án	Ghi chú
Specification(A)	Services\Listing\Upgrade\Specifications\CanUpgradeSpecification	Xác nhận yêu cầu nâng cấp là hợp lệ nếu: không phải yêu cầu gia hạn và độ ưu tiên (priority) của gói mới phải lớn hơn độ ưu tiên của gói hiện tại đang dùng.
Specification(B)	Services\Listing\Upgrade\Specifications\CanRenewSpecification	Xác nhận yêu cầu gia hạn là hợp lệ nếu: gói mới giống hệt gói cũ và tin đăng vẫn đang kích hoạt gói này.
Context	Services\Listing\Upgrade\UpgradeEligibilityPolicy	Tiến hành kiểm định quyền sở hữu tin đăng, trạng thái tin đăng phải là ACTIVE, kiểm tra gói mới phải ở trạng thái đang mở (is_active), và cuối cùng là phải thỏa mãn một trong hai Specification trên.

- Code triển khai

UpgradeEligibilityPolicy

```

final class UpgradeEligibilityPolicy buith, 5/15/2026 4:04 PM • feat: implement modular list:
{
    1 usage  🧑 buith
    public function __construct(
        private readonly CanRenewSpecification $canRenew,
        private readonly CanUpgradeSpecification $canUpgrade,
    ) {}

    7 usages  🧑 buith
    public function assertEligible(UpgradeContext $context): void
    {
        if ($context->listing->owner_id !== $context->user->id) {
            throw new BusinessException( errorCode: ErrorCode::ListingNotOwned);
        }
    }
}

```

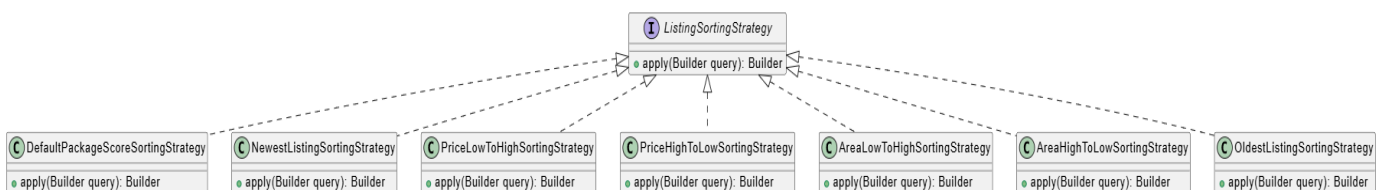
3.2.3.4. Thuật toán sắp xếp hiển thị

- **Chức năng đã sử dụng các Design Patterns:** Strategy Pattern, Factory Method Pattern, Adapter Pattern

a. Strategy Pattern

- **Vấn đề cần giải quyết:**
 - Trang danh sách tin đăng của nền tảng bất động sản yêu cầu sắp xếp linh hoạt theo nhiều tiêu chí khác nhau: Mặc định (ưu tiên gói VIP x điểm chất lượng x hệ số suy giảm thời gian đăng tin), Sắp xếp theo giá tăng/giảm dần, Diện tích tăng/giảm dần, Ngày đăng mới nhất - cũ nhất.
 - Nếu xử lý tất cả các biến thể sắp xếp bằng một câu lệnh switch-case đơn khối trong Repository, bất kỳ thay đổi nhỏ nào ở một biến thể sắp xếp cũng đòi hỏi phải chạy regression trên tất cả các biến thể khác, vi phạm nghiêm trọng nguyên tắc Open/Closed (OCP).

- **Sơ đồ class**



Thành phần	Lớp cụ thể trong dự án	Ghi chú
Context	Impl\ListingServiceImpl	Service gọi Factory để nhận Strategy, sau đó chuyển cho Repository.

Strategy(Interface)	ListingSortingStrategy	Định nghĩa phương thức apply(Builder \$query): Builder.
Concrete Strategy A	Sorting\Strategies\DefaultPackageScoreSortingStrategy	Sắp xếp mặc định theo công thức điểm ưu tiên gói tin x điểm chất lượng x hệ số suy giảm thời gian.
Concrete Strategy B	Listing\Sorting\Strategies\NewestListingSortingStrategy	Sắp xếp theo ngày đăng giảm dần (mới nhất đầu).
Concrete Strategy C	Listing\Sorting\Strategies\OldestListingSortingStrategy	Sắp xếp theo ngày đăng tăng dần (cũ nhất đầu)
Concrete Strategy D	Listing\Sorting\Strategies\PriceLowToHighSortingStrategy	Sắp xếp theo giá tăng dần
Concrete Strategy E	Listing\Sorting\Strategies\PriceHighToLowSortingStrategy	Sắp xếp theo giá giảm dần
Concrete Strategy F	Listing\Sorting\Strategies\AreaLowToHighSortingStrategy	Sắp xếp theo diện tích tăng dần.
Concrete Strategy G	Listing\Sorting\Strategies\AreaHighToLowSortingStrategy	Sắp xếp theo diện tích giảm dần

- Code triển khai

ListingSortingStrategy

```
interface ListingSortingStrategy buith, 5/27/2026
{
    7 implementations  🐞 buith
    public function apply(Builder $query): Builder;
}
```

DefaultPackageScoreSortingStrategy

```
final class DefaultPackageScoreSortingStrategy implements ListingSortingStrategy buith, 5/27/2026 11:09 PM • feat: implement fl
{
    🐞 buith
    public function apply(Builder $query): Builder
    {
        $driver = DB::connection()->getDriverName();
        $hoursSincePublished = $driver === 'sqlite'
            ? "((julianday('now') - julianday(COALESCE(listings.published_at, listings.created_at))) * 24.0)"
            : 'TIMESTAMPDIFF(HOUR, COALESCE(listings.published_at, listings.created_at), NOW())';

        $finalScoreFormula = $driver === 'sqlite'
            ? "(COALESCE(listings.score, 0) * COALESCE(packages.multiplier, 1.0) * (1.0 / (1.0 + {$hoursSincePublished} / 24.0)))"
            : "(COALESCE(listings.score, 0) * COALESCE(packages.multiplier, 1.0) * (1.0 / (1.0 + {$hoursSincePublished} / 24.0)))";

        return $query
            ->selectRaw( expression: "
                COALESCE(packages.priority, 1) AS pkg_priority,
                {$finalScoreFormula} AS final_score
            ")
            ->leftJoin( table: 'packages', first: 'listings.package_id', operator: '=', second: 'packages.id')
            ->orderByDesc( column: 'pkg_priority')
            ->orderByDesc( column: 'final_score');
    }
}
```

3.2.3.5. Tìm kiếm

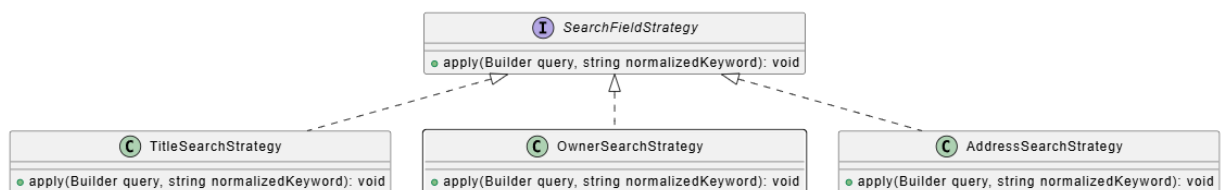
- Chức năng đã sử dụng các Design Patterns: Strategy Pattern

a. Strategy Pattern

- **Vấn đề cần giải quyết:**

- Người dùng cần có khả năng tìm kiếm tin đăng qua các trường dữ liệu khác nhau: theo tiêu đề tin, theo tên người đăng, và theo địa chỉ bất động sản. Mỗi loại tìm kiếm có cách thức truy vấn hoàn toàn khác nhau (một số trường tìm liên kết, một số khác tìm trong cùng bảng).
- Nếu nhồi nhét tất cả các logic tìm kiếm này vào một hàm duy nhất, code sẽ phát triển thành một khối if-else đồ sộ và việc thêm một trường tìm kiếm mới đồng nghĩa với việc phải can thiệp sâu vào các lớp Repository chính.

- **Sơ đồ**



Thành phần	Lớp cụ thể trong dự án	Ghi chú
Context	Impl\ListingServiceImpl	Service gọi Factory để nhận Strategy, sau đó chuyển cho Repository.
Strategy(Interface)	Search\SearchFieldStrategy	Định nghĩa apply(Builder \$query, string \$normalizedKeyword).
Concrete Strategy A	Filter\Search\TitleSearchStrategy	Tìm theo tiêu đề tin đăng, có mở rộng loại nhu cầu nếu từ khoá chứa "cho thuê"/"mua bán".
Concrete Strategy B	Filter\Search\OwnerSearchStrategy	Tìm theo tên người đăng tin (owner).
Concrete Strategy C	Filter\Search\AddressSearchStrategy	Tìm theo địa chỉ bất động sản.

- Code triển khai

SearchFieldStrategy

```
interface SearchFieldStrategy  NguyenHoangViet1501, 6/20/2026 10:34 PM • refac
{
    3 implementations  🐞 NguyenHoangViet1501
    public function apply(Builder $query, string $normalizedKeyword): void;
}
```

OwnerSearchStrategy

```
final class OwnerSearchStrategy implements SearchFieldStrategy  NguyenHoangViet1501, 6/20/2026 10:3
{
    🐞 NguyenHoangViet1501
    public function apply(Builder $query, string $normalizedKeyword): void
    {
        $like = '%'.$normalizedKeyword.'%';

        $query->whereHas( relation: 'owner', function ($ownerQuery) use ($like) {
            $ownerQuery
                ->whereRaw(ListingSearchExpression::column( column: 'full_name').' LIKE ?', [$like])
                ->orWhereRaw(ListingSearchExpression::column( column: 'email').' LIKE ?', [$like])
                ->orWhere('phone', 'like', $like);
        });
    }
}
```

AddressSearchStrategy

```

final class AddressSearchStrategy implements SearchFieldStrategy NguyenHoangViet1501, 6/20/2026 10:34 PM
{
    NguyenHoangViet1501
    public function apply(Builder $query, string $normalizedKeyword): void
    {
        $like = '%'.$normalizedKeyword.'%';

        $query->whereHas( relation: 'property', function ($propertyQuery) use ($like) {
            $propertyQuery
                ->whereRaw(ListingSearchExpression::column( column: 'address_detail').' LIKE ?', [$like])
                ->orWhereRaw(ListingSearchExpression::column( column: 'project_name').' LIKE ?', [$like])
                ->orWhereRaw(ListingSearchExpression::column( column: 'street_code').' LIKE ?', [$like])
                ->orWhereRaw(ListingSearchExpression::column( column: 'province_code').' LIKE ?', [$like])
                ->orWhereRaw(ListingSearchExpression::column( column: 'ward_code').' LIKE ?', [$like])
                ->orWhereRaw(ListingSearchExpression::column( column: 'province').' LIKE ?', [$like])
                ->orWhereRaw(ListingSearchExpression::column( column: 'ward').' LIKE ?', [$like]);
        });
    }
}

```

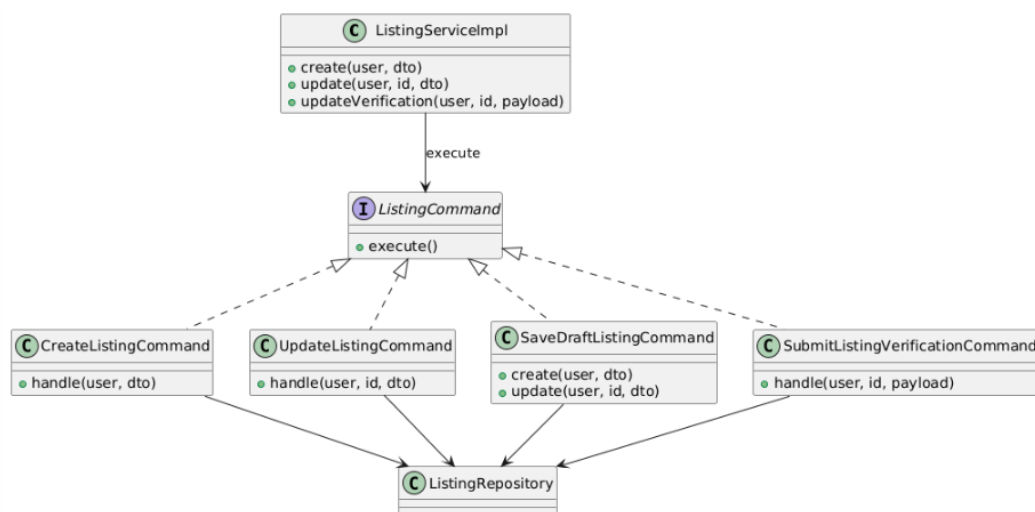
3.2.3.6. Tạo tin đăng

- **Chức năng đã sử dụng các Design Patterns:** Command Pattern, Chain of Responsibility Pattern, State Pattern, Observer Pattern, Adapter Pattern

a. Command Pattern

- **Vấn đề cần giải quyết:**
 - Chức năng đăng tin có nhiều hành động nghiệp vụ: tạo tin, cập nhật tin, lưu nháp, gửi/cập nhật xác thực. Mỗi hành động đều gồm nhiều bước như validate, lưu property, lưu listing, lưu media, tính điểm, xử lý trạng thái và phát event.
 - Nếu toàn bộ logic nằm trong ListingServiceImpl, service sẽ phình to và khó mở rộng. Command Pattern đóng gói từng hành động thành một command object riêng. Invoker chỉ gọi command, không cần biết chi tiết xử lý bên trong.

- Sơ đồ



Thành phần	Lớp cụ thể trong dự án	Ghi chú
------------	------------------------	---------

Invoker	App\Services\Listing\impl ListingServiceImpl	Đóng vai trò invoker, chọn command phù hợp với thao tác người dùng
Command(Interface)	(Không dùng interface riêng do tận dụng DI & class-based command của Laravel)	Laravel Service/Command thường viết trực tiếp dạng Concrete Class
ConcreteCommand	CreateListingCommand, UpdateListingCommand, SaveDraftListingCommand, SubmitListingVerificationCommand	Mỗi class đóng gói một hành động nghiệp vụ
Receiver	ListingRepository	Cung cấp thao tác lưu trữ và các service phụ trợ để command thực thi nghiệp vụ

- **Code triển khai**

```

public function __construct(
    private readonly ListingRepository $listingRepository,
    private readonly CreateListingCommand $createListingCommand,
    private readonly UpdateListingCommand $updateListingCommand,
    private readonly SaveDraftListingCommand $saveDraftListingCommand,
    private readonly SubmitListingVerificationCommand $submitListingVerificationCommand,
    private readonly UnlistListingCommand $unlistListingCommand,
    private readonly ListingStatusStateFactory $statusStateFactory,
    private readonly UpgradeEligibilityPolicy $upgradeEligibilityPolicy,
    private readonly UpgradeListingCommand $upgradeListingCommand,
    private readonly CreateUpgradePaymentCommand $createUpgradePaymentCommand,
    private readonly ListingVerificationService $listingVerificationService,
    private readonly ApproveListingCommand $approveListingCommand,
    private readonly RejectListingCommand $rejectListingCommand,
    private readonly LockListingCommand $lockListingCommand,
) {}

```

```

public function create(User $user, CreateListingDto $dto): Listing
{
    if ($dto->saveAsDraft) {
        return $this->saveDraftListingCommand->create($user, $dto);
    }

    return $this->createListingCommand->handle($user, $dto);
}

public function update(User $user, int $id, CreateListingDto $dto): Listing
{
    if ($dto->saveAsDraft) {
        return $this->saveDraftListingCommand->update($user, $id, $dto);
    }

    return $this->updateListingCommand->handle($user, $id, $dto);
}

public function updateVerification(User $user, int $id, array $payload): Listing
{
    $updated = $this->listingVerificationService->requestVerification($user, $id, $payload);

    $this->clearPublicListingsCache();

    return $updated;
}

```

final class CreateListingCommand

2 references

```
public function handle(User $user, CreateListingDto $dto): Listing
{
    Log::debug('[CreateListingCommand] handle() called', [
        'user_id' => $user->id,
        'demand_type' => $dto->demandType,
        'title' => $dto->title,
        'images_count' => count($dto->images),
        'has_video' => $dto->video !== null,
        'request_verification' => $dto->requestVerification,
    ]);

    $this->validationPipeline->validate(new ListingSubmissionValidationContext($user, $dto));

    $listing = DB::transaction(function () use ($user, $dto) {
        $property = $this->listingRepository->createProperty($this->propertyAttributes($dto));

        if ($dto->attributeIds !== []) {
            $property->attributes()->sync($dto->attributeIds);
        }

        $listingStatus = $this->statusStateFactory->initialForSave($dto->saveAsDraft)->value();

        $listing = $this->listingRepository->createListing([
            'property_id' => $property->id,
            'owner_id' => $user->id,
            'demand_type' => $dto->demandType,
            'title' => $dto->title,
            'description' => $dto->description,
            'status' => $listingStatus,
            'package_id' => $dto->packageId,
            'score' => $this->calculateContentScore($dto),
            'is_verified' => ListingVerificationStatusResolver::forSubmission(
```

```

final class SubmitListingVerificationCommand
0 references
public function handle(User $user, int $id, array $payload): Listing
{
    $updated = DB::transaction(function () use ($user, $id, $payload) {
        $listing = Listing::query()->lockForUpdate()->findOrFail($id);

        if ((int) $listing->owner_id !== (int) $user->id) {
            throw new BusinessException(ErrorCode::AuthForbidden);
        }

        if ($listing->demand_type === 'RENT') {
            throw new BusinessException(ErrorCode::BadRequest, 'Tin cho thuê không hỗ trợ xác thực bất động sản.');
        }

        $identityCardFront = $payload['identity_card_front'] ?? null;
        $identityCardBack = $payload['identity_card_back'] ?? null;
        $legalDocuments = $payload['legal_documents'] ?? [];
        if (! ListingVerificationStatusResolver::hasCompleteDocuments($identityCardFront, $identityCardBack, $legalDocuments)) {
            throw new BusinessException(ErrorCode::ValidationError, 'Can tải lên đầy đủ CCCD mặt trước, mặt sau và i
        }

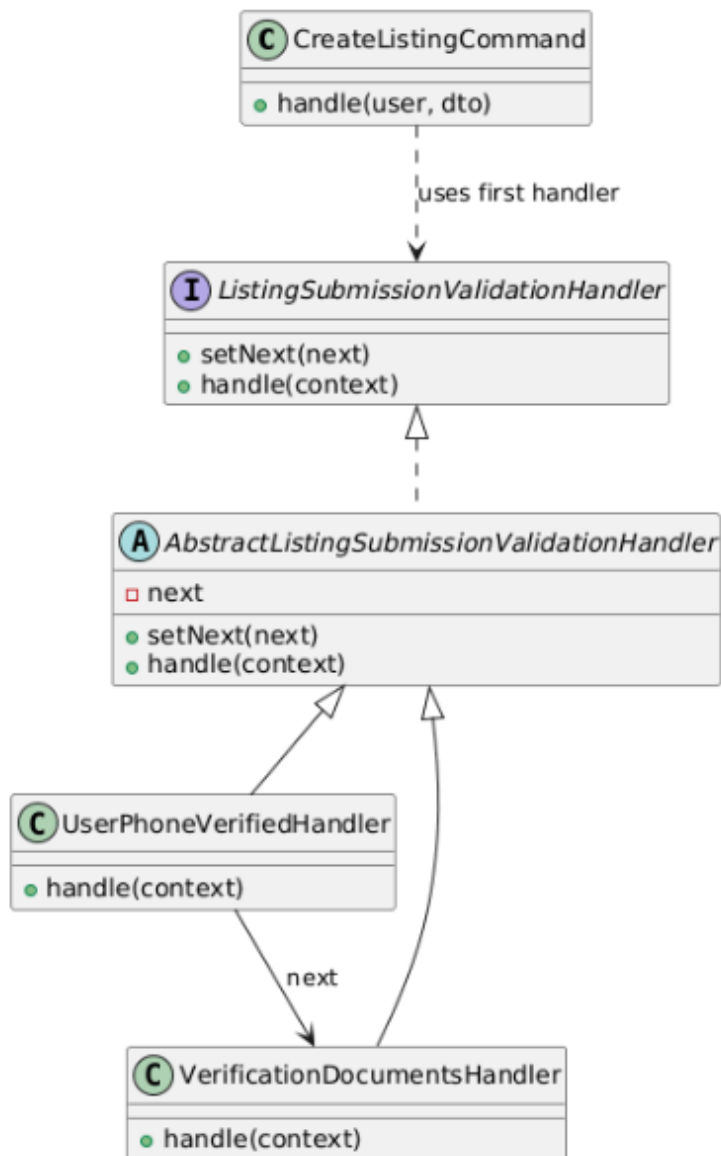
        $requestVerification = true;

        $this->listingRepository->updateListing($listing->id, [
            'request_verification' => $requestVerification,
            'is_verified' => ListingVerificationStatusResolver::forSubmission(
                $listing->demand_type,
                $identityCardFront,
                $identityCardBack,
                $legalDocuments,
                $listing->is_verified,
            )->value,
        ]);
    });
}

```

b. Chain of Responsibility Pattern

- Vấn đề cần giải thích:
 - Trước khi lưu một tin đăng, hệ thống cần kiểm tra nhiều điều kiện nghiệp vụ theo thứ tự. Ví dụ: người dùng phải có số điện thoại, nếu có yêu cầu xác thực BĐS thì phải tải đủ CCCD mặt trước, CCCD mặt sau và ít nhất một giấy tờ pháp lý. Các rule này có thể tăng thêm trong tương lai như kiểm tra quota đăng tin, giới hạn số ảnh hoặc kiểm tra tài khoản bị khóa.
 - Nếu tất cả rule được viết trong một hàm validate() lớn, mỗi lần thêm rule mới phải sửa trực tiếp hàm cũ, làm tăng rủi ro regression. Chain of Responsibility giúp tách mỗi rule thành một handler riêng và nối chúng thành chuỗi xử lý.
- Sơ đồ



Thành phần	Lớp cụ thể trong dự án	Ghi chú
Handler Interface	ListingSubmissionValidationHandler	Khai báo <code>setNext()</code> và <code>handle()</code>
Base Handler	AbstractListingSubmissionValidationHandler	Cung cấp logic chung: chạy <code>validate(\$context)</code> của handler hiện tại rồi chuyển tiếp sang <code>\$next</code>
ConcreteHandlerA	UserPhoneVerifiedHandler	Nếu người dùng chưa có số điện thoại, ném <code>BusinessException(ErrorCode::AuthPhoneNotVerified)</code>
ConcreteHandlerB	VerificationDocumentsHandler	Nếu người dùng bật <code>requestVerification</code> nhưng thiếu giấy tờ, ném lỗi <code>validation</code> nghiệp vụ

Pipeline	ListingSubmissionValidationPipeline	Lắp chuỗi UserPhoneVerifiedHandler -> VerificationDocumentsHandler và gọi handler đầu tiên
Context	ListingSubmissionValidationContext	Mang theo User và CreateListingDto qua toàn bộ chain

- Code triển khai

CreateListingCommand

```
public function handle(User $user, CreateListingDto $dto): Listing
{
    Log::debug('[CreateListingCommand] handle() called', [
        'user_id' => $user->id,
        'demand_type' => $dto->demandType,
        'title' => $dto->title,
        'images_count' => count($dto->images),
        'has_video' => $dto->video !== null,
        'request_verification' => $dto->requestVerification,
    ]);

    $this->validationPipeline->validate(new ListingSubmissionValidationContext($user, $dto));
}
```

ListingSubmissionValidationHandler

```
interface ListingSubmissionValidationHandler
{
    1 reference | 1 override
    public function setNext(?ListingSubmissionValidationHandler $next): ListingSubmissionValidationHandler;

    2 references | 1 override
    public function handle(ListingSubmissionValidationContext $context): void;
}
```

AbstractListingSubmissionValidationHandler

```

abstract class AbstractListingSubmissionValidationHandler implements ListingSubmissionValidationHandler
{
    2 references
    private ?ListingSubmissionValidationHandler $next = null;

    1 reference | 0 overrides
    public function setNext(?ListingSubmissionValidationHandler $next): ListingSubmissionValidationHandler
    {
        $this->next = $next;

        return $next ?? $this;
    }

    2 references | 0 overrides
    public function handle(ListingSubmissionValidationContext $context): void
    {
        $this->validate($context);
        $this->next?->handle($context);
    }

    1 reference | 2 overrides
    abstract protected function validate(ListingSubmissionValidationContext $context): void;
}

```

UserPhoneVerifiedHandler

```

final class UserPhoneVerifiedHandler extends AbstractListingSubmissionValidationHandler
{
    1 reference | prototype
    protected function validate(ListingSubmissionValidationContext $context): void
    {
        if (! $context->user->phone || trim((string) $context->user->phone) === '') {
            throw new BusinessException(ErrorCode::AuthPhoneNotVerified);
        }
    }
}

```

VerificationDocumentsHandler

```

final class VerificationDocumentsHandler extends AbstractListingSubmissionValidationHandler
{
    1 reference | prototype
    protected function validate(ListingSubmissionValidationContext $context): void
    {
        if (! $context->dto->requestVerification) {
            return;
        }

        if (! ListingVerificationStatusResolver::hasCompleteDocuments(
            $context->dto->identityCardFront,
            $context->dto->identityCardBack,
            $context->dto->legalDocuments,
        )) {
            throw new BusinessException(ErrorCode::ValidationError, 'Can tai len day du CCCD mat truoc, mat sau va it nhat mot anh giay');
        }
    }
}

```

ListingSubmissionValidationPipeline

```

final class ListingSubmissionValidationPipeline
{
    0 references
    public function __construct(
        private readonly UserPhoneVerifiedHandler $userPhoneVerifiedHandler,
        private readonly VerificationDocumentsHandler $verificationDocumentsHandler,
    ) {}

    1 reference
    public function validate(ListingSubmissionValidationContext $context): void
    {
        $this->userPhoneVerifiedHandler
            ->setNext($this->verificationDocumentsHandler);

        $this->userPhoneVerifiedHandler->handle($context);
    }
}

```

ListingSubmissionValidationContext

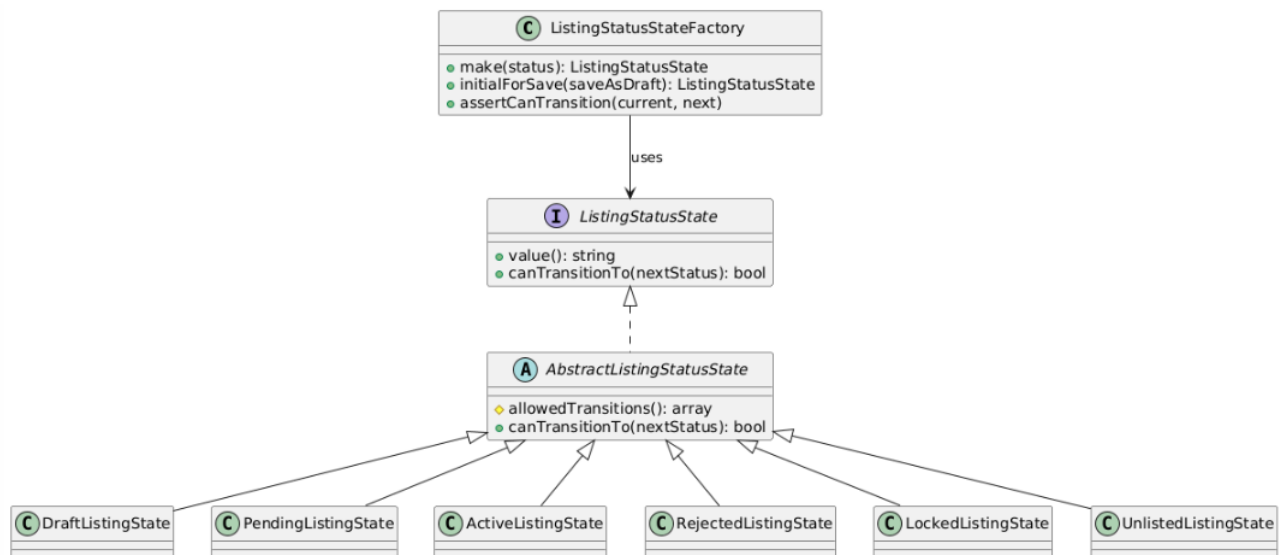
```

final readonly class ListingSubmissionValidationContext
{
    1 reference
    public function __construct(
        public User $user,
        public CreateListingDto $dto,
    ) {}
}

```

c. State Pattern

- Vấn đề cần giải quyết:
 - Tin đăng có nhiều trạng thái: DRAFT, PENDING, ACTIVE, REJECTED, LOCKED, UNLISTED. Mỗi trạng thái chỉ cho phép chuyển sang một số trạng thái hợp lệ.
 - Nếu xử lý bằng nhiều if/else hoặc switch, logic trạng thái dễ bị rối rắm và khó kiểm soát → State Pattern tách từng trạng thái thành class riêng. Mỗi state tự biết giá trị của mình và những trạng thái tiếp theo được phép chuyển.
- Sơ đồ class



Thành phần	Lớp cụ thể trong dự án	Ghi chú
Context	ListingStatusStateFactory	Tạo state tương ứng, xác định trạng thái ban đầu và kiểm tra chuyển trạng thái hợp lệ
State Interface	ListingStatusState	Interface chung cho các trạng thái
Base State	AbstractListingStatusState	Cung cấp logic kiểm tra transition chung
ConcreteState	DraftListingState, PendingListingState, ActiveListingState, RejectedListingState, LockedListingState, UnlistedListingState	Tự định nghĩa value() và danh sách trạng thái được phép chuyển

- Code triển khai

ListingStatusState

```

interface ListingStatusState
{
    2 references | 6 overrides
    public function value(): string;

    9 references | 1 override
    public function canTransitionTo(string $nextStatus): bool;
}
  
```

ListingStatusStateFactory

```
final class ListingStatusStateFactory
{
    10 references
    public function make(string $status): ListingStatusState
    {
        return match ($status) {
            'DRAFT' => new DraftListingState,
            'PENDING' => new PendingListingState,
            'ACTIVE' => new ActiveListingState,
            'REJECTED' => new RejectedListingState,
            'LOCKED' => new LockedListingState,
            'UNLISTED' => new UnlistedListingState,
            default => throw new BusinessException(ErrorCode::BadRequest, "Trang thai listing {$status} không hợp lệ."),
        };
    }

    2 references
    public function initialForSave(bool $saveAsDraft): ListingStatusState
    {
        return $saveAsDraft ? new DraftListingState : new PendingListingState;
    }

    1 reference
    public function assertCanTransition(string $currentStatus, string $nextStatus): void
    {
        if ($currentStatus === $nextStatus) {
            throw new BusinessException(ErrorCode::BadRequest, "Listing đã ở trạng thái {$nextStatus}.");
        }

        $currentState = $this->make($currentStatus);

        if (!$currentState->canTransitionTo($nextStatus)) {
            throw new BusinessException(ErrorCode::BadRequest, "Trang thái hiện tại của listing không hỗ trợ chuyển sang trạng thái {$nextStatus}.");
        }
    }
}
```

AbstractListingStatusState

```
abstract class AbstractListingStatusState implements ListingStatusState
{
    /**
     * @return string[]
     */
    1 reference | 6 overrides
    abstract protected function allowedTransitions(): array;

    9 references | 0 overrides
    public function canTransitionTo(string $nextStatus): bool
    {
        return in_array($nextStatus, $this->allowedTransitions(), true);
    }
}
```

DraftListingState

```
final class DraftListingState extends AbstractListingStatusState
{
    2 references
    public function value(): string
    {
        return 'DRAFT';
    }

    1 reference | prototype
    protected function allowedTransitions(): array
    {
        return ['PENDING'];
    }
}
```

PendingListingState

```
final class PendingListingState extends AbstractListingStatusState
{
    2 references
    public function value(): string
    {
        return 'PENDING';
    }

    1 reference | prototype
    protected function allowedTransitions(): array
    {
        return ['ACTIVE', 'REJECTED'];
    }
}
```

ActiveListingState

```
final class ActiveListingState extends AbstractListingStatusState
{
    2 references
    public function value(): string
    {
        return 'ACTIVE';
    }

    1 reference | prototype
    protected function allowedTransitions(): array
    {
        return ['LOCKED', 'REJECTED', 'UNLISTED'];
    }
}
```

RejectedListingState

```
final class RejectedListingState extends AbstractListingStatusState
{
    2 references
    public function value(): string
    {
        return 'REJECTED';
    }

    1 reference | prototype
    protected function allowedTransitions(): array
    {
        return ['ACTIVE', 'PENDING'];
    }
}
```

LockedListingState

```
final class LockedListingState extends AbstractListingStatusState
{
    2 references
    public function value(): string
    {
        return 'LOCKED';
    }

    1 reference | prototype
    protected function allowedTransitions(): array
    {
        return ['ACTIVE'];
    }
}
```

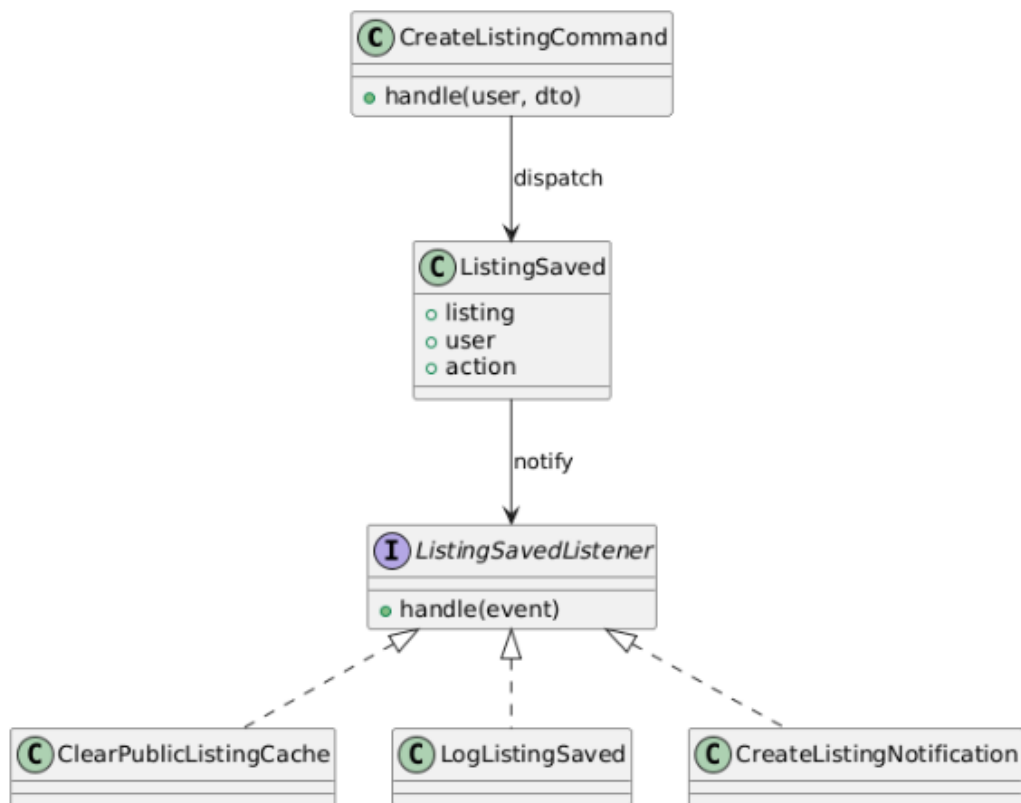
UnlistedListingState

```
final class UnlistedListingState extends AbstractListingStatusState
{
    2 references
    public function value(): string
    {
        return 'UNLISTED';
    }

    1 reference | prototype
    protected function allowedTransitions(): array
    {
        return [];
    }
}
```

d. Observer Pattern

- Vấn đề cần giải quyết:
 - Sau khi tạo hoặc cập nhật tin, hệ thống cần xử lý nhiều side effect như xóa cache, ghi log và tạo notification. Nếu command gọi trực tiếp tất cả xử lý phụ này, command sẽ biết quá nhiều observer cụ thể và vi phạm SRP/OCP → Observer Pattern được áp dụng theo hướng event-driven: command chỉ phát event, listener nào quan tâm thì tự xử lý.
- Sơ đồ class



Thành phần	Lớp cụ thể trong dự án	Ghi chú
Subject	App\Events\Listing\ListingSaved	Mang dữ liệu listing, user và action
Publisher	CreateListingCommand, UpdateListingCommand, SubmitListingVerificationCommand	Phát event sau khi lưu dữ liệu
Observer Interface	Laravel listener contract qua method handle()	Project không cần interface riêng, listener được Laravel gọi bằng handle()
ConcreteObserver	ClearPublicListingCache, LogListingSaved, CreateListingNotification	Các listener xử lý side effect độc lập

- Code triển khai

CreateListingCommand

```
final class CreateListingCommand
{
    public function handle(User $user, CreateListingDto $dto): Listing
    {
        'appointment_contact_name' => $dto->appointmentContactName,
        'appointment_contact_phone' => $dto->appointmentContactPhone,
        'appointment_contact_email' => $dto->appointmentContactEmail,
        'appointment_note' => $dto->appointmentNote,
        'submitted_at' => $dto->saveAsDraft ? null : now(),
    ];

    $this->saveImages($listing, $dto->images);
    $this->saveVideo($listing, $dto->video);
    $this->saveVerificationDocuments($listing, $dto);

    return $listing->load([
        'property.attributes.group',
        'images',
        'videos',
        'verificationDocuments',
        'appointments',
    ]);
    });

    ListingSaved::dispatch($listing, $user, $dto->saveAsDraft ? 'draft_created' : 'created');

    return $listing;
}
```

ListingSaved

```
final class ListingSaved
{
    use Dispatchable, SerializesModels;

    0 references
    public function __construct(
        public readonly Listing $listing,
        public readonly ?User $user,
        public readonly string $action,
    ) {}
}
```

```
final class ClearPublicListingCache
{
    0 references
    public function handle(ListingPackageUpgraded|ListingSaved $event): void
    {
        Cache::tags(['listings:public'])->flush();
    }
}
```

CreateListingNotification

```
final class CreateListingNotification
{
    0 references
    public function handle(ListingSaved $event): void
    {
        $listing = $event->listing->loadMissing('owner');
        $ownerId = (int) $listing->owner_id;

        if ($ownerId <= 0) {
            return;
        }

        $payload = match ($event->action) {
            'updated' => $this->updatedPendingPayload($listing),
            'unlisted' => $this->statusPayload($listing, 'UNLISTED'),
            'admin_status_changed' => $this->statusPayload($listing, (string) $listing->status),
            default => null,
        };

        if ($payload === null) {
            return;
        }

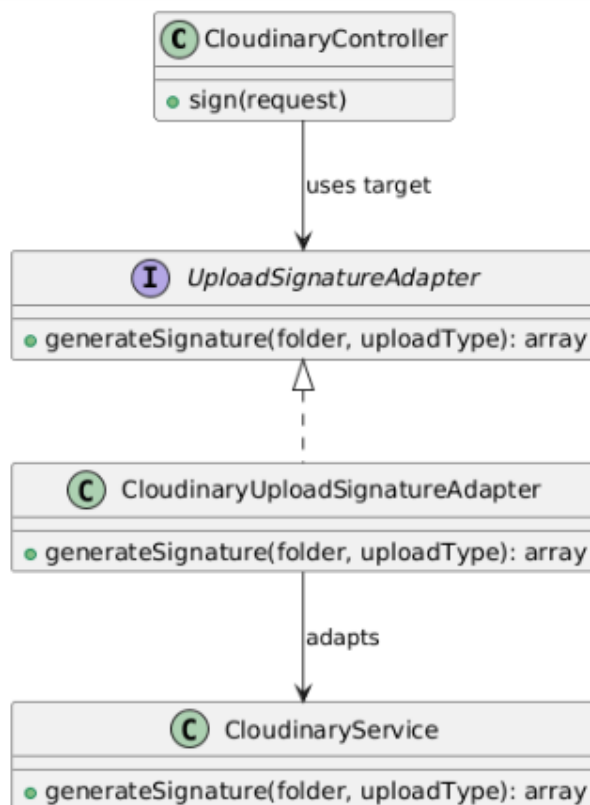
        $this->notificationService->create(
            userId: $ownerId,
            type: $payload['type'],
            title: $payload['title'],
            message: $payload['message'],
            data: $payload['data'],
        );
    }
}
```

LogListingSaved

```
final class LogListingSaved
{
    0 references
    public function handle(ListingSaved $event): void
    {
        Log::info('[Listing] Listing saved', [
            'listing_id' => $event->listing->id,
            'user_id' => $event->user->id,
            'action' => $event->action,
            'status' => $event->listing->status,
        ]);
    }
}
```

e. Adapter Pattern

- Vấn đề cần giải quyết:
 - Chức năng đăng tin cần upload ảnh, video và giấy tờ lên Cloudinary.
 - Nếu controller phụ thuộc trực tiếp vào CloudinaryService, hệ thống sẽ bị gắn chặt với một provider. Khi đổi provider upload, nhiều nơi phải sửa code. → Adapter Pattern tạo một interface chung cho việc sinh upload signature, còn adapter cụ thể bọc Cloudinary.
- Sơ đồ class



Thành phần	Lớp cụ thể trong dự án	Ghi chú
Client	CloudinaryController	Gọi UploadSignatureAdapter, không phụ thuộc trực tiếp Cloudinary
Target Interface	UploadSignatureAdapter	Interface mà client phụ thuộc vào
Adapter	CloudinaryUploadSignatureAdapter	Bọc CloudinaryService và chuyển lời gọi
Adaptee	CloudinaryService	Xử lý chi tiết tạo chữ ký upload Cloudinary

- Code triển khai

```

namespace App\Http\Controllers\Api\Cloudinary;
final class CloudinaryController
{
    /**
     * 1 reference
     * @param Request $request
     * @param string $request_query_param_type = 'avatar' | 'listing'
     */
    public function sign(Request $request): JsonResponse
    {
        $request->validate([
            'type' => ['required', 'string', Rule::in(ListingPostingOptions::values('upload_types'))],
        ]);

        $uploadType = $request->query('type', 'avatar');

        /** @var User $user */
        $user = $this->authFactory->guard('api')->user();
        $folder = match ($uploadType) {
            'avatar' => 'propify/avatars',
            'listing' => 'propify/listings',
            default => 'propify/avatars',
        };

        $signatureData = $this->uploadSignatureAdapter->generateSignature($folder, $uploadType);

        return ApiResponse::success(
            data: $signatureData,
            message: 'Signature generated successfully.'
        );
    }
}

interface UploadSignatureAdapter
{
    /**
     * 1 reference | 1 override
     */
    public function generateSignature(string $folder, string $uploadType): array;
}

```

CloudinaryUploadSignatureAdapter

```

final class CloudinaryUploadSignatureAdapter implements UploadSignatureAdapter
{
    0 references
    public function __construct(
        private readonly CloudinaryService $cloudinaryService,
    ) {}

    1 reference
    public function generateSignature(string $folder, string $uploadType): array
    {
        return $this->cloudinaryService->generateSignature($folder, $uploadType);
    }
}

```

```

final class CloudinaryServiceImpl implements CloudinaryService
{
    1 reference
    public function generateSignature(string $folder, string $uploadType): array
    {
        $timestamp = time();

        $paramsToSign = [
            'folder' => $folder,
            'timestamp' => $timestamp,
        ];
        ksort($paramsToSign);

        $stringToSign = collect($paramsToSign)
            ->map(fn ($value, $key) => "{$key}={$value}")
            ->join('&');

        $signature = sha1($stringToSign.$this->apiSecret);

        Log::info('Cloudinary signature generated', [
            'folder' => $folder,
            'upload_type' => $uploadType,
            'timestamp' => $timestamp,
        ]);

        return [
            'signature' => $signature,
            'api_key' => $this->apiKey,
            'cloud_name' => $this->cloudName,
            'timestamp' => $timestamp,
            'folder' => $folder,
            'resource_type' => 'auto',
        ];
    }
}

```

3.2.3.7. Admin duyệt, từ chối, khóa tin đăng

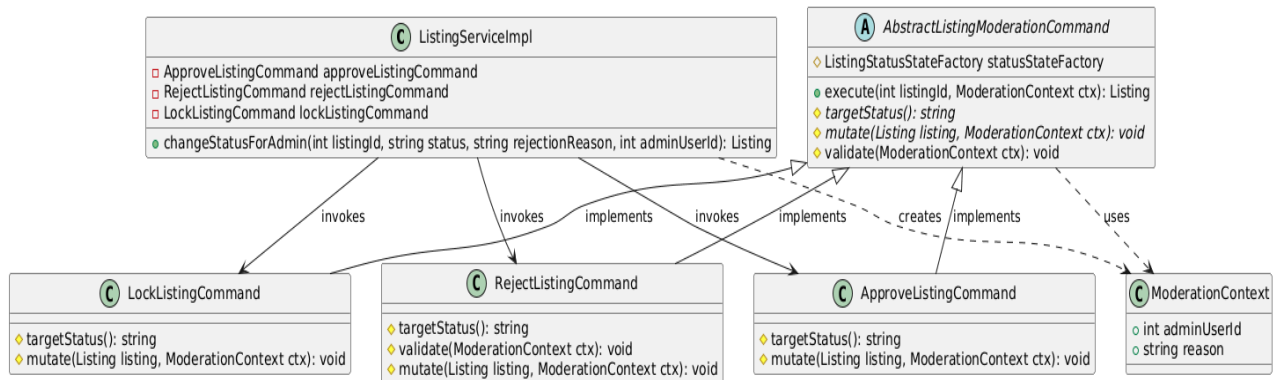
- **Chức năng đã sử dụng các Design Patterns:** Command Pattern, State Pattern, Observer Pattern

a. Command Pattern

- **Vấn đề cần giải quyết:**

- Một hàm `changeStatus` xử lý cả duyệt/từ chối/khóa với nhiều if theo status; phần khung lặp lại (mở giao dịch, ghi lịch sử, phát sự kiện) bị trộn lẫn và rối. → Mỗi thao tác là một Command, dùng chung một lớp base chứa khung

- Sơ đồ class



Thành phần	Lớp cụ thể trong dự án	Tác dụng
Invoker	App\Services\Listing\impl\ListingServiceImpl (changeStatusForAdmin)	Chọn Command theo trạng thái đích.
Command (base)	Listing\Moderation\AbstractListingModerationCommand	Khung execute() chung: kiểm tra chuyển trạng thái → mutate → lịch sử → sự kiện.
ConcreteCommand	Moderation\ApproveListingCommand	Duyệt → ACTIVE.
ConcreteCommand	Moderation\RejectListingCommand	Từ chối → REJECTED (bắt buộc lý do).
ConcreteCommand	Moderation\LockListingCommand	Khóa → LOCKED.
DTO	Moderation\ModerationContext	adminUserId + reason.

- Code triển khai

AbstractListingModerationCommand:

app/Services/Listing/Moderation/AbstractListingModerationCommand.php (1–84)

```

abstract class AbstractListingModerationCommand
{
    /** Quan hệ được nạp lại sau khi đổi trạng thái (giữ nguyên danh sách gốc). */
    private const RELATIONS = [
        'property.attributes.group',
        'images',
        'videos',
        'verificationDocuments',
        'appointmentsSlots',
        'appointments',
        'owner',
        'approver',
        'package',
    ];

    public function __construct(
        protected readonly ListingStatusStateFactory $statusStateFactory,
    ) {}

    final public function execute(int $listingId, ModerationContext $ctx): Listing
    {
        $this->validate($ctx);

        $listing = DB::transaction(function () use ($listingId, $ctx) {
            $listing = Listing::query()->lockForUpdate()->find($listingId);
            if (!$listing) {
                throw new BusinessException(ErrorCode::ListingNotFound);
            }
        });

        $this->statusStateFactory->assertCanTransition($listing->status,
            $this->targetStatus());

        $this->mutate($listing, $ctx);
        $listing->save();

        ListingStatusHistory::create([
            'user_id' => $ctx->adminUserId,
            'listing_id' => $listing->id,
            'action' => $this->targetStatus(),
            'reason' => $ctx->reason,
        ]);

        return $listing;
    });

    $loaded = $listing->load(self::RELATIONS);

    ListingSaved::dispatch($loaded, null, 'admin_status_changed');

    return $loaded;
}

/** Trạng thái đích của thao tác (ACTIVE / REJECTED / LOCKED). */
abstract protected function targetStatus(): string;

/** Phần thay đổi listing khác nhau giữa các lệnh. */
abstract protected function mutate(Listing $listing, ModerationContext $ctx): void;

/** Kiểm tra tiền điều kiện trước transaction (mặc định không có). */
protected function validate(ModerationContext $ctx): void {}
}

```

ApproveListingCommand:

app/Services/Listing/Moderation/ApproveListingCommand.php (1–23)

```
final class ApproveListingCommand extends AbstractListingModerationCommand
{
    protected function targetStatus(): string
    {
        return 'ACTIVE';
    }

    protected function mutate(Listing $listing, ModerationContext $ctx): void
    {
        $listing->status = 'ACTIVE';
        $listing->published_at = now();
        $listing->approved_by = $ctx->adminUserId;
    }
}
```

RejectListingCommand:

app/Services/Listing/Moderation/RejectListingCommand.php (1–31)

```
final class RejectListingCommand extends AbstractListingModerationCommand
{
    protected function targetStatus(): string
    {
        return 'REJECTED';
    }

    protected function validate(ModerationContext $ctx): void
    {
        if (trim((string) $ctx->reason) === '') {
            throw new BusinessException(ErrorCode::BadRequest, 'Vui lòng nhập lý do từ chối.');
```

LockListingCommand:

app/Services/Listing/Moderation/LockListingCommand.php (1–21)

```
final class LockListingCommand extends AbstractListingModerationCommand
{
    protected function targetStatus(): string
    {
        return 'LOCKED';
    }

    protected function mutate(Listing $listing, ModerationContext $ctx): void
    {
        $listing->status = 'LOCKED';
    }
}
```


ModerationContext: app/Services/Listing/Moderation/ModerationContext.php (1–14)

```
final class ModerationContext
{
    public function __construct(
        public readonly int $adminUserId,
        public readonly ?string $reason = null,
    ) {}
}
```

Điều phối: app/Services/Listing/impl/ListingServiceImpl.php (dòng 299–315)

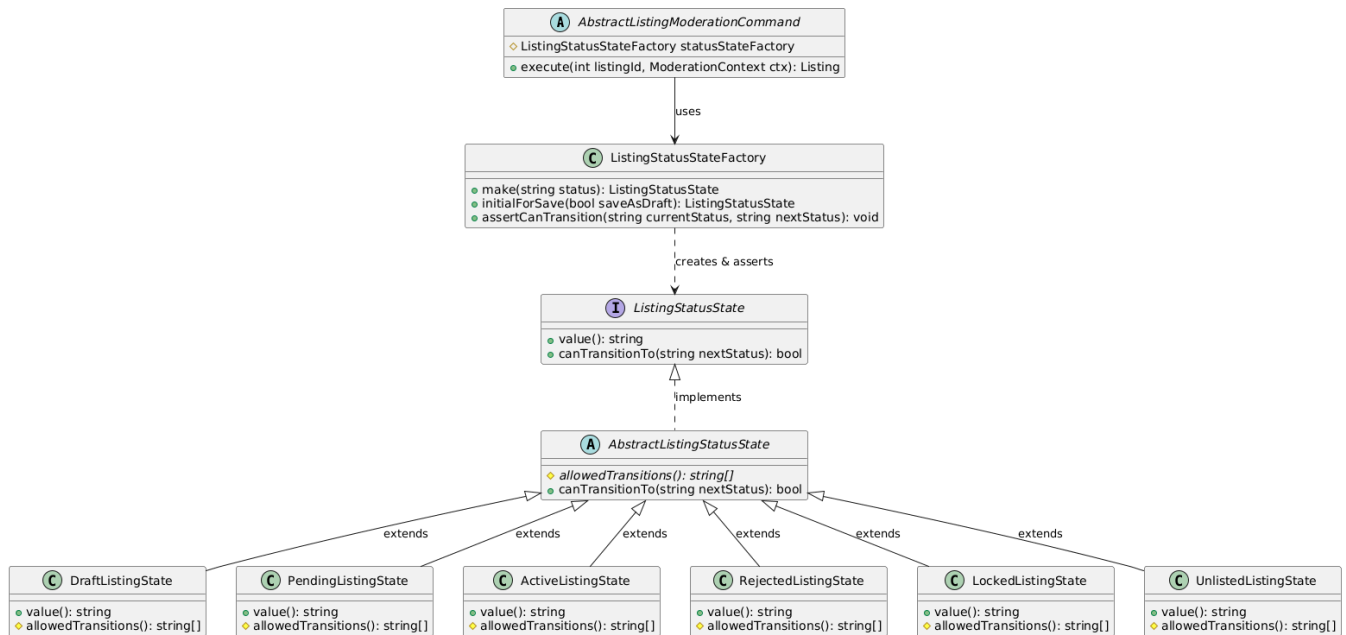
```
public function changeStatusForAdmin(int $listingId, string $status, ?string
$rejectionReason = null, ?int $adminUserId = null): Listing
{
    if ($adminUserId === null) {
        throw new BusinessException(ErrorCode::AuthForbidden);
    }

    // Điều phối theo trạng thái đích (Command). Mỗi lệnh dùng chung khung kiểm duyệt
    // (Template Method) + State::assertCanTransition; nghiệp vụ giữ nguyên như cũ.
    $ctx = new ModerationContext($adminUserId, $rejectionReason);

    return match ($status) {
        'ACTIVE' => $this->approveListingCommand->execute($listingId, $ctx),
        'REJECTED' => $this->rejectListingCommand->execute($listingId, $ctx),
        'LOCKED' => $this->lockListingCommand->execute($listingId, $ctx),
        default => throw new BusinessException(ErrorCode::BadRequest),
    };
}
```

b. State Pattern

- **Vấn đề cần giải quyết:**
 - Tránh chuyển trạng thái tin không hợp lệ (ví dụ PENDING→LOCKED tùy tiện). → State machine khai báo rõ mỗi trạng thái được chuyển sang đâu và chặn bước sai.
- **Sơ đồ class**



Thành phần	Lớp cụ thể trong dự án	Tác dụng
State	Listing\State\ListingStatusState	value()/canTransitionTo(nextStatus).
Factory	State\ListingStatusStateFactory	make(status) + assertCanTransition(from, to).
ConcreteState	State\{Draft,Pending,Active,Rejected,Locked,Unlisted}ListingState	Mỗi trạng thái khai báo chuyển đổi hợp lệ.

- Code triển khai

ListingStatusState: app/Services/Listing/State/ListingStatusState.php (1–10)

```

interface ListingStatusState
{
    public function value(): string;

    public function canTransitionTo(string $nextStatus): bool;
}
  
```

ListingStatusStateFactory:

app/Services/Listing/State/ListingStatusStateFactory.php (1–40)

```

final class ListingStatusStateFactory
{
    public function make(string $status): ListingStatusState
    {
        return match ($status) {
            'DRAFT' => new DraftListingState,
            'PENDING' => new PendingListingState,
            'ACTIVE' => new ActiveListingState,
            'REJECTED' => new RejectedListingState,
            'LOCKED' => new LockedListingState,
            'UNLISTED' => new UnlistedListingState,
            default => throw new BusinessException(ErrorCode::BadRequest, "Trang thai
            listing {$status} không hợp lệ."),
        };
    }

    public function initialForSave(bool $saveAsDraft): ListingStatusState
    {
        return $saveAsDraft ? new DraftListingState : new PendingListingState;
    }

    public function assertCanTransition(string $currentStatus, string $nextStatus): void
    {
        if ($currentStatus === $nextStatus) {
            throw new BusinessException(ErrorCode::BadRequest, "Listing đã ở trạng thái
            {$nextStatus}.");
        }

        $currentState = $this->make($currentStatus);

        if (!$currentState->canTransitionTo($nextStatus)) {
            throw new BusinessException(ErrorCode::BadRequest, "Trang thái hiện tại của
            listing không hỗ trợ chuyển sang trạng thái {$nextStatus}.");
        }
    }
}

```

Các ConcreteState: app/Services/Listing/State/*ListingState.php (mỗi file một trạng thái)

```

final class ActiveListingState extends AbstractListingStatusState
{
    public function value(): string
    {
        return 'ACTIVE';
    }

    protected function allowedTransitions(): array
    {
        return ['LOCKED', 'REJECTED', 'UNLISTED'];
    }
}

```

```

final class DraftListingState extends AbstractListingStatusState
{
    public function value(): string
    {
        return 'DRAFT';
    }

    protected function allowedTransitions(): array
    {
        return ['PENDING'];
    }
}

final class LockedListingState extends AbstractListingStatusState
{
    public function value(): string
    {
        return 'LOCKED';
    }

    protected function allowedTransitions(): array
    {
        return ['ACTIVE'];
    }
}

final class PendingListingState extends AbstractListingStatusState
{
    public function value(): string
    {
        return 'PENDING';
    }

    protected function allowedTransitions(): array
    {
        return ['ACTIVE', 'REJECTED'];
    }
}

```

```

final class RejectedListingState extends AbstractListingStatusState
{
    public function value(): string
    {
        return 'REJECTED';
    }

    protected function allowedTransitions(): array
    {
        return ['ACTIVE', 'PENDING'];
    }
}

final class UnlistedListingState extends AbstractListingStatusState
{
    public function value(): string
    {
        return 'UNLISTED';
    }

    protected function allowedTransitions(): array
    {
        return [];
    }
}

```

Gọi assertCanTransition:

app/Services/Listing/Moderation/AbstractListingModerationCommand.php (trong execute())

```

final public function execute(int $listingId, ModerationContext $ctx): Listing
{
    $this->validate($ctx);

    $listing = DB::transaction(function () use ($listingId, $ctx) {
        $listing = Listing::query()->lockForUpdate()->find($listingId);
        if (! $listing) {
            throw new BusinessException(ErrorCode::ListingNotFound);
        }

        $this->statusStateFactory->assertCanTransition($listing->status,
            $this->targetStatus());

        $this->mutate($listing, $ctx);
        $listing->save();

        ListingStatusHistory::create([
            'user_id' => $ctx->adminUserId,
            'listing_id' => $listing->id,
            'action' => $this->targetStatus(),
            'reason' => $ctx->reason,
        ]);

        return $listing;
    });

    $loaded = $listing->load(self::RELATIONS);

    ListingSaved::dispatch($loaded, null, 'admin_status_changed');

    return $loaded;
}

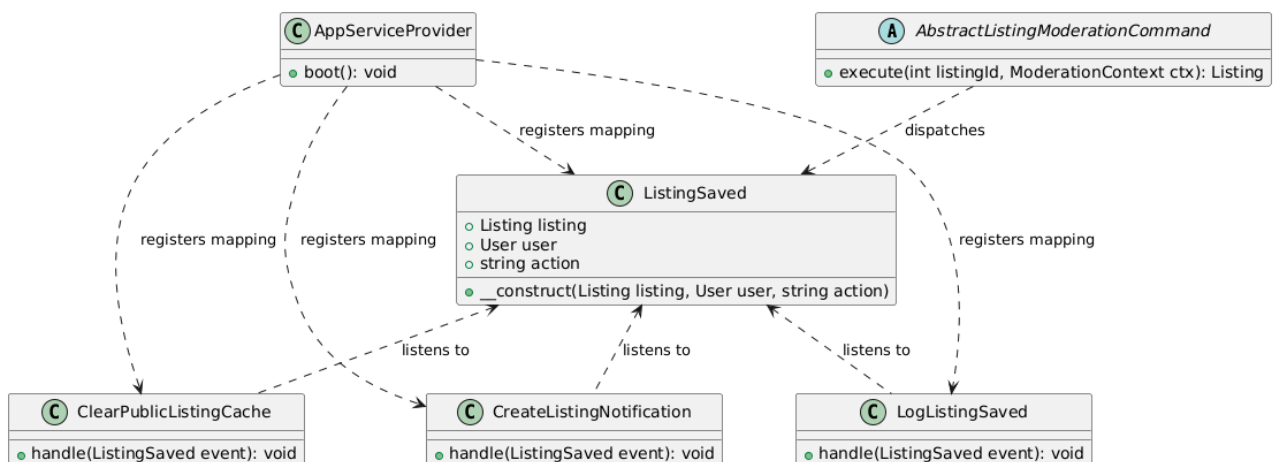
```

c. Observer Pattern

- **Vấn đề cần giải quyết:**

- Sau khi đổi trạng thái tin cần xóa cache công khai, ghi log, tạo thông báo. → phát sự kiện ListingSaved.

- **Sơ đồ class**



Thành phần	Lớp cụ thể trong dự án	Tác dụng
Event	App\Events\Listing\ListingSaved	Sự kiện sau khi lưu/đổi trạng thái tin.
Listener	Listeners\Listing\{ClearPublicListingCache, LogListingSaved, CreateListingNotification }	Xử lý phụ tách rời nghiệp vụ.

- Code triển khai

Phát ListingSaved:

app/Services/Listing/Moderation/AbstractListingModerationCommand.php (trong execute())

```

final public function execute(int $listingId, ModerationContext $ctx): Listing
{
    $this->validate($ctx);

    $listing = DB::transaction(function () use ($listingId, $ctx) {
        $listing = Listing::query()->lockForUpdate()->find($listingId);
        if (! $listing) {
            throw new BusinessException(ErrorCode::ListingNotFound);
        }

        $this->statusStateFactory->assertCanTransition($listing->status,
            $this->targetStatus());

        $this->mutate($listing, $ctx);
        $listing->save();

        ListingStatusHistory::create([
            'user_id' => $ctx->adminUserId,
            'listing_id' => $listing->id,
            'action' => $this->targetStatus(),
            'reason' => $ctx->reason,
        ]);

        return $listing;
    });

    $loaded = $listing->load(self::RELATIONS);

    ListingSaved::dispatch($loaded, null, 'admin_status_changed');

    return $loaded;
}

```

Đăng ký listener: app/Providers/AppServiceProvider.php (dòng 222–224)

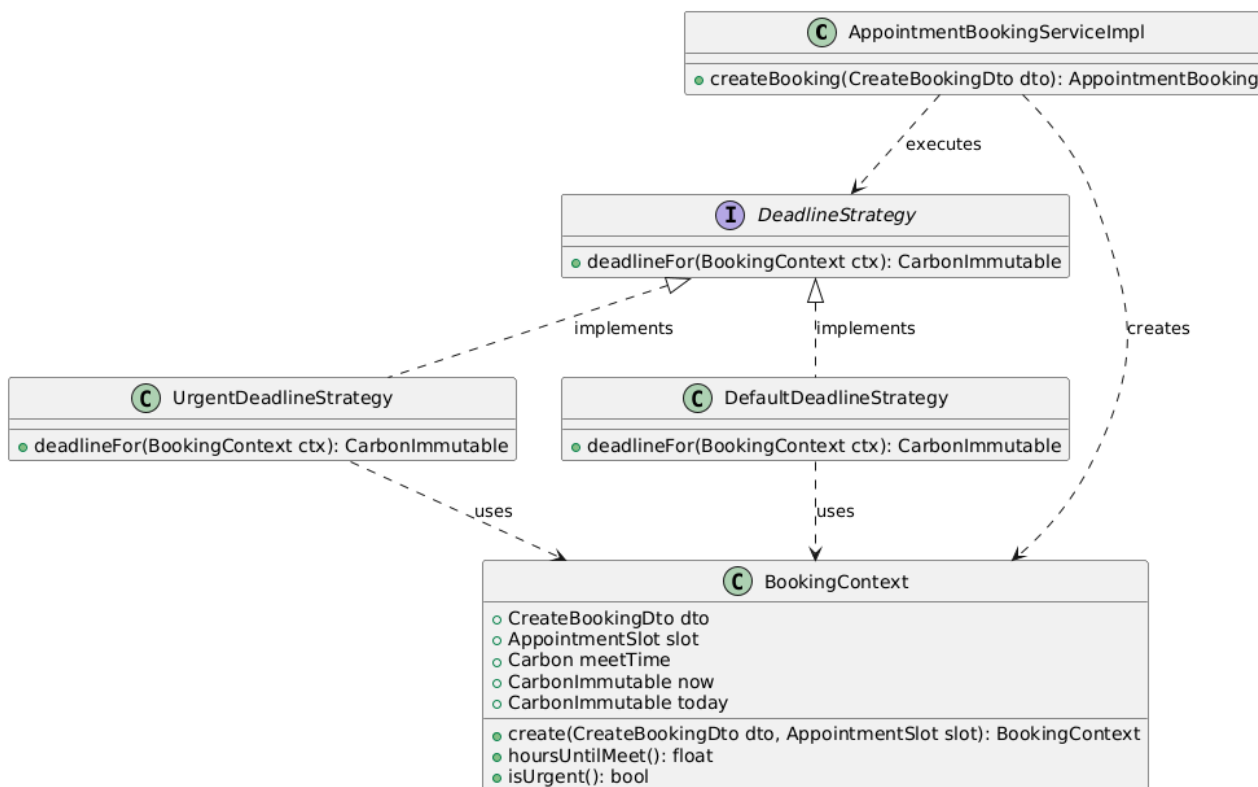
```
Event::listen(ListingSaved::class, ClearPublicListingCache::class);
Event::listen(ListingSaved::class, LogListingSaved::class);
Event::listen(ListingSaved::class, CreateListingNotification::class);
```

3.2.3.8. Đặt lịch hẹn

- **Chức năng đã sử dụng các Design Patterns:** Strategy Pattern, Observer Pattern

a. Strategy Pattern

- **Vấn đề cần giải quyết:**
 - Hạn xác nhận lịch có nhiều cách tính tùy tình huống: đặt gấp (còn dưới 6 giờ tới giờ hẹn) và thông thường.
 - Nếu nhồi if (đặt gấp) ... else ... ngay trong hàm tạo lịch, mỗi lần thêm/đổi cách tính phải sửa vào giữa hàm, khó kiểm thử riêng và vi phạm Open/Closed (OCP)
- **Sơ đồ class**



Thành phần	Lớp cụ thể trong dự án	Tác dụng
Context	App\Services\Appointment\Impl\AppointmentBookingServiceImpl (createBooking)	Chọn Strategy theo BookingContext::isUrgent() rồi gọi deadlineFor().

Strategy	App\Services\Appointment\Booking\Deadline\DeadlineStrategy	Interface khai báo deadlineFor(ctx) để mọi cách tính tuân theo.
ConcreteStrategyA	Booking\Deadline\UrgentDeadlineStrategy	Tính hạn khi đặt gấp.
ConcreteStrategyB	Booking\Deadline\DefaultDeadlineStrategy	Tính hạn mặc định (+6 giờ).
Value Object	Booking\BookingContext	Gói dữ liệu đầu vào (dto/slot/meetTime/now/today) cho strategy.

- **Code triển khai**

DeadlineStrategy (Strategy):

app/Services/Appointment/Booking/Deadline/DeadlineStrategy.php (dòng 1–14)

```
interface DeadlineStrategy
{
    public function deadlineFor(BookingContext $ctx): CarbonImmutable;
}
```

UrgentDeadlineStrategy:

app/Services/Appointment/Booking/Deadline/UrgentDeadlineStrategy.php (1–20)

```
final class UrgentDeadlineStrategy implements DeadlineStrategy
{
    public function deadlineFor(BookingContext $ctx): CarbonImmutable
    {
        $hoursUntilMeet = $ctx->hoursUntilMeet();

        return $ctx->now->addHours(max($hoursUntilMeet - 1, 1));
    }
}
```

DefaultDeadlineStrategy:

app/Services/Appointment/Booking/Deadline/DefaultDeadlineStrategy.php (1–17)

```
final class DefaultDeadlineStrategy implements DeadlineStrategy
{
    public function deadlineFor(BookingContext $ctx): CarbonImmutable
    {
        return $ctx->now->addHours(6);
    }
}
```

BookingContext: app/Services/Appointment/Booking/BookingContext.php (1–56)

```

final class BookingContext
{
    public function __construct(
        public readonly CreateBookingDto $dto,
        public readonly AppointmentSlot $slot,
        public readonly Carbon $meetTime,
        public readonly CarbonImmutable $now,
        public readonly CarbonImmutable $today,
    ) {}

    /**
     * Dựng context từ DTO + slot đã tải. meet_time = ngày hẹn + start_time của slot.
     */
    public static function create(CreateBookingDto $dto, AppointmentSlot $slot): self
    {
        return new self(
            dto: $dto,
            slot: $slot,
            meetTime: Carbon::parse($dto->date.' '.$slot->start_time),
            now: CarbonImmutable::now(),
            today: CarbonImmutable::today(),
        );
    }

    /**
     * Số giờ (có dấu) từ bây giờ đến giờ hẹn – giữ nguyên công thức gốc của service.
     */
    public function hoursUntilMeet(): float
    {
        return $this->now->diffInHours($this->meetTime, absolute: false);
    }

    /**
     * AF-01 (SRS trang 68): đặt gấp khi còn dưới 6 tiếng trước giờ hẹn.
     */
    public function isUrgent(): bool
    {
        return $this->hoursUntilMeet() < 6;
    }
}

```

Chọn & gọi Strategy:

app/Services/Appointment/Impl/AppointmentBookingServiceImpl.php (dòng 53–57)

```

// 3. Tính hạn xác nhận (Strategy): đặt gấp (<6h) vs mặc định.
$strategy = $ctx->isUrgent()
    ? new UrgentDeadlineStrategy
    : new DefaultDeadlineStrategy;
$confirmDeadline = $strategy->deadlineFor($ctx);

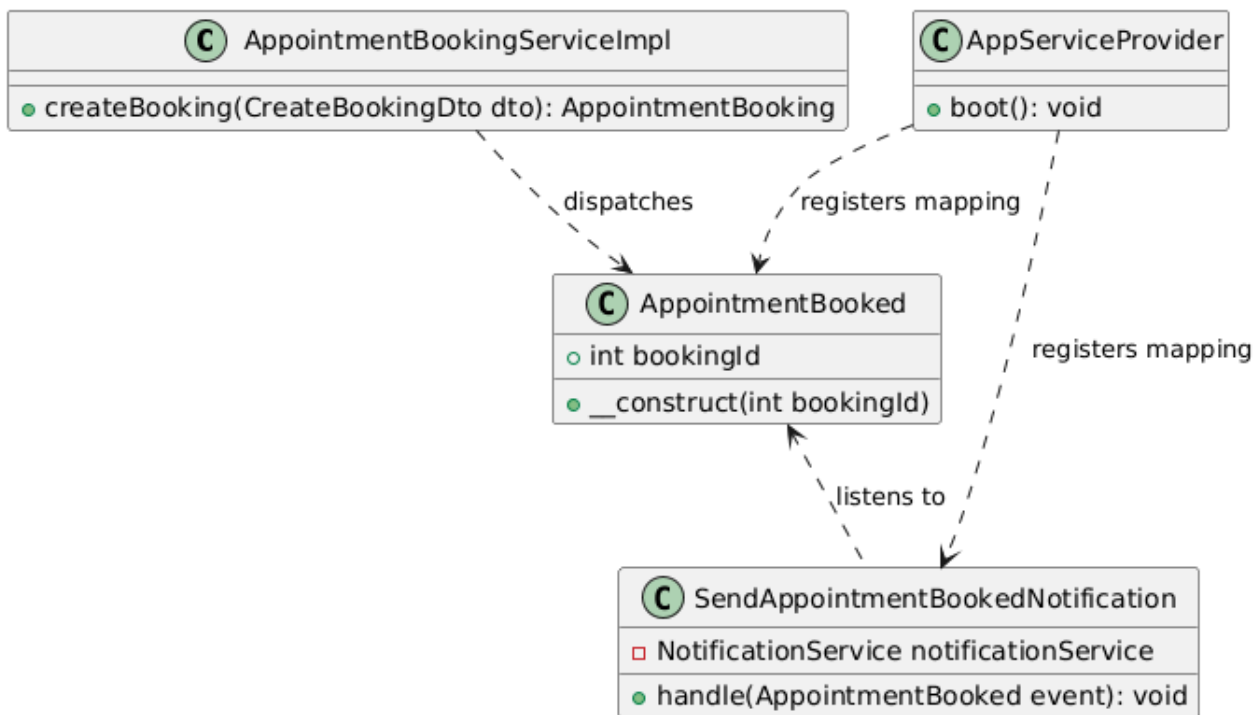
```

b. Observer Pattern

- Vấn đề cần giải quyết:

- Sau khi đặt lịch cần gửi thông báo/email cho chủ nhà.
- Nếu gọi thẳng các tác vụ này trong hàm tạo lịch, nghiệp vụ chính phình to và phụ thuộc nhiều dịch vụ.
- Thêm một kênh thông báo lại phải sửa nghiệp vụ. → Nghiệp vụ chỉ phát sự kiện, các listener tự lắng nghe và xử lý

- Sơ đồ class



Thành phần	Lớp cụ thể trong dự án	Tác dụng
Subject/Event	App\Events\Appointment\AppointmentBooked	Sự kiện phát khi tạo lịch thành công (mang bookingId).
Observer/Listener	App\Listeners\Appointment\SendAppointmentBookedNotification	Nhận sự kiện, gửi thông báo/email cho chủ nhà.
Publisher	AppointmentBookingServiceImpl (createBooking)	Phát AppointmentBooked::dispatch().

- Code triển khai

Sự kiện AppointmentBooked: app/Events/Appointment/AppointmentBooked.php (toàn bộ)

```

final class AppointmentBooked implements ShouldDispatchAfterCommit
{
    use Dispatchable, SerializesModels;

    public function __construct(
        public readonly int $bookingId,
    ) {}
}

```

Listener: app/Listeners/Appointment/SendAppointmentBookedNotification.php (toàn bộ)

```
final class SendAppointmentBookedNotification implements ShouldQueue
{
    public function __construct(
        private readonly NotificationService $notificationService,
    ) {}

    public function handle(AppointmentBooked $event): void
    {
        $booking = AppointmentBooking::query()
            ->with(['slot.listing.property', 'slot.poster', 'viewer'])
            ->findOrFail($event->bookingId);

        $poster = $booking->slot->poster;

        if (!$poster) {
            return;
        }

        $this->notificationService->send(
            user: $poster,
            type: NotificationType::APPOINTMENT_BOOKED,
            data: [
                'booking_id' => $booking->id,
                'viewer_name' => $booking->full_name,
                'viewer_phone' => $booking->phone_number ?: $booking->viewer->phone,
                'viewer_email' => $booking->email ?: $booking->viewer->email,
                'meet_time' => $booking->meet_time->format('H:i - d/m/Y'),
                'listing_id' => $booking->slot->listing_id,
                'listing_title' => $booking->slot->listing->title
                    ?? $booking->slot->listing->property->title
                    ?? 'Tin đăng',
            ],
            channels: [NotificationChannelType::EMAIL, NotificationChannelType::DATABASE],
        );
    }
}
```

Phát sự kiện:

app/Services/Appointment/Impl/AppointmentBookingServiceImpl.php (dòng 78)

```
AppointmentBooked::dispatch($booking->id);
```

Đăng ký listener: app/Providers/AppServiceProvider.php (dòng 229)

```
Event::listen(AppointmentBooked::class, SendAppointmentBookedNotification::class);
```

3.2.3.9. Xác nhận / Từ chối / Hủy lịch hẹn

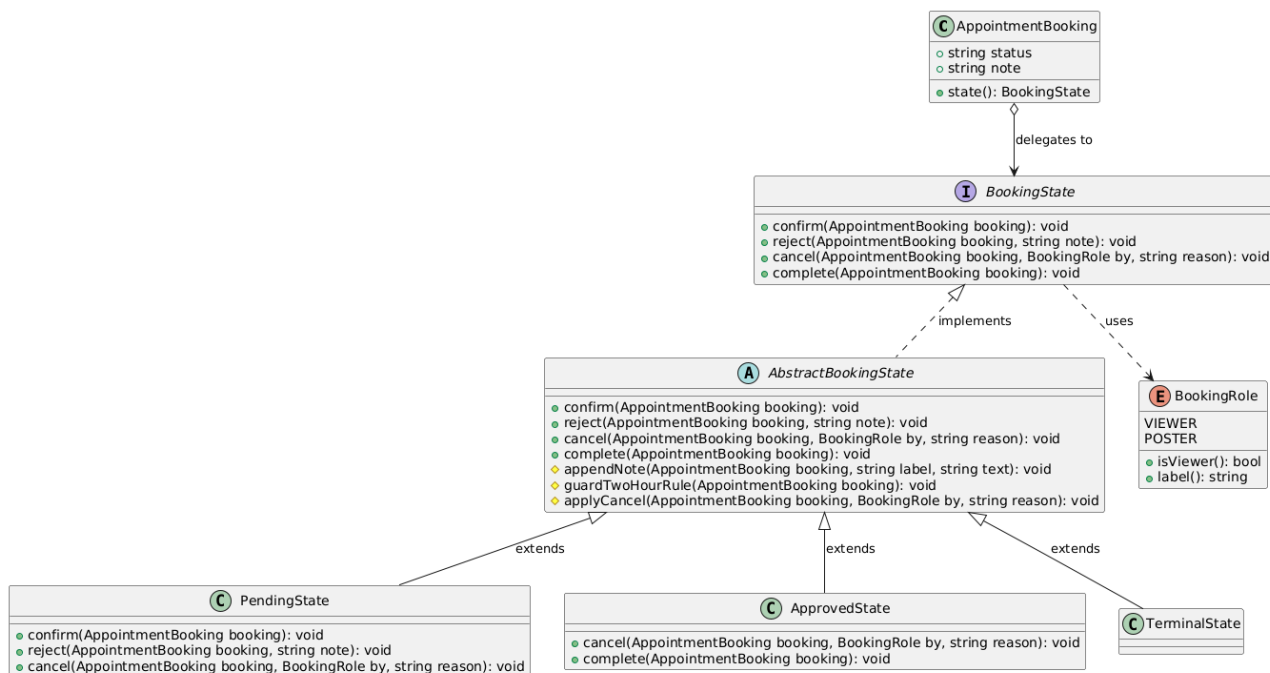
- **Chức năng đã sử dụng các Design Patterns:** State Pattern, Command Pattern, Observer Pattern

a. State Pattern

- **Vấn đề cần giải quyết:**

- Lịch hẹn có nhiều trạng thái (chờ duyệt, đã duyệt, kết thúc); mỗi trạng thái chỉ cho phép một số thao tác.
- Nếu code if (status == 'PENDING') ... if (status == 'APPROVED') ... khắp service, luật chuyển trạng thái bị phân tán, dễ cho thao tác sai trạng thái và khó nắm tổng thể.

- **Sơ đồ class**



Thành phần	Lớp cụ thể trong dự án	Tác dụng
Context	App\Models\AppointmentBooking (state())	Giữ status, trả về đối tượng State tương ứng.
State	Appointment\State\BookingState	Interface: confirm/reject/cancel/complete.
Base	State\AbstractBookingState	Mặc định cấm mọi thao tác + tiện ích chung (ghép note, quy tắc 2 giờ).
ConcreteState	State\PendingState	Chờ duyệt: cho confirm/reject/cancel.
ConcreteState	State\ApprovedState	Đã duyệt: cho cancel/complete.
ConcreteState	State\TerminalState	Kết thúc (hủy/hết hạn/hoàn thành): cấm thao tác.
Enum	State\BookingRole	Vai trò người hủy (VIEWER/POSTER).

- Code triển khai**

BookingState: app/Services/Appointment/State/BookingState.php (1–37)

```

interface BookingState
{
    /** Chủ nhà xác nhận (PENDING → APPROVED). */
    public function confirm(AppointmentBooking $booking): void;

    /** Chủ nhà từ chối (PENDING → CANCELLED_BY_POSTER), ghi lý do vào note nếu có. */
    public function reject(AppointmentBooking $booking, ?string $note): void;

    /**
     * Khách hoặc chủ nhà hủy (→ CANCELLED_BY_VIEWER / CANCELLED_BY_POSTER).
     * Áp quy tắc 2 giờ (BR-01).
     *
     * @throws BusinessException
     */
    public function cancel(AppointmentBooking $booking, BookingRole $by, string $reason): void;

    /**
     * Hệ thống tự hoàn thành khi đã qua giờ kết thúc hẹn (APPROVED → COMPLETED).
     *
     * @throws BusinessException
     */
    public function complete(AppointmentBooking $booking): void;
}

```

AbstractBookingState: app/Services/Appointment/State/AbstractBookingState.php
(1–76)

```

18 abstract class AbstractBookingState implements BookingState
19 {
20     public function confirm(AppointmentBooking $booking): void
21     {
22         throw new BusinessException(ErrorCode::BookingNotPending);
23     }
24
25     public function reject(AppointmentBooking $booking, ?string $note): void
26     {
27         throw new BusinessException(ErrorCode::BookingNotPending);
28     }
29
30     public function cancel(AppointmentBooking $booking, BookingRole $by, string $reason): void
31     {
32         throw new BusinessException(ErrorCode::BookingNotPending);
33     }
34
35     public function complete(AppointmentBooking $booking): void
36     {
37         throw new BusinessException(ErrorCode::BookingNotPending);
38     }
39
40     /**
41      * Ghép note theo định dạng "[{label}] {text}", giữ note cũ nếu có (ngăn cách " | ").
42      */
43     protected function appendNote(AppointmentBooking $booking, string $label, string $text): void
44     {
45         $existingNote = $booking->note ? $booking->note.' | ' : '';
46         $booking->note = $existingNote."[{label}] ".$text;
47     }

```

```

48
49 /**
50  * BR-01: chỉ được hủy khi còn ít nhất 2 tiếng trước giờ hẹn.
51  */
52 protected function guardTwoHourRule(AppointmentBooking $booking): void
53 {
54     $now = Carbon::now();
55     $meetTime = Carbon::parse($booking->meet_time);
56
57     if ($now->diffInHours($meetTime, false) < 2) {
58         throw new BusinessException(ErrorCode::BookingTooLateToCancel);
59     }
60 }
61
62 /**
63  * Logic hủy dùng chung cho PENDING và APPROVED: kiểm tra 2 giờ, đặt trạng thái
64  * theo vai trò người hủy, ghi note.
65  */
66 protected function applyCancel(AppointmentBooking $booking, BookingRole $by, string $reason): void
67 {
68     $this->guardTwoHourRule($booking);
69
70     $booking->status = $by->isViewer()
71         ? BookingStatus::CANCELLED_BY_VIEWER->value
72         : BookingStatus::CANCELLED_BY_POSTER->value;
73
74     $this->appendNote($booking, $by->label().' hủy', $reason);
75 }
76 }
77

```

PendingState: app/Services/Appointment/State/PendingState.php (1–31)

```

final class PendingState extends AbstractBookingState
{
    public function confirm(AppointmentBooking $booking): void
    {
        $booking->status = BookingStatus::APPROVED->value;
    }

    public function reject(AppointmentBooking $booking, ?string $note): void
    {
        $booking->status = BookingStatus::CANCELLED_BY_POSTER->value;

        if ($note) {
            $this->appendNote($booking, 'Chủ nhà từ chối', $note);
        }
    }

    public function cancel(AppointmentBooking $booking, BookingRole $by, string $reason): void
    {
        $this->applyCancel($booking, $by, $reason);
    }
}

```

ApprovedState: app/Services/Appointment/State/ApprovedState.php (1–23)

```

final class ApprovedState extends AbstractBookingState
{
    public function cancel(AppointmentBooking $booking, BookingRole $by, string $reason): void
    {
        $this->applyCancel($booking, $by, $reason);
    }

    public function complete(AppointmentBooking $booking): void
    {
        $booking->status = BookingStatus::COMPLETED->value;
    }
}

```

TerminalState: app/Services/Appointment/State/TerminalState.php (1–9)

```
final class TerminalState extends AbstractBookingState {}
```

BookingRole: app/Services/Appointment/State/BookingRole.php (1–25)

```
enum BookingRole
{
    case VIEWER;
    case POSTER;

    public function isViewer(): bool
    {
        return $this === self::VIEWER;
    }

    /**
     * Nhân hiển thị dùng khi ghép note (giữ nguyên chuỗi gốc của service).
     */
    public function label(): string
    {
        return $this === self::VIEWER ? 'Khách thuê' : 'Chủ nhà';
    }
}
```

Model state(): app/Models/AppointmentBooking.php (hàm state())

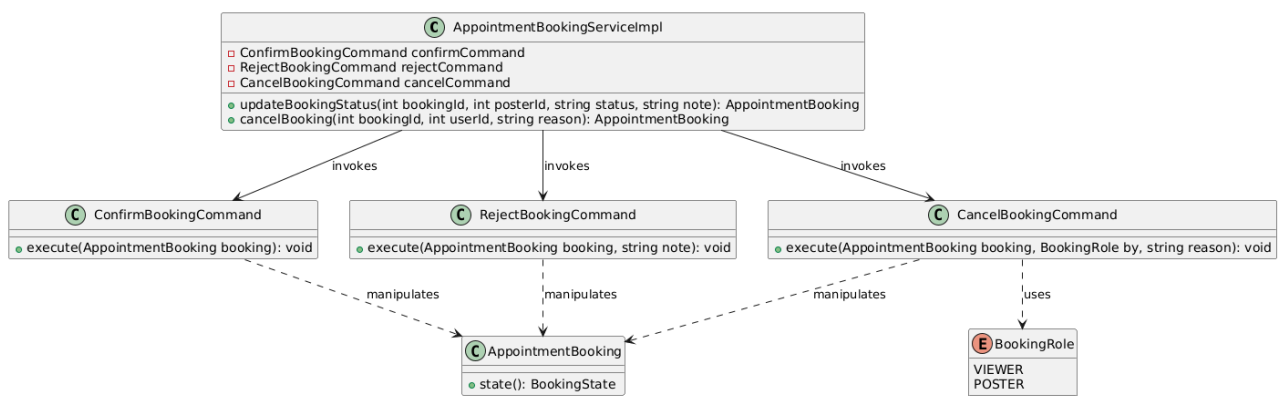
```
public function state(): BookingState
{
    return match (BookingStatus::from($this->status)) {
        BookingStatus::PENDING => new PendingState,
        BookingStatus::APPROVED => new ApprovedState,
        default => new TerminalState,
    };
}
```

b. Command Pattern

- **Vấn đề cần giải quyết:**

- Xác nhận, từ chối, hủy là các thao tác khác nhau.
- Nếu gộp chung một hàm, khó gắn log/thông báo riêng và khó kiểm thử từng thao tác. → Mỗi thao tác được đóng gói thành một Command.

- **Sơ đồ class**



Thành phần	Lớp cụ thể trong dự án	Tác dụng
------------	------------------------	----------

Invoker	AppointmentBookingServiceImpl (updateBookingStatus/cancelBooking)	Điều phối, chọn Command theo input.
ConcreteCommand	Appointment\Command\ConfirmBookingCommand	execute(b): state().confirm + lưu + phát sự kiện.
ConcreteCommand	Command\RejectBookingCommand	execute(b, note): state().reject + lưu + sự kiện.
ConcreteCommand	Command\CancelBookingCommand	execute(b, by, reason): state().cancel + lưu.
Receiver	AppointmentBooking (qua State)	Đối tượng bị thao tác.

- **Code triển khai**

ConfirmBookingCommand:

app/Services/Appointment/Command/ConfirmBookingCommand.php (1–22)

```
final class ConfirmBookingCommand
{
    public function execute(AppointmentBooking $booking): void
    {
        $booking->state()->confirm($booking);
        $booking->save();

        AppointmentBookingStatusUpdated::dispatch($booking->id, BookingStatus::APPROVED->value);
    }
}
```

RejectBookingCommand:

app/Services/Appointment/Command/RejectBookingCommand.php (1–21)

```
final class RejectBookingCommand
{
    public function execute(AppointmentBooking $booking, ?string $note): void
    {
        $booking->state()->reject($booking, $note);
        $booking->save();

        AppointmentBookingStatusUpdated::dispatch($booking->id, BookingStatus::CANCELLED_BY_POSTER->value);
    }
}
```

CancelBookingCommand:

app/Services/Appointment/Command/CancelBookingCommand.php (1–20)

```
final class CancelBookingCommand
{
    public function execute(AppointmentBooking $booking, BookingRole $by, string $reason): void
    {
        $booking->state()->cancel($booking, $by, $reason);
        $booking->save();
    }
}
```

Điều phối xác nhận/từ chối:

app/Services/Appointment/Impl/AppointmentBookingServiceImpl.php (dòng 93–116)

```
public function updateBookingStatus(int $bookingId, int $posterId, string $status, ?string $note): AppointmentBooking
{
    // 1. Kiểm tra booking có tồn tại không
    $booking = $this->findActiveBookingOrFail($bookingId);

    // 2. Kiểm tra người gọi API phải là poster (chủ nhà) của slot
    $slot = AppointmentSlot::find($booking->slot_id);
    if (!$slot || $slot->poster_id !== $posterId) {
        throw new BusinessException(ErrorCode::BookingNotOwner);
    }

    // 3. Điều phối theo status (Command). State sẽ tự kiểm tra booking đang PENDING;
    // nếu không, ném BookingNotPending – giữ nguyên hành vi gốc.
    match ($status) {
        BookingStatus::APPROVED->value => $this->confirmCommand->execute($booking),
        BookingStatus::CANCELLED_BY_POSTER->value => $this->rejectCommand->execute($booking, $note),
        default => throw new BusinessException(ErrorCode::BookingNotPending),
    };

    // 4. Load relationships để resource trả về đầy đủ
    $booking->load('slot.listing');

    return $booking;
}
```

Điều phối hủy:

app/Services/Appointment/Impl/AppointmentBookingServiceImpl.php (dòng 118–139)

```
public function cancelBooking(int $bookingId, int $userId, string $reason): AppointmentBooking
{
    // 1. Tìm booking
    $booking = $this->findActiveBookingOrFail($bookingId);

    // 2. Xác định vai trò của người thực hiện hủy
    $slot = AppointmentSlot::find($booking->slot_id);
    $isViewer = ($booking->viewer_id === $userId);
    $isPoster = ($slot && $slot->poster_id === $userId);

    if (!$isViewer && !$isPoster) {
        throw new BusinessException(ErrorCode::BookingNotOwner);
    }

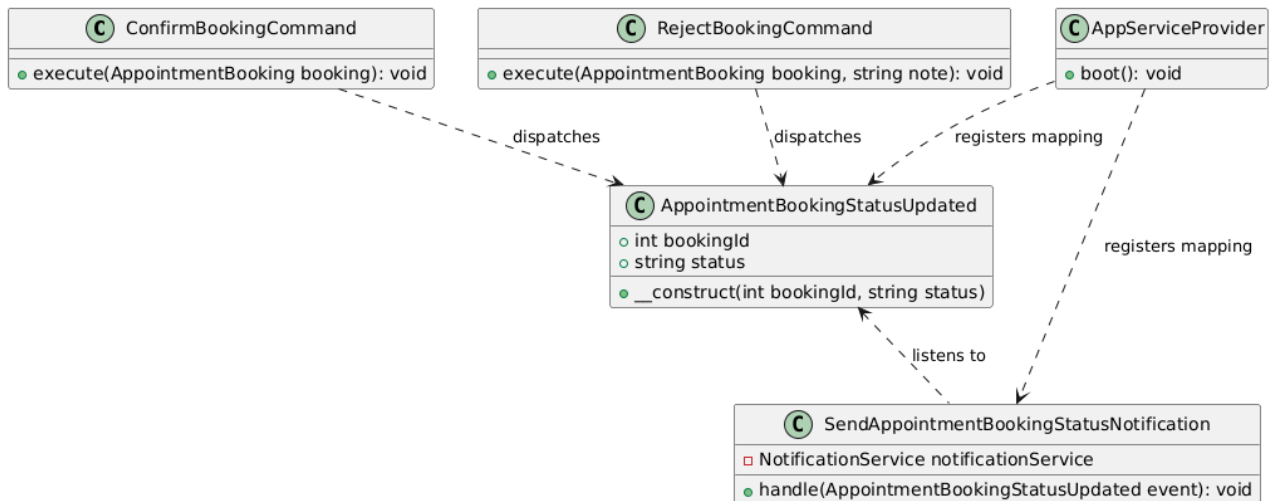
    // 3. Hủy qua Command + State (kiểm tra trạng thái hợp lệ + quy tắc 2 giờ, ghi note).
    $role = $isViewer ? BookingRole::VIEWER : BookingRole::POSTER;
    $this->cancelCommand->execute($booking, $role, $reason);

    $booking->load('slot.listing');

    return $booking;
}
```

c. Observer Pattern

- **Vấn đề cần giải quyết:**
 - Sau khi xác nhận/từ chối cần báo cho người xem. → phát sự kiện AppointmentBookingStatusUpdated, listener xử lý.
- **Sơ đồ class**



Thành phần	Lớp cụ thể trong dự án	Tác dụng
Event	App\Events\Appointment\AppointmentBookingStatusUpdated	Mang bookingId + status mới.
Listener	App\Listeners\Appointment\SendAppointmentBookingStatusNotification	Báo người xem khi trạng thái đổi.
ConcreteCommand	Command\RejectBookingCommand	execute(b, note): state().reject + lưu + sự kiện.
ConcreteCommand	Command\CancelBookingCommand	execute(b, by, reason): state().cancel + lưu.
Receiver	AppointmentBooking (qua State)	Đối tượng bị thao tác.

- Code triển khai**

Sự kiện: app/Events/Appointment/AppointmentBookingStatusUpdated.php (toàn bộ)

```

final class AppointmentBookingStatusUpdated implements ShouldDispatchAfterCommit
{
    use Dispatchable, SerializesModels;

    public function __construct(
        public readonly int $bookingId,
        public readonly string $status,
    ) {}
}
  
```

Listener:

app/Listeners/Appointment/SendAppointmentBookingStatusNotification.php (toàn bộ)

```

final class SendAppointmentBookingStatusNotification implements ShouldQueue
{
    public function __construct(
        private readonly NotificationService $notificationService,
    ) {}

    public function handle(AppointmentBookingStatusUpdated $event): void
    {
        $booking = AppointmentBooking::query()
            ->with(['slot.listing.property', 'slot.poster', 'viewer'])
            ->findOrFail($event->bookingId);

        $viewer = $booking->viewer;

        if (! $viewer) {
            return;
        }

        $status = BookingStatus::from($event->status);

        $this->notificationService->send(
            user: $viewer,
            type: NotificationType::APPOINTMENT_STATUS_UPDATED,
            data: [
                'booking_id' => $booking->id,
                'status' => $status->value,
                'listing_id' => $booking->slot->listing_id,
                'listing_title' => $booking->slot->listing->title
                    ?? $booking->slot->listing->property->title
                    ?? 'Tin đăng',
                'meet_time' => $booking->meet_time->format('H:i - d/m/Y'),
                'poster_name' => $booking->slot->poster->full_name ?? 'Người đăng tin',
            ],
            channels: [NotificationChannelType::EMAIL, NotificationChannelType::DATABASE],
        );
    }
}

```

Phát khi xác nhận:

app/Services/Appointment/Command/ConfirmBookingCommand.php (dòng 20)

```

13 final class ConfirmBookingCommand
14 {
15     public function execute(AppointmentBooking $booking): void
16     {
17         $booking->state()->confirm($booking);
18         $booking->save();
19
20         AppointmentBookingStatusUpdated::dispatch($booking->id, BookingStatus::APPROVED->value);
21     }
22 }

```

Phát khi từ chối:

app/Services/Appointment/Command/RejectBookingCommand.php (dòng 19)

```

12 final class RejectBookingCommand
13 {
14     public function execute(AppointmentBooking $booking, ?string $note): void
15     {
16         $booking->state()->reject($booking, $note);
17         $booking->save();
18
19         AppointmentBookingStatusUpdated::dispatch($booking->id, BookingStatus::CANCELLED_BY_POSTER->value);
20     }
21 }

```

Đăng ký listener: app/Providers/AppServiceProvider.php (dòng 230)

```

230 | | Event::listen(AppointmentBookingStatusUpdated::class, SendAppointmentBookingStatusNotification::class);

```

3.2.3.10. Tự động hoàn thành lịch hẹn

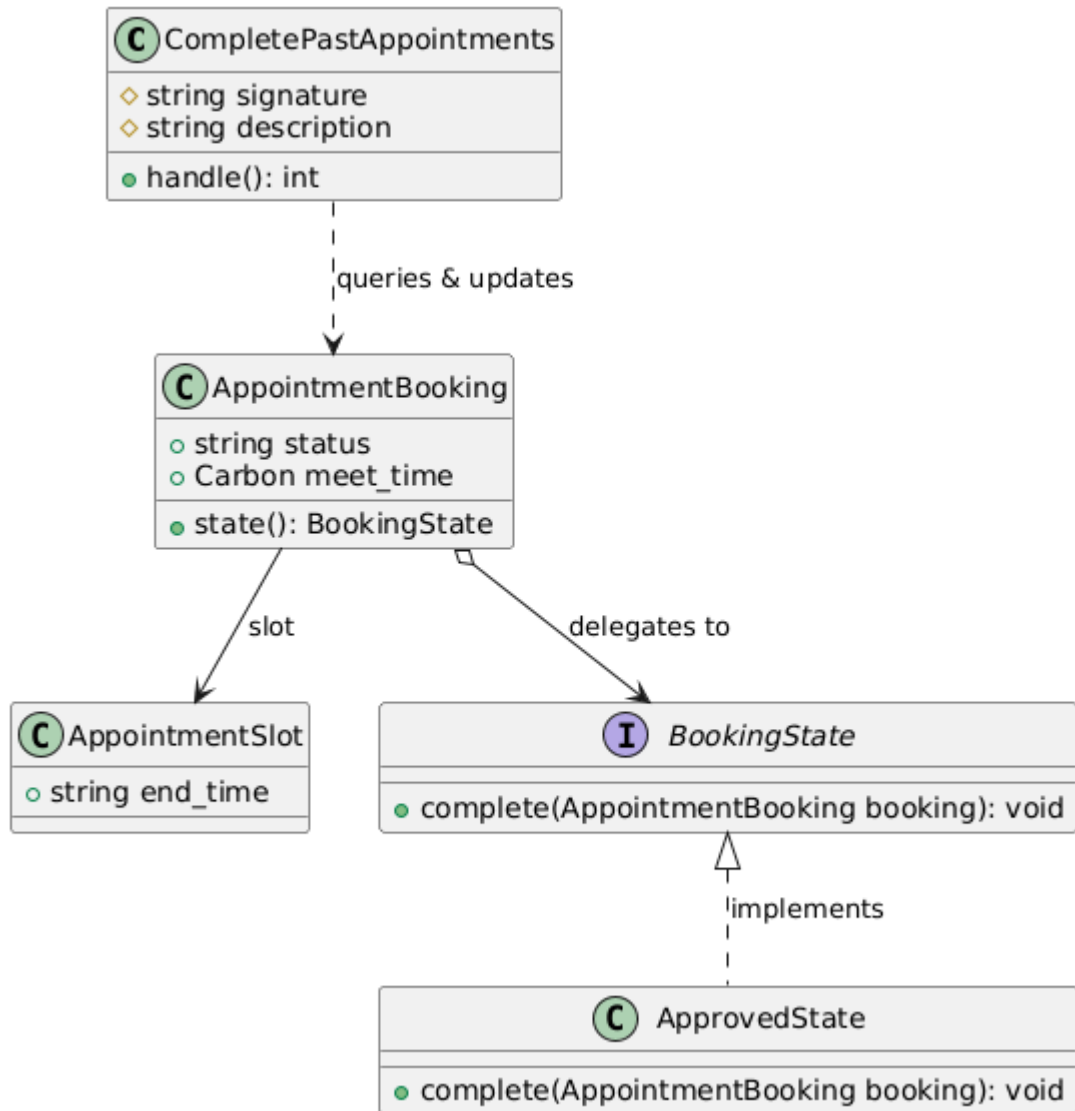
- Chức năng đã sử dụng các Design Patterns: State Pattern

a. State Pattern

- **Vấn đề cần giải quyết:**

- Lịch đã xác nhận khi đã qua giờ kết thúc hẹn (ngày hẹn + end_time của slot) cần tự chuyển sang COMPLETED.
- Nếu lệnh định kỳ tự update(status='COMPLETED') thẳng vào DB, luật 'trạng thái nào được hoàn thành' lại nằm ngoài State, dễ lệch với confirm/reject/cancel. → Để State (ApprovedState::complete) quyết định.

- **Sơ đồ class**



Thành phần	Lớp cụ thể trong dự án	Tác dụng
Invoker (Console)	App\Console\Commands\CompletePastAppointments	Lệnh chạy mỗi phút: tìm lịch APPROVED đã qua giờ kết thúc, gọi state().complete().

Context	App\Models\AppointmentBooking	state() trả State theo status; có meet_time + quan hệ slot.
State	Appointment\State\BookingState (complete)	Khai báo complete(b).
ConcreteState	State\ApprovedState (complete)	Đặt status = COMPLETED (chỉ trạng thái đã duyệt mới cho phép).
(dữ liệu giờ)	App\Models\AppointmentSlot	Cung cấp end_time để tính thời điểm kết thúc.

- **Code triển khai**

Lệnh tự hoàn thành: app/Console/Commands/CompletePastAppointments.php (1–54)

```
final class CompletePastAppointments extends Command
{
    protected $signature = 'appointments:complete-past';

    protected $description = 'Đánh dấu HOÀN THÀNH cho các lịch hẹn đã xác nhận đã qua giờ kết thúc.';

    public function handle(): int
    {
        $now = Carbon::now();
        $completed = 0;

        AppointmentBooking::query()
            ->where('status', BookingStatus::APPROVED->value)
            ->where('is_deleted', false)
            ->where('meet_time', '<=', $now) // giờ bắt đầu đã qua → mới có khả năng kết thúc
            ->with('slot')
            ->chunkById(200, function ($bookings) use ($now, &$completed) {
                foreach ($bookings as $booking) {
                    if (!$booking->slot) {
                        continue;
                    }

                    $endTime = Carbon::parse($booking->meet_time->toDateString().' '.$booking->slot->end_time);

                    if ($endTime->lessThanOrEqualTo($now)) {
                        // Chuyển trạng thái qua State (ApprovedState::complete) cho nhất quán.
                        $booking->state()->complete($booking);
                        $booking->save();
                        $completed++;
                    }
                }
            });

        $this->info("Completed {$completed} past appointment(s).");

        return self::SUCCESS;
    }
}
```

ApprovedState::complete: app/Services/Appointment/State/ApprovedState.php
(dòng 19–22)

```

12 final class ApprovedState extends AbstractBookingState
13 {
14     public function cancel(AppointmentBooking $booking, BookingRole $by, string $reason): void
15     {
16         $this->applyCancel($booking, $by, $reason);
17     }
18
19     public function complete(AppointmentBooking $booking): void
20     {
21         $booking->status = BookingStatus::COMPLETED->value;
22     }
23 }

```

BookingState::complete: app/Services/Appointment/State/BookingState.php (dòng 32–36)

```

15 interface BookingState
16 {
17     /** Chủ nhà xác nhận (PENDING → APPROVED). */
18     public function confirm(AppointmentBooking $booking): void;
19
20     /** Chủ nhà từ chối (PENDING → CANCELLED_BY_POSTER), ghi lý do vào note nếu có. */
21     public function reject(AppointmentBooking $booking, ?string $note): void;
22
23     /**
24      * Khách hoặc chủ nhà hủy (→ CANCELLED_BY_VIEWER / CANCELLED_BY_POSTER).
25      * Áp quy tắc 2 giờ (BR-01).
26      *
27      * @throws BusinessException
28      */
29     public function cancel(AppointmentBooking $booking, BookingRole $by, string $reason): void;
30
31     /**
32      * Hệ thống tự hoàn thành khi đã qua giờ kết thúc hẹn (APPROVED → COMPLETED).
33      *
34      * @throws BusinessException
35      */
36     public function complete(AppointmentBooking $booking): void;
37 }

```

Lên lịch chạy: routes/console.php (dòng 24)

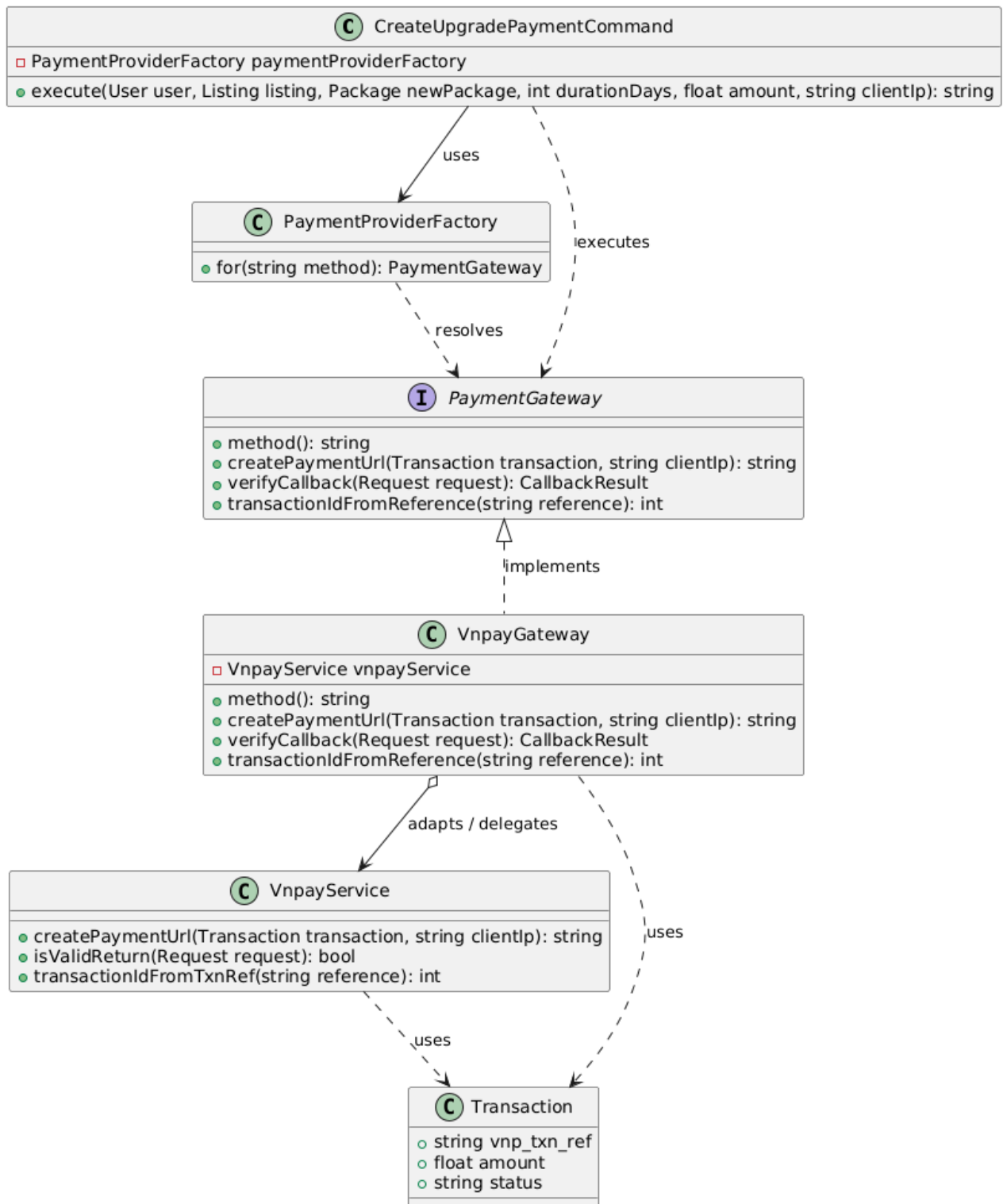
```

23 // — Appointment Completion: Lịch đã xác nhận, qua giờ kết thúc → HOÀN THÀNH —
24 Schedule::command('appointments:complete-past')->everyMinute();
--

```

3.2.3.11. Tạo yêu cầu thanh toán VNPay

- Chức năng đã sử dụng các Design Patterns: Adapter Pattern, Factory Pattern
 - a. Adapter Pattern
- Vấn đề cần giải quyết:
 - Nếu gọi thẳng API VNPay (ký HMAC, dựng URL) rải khắp use case, code phụ thuộc cứng vào một cổng.
 - Muốn thêm Momo/ZaloPay phải sửa nhiều nơi. → Bọc cổng VNPay sau một interface nội bộ thống nhất.
- Sơ đồ class



Thành phần	Lớp cụ thể trong dự án	Tác dụng
Target	App\Services\Payment\Gateway \PaymentGateway	Interface nội bộ: method/createPaymentUrl/verifyCa llback/transactionIdFromReference .

Adapter	Gateway\VnpayGateway	Triển khai Target bằng cách ủy thác cho VnpayService.
Adaptee	Payment\VnpayService	Logic kỹ thuật VNPay (ký HMAC, dựng URL, kiểm chữ ký).
DTO	Gateway\CallbackResult	Dữ liệu callback đã chuẩn hóa.
Client	Listing\Upgrade\CreateUpgradePaymentCommand	Dùng PaymentGateway để tạo URL thanh toán.

- **Code triển khai**

PaymentGateway (Target): app/Services/Payment/Gateway/PaymentGateway.php (1–25)

```

interface PaymentGateway
{
    /** Mã phương thức ('VNPAY', ...). */
    public function method(): string;

    /** Tạo URL chuyển hướng sang cổng thanh toán cho một giao dịch. */
    public function createPaymentUrl(Transaction $transaction, string $clientId): string;

    /** Chuẩn hoá dữ liệu callback từ cổng về CallbackResult nội bộ. */
    public function verifyCallback(Request $request): CallbackResult;

    /** Tách id giao dịch nội bộ từ mã tham chiếu của cổng. */
    public function transactionIdFromReference(string $reference): ?int;
}

```

VnpayGateway (Adapter): app/Services/Payment/Gateway/VnpayGateway.php (1–52)

```

final class VnpayGateway implements PaymentGateway
{
    public function __construct(
        private readonly VnpayService $vnpayService,
    ) {}

    public function method(): string
    {
        return 'VNPAY';
    }

    public function createPaymentUrl(Transaction $transaction, string $clientId): string
    {
        return $this->vnpayService->createPaymentUrl($transaction, $clientId);
    }

    public function verifyCallback(Request $request): CallbackResult
    {
        $reference = (string) $request->query('vnp_TxnRef', '');

        return new CallbackResult(
            isValidSignature: $this->vnpayService->isValidReturn($request),
            responseCode: $request->query('vnp_ResponseCode'),
            transactionStatus: $request->query('vnp_TransactionStatus'),
            reference: $reference,
            gatewayFields: [
                'vnp_txn_ref' => $reference,
                'vnp_transaction_no' => $request->query('vnp_TransactionNo'),
                'vnp_bank_code' => $request->query('vnp_BankCode'),
                'vnp_response_code' => $request->query('vnp_ResponseCode'),
                'vnp_pay_date' => $request->query('vnp_PayDate'),
            ],
        );
    }

    public function transactionIdFromReference(string $reference): ?int
    {
        return $this->vnpayService->transactionIdFromTxnRef($reference);
    }
}

```

VnpayService (Adaptee): app/Services/Payment/VnpayService.php (1–79)

```

9   final class VnpayService
10  {
11      public function createPaymentUrl(Transaction $transaction, string $clientId): string
12      {
13          $tmnCode = (string) config('vnpay.tmn_code');
14          $hashSecret = (string) config('vnpay.hash_secret');
15          $paymentUrl = (string) config('vnpay.payment_url');
16
17          if ($tmnCode === '' || $hashSecret === '' || $paymentUrl === '') {
18              throw new RuntimeException('VNPAY configuration is missing.');
```

```
19          }
20
21          $txnRef = $this->txnRef($transaction);
22          $amount = (int) round((float) $transaction->amount * 100);
23
24          $params = [
25              'vnp_Version' => '2.1.0',
26              'vnp_Command' => 'pay',
27              'vnp_TmnCode' => $tmnCode,
28              'vnp_Amount' => $amount,
29              'vnp_CurrCode' => 'VND',
30              'vnp_TxnRef' => $txnRef,
31              'vnp_OrderInfo' => 'Thanh toan goi tin Propify #'.$transaction->id,
32              'vnp_OrderType' => 'billpayment',
33              'vnp_Locale' => 'vn',
34              'vnp_ReturnUrl' => (string) config('vnpay.return_url'),
35              'vnp_IpAddr' => $clientId ?: '127.0.0.1',
36              'vnp_CreateDate' => now()->format('YmdHis'),
37              'vnp_ExpireDate' => ($transaction->expires_at ?? now()->addMinutes(15))->format('YmdHis'),
38          ];
39
40          $bankCode = (string) config('vnpay.bank_code');
41          if ($bankCode !== '') {
42              $params['vnp_BankCode'] = $bankCode;
43          }
44
45          ksort($params);
46          $query = http_build_query($params, '', '&', PHP_QUERY_RFC1738);
47          $secureHash = hash_hmac('sha512', $query, $hashSecret);
48
49          return $paymentUrl.'?'.$query.'&vnp_SecureHash='.$secureHash;
50      }
51
52      public function isValidReturn(Request $request): bool
53      {
54          $params = $request->query();
55          $secureHash = (string) ($params['vnp_SecureHash'] ?? '');
56
57          unset($params['vnp_SecureHash'], $params['vnp_SecureHashType']);
58          ksort($params);
59
60          $query = http_build_query($params, '', '&', PHP_QUERY_RFC1738);
61          $expected = hash_hmac('sha512', $query, (string) config('vnpay.hash_secret'));
62
63          return $secureHash !== '' && hash_equals($expected, $secureHash);
64      }
65
66      public function transactionIdFromTxnRef(string $txnRef): ?int
67      {
68          if (preg_match('/^PFY(\d+)/', $txnRef, $matches) !== 1) {
69              return null;
70          }
71
72          return (int) $matches[1];
73      }
74
75      private function txnRef(Transaction $transaction): string
76      {
77          return 'PFY'.$transaction->id;
78      }
79  }

```

183

CallbackResult: app/Services/Payment/Gateway/CallbackResult.php (1–19)

```
final class CallbackResult
{
    public function __construct(
        public readonly bool $isValidSignature,
        public readonly ?string $responseCode,
        public readonly ?string $transactionStatus,
        public readonly string $reference,
        /** @var array<string, mixed> Các cột cần lưu lại để đối soát (vnp_*). */
        public readonly array $gatewayFields,
    ) {}
}
```

Client tạo thanh toán:

app/Services/Listing/Upgrade/CreateUpgradePaymentCommand.php (1–45)

```
final class CreateUpgradePaymentCommand
{
    public function __construct(
        private readonly PaymentProviderFactory $paymentProviderFactory,
    ) {}

    public function execute(
        User $user,
        Listing $listing,
        Package $newPackage,
        int $durationDays,
        float $amount,
        string $clientId
    ): string {
        $paymentExpiresAt = now()->addMinutes(15);

        $transaction = Transaction::create([
            'user_id' => $user->id,
            'listing_id' => $listing->id,
            'package_id' => $newPackage->id,
            'amount' => $amount,
            'duration_days' => $durationDays,
            'payment_method' => 'VNPAY',
            'status' => 'PENDING',
            'transaction_date' => now(),
            'expires_at' => $paymentExpiresAt,
        ]);

        $transaction->update(['vnp_txn_ref' => 'PFY'.$transaction->id]);
        ExpirePendingTransactionJob::dispatch($transaction->id)->delay($paymentExpiresAt);

        return $this->paymentProviderFactory->for('VNPAY')->createPaymentUrl($transaction->fresh(), $clientId);
    }
}
```

Gọi từ service: app/Services/Listing/impl/ListingServiceImpl.php (dòng 332–344)

```
public function createUpgradePayment(User $user, int $listingId, int $packageId, int $durationDays, string $clientId): string
{
    [$listing, $newPackage, $pricing, $context] = $this->prepareUpgrade($user, $listingId, $packageId, $durationDays);

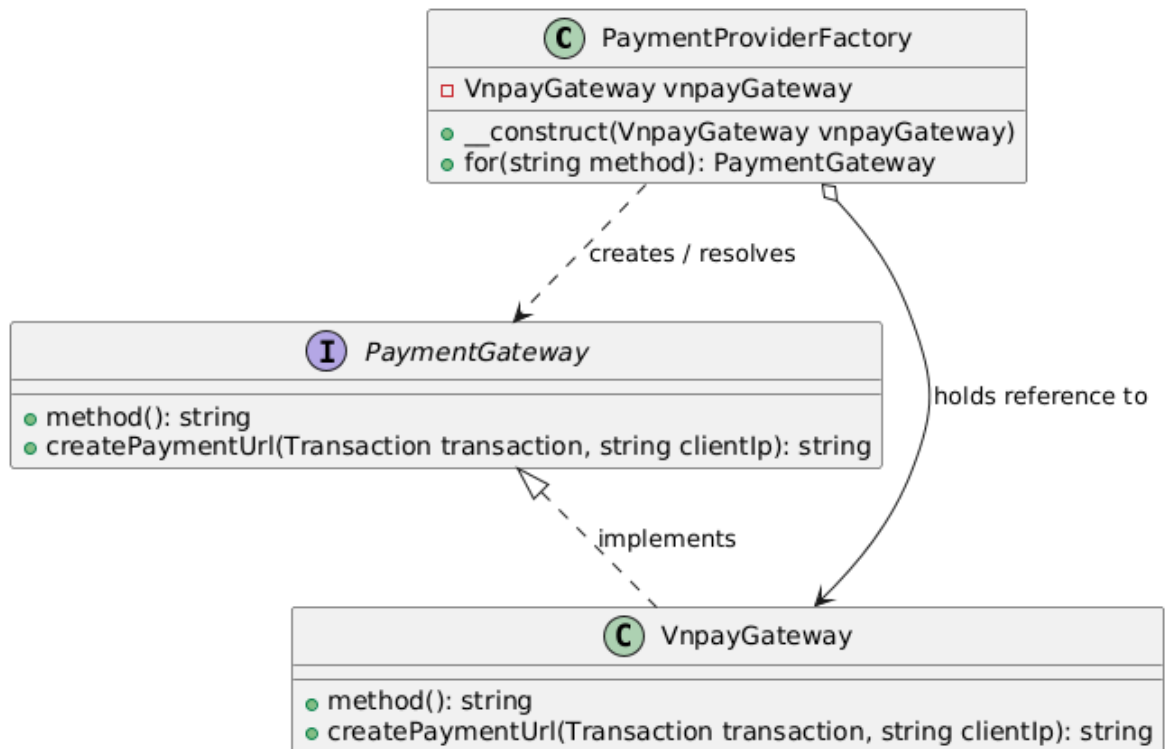
    return $this->createUpgradePaymentCommand->execute(
        user: $user,
        listing: $listing,
        newPackage: $newPackage,
        durationDays: $durationDays,
        amount: (float) $pricing->price,
        clientId: $clientId,
    );
}
```

b. Factory Pattern

- Vấn đề cần giải quyết:

- Chọn công thanh toán theo phương thức tại runtime mà không new cứng. → PaymentProviderFactory.

- Sơ đồ class



Thành phần	Lớp cụ thể trong dự án	Tác dụng
Creator/Factory	Gateway\PaymentProviderFactory	for(method) trả về PaymentGateway.
Product	Gateway\PaymentGateway	Interface sản phẩm.
ConcreteProduct	Gateway\VnpayGateway	Sản phẩm cụ thể cho VNPAY.

- Code triển khai

PaymentProviderFactory:

app/Services/Payment/Gateway/PaymentProviderFactory.php (1–24)

```

final class PaymentProviderFactory
{
    public function __construct(
        private readonly VnpayGateway $vnpayGateway,
    ) {}

    public function for(string $method): PaymentGateway
    {
        return match (strtoupper($method)) {
            'VNPAY' => $this->vnpayGateway,
            default => throw new RuntimeException("Unsupported payment method: {$method}"),
        };
    }
}
    
```

3.2.3.12. Xử lý callback thanh toán VNPay

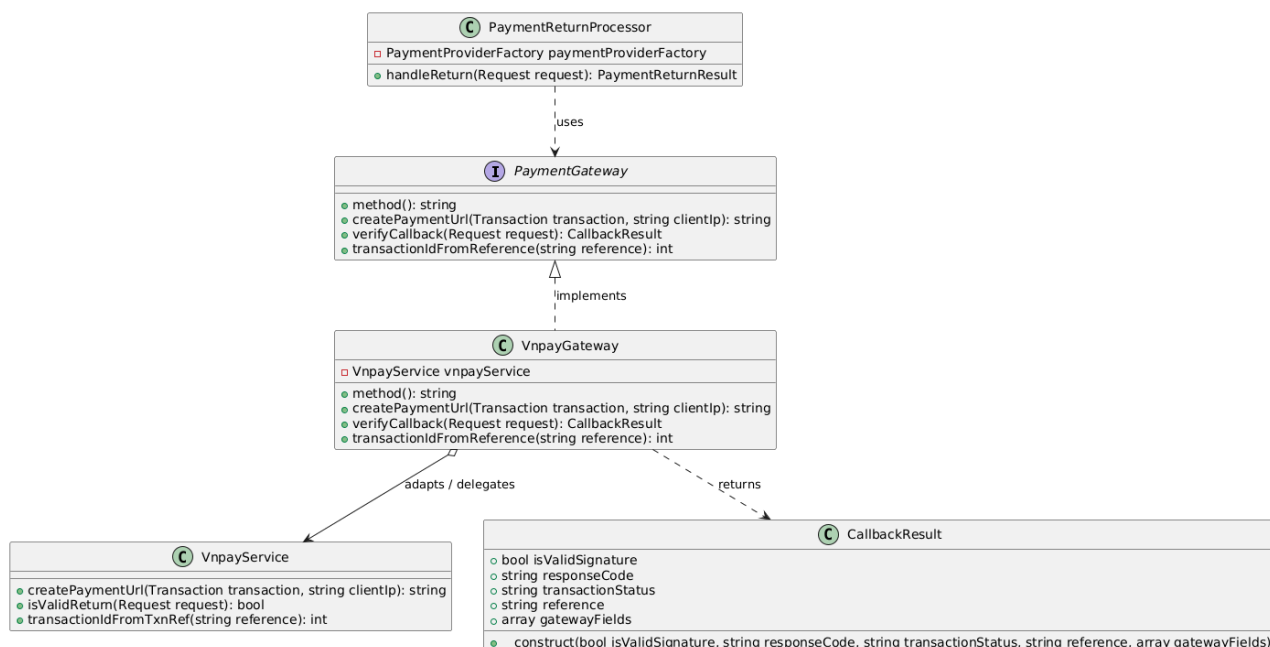
- Chức năng đã sử dụng các Design Patterns: Adapter Pattern, State Pattern, Observer Pattern

a. Adapter Pattern

- **Vấn đề cần giải quyết:**

- Dữ liệu callback thô của VNPay (các trường vnp_*) cần chuẩn hóa về dạng nội bộ để xử lý đồng nhất. → VnpayGateway::verifyCallback chuyển request VNPay thành CallbackResult.

- **Sơ đồ class**



Thành phần	Lớp cụ thể trong dự án	Tác dụng
Target	Gateway\PaymentGateway (verifyCallback)	Khuôn chuẩn hóa callback.
Adapter	Gateway\VnpayGateway	Bóc vnp_*, kiểm chữ ký → CallbackResult.
DTO	Gateway\CallbackResult	isValidSignature/responseCode/transactionStatus/reference/gatewayFields.

- **Code triển khai**

VnpayGateway::verifyCallback:

app/Services/Payment/Gateway/VnpayGateway.php (dòng 28–46)

```

28
29     public function verifyCallback(Request $request): CallbackResult
30     {
31         $reference = (string) $request->query('vnp_TxnRef', '');
32
33         return new CallbackResult(
34             isValidSignature: $this->vnpayService->isValidReturn($request),
35             responseCode: $request->query('vnp_ResponseCode'),
36             transactionStatus: $request->query('vnp_TransactionStatus'),
37             reference: $reference,
38             gatewayFields: [
39                 'vnp_txn_ref' => $reference,
40                 'vnp_transaction_no' => $request->query('vnp_TransactionNo'),
41                 'vnp_bank_code' => $request->query('vnp_BankCode'),
42                 'vnp_response_code' => $request->query('vnp_ResponseCode'),
43                 'vnp_pay_date' => $request->query('vnp_PayDate'),
44             ],
45         );
46     }

```

CallbackResult: app/Services/Payment/Gateway/CallbackResult.php (1–19)

```

final class CallbackResult
{
    public function __construct(
        public readonly bool $isValidSignature,
        public readonly ?string $responseCode,
        public readonly ?string $transactionStatus,
        public readonly string $reference,
        /** @var array<string, mixed> Các cột cần lưu lại để đối soát (vnp_*). */
        public readonly array $gatewayFields,
    ) {}
}

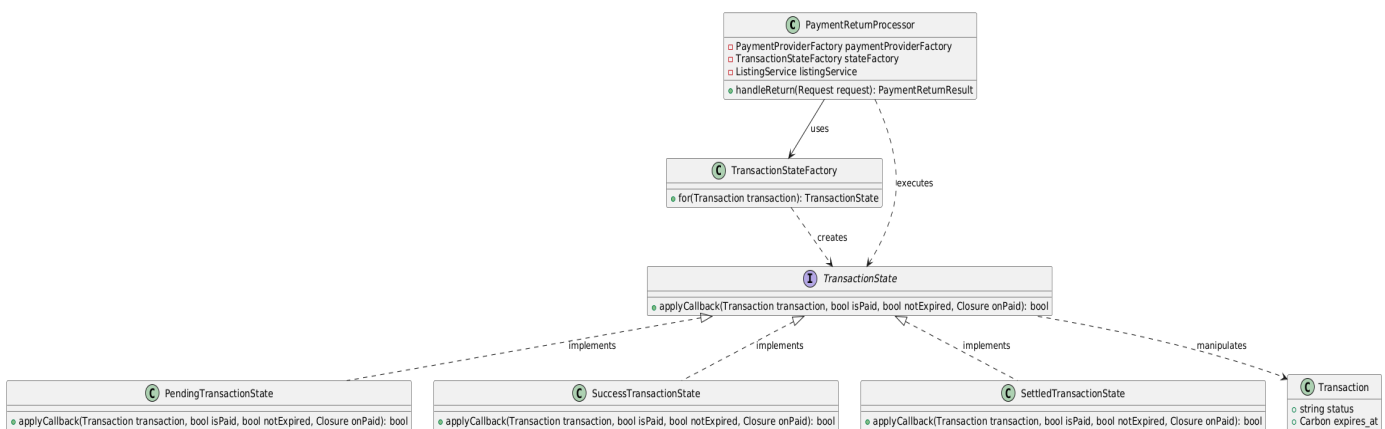
```

b. State Pattern

- Vấn đề cần giải quyết:

- Xử lý callback có nhiều nhánh: đã thanh toán + còn hạn → hoàn tất; đã thanh toán + hết hạn → hết hạn; không hợp lệ → thất bại; đã thành công → bỏ qua (idempotent)
- Một khối if/elseif dài để sót nhánh. → Mỗi trạng thái giao dịch là một State.

- Sơ đồ class



Thành phần	Lớp cụ thể trong dự án	Tác dụng
Context	Payment\PaymentReturnProcessor (handleReturn)	Điều phối luồng callback; gọi State theo status giao dịch.
Factory	Payment\State\TransactionStateFactory	for(t) trả State theo status.
State	State\TransactionState	applyCallback(t, isPaid, notExpired, onPaid).
ConcreteState	State\PendingTransactionState	Chờ: hoàn tất/hết hạn/thất bại.
ConcreteState	State\SuccessTransactionState	Đã thành công: idempotent.
ConcreteState	State\SettledTransactionState	Đã chốt: bỏ qua.

- **Code triển khai**

TransactionState: app/Services/Payment/State/TransactionState.php (1–22)

```
interface TransactionState
{
    /**
     * @param bool $isPaid Callback xác nhận đã thanh toán hợp lệ.
     * @param bool $notExpired Giao dịch còn hạn (chưa hết hạn).
     * @param Closure $onPaid Hành động hoàn tất nâng cấp khi thanh toán thành công.
     * @return bool Giao dịch có được coi là hoàn tất không.
     */
    public function applyCallback(Transaction $transaction, bool $isPaid, bool $notExpired, Closure $onPaid): bool;
}
```

PendingTransactionState:

app/Services/Payment/State/PendingTransactionState.php (1–34)

```
final class PendingTransactionState implements TransactionState
{
    public function applyCallback(Transaction $transaction, bool $isPaid, bool $notExpired, Closure $onPaid): bool
    {
        if ($isPaid && $notExpired) {
            $onPaid($transaction);

            return true;
        }

        if ($isPaid) {
            $transaction->update(['status' => 'EXPIRED']);

            return false;
        }

        $transaction->update(['status' => 'FAILED']);

        return false;
    }
}
```

SuccessTransactionState: app/Services/Payment/State/SuccessTransactionState.php (1–18)

```
final class SuccessTransactionState implements TransactionState
{
    public function applyCallback(Transaction $transaction, bool $isPaid, bool $notExpired, Closure $onPaid): bool
    {
        return $isPaid;
    }
}
```


SettledTransactionState: app/Services/Payment/State/SettledTransactionState.php
(1–18)

```
final class SettledTransactionState implements TransactionState
{
    public function applyCallback(Transaction $transaction, bool $isPaid, bool $notExpired, Closure $onPaid): bool
    {
        return false;
    }
}
```

TransactionStateFactory: app/Services/Payment/State/TransactionStateFactory.php
(1–20)

```
final class TransactionStateFactory
{
    public function for(Transaction $transaction): TransactionState
    {
        return match ($transaction->status) {
            'PENDING' => new PendingTransactionState,
            'SUCCESS' => new SuccessTransactionState,
            default => new SettledTransactionState,
        };
    }
}
```

PaymentReturnProcessor: app/Services/Payment/PaymentReturnProcessor.php (1–52)

```
final class PaymentReturnProcessor
{
    public function __construct(
        private readonly PaymentProviderFactory $paymentProviderFactory,
        private readonly TransactionStateFactory $stateFactory,
        private readonly ListingService $listingService,
    ) {}

    public function handleReturn(Request $request): PaymentReturnResult
    {
        $gateway = $this->paymentProviderFactory->for('VNPAY');
        $result = $gateway->verifyCallback($request);

        $transactionId = $gateway->transactionIdFromReference($result->reference);
        $transaction = $transactionId ? Transaction::find($transactionId) : null;

        $isCompleted = false;

        if ($transaction) {
            $transaction->update($result->gatewayFields);

            $isPaid = $result->isValidSignature
                && $result->responseCode === '00'
                && $result->transactionStatus === '00';
            $notExpired = ! $transaction->expires_at || $transaction->expires_at->isFuture();

            $isCompleted = $this->stateFactory->for($transaction)->applyCallback(
                $transaction,
                $isPaid,
                $notExpired,
                fn (Transaction $t) => $this->listingService->completePaidUpgrade($t),
            );
        }

        return new PaymentReturnResult($isCompleted, $transaction?->id);
    }
}
```

Controller nhận callback:

app/Http/Controllers/Api/V1/Payment/VnpayReturnController.php (hàm __invoke, dòng 15)

```
public function __invoke(Request $request): Response
{
    $result = $this->paymentReturnProcessor->handleReturn($request);

    $status = $result->isCompleted ? 'success' : 'failed';
    $frontendUrl = rtrim((string) config('app.frontend_url'), '/');
    $redirectUrl = $frontendUrl.'/payment/vnpay-result?status='.$status.'&transaction_id='.$( $result->transactionId ?? '' );

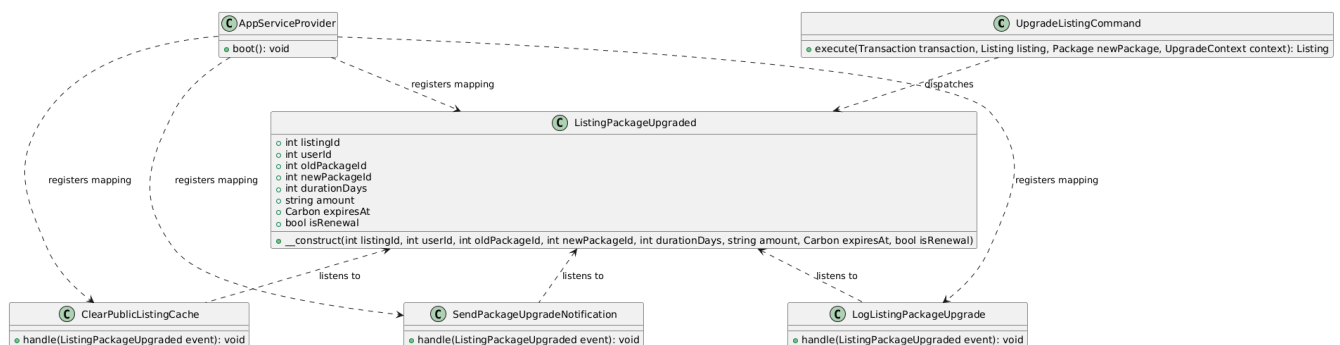
    return $this->redirectResponse($result->isCompleted, $redirectUrl);
}
```

c. Observer Pattern

- Vấn đề cần giải quyết:

- Khi nâng cấp gói hoàn tất cần xóa cache, ghi log, gửi thông báo. → phát sự kiện ListingPackageUpgraded.

- Sơ đồ class



Thành phần	Lớp cụ thể trong dự án	Tác dụng
Event	App\Events\Listing\ListingPackageUpgraded	Sự kiện nâng cấp gói thành công.
Listener	Listeners\Listing\{ClearPublicListingCache, LogListingPackageUpgrade, SendPackageUpgradeNotification}	Xử lý phụ.

- Code triển khai

Đăng ký listener: app/Providers/AppServiceProvider.php (dòng 225–227)

```
225 Event::listen(ListingPackageUpgraded::class, ClearPublicListingCache::class);
226 Event::listen(ListingPackageUpgraded::class, LogListingPackageUpgrade::class);
227 Event::listen(ListingPackageUpgraded::class, SendPackageUpgradeNotification::class);
```

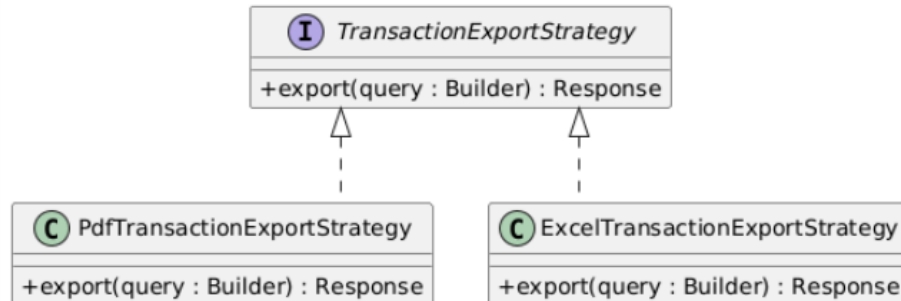
3.2.3.13. Xem báo cáo và xuất dữ liệu

- Chức năng đã sử dụng các Design Patterns: Strategy Pattern, Factory Pattern

b. Strategy Pattern

- Vấn đề cần giải quyết:

- Tránh sử dụng nhiều câu lệnh if-else hoặc switch-case để xử lý từng loại báo cáo. Dễ mở rộng khi thêm loại báo cáo mới mà không ảnh hưởng đến mã nguồn hiện có (tuân thủ Open/Closed Principle).
- Sơ đồ class



Thành phần	Lớp cụ thể trong dự án	Tác dụng
Strategy	TransactionExportStrategy	Mọi class xuất file bắt buộc phải có hàm export() để thực thi xuất file, và hàm format() để khai báo mình đại diện cho định dạng nào.
ConcreteStrategy A	PdfExportStrategy	Xuất file dưới định dạng PDF
ConcreteStrategy B	ExcelExportStrategy	Xuất file dưới định dạng Excel

- Code triển khai

TransactionExportStrategy

```

namespace App\Services\Payment\Export;

use Illuminate\Database\Eloquent\Builder;
use Symfony\Component\HttpFoundation\Response;

interface TransactionExportStrategy
{
    /**
     * Xuất báo cáo giao dịch dựa trên câu truy vấn đã lọc.
     */
    public function export(Builder $query): Response;
}
  
```

PdfExportStrategy

```

final class PdfTransactionExportStrategy implements TransactionExportStrategy
{
    public function export(Builder $query): Response
    {
        if (!class_exists(\Barryvdh\DomPDF\Facade\Pdf::class)) {
            throw new BusinessException(
                ErrorCode::BadRequest,
                "Chức năng xuất PDF yêu cầu thư viện DomPDF. Vui lòng chạy lệnh 'composer require barryvdh/laravel-dompdf'"
            );
        }
        $transactions = $query->get();
        $pdf = \Barryvdh\DomPDF\Facade\Pdf::loadView('admin.exports.transactions_pdf', compact('transactions'));
        $pdf->setPaper('a4', 'landscape');

        return new Response($pdf->output(), 200, [
            'Content-Type' => 'application/pdf',
            'Content-Disposition' => 'attachment; filename="bao_cao_giao_dich_" . date('Ymd_His') . '.pdf',
        ]);
    }
}

```

ExcelExportStrategy

```

final class ExcelTransactionExportStrategy implements TransactionExportStrategy
{
    public function export(Builder $query): Response
    {
        $transactions = $query->get();

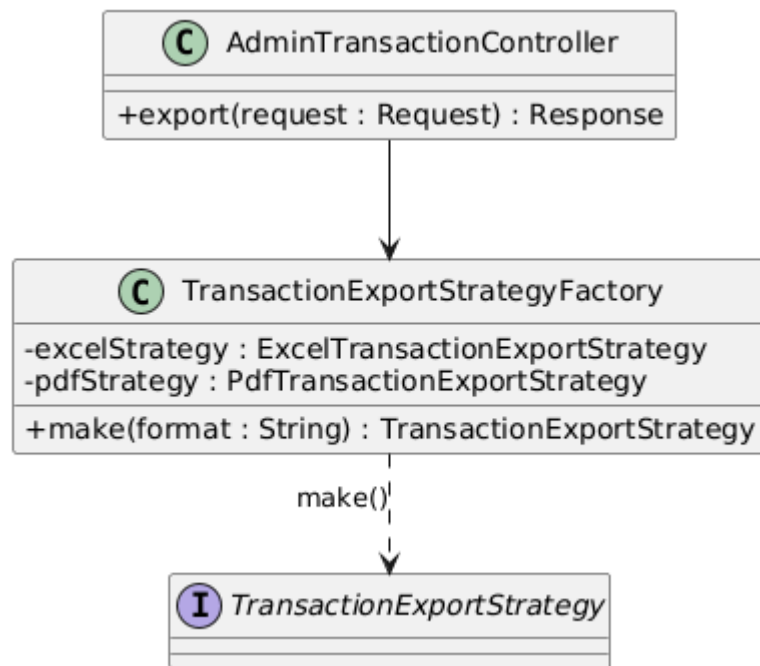
        $html = view('admin.exports.transactions_excel', compact('transactions'))->render();

        return new Response($html, 200, [
            'Content-Type' => 'application/vnd.ms-excel; charset=utf-8',
            'Content-Disposition' => 'attachment; filename="bao_cao_giao_dich_" . date('Ymd_His') . '.xls"',
            'Cache-Control' => 'no-cache, must-revalidate',
            'Pragma' => 'no-cache',
            'Expires' => '0',
        ]);
    }
}

```

c. *Strategy Pattern*

- **Vấn đề cần giải quyết:**
 - Làm thế nào để biết cách khởi tạo Strategy nào tại runtime mà không cần phải viết code cứng new
 - Client không nên biết chi tiết cấu tạo bên trong hoặc các tham số khởi dựng phức tạp của từng Strategy cụ thể
- **Sơ đồ class**



Thành phần	Lớp cụ thể trong dự án	Tác dụng
Client	AdminTransactionController	Gọi Factory để lấy đúng Strategy xuất file (dựa theo format) thay vì dùng if/else thủ công.
Creator / Factory	TransactionExportStrategyFactory	Xử lý logic khởi tạo: Nhận chuỗi format ('pdf', 'excel') và sinh ra/trả về Strategy tương ứng.
Strategy	TransactionExportStrategy	Chuẩn hóa kiểu dữ liệu đầu ra của Factory, giúp Client gọi hàm export() mà không cần biết đó là PDF hay Excel.

- Code triển khai

AdminTransactionController

```

public function export(Request $request): Response
{
    $validated = $request->validate([
        'format' => 'required|string|in:excel,pdf,xlsx,xls',
        'status' => 'nullable|string|in:PENDING,SUCCESS,FAILED,EXPIRED',
        'payment_method' => 'nullable|string|max:50',
        'package_id' => 'nullable|integer|exists:packages,id',
        'from_date' => 'nullable|date',
        'to_date' => 'nullable|date|after_or_equal:from_date',
        'min_amount' => 'nullable|numeric|min:0',
        'max_amount' => 'nullable|numeric|min:0|gte:min_amount',
        'search' => 'nullable|string|max:255',
    ]);

    $query = Transaction::query()
        ->with(['user', 'listing', 'package', 'notes.admin'])
        ->latest('transaction_date');

    $query = $this->applyFilters($query, $validated);

    $strategy = $this->exportFactory->make($validated['format']);

    return $strategy->export($query);
}

```

TransactionExportStrategyFactory

```

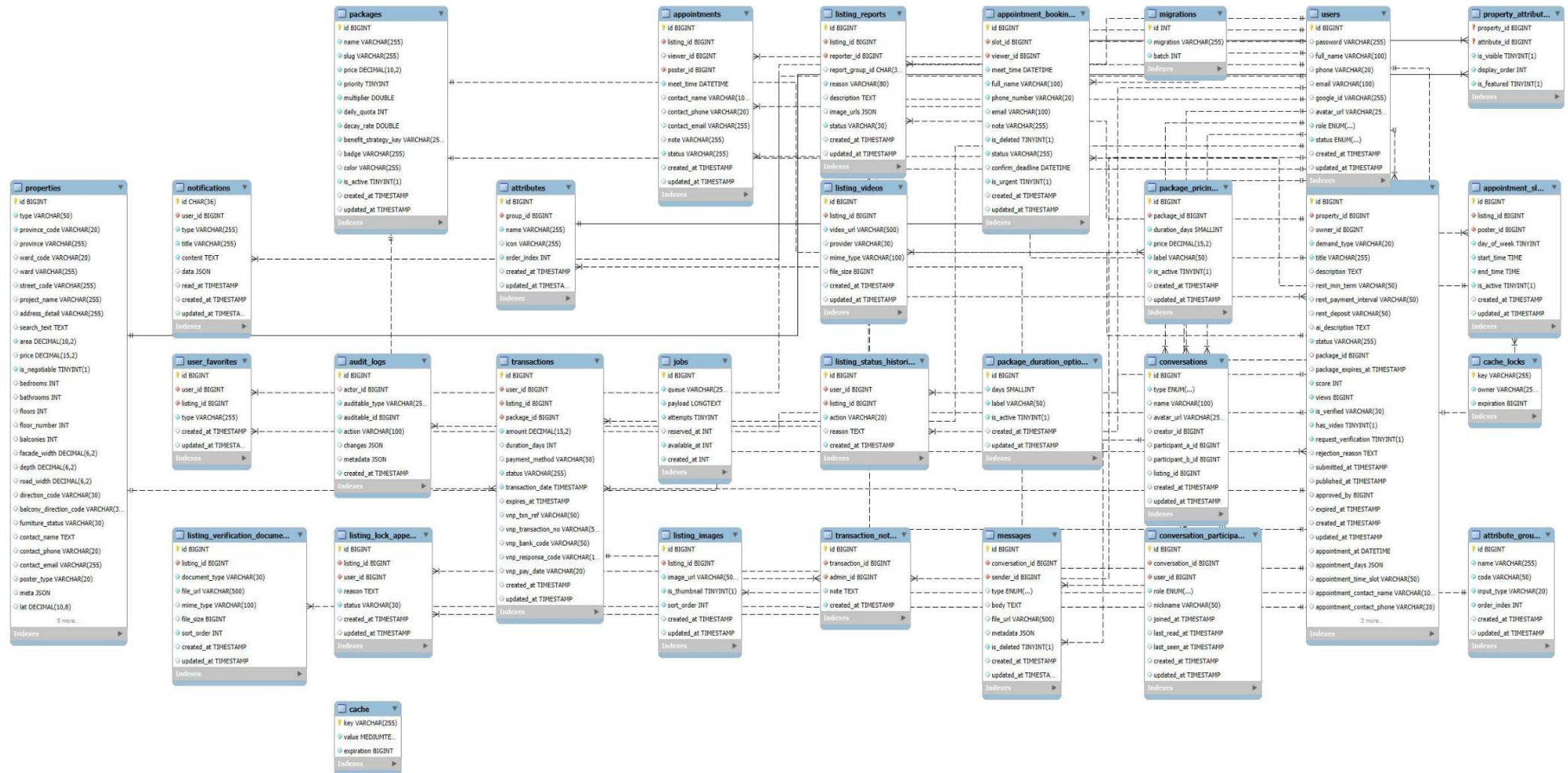
final class TransactionExportStrategyFactory
{
    public function __construct(
        private readonly ExcelTransactionExportStrategy $excelStrategy,
        private readonly PdfTransactionExportStrategy $pdfStrategy,
    ) {}

    public function make(string $format): TransactionExportStrategy
    {
        return match (strtolower($format)) {
            'excel', 'xls', 'xlsx' => $this->excelStrategy,
            'pdf' => $this->pdfStrategy,
            default => throw new BusinessException(
                ErrorCode::BadRequest,
                "Định dạng xuất file {$format} không được hỗ trợ. Chỉ hỗ trợ excel hoặc pdf."
            ),
        };
    }
}

```

3.3. Thiết kế cơ sở dữ liệu (ERD, bảng, mối quan hệ)

3.3.1. Sơ đồ ERD



Hình 42: Sơ đồ ERD

3.3.2. Danh sách bảng chính

Tên bảng	Mô tả
Attribute_groups	Lưu thông tin các nhóm thuộc tính của bất động sản (ví dụ: Thông tin chung, Tiện ích, Nội thất...).
Attributes	Lưu danh sách các thuộc tính thuộc từng nhóm thuộc tính để mô tả bất động sản.
Properties	Lưu thông tin chi tiết của các bất động sản do người dùng tạo trên hệ thống.
Property_attributes	Lưu giá trị các thuộc tính được gán cho từng bất động sản.
Users	Lưu thông tin tài khoản người dùng, bao gồm chủ sở hữu, khách hàng và quản trị viên.
Listings	Lưu thông tin các tin đăng bất động sản được hiển thị trên hệ thống.
Listing_images	Lưu danh sách hình ảnh minh họa của từng tin đăng.
Listing_videos	Lưu video giới thiệu hoặc video thực tế của tin đăng bất động sản.
Listing_reports	Lưu các báo cáo, phản ánh của người dùng đối với tin đăng vi phạm hoặc có dấu hiệu bất thường.
Listing_lock_appeals	Lưu thông tin khiếu nại của người dùng đối với các tin đăng bị khóa.
Listing_status_histories	Lưu lịch sử thay đổi trạng thái của các tin đăng trong quá trình kiểm duyệt và hoạt động.
Listing_verification_documents	Lưu các giấy tờ, tài liệu xác minh quyền sở hữu hoặc tính hợp pháp của tin đăng.
Packages	Lưu thông tin các gói dịch vụ đăng tin mà hệ thống cung cấp.
Package_pricings	Lưu bảng giá tương ứng với từng gói dịch vụ và từng hình thức thanh toán.
Package_duration_options	Lưu các tùy chọn thời hạn sử dụng của từng gói dịch vụ.
Transactions	Lưu thông tin các giao dịch thanh toán khi người dùng mua gói dịch vụ hoặc thực hiện các khoản thanh toán khác.
Transaction_notes	Lưu ghi chú hoặc nội dung xử lý liên quan đến từng giao dịch thanh toán.
Appointments	Lưu thông tin các lịch hẹn xem bất động sản giữa khách hàng và chủ sở hữu.

Appointment_slots	Lưu các khung thời gian có thể đặt lịch xem bất động sản.
Appointment_bookings	Lưu thông tin đặt lịch cụ thể của khách hàng đối với từng khung giờ xem nhà.
Conversations	Lưu thông tin các cuộc hội thoại giữa người dùng trên hệ thống.
Conversation_participants	Lưu danh sách những người tham gia trong từng cuộc hội thoại.
Messages	Lưu nội dung tin nhắn được trao đổi trong từng cuộc hội thoại.
Notifications	Lưu các thông báo được gửi đến người dùng từ hệ thống hoặc các sự kiện phát sinh.
User_favorites	Lưu danh sách các bất động sản hoặc tin đăng được người dùng đánh dấu yêu thích.
Audit_logs	Lưu nhật ký hoạt động của người dùng và hệ thống nhằm phục vụ kiểm tra và truy vết.
Cache	Lưu dữ liệu bộ nhớ đệm nhằm tăng tốc độ truy xuất của hệ thống.
Cache_locks	Lưu thông tin khóa bộ nhớ đệm để tránh xung đột khi nhiều tiến trình cùng truy cập dữ liệu.
Jobs	Lưu danh sách các tác vụ xử lý bất đồng bộ (Queue Jobs) đang chờ hoặc đang được thực hiện.
Migrations	Lưu lịch sử các lần cập nhật cấu trúc cơ sở dữ liệu thông qua Migration.

3.3.3. Các mối quan hệ chính

Bảng cha	Bảng con	Quan hệ	Ý nghĩa
Attribute_groups	Attributes	1 - N	Một nhóm thuộc tính có thể chứa nhiều thuộc tính.
Users	Audit_logs	1 - N	Một người dùng có thể phát sinh nhiều bản ghi nhật ký hoạt động.
Users	Conversations	1 - N	Một người dùng có thể là người tạo nhiều cuộc hội thoại.
Users	Conversations	1 - N	Một người dùng có thể là người tham gia A trong nhiều cuộc hội thoại.

Users	Conversations	1 - N	Một người dùng có thể là người tham gia B trong nhiều cuộc hội thoại.
Users	Listings	1 - N	Một quản trị viên có thể phê duyệt nhiều tin đăng.
Users	Listings	1 - N	Một người dùng có thể sở hữu nhiều tin đăng bất động sản.
Packages	Listings	1 - N	Một gói dịch vụ có thể được áp dụng cho nhiều tin đăng.
Properties	Listings	1 - N	Một bất động sản có thể được tạo thành nhiều tin đăng.
Conversations	Messages	1 - N	Một cuộc hội thoại bao gồm nhiều tin nhắn.
Users	Messages	1 - N	Một người dùng có thể gửi nhiều tin nhắn.
Users	Notifications	1 - N	Một người dùng có thể nhận nhiều thông báo từ hệ thống.
Packages	Package_pricings	1 - N	Một gói dịch vụ có nhiều mức giá hoặc phương án thanh toán.
Attributes	Property_attributes	1 - N	Một thuộc tính có thể được sử dụng cho nhiều bất động sản.
Properties	Property_attributes	1 - N	Một bất động sản có nhiều thuộc tính mô tả.
Listings	Transactions	1 - N	Một tin đăng có thể phát sinh nhiều giao dịch thanh toán.
Packages	Transactions	1 - N	Một gói dịch vụ có thể được thanh toán nhiều lần.
Users	Transactions	1 - N	Một người dùng có thể thực hiện nhiều giao dịch.
Listings	User_favorites	1 - N	Một tin đăng có thể được nhiều người dùng thêm vào danh sách yêu thích.
Users	User_favorites	1 - N	Một người dùng có thể lưu nhiều tin đăng yêu thích.
Listings	Appointment_slots	1 - N	Một tin đăng có nhiều khung giờ xem nhà.
Users	Appointment_slots	1 - N	Chủ tin có thể tạo nhiều khung giờ xem nhà.

Listings	Appointments	1 - N	Một tin đăng có thể có nhiều lịch hẹn xem nhà.
Users	Appointments	1 - N	Chủ tin có thể tiếp nhận nhiều lịch hẹn.
Users	Appointments	1 - N	Một khách hàng có thể đặt nhiều lịch xem nhà.
Conversations	Conversation_participants	1 - N	Một cuộc hội thoại có nhiều thành viên tham gia.
Users	Conversation_participants	1 - N	Một người dùng có thể tham gia nhiều cuộc hội thoại.
Listings	Listing_images	1 - N	Một tin đăng có thể có nhiều hình ảnh minh họa.
Listings	Listing_lock_appeals	1 - N	Một tin đăng có thể có nhiều lần khiếu nại khi bị khóa.
Users	Listing_lock_appeals	1 - N	Một người dùng có thể gửi nhiều đơn khiếu nại về tin đăng.
Listings	Listing_reports	1 - N	Một tin đăng có thể bị báo cáo nhiều lần bởi nhiều người dùng.
Users	Listing_reports	1 - N	Một người dùng có thể gửi nhiều báo cáo vi phạm.
Listings	Listing_status_histories	1 - N	Một tin đăng có thể có nhiều lần thay đổi trạng thái trong quá trình hoạt động.
Users	Listing_status_histories	1 - N	Một quản trị viên hoặc nhân viên có thể cập nhật trạng thái của nhiều tin đăng.
Listings	Listing_verification_documents	1 - N	Một tin đăng có thể có nhiều tài liệu xác minh.
Listings	Listing_videos	1 - N	Một tin đăng có thể có nhiều video giới thiệu.
Users	Transaction_notes	1 - N	Một quản trị viên có thể tạo nhiều ghi chú xử lý giao dịch.
Transactions	Transaction_notes	1 - N	Một giao dịch có thể có nhiều ghi chú xử lý.
Appointment_slots	Appointment_bookings	1 - N	Một khung giờ xem nhà có thể được đặt bởi nhiều khách hàng (theo quy tắc của hệ thống).

Users	Appointment_bookings	1 - N	Một khách hàng có thể đặt nhiều lịch xem nhà tại các khung giờ khác nhau.
-------	----------------------	-------	---

Bảng A	Bảng B	Bảng trung gian	Ý nghĩa
Properties	Attributes	Property_attributes	Một bất động sản có nhiều thuộc tính và một thuộc tính có thể áp dụng cho nhiều bất động sản.
Users	Listings	User_favorites	Một người dùng có thể yêu thích nhiều tin đăng và một tin đăng có thể được nhiều người dùng yêu thích.

3.3.4. Chi tiết bảng dữ liệu

Bảng USERS - Người dùng

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT UNSIGNED	PK, Auto Increment	Mã người dùng
password	VARCHAR(255)	NULL	Mật khẩu
full_name	VARCHAR(100)	NULL	Họ và tên
phone	VARCHAR(20)	NULL	Số điện thoại
email	VARCHAR(100)	NULL	Địa chỉ email
google_id	VARCHAR(255)	NULL	Mã tài khoản Google
avatar_url	VARCHAR(255)	NULL	Ảnh đại diện
role	ENUM('USER','ADMIN')	NOT NULL, DEFAULT 'USER'	Vai trò
status	ENUM('A','P','IA','BAN')	NOT NULL, DEFAULT 'A'	Trạng thái tài khoản
created_at	TIMESTAMP	NULL	Thời gian tạo
updated_at	TIMESTAMP	NULL	Thời gian cập nhật

Bảng USER_FAVORITES – Danh sách tin đăng yêu thích

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT UNSIGNED	PK, Auto Increment	Mã bản ghi yêu thích

user_id	BIGINT UNSIGNED	FK → USERS(id), NOT NULL	Người dùng
listing_id	BIGINT UNSIGNED	FK → LISTINGS(id), NOT NULL	Tin đăng
type	VARCHAR(255)	NOT NULL, Default 'FAVORITE'	Loại bản ghi (Yêu thích hoặc Đã xem)
created_at	TIMESTAMP	NULL	Thời gian tạo
updated_at	TIMESTAMP	NULL	Thời gian cập nhật

Bảng PROPERTIES – Bất động sản

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT UNSIGNED	PK, Auto Increment	Mã bất động sản
type	VARCHAR(50)	NOT NULL	Loại bất động sản
province_code	VARCHAR(20)	NOT NULL	Mã tỉnh/thành phố
province	VARCHAR(255)	NULL	Tên tỉnh/thành phố
ward_code	VARCHAR(20)	NULL	Mã phường/xã
ward	VARCHAR(255)	NULL	Tên phường/xã
street_code	VARCHAR(255)	NULL	Mã tuyến đường
project_name	VARCHAR(255)	NULL	Tên dự án
address_detail	VARCHAR(255)	NULL	Địa chỉ chi tiết
search_text	TEXT	NULL	Chuỗi hỗ trợ tìm kiếm Full-text
area	DECIMAL(10,2)	NOT NULL	Diện tích (m ²)
price	DECIMAL(15,2)	NULL	Giá bán hoặc giá cho thuê
is_negotiable	TINYINT(1)	NOT NULL, Default 0	Cho phép thương lượng giá
bedrooms	INT	NULL	Số phòng ngủ
bathrooms	INT	NULL	Số phòng tắm
floors	INT	NULL	Tổng số tầng
floor_number	INT	NULL	Tầng của căn hộ
balconies	INT	NULL	Số lượng ban công
facade_width	DECIMAL(6,2)	NULL	Chiều rộng mặt tiền (m)

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
depth	DECIMAL(6,2)	NULL	Chiều sâu (m)
road_width	DECIMAL(6,2)	NULL	Chiều rộng đường trước nhà (m)
direction_code	VARCHAR(30)	NULL	Hướng của bất động sản
balcony_direction_code	VARCHAR(30)	NULL	Hướng ban công
furniture_status	VARCHAR(30)	NULL	Tình trạng nội thất
contact_name	TEXT	NULL	Tên người liên hệ
contact_phone	VARCHAR(20)	NULL	Số điện thoại liên hệ
contact_email	VARCHAR(255)	NULL	Email liên hệ
poster_type	VARCHAR(20)	NULL	Loại người đăng tin (Chủ sở hữu/Môi giới)
meta	JSON	NULL	Thông tin mở rộng của bất động sản
lat	DECIMAL(10,8)	NULL	Vĩ độ
lng	DECIMAL(11,8)	NULL	Kinh độ
created_at	TIMESTAMP	NULL	Thời gian tạo
updated_at	TIMESTAMP	NULL	Thời gian cập nhật
amenities	JSON	NULL	Danh sách tiện ích
legal_paper_types	JSON	NULL	Danh sách giấy tờ pháp lý
public_info_agreed	TINYINT(1)	NOT NULL, Default 0	Xác nhận đồng ý công khai thông tin

Bảng PROPERTY_ATTRIBUTES

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT UNSIGNED	PK, Auto Increment	Mã thuộc tính bất động sản
property_id	BIGINT UNSIGNED	FK → PROPERTIES(id), NOT NULL	Mã bất động sản

attribute_id	BIGINT UNSIGNED	FK → ATTRIBUTES(id), NOT NULL	Mã thuộc tính
created_at	TIMESTAMP	NULL	Thời gian tạo
updated_at	TIMESTAMP	NULL	Thời gian cập nhật

Bảng LISTINGS – Tin đăng

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT UNSIGNED	PK, Auto Increment	Mã tin đăng
property_id	BIGINT UNSIGNED	FK → PROPERTIES(id) , NOT NULL	Mã bất động sản
owner_id	BIGINT UNSIGNED	FK → USERS(id)	Người đăng tin
demand_type	VARCHAR(20)	NOT NULL	Loại giao dịch (SALE/RENT)
title	VARCHAR(255)	NOT NULL	Tiêu đề tin đăng
description	TEXT	NULL	Nội dung mô tả
rent_min_term	VARCHAR(50)	NULL	Thời gian thuê tối thiểu
rent_payment_interval	VARCHAR(50)	NULL	Chu kỳ thanh toán tiền thuê
rent_deposit	VARCHAR(50)	NULL	Mức tiền đặt cọc
ai_description	TEXT	NULL	Mô tả do AI sinh
status	VARCHAR(255)	NOT NULL, Default 'DRAFT'	Trạng thái tin đăng
package_id	BIGINT UNSIGNED	FK → PACKAGES(id)	Gói tin đăng
package_expires_at	TIMESTAMP	NULL	Thời điểm hết hạn gói

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
score	INT	NOT NULL, Default 0	Điểm đánh giá xếp hạng
views	BIGINT UNSIGNED	NOT NULL, Default 0	Lượt xem
is_verified	VARCHAR(30)	NOT NULL	Trạng thái xác minh
has_video	TINYINT(1)	NOT NULL, Default 0	Có video hay không
request_verification	TINYINT(1)	NOT NULL, Default 0	Yêu cầu xác minh
rejection_reason	TEXT	NULL	Lý do từ chối
submitted_at	TIMESTAMP	NULL	Thời gian gửi duyệt
published_at	TIMESTAMP	NULL	Thời gian đăng công khai
approved_by	BIGINT UNSIGNED	FK → USERS(id)	Quản trị viên phê duyệt
expired_at	TIMESTAMP	NULL	Thời gian hết hạn
created_at	TIMESTAMP	NULL	Thời gian tạo
updated_at	TIMESTAMP	NULL	Thời gian cập nhật
appointment_at	DATETIME	NULL	Thời gian hẹn xem mặc định
appointment_days	JSON	NULL	Danh sách ngày hỗ trợ xem nhà
appointment_time_slot	VARCHAR(50)	NULL	Khung giờ xem nhà
appointment_contact_name	VARCHAR(100)	NULL	Người liên hệ đặt lịch
appointment_contact_phone	VARCHAR(20)	NULL	Số điện thoại liên hệ
appointment_contact_email	VARCHAR(255)	NULL	Email liên hệ
appointment_note	VARCHAR(255)	NULL	Ghi chú đặt lịch

Bảng LISTING_IMAGES – Hình ảnh tin đăng

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT UNSIGNED	PK, Auto Increment	Mã hình ảnh
listing_id	BIGINT UNSIGNED	FK → LISTINGS(id), NOT NULL	Tin đăng sở hữu ảnh
image_url	VARCHAR(500)	NOT NULL	Đường dẫn hình ảnh
is_thumbnail	TINYINT(1)	NOT NULL, Default 0	Đánh dấu ảnh đại diện
sort_order	INT	NOT NULL, Default 0	Thứ tự hiển thị
created_at	TIMESTAMP	NULL	Thời gian tạo
updated_at	TIMESTAMP	NULL	Thời gian cập nhật

Bảng LISTING_VIDEOS – Video tin đăng

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT UNSIGNED	PK, Auto Increment	Mã video
listing_id	BIGINT UNSIGNED	FK → LISTINGS(id), NOT NULL	Tin đăng chứa video
video_url	VARCHAR(500)	NOT NULL	Đường dẫn video
provider	VARCHAR(30)	NULL	Nguồn video (LOCAL, YOUTUBE, VIMEO)
mime_type	VARCHAR(100)	NULL	Kiểu MIME của video
file_size	BIGINT UNSIGNED	NULL	Kích thước tệp video (Byte)
created_at	TIMESTAMP	NULL	Thời gian tạo
updated_at	TIMESTAMP	NULL	Thời gian cập nhật

Bảng LISTING_REPORTS – Báo cáo tin đăng

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT UNSIGNED	PK, Auto Increment	Mã báo cáo
listing_id	BIGINT UNSIGNED	FK → LISTINGS(id), NOT NULL	Tin đăng bị báo cáo
reporter_id	BIGINT UNSIGNED	FK → USERS(id), NOT NULL	Người gửi báo cáo
report_group_id	CHAR(36)	NULL	Mã nhóm báo cáo để gom các báo cáo cùng nội dung
reason	VARCHAR(80)	NOT NULL	Lý do báo cáo (Sai giá, Sai địa chỉ, Tin trùng,...)
description	TEXT	NULL	Nội dung mô tả chi tiết của báo cáo
image_urls	JSON	NULL	Danh sách hình ảnh minh chứng
status	VARCHAR(30)	NOT NULL, Default 'WARNING'	Trạng thái xử lý báo cáo
created_at	TIMESTAMP	NULL	Thời gian tạo báo cáo
updated_at	TIMESTAMP	NULL	Thời gian cập nhật

Bảng LISTING_STATUS_HISTORIES – Lịch sử thay đổi trạng thái tin đăng

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT UNSIGNED	PK, Auto Increment	Mã lịch sử
user_id	BIGINT UNSIGNED	FK → USERS(id), NOT NULL	Người thực hiện thay đổi trạng thái
listing_id	BIGINT UNSIGNED	FK → LISTINGS(id), NOT NULL	Tin đăng được thay đổi
action	VARCHAR(20)	NOT NULL	Hành động thực hiện (ACTIVE,

			REJECTED, LOCKED,...)
reason	TEXT	NULL	Lý do thay đổi trạng thái
created_at	TIMESTAMP	NOT NULL	Thời gian thực hiện

Bảng LISTING_VERIFICATION_DOCUMENTS – Tài liệu xác minh tin đăng

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT UNSIGNED	PK, Auto Increment	Mã tài liệu xác minh
listing_id	BIGINT UNSIGNED	FK → LISTINGS(id), NOT NULL	Tin đăng cần xác minh
document_type	VARCHAR(30)	NOT NULL	Loại tài liệu (CCCD mặt trước, CCCD mặt sau, Giấy tờ pháp lý...)
file_url	VARCHAR(500)	NOT NULL	Đường dẫn tệp
mime_type	VARCHAR(100)	NULL	Kiểu MIME của tệp
file_size	BIGINT UNSIGNED	NULL	Kích thước tệp (Byte)
sort_order	INT	NOT NULL, Default 0	Thứ tự hiển thị
created_at	TIMESTAMP	NULL	Thời gian tải lên
updated_at	TIMESTAMP	NULL	Thời gian cập nhật

Bảng LISTING_LOCK_APPEALS – Khiếu nại khóa tin đăng

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT UNSIGNED	PK, Auto Increment	Mã khiếu nại
listing_id	BIGINT UNSIGNED	FK → LISTINGS(id), NOT NULL	Tin đăng bị khóa
user_id	BIGINT UNSIGNED	FK → USERS(id), NOT NULL	Người gửi khiếu nại
reason	TEXT	NOT NULL	Nội dung khiếu nại

status	VARCHAR(30)	NOT NULL, Default 'PENDING'	Trạng thái xử lý khiếu nại
created_at	TIMESTAMP	NULL	Thời gian gửi khiếu nại
updated_at	TIMESTAMP	NULL	Thời gian cập nhật

Bảng PACKAGES – Gói đăng tin

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT UNSIGNED	PK, Auto Increment	Mã gói đăng tin
name	VARCHAR(255)	NOT NULL	Tên gói đăng tin
slug	VARCHAR(255)	NOT NULL, UNIQUE	Mã định danh duy nhất của gói
price	DECIMAL(10,2)	NOT NULL, Default 0.00	Giá của gói
priority	TINYINT UNSIGNED	NOT NULL, Default 1	Mức độ ưu tiên hiển thị
multiplier	DOUBLE	NOT NULL, Default 1	Hệ số tăng điểm hiển thị
daily_quota	INT UNSIGNED	NOT NULL, Default 100	Số lượt hiển thị mỗi ngày
decay_rate	DOUBLE	NOT NULL, Default 0.05	Tỷ lệ giảm điểm hiển thị theo thời gian
benefit_strategy_key	VARCHAR(255)	NOT NULL	Chiến lược áp dụng quyền lợi của gói
badge	VARCHAR(255)	NULL	Huy hiệu hiển thị của gói
color	VARCHAR(255)	NULL	Màu đại diện của gói
is_active	TINYINT(1)	NOT NULL, Default 1	Trạng thái hoạt động của gói
created_at	TIMESTAMP	NULL	Thời gian tạo
updated_at	TIMESTAMP	NULL	Thời gian cập nhật

Bảng PACKAGE_PRICINGS – Bảng giá gói đăng tin

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT UNSIGNED	PK, Auto Increment	Mã bảng giá
package_id	BIGINT UNSIGNED	FK → PACKAGES(id), NOT NULL	Gói đăng tin
duration_days	SMALLINT UNSIGNED	NOT NULL	Thời hạn sử dụng (ngày)
price	DECIMAL(15,2)	NOT NULL	Giá tương ứng với thời hạn
label	VARCHAR(50)	NOT NULL	Nhãn hiển thị (Ví dụ: 7 ngày, 1 tháng)
is_active	TINYINT(1)	NOT NULL, Default 1	Trạng thái áp dụng
created_at	TIMESTAMP	NULL	Thời gian tạo
updated_at	TIMESTAMP	NULL	Thời gian cập nhật

Bảng PACKAGE_DURATION_OPTIONS – Danh sách thời hạn gói

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT UNSIGNED	PK, Auto Increment	Mã thời hạn
days	SMALLINT UNSIGNED	NOT NULL, UNIQUE	Số ngày sử dụng
label	VARCHAR(50)	NOT NULL	Tên thời hạn hiển thị
is_active	TINYINT(1)	NOT NULL, Default 1	Trạng thái sử dụng
created_at	TIMESTAMP	NULL	Thời gian tạo
updated_at	TIMESTAMP	NULL	Thời gian cập nhật

Bảng TRANSACTIONS – Giao dịch thanh toán

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT UNSIGNED	PK, Auto Increment	Mã giao dịch
user_id	BIGINT UNSIGNED	FK → USERS(id), NOT NULL	Người thực hiện thanh toán
listing_id	BIGINT UNSIGNED	FK → LISTINGS(id), NOT NULL	Tin đăng được thanh toán
package_id	BIGINT UNSIGNED	FK → PACKAGES(id), NOT NULL	Gói đã mua
amount	DECIMAL(15,2)	NOT NULL	Số tiền thanh toán
duration_days	INT UNSIGNED	NULL	Số ngày hiệu lực
payment_method	VARCHAR(50)	NULL	Phương thức thanh toán
status	VARCHAR(255)	NOT NULL, Default 'PENDING'	Trạng thái giao dịch
transaction_date	TIMESTAMP	NOT NULL	Thời điểm giao dịch
expires_at	TIMESTAMP	NULL	Thời điểm hết hiệu lực của gói
vnp_txn_ref	VARCHAR(50)	NULL	Mã giao dịch tại VNPay
vnp_transaction_no	VARCHAR(50)	NULL	Số giao dịch VNPay
vnp_bank_code	VARCHAR(50)	NULL	Mã ngân hàng thanh toán
vnp_response_code	VARCHAR(10)	NULL	Mã phản hồi từ VNPay
vnp_pay_date	VARCHAR(20)	NULL	Thời điểm thanh toán VNPay
created_at	TIMESTAMP	NULL	Thời gian tạo
updated_at	TIMESTAMP	NULL	Thời gian cập nhật

Bảng TRANSACTION_NOTES – Ghi chú giao dịch

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT UNSIGNED	PK, Auto Increment	Mã ghi chú
transaction_id	BIGINT UNSIGNED	FK → TRANSACTIONS(id), NOT NULL	Giao dịch được ghi chú
admin_id	BIGINT UNSIGNED	FK → USERS(id), NOT NULL	Quản trị viên tạo ghi chú
note	TEXT	NOT NULL	Nội dung ghi chú
created_at	TIMESTAMP	NOT NULL	Thời gian tạo ghi chú

Bảng CONVERSATIONS – Cuộc trò chuyện

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT UNSIGNED	PK, Auto Increment	Mã cuộc hội thoại
type	ENUM('private', 'group')	NOT NULL, Default 'private'	Loại cuộc hội thoại (Cá nhân hoặc Nhóm)
name	VARCHAR(100)	NULL	Tên cuộc hội thoại (đối với nhóm)
avatar_url	VARCHAR(255)	NULL	Ảnh đại diện cuộc hội thoại
creator_id	BIGINT UNSIGNED	FK → USERS(id)	Người tạo cuộc hội thoại
participant_a_id	BIGINT UNSIGNED	FK → USERS(id)	Người tham gia thứ nhất
participant_b_id	BIGINT UNSIGNED	FK → USERS(id)	Người tham gia thứ hai
listing_id	BIGINT UNSIGNED	NULL	Tin đăng liên quan đến cuộc hội thoại
created_at	TIMESTAMP	NULL	Thời gian tạo cuộc hội thoại

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
updated_at	TIMESTAMP	NULL	Thời gian cập nhật gần nhất

Bảng CONVERSATION_PARTICIPANTS – Thành viên cuộc trò chuyện

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT UNSIGNED	PK, Auto Increment	Mã thành viên
conversation_id	BIGINT UNSIGNED	FK → CONVERSATIONS(id), NOT NULL	Cuộc hội thoại
user_id	BIGINT UNSIGNED	FK → USERS(id), NOT NULL	Người tham gia
role	ENUM('admin', 'member')	NOT NULL, Default 'member'	Vai trò trong cuộc hội thoại
nickname	VARCHAR(50)	NULL	Biệt danh trong cuộc hội thoại
joined_at	TIMESTAMP	NULL	Thời điểm tham gia
last_read_at	TIMESTAMP	NULL	Thời điểm đọc tin nhắn gần nhất
last_seen_at	TIMESTAMP	NULL	Thời điểm hoạt động gần nhất
created_at	TIMESTAMP	NULL	Thời gian tạo bản ghi
updated_at	TIMESTAMP	NULL	Thời gian cập nhật

Bảng MESSAGES – Tin nhắn

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT UNSIGNED	PK, Auto Increment	Mã tin nhắn
conversation_id	BIGINT UNSIGNED	FK → CONVERSATIONS(id), NOT NULL	Cuộc hội thoại chứa tin nhắn

sender_id	BIGINT UNSIGNED	FK → USERS(id), NOT NULL	Người gửi tin nhắn
type	ENUM('text','image','file','system')	Default 'text'	Loại tin nhắn
body	TEXT	NULL	Nội dung tin nhắn
file_url	VARCHAR(500)	NULL	Đường dẫn tệp hoặc hình ảnh đính kèm
metadata	JSON	NULL	Dữ liệu mở rộng của tin nhắn
is_deleted	TINYINT(1)	NOT NULL, Default 0	Trạng thái xóa mềm của tin nhắn
created_at	TIMESTAMP	NULL	Thời gian gửi tin nhắn
updated_at	TIMESTAMP	NULL	Thời gian chỉnh sửa tin nhắn

Bảng APPOINTMENTS – Lịch hẹn xem bất động sản

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT UNSIGNED	PK, Auto Increment	Mã lịch hẹn
listing_id	BIGINT UNSIGNED	FK → LISTINGS(id), NOT NULL	Tin đăng được đặt lịch xem
viewer_id	BIGINT UNSIGNED	FK → USERS(id)	Người xem bất động sản
poster_id	BIGINT UNSIGNED	FK → USERS(id), NOT NULL	Người đăng tin
meet_time	DATETIME	NOT NULL	Thời gian hẹn xem bất động sản
contact_name	VARCHAR(100)	NULL	Họ và tên người liên hệ
contact_phone	VARCHAR(20)	NULL	Số điện thoại liên hệ

contact_email	VARCHAR(255))	NULL	Email liên hệ
note	VARCHAR(255))	NULL	Ghi chú của người đặt lịch
status	VARCHAR(255))	NOT NULL, Default 'PENDING'	Trạng thái lịch hẹn (PENDING, CONFIRMED, DECLINED, CANCELLED, COMPLETED)
created_at	TIMESTAMP	NULL	Thời gian tạo lịch hẹn
updated_at	TIMESTAMP	NULL	Thời gian cập nhật lịch hẹn

Bảng APPOINTMENT_BOOKINGS – Phiếu đặt lịch xem bất động sản

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT UNSIGNED	PK, Auto Increment	Mã phiếu đặt lịch
slot_id	BIGINT UNSIGNED	FK → APPOINTMENT_SLOTS(id), NOT NULL	Khung giờ được đặt
viewer_id	BIGINT UNSIGNED	FK → USERS(id), NOT NULL	Người đặt lịch xem
meet_time	DATETIME	NOT NULL	Thời gian xem nhà cụ thể được chọn
full_name	VARCHAR(100)	NOT NULL	Họ và tên người đặt lịch
phone_number	VARCHAR(20)	NOT NULL	Số điện thoại người đặt lịch
email	VARCHAR(100)	NULL	Email người đặt lịch

note	VARCHAR(255)	NULL	Ghi chú bổ sung
is_deleted	TINYINT(1)	NOT NULL, Default 0	Trạng thái xóa mềm
status	VARCHAR(255)	NOT NULL, Default 'PENDING'	Trạng thái xử lý phiếu đặt lịch
confirm_deadline	DATETIME	NULL	Thời hạn người đăng tin phải xác nhận lịch hẹn
is_urgent	TINYINT(1)	NOT NULL, Default 0	Đánh dấu lịch hẹn gấp (thời gian đặt sát giờ hẹn)
created_at	TIMESTAMP	NULL	Thời gian tạo
updated_at	TIMESTAMP	NULL	Thời gian cập nhật

Bảng APPOINTMENT_SLOTS – Khung giờ xem bất động sản

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT UNSIGNED	PK, Auto Increment	Mã khung giờ
listing_id	BIGINT UNSIGNED	FK → LISTINGS(id), NOT NULL	Tin đăng áp dụng khung giờ
poster_id	BIGINT UNSIGNED	FK → USERS(id), NOT NULL	Người đăng tin thiết lập khung giờ
day_of_week	TINYINT	NOT NULL	Thứ trong tuần (1 = Thứ Hai, ..., 7 = Chủ Nhật)
start_time	TIME	NOT NULL	Giờ bắt đầu
end_time	TIME	NOT NULL	Giờ kết thúc

is_active	TINYINT(1)	NOT NULL, Default 1	Trạng thái hoạt động của khung giờ (1: Hoạt động, 0: Xóa mềm)
created_at	TIMESTAMP	NULL	Thời gian tạo
updated_at	TIMESTAMP	NULL	Thời gian cập nhật

Bảng NOTIFICATIONS – Thông báo

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	CHAR(36)	PK	Mã định danh thông báo (UUID)
user_id	BIGINT UNSIGNED	FK → USERS(id), NOT NULL	Người nhận thông báo
type	VARCHAR(255)	NOT NULL	Loại thông báo
title	VARCHAR(255)	NOT NULL	Tiêu đề thông báo
content	TEXT	NOT NULL	Nội dung thông báo
data	JSON	NULL	Dữ liệu bổ sung của thông báo
read_at	TIMESTAMP	NULL	Thời điểm người dùng đã đọc thông báo
created_at	TIMESTAMP	NULL	Thời gian tạo thông báo
updated_at	TIMESTAMP	NULL	Thời gian cập nhật

Bảng AUDIT_LOGS – Nhật ký hệ thống

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT UNSIGNED	PK, Auto Increment	Mã nhật ký
actor_id	BIGINT UNSIGNED	FK → USERS(id)	Người thực hiện thao tác
auditable_type	VARCHAR(255)	NOT NULL	Loại đối tượng được tác động

auditable_id	BIGINT UNSIGNED	NOT NULL	Mã đối tượng được tác động
action	VARCHAR(100)	NOT NULL	Hành động được thực hiện
changes	JSON	NULL	Danh sách dữ liệu thay đổi trước và sau
metadata	JSON	NULL	Dữ liệu bổ sung của thao tác
created_at	TIMESTAMP	NOT NULL	Thời gian ghi nhận nhật ký

Bảng ATTRIBUTES – Thuộc tính

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT UNSIGNED	PK, Auto Increment	Mã thuộc tính
attribute_group_id	BIGINT UNSIGNED	FK → ATTRIBUTE_GROUPS(id), NOT NULL	Mã nhóm thuộc tính
name	VARCHAR(255)	NOT NULL	Tên thuộc tính
code	VARCHAR(50)	NOT NULL	Mã thuộc tính
icon	VARCHAR(255)	NULL	Biểu tượng
order_index	INT	NOT NULL, DEFAULT 0	Thứ tự hiển thị
created_at	TIMESTAMP	NULL	Thời gian tạo
updated_at	TIMESTAMP	NULL	Thời gian cập nhật

Bảng ATTRIBUTE_GROUPS – Nhóm thuộc tính

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT UNSIGNED	PK, Auto Increment	Mã nhóm thuộc tính

name	VARCHAR(255)	NOT NULL	Tên nhóm thuộc tính
code	VARCHAR(50)	UNIQUE, NOT NULL	Mã nhóm thuộc tính
input_type	VARCHAR(20)	NOT NULL, DEFAULT 'checkbox'	Kiểu nhập liệu
order_index	INT	NOT NULL, DEFAULT 0	Thứ tự hiển thị
created_at	TIMESTAMP	NULL	Thời gian tạo
updated_at	TIMESTAMP	NULL	Thời gian cập nhật

Bảng CACHE_LOCKS – Khóa bộ nhớ đệm

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
key	VARCHAR(255)	PK	Khóa định danh của cache
owner	VARCHAR(255)	NOT NULL	Định danh tiến trình đang giữ khóa
expiration	BIGINT	NOT NULL	Thời điểm hết hiệu lực của khóa

Bảng JOBS – Hàng đợi tác vụ

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT UNSIGNED	PK, Auto Increment	Mã tác vụ
queue	VARCHAR(255)	NOT NULL	Tên hàng đợi (Queue)
payload	LONGTEXT	NOT NULL	Dữ liệu của tác vụ được lưu dưới dạng JSON
attempts	TINYINT UNSIGNED	NOT NULL	Số lần hệ thống đã thử thực hiện tác vụ

reserved_at	INT UNSIGNED	NULL	Thời điểm worker nhận xử lý tác vụ (Unix Timestamp)
available_at	INT UNSIGNED	NOT NULL	Thời điểm tác vụ được phép xử lý
created_at	INT UNSIGNED	NOT NULL	Thời điểm tạo tác vụ

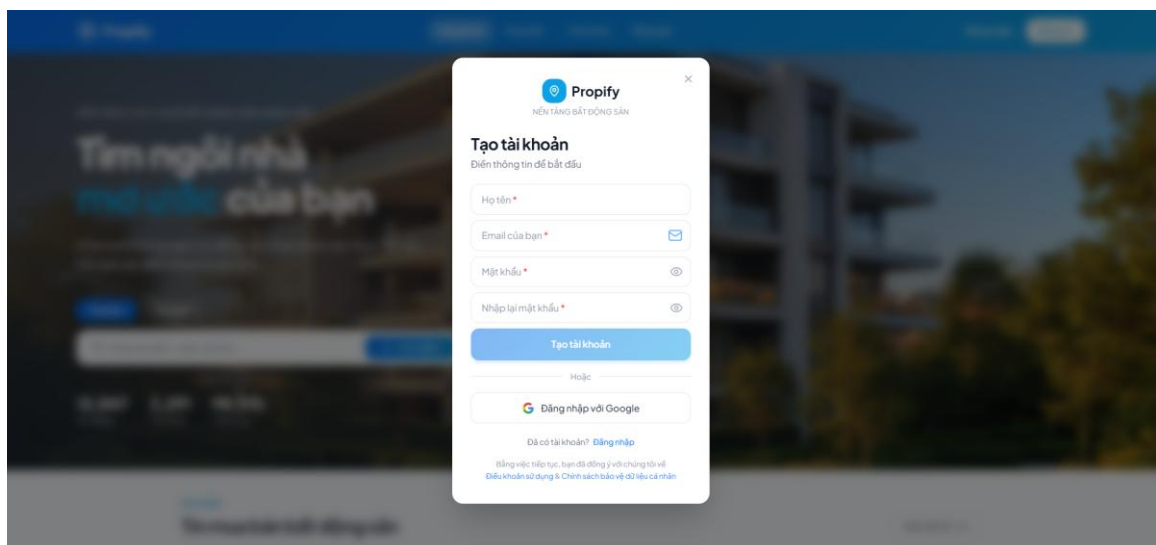
Bảng MIGRATIONS – Lịch sử Migration

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	INT UNSIGNED	PK, Auto Increment	Mã migration
migration	VARCHAR(255)	NOT NULL	Tên file migration đã thực thi
batch	INT	NOT NULL	Số thứ tự của lần thực hiện migration

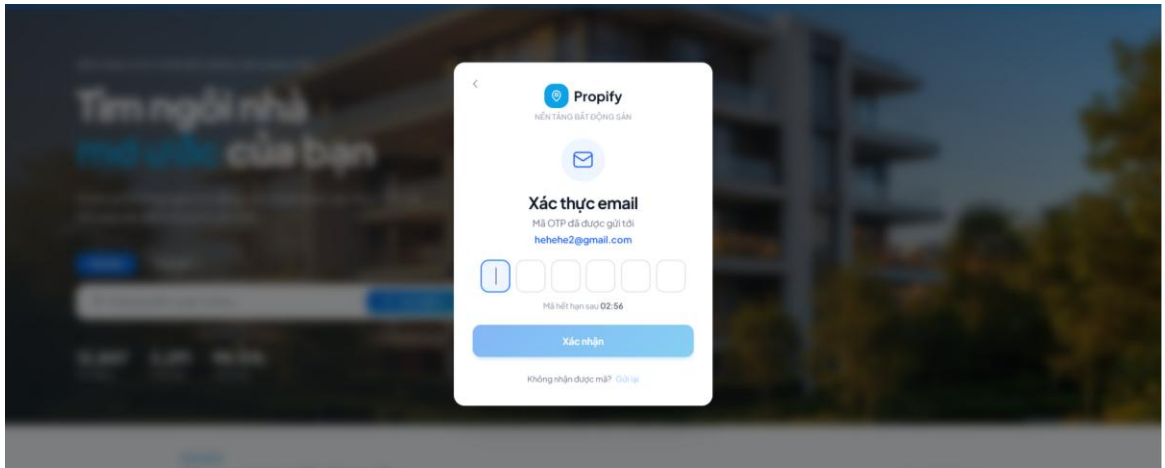
3.4. Thiết kế giao diện người dùng (Mockup / Wireframe)

3.4.1. Màn hình Đăng ký

Form đăng ký gồm các trường: Họ tên (2-100 ký tự), Email (validate realtime), Mật khẩu (≥ 8 ký tự, bao gồm chữ hoa, chữ thường và số, có nút show/hide), Xác nhận mật khẩu. Nút Đăng ký được disable trong khi xử lý với hiệu ứng loading. Sau khi submit thành công, hiển thị màn hình nhập OTP 6 ô với countdown 60 giây và nút Gửi lại.



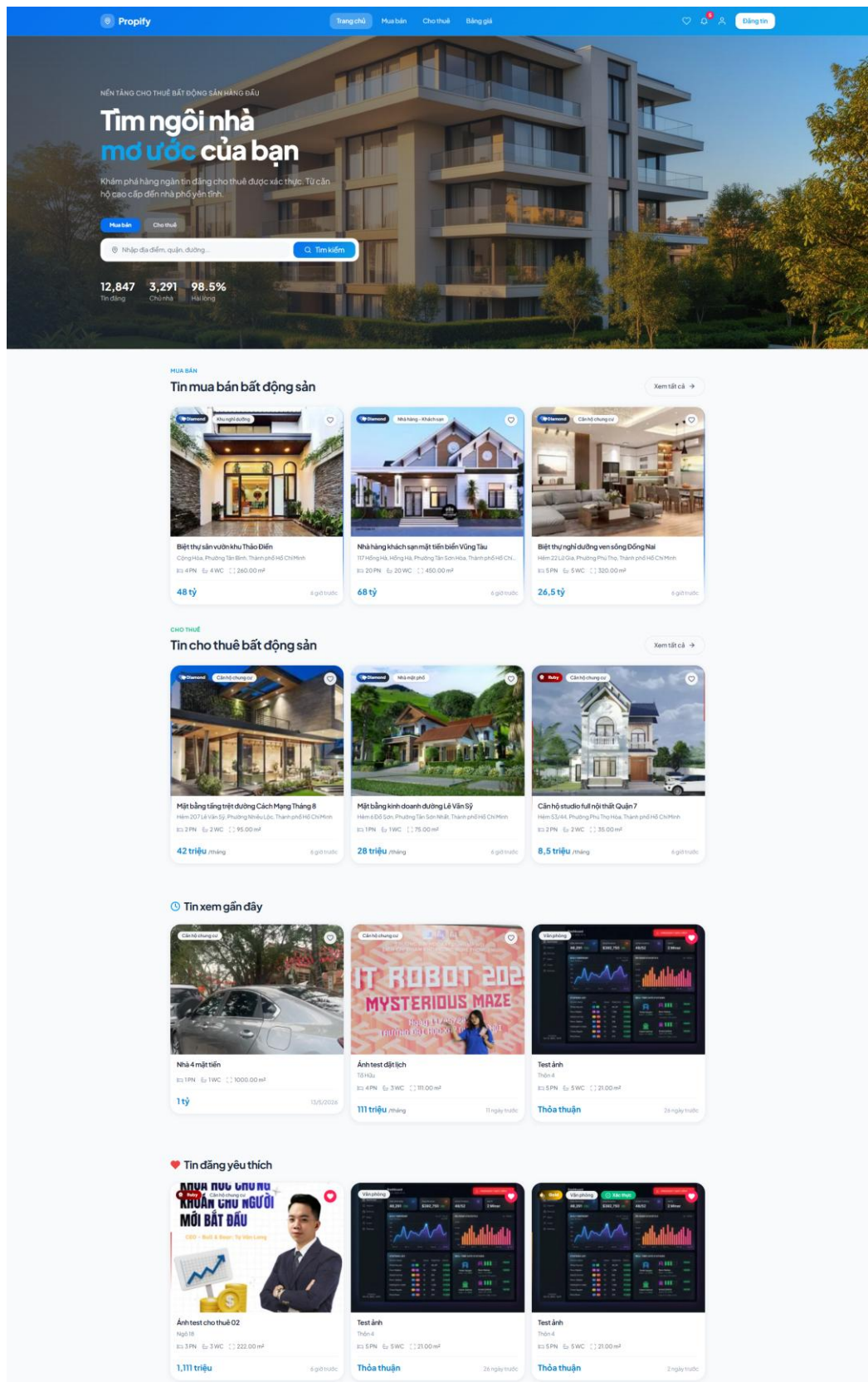
Hình 43: Màn hình trang Đăng ký



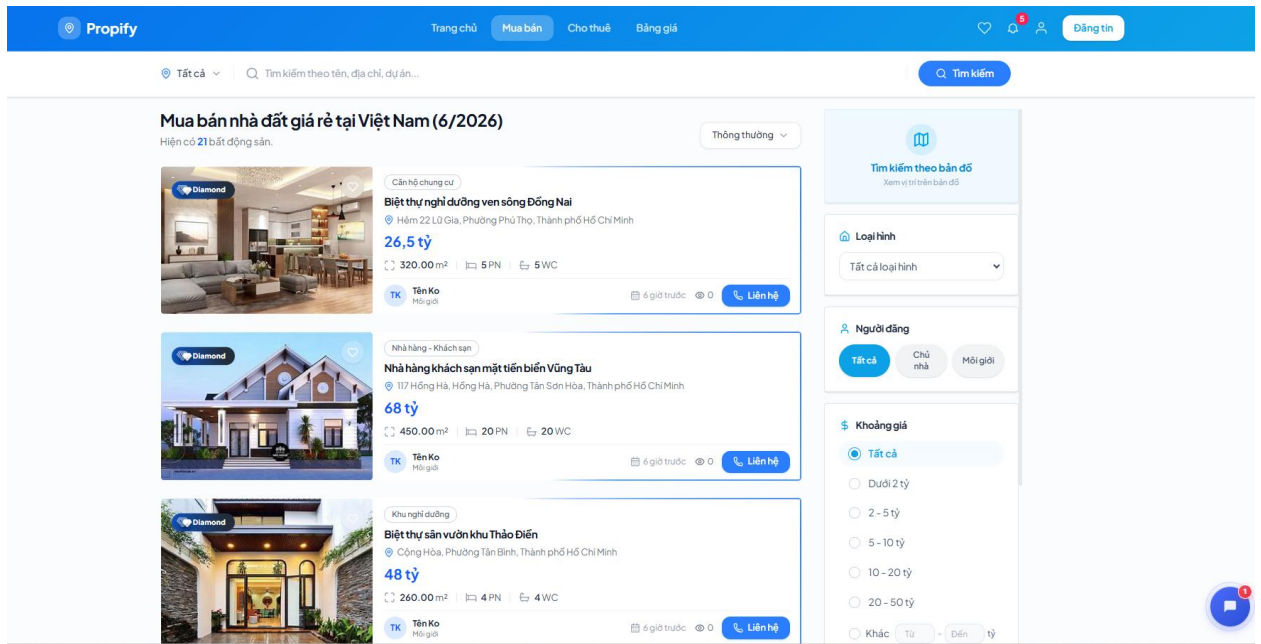
Hình 44: Màn hình trang nhập OTP

3.4.2. Trang chủ và Tìm kiếm

Trang chủ gồm: thanh tìm kiếm thông minh (địa danh, tên tin đăng), bộ lọc nhanh (loại hình, khoảng giá, diện tích), danh sách tin. Thứ tự ưu tiên hiển thị: gói trả phí cao nhất → tin xác thực → thời gian đăng mới nhất.



Hình 45: Màn hình Trang chủ



Hình 46: Màn hình trang Mua bán/Cho thuê

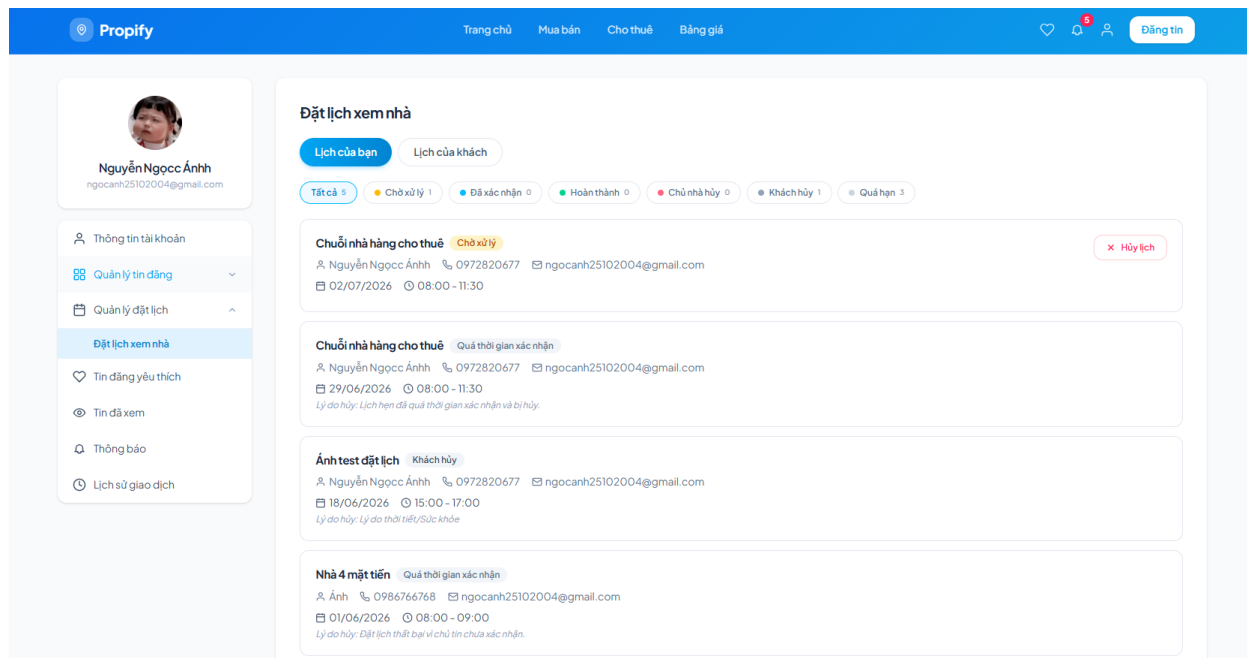
3.4.3. Màn hình Tạo/sửa tin đăng

Form tạo tin đăng được chia thành các section: (1) Media (upload ảnh tối thiểu 1, tối đa 10; video tùy chọn MP4 $\leq 100\text{MB}$); (2) Thông tin cơ bản (nhu cầu, tiêu đề, mô tả, loại nhà đất, giá, diện tích); (3) Địa chỉ (tỉnh/thành $\rightarrow$ quận/huyện $\rightarrow$ phường/xã $\rightarrow$ đường phố, địa chỉ chi tiết); (4) Thông tin chi tiết BĐS (số phòng, hướng nhà, tiện ích); (5) Thông tin liên hệ và giấy tờ pháp lý.

The screenshot shows the 'Đăng tin bất động sản' form on the Propify website. The form is divided into several sections: 1. Media upload: 'Hình ảnh, Video' section with a '+ Kéo thả tối thiểu 1 ảnh vào đây hoặc Chọn tệp ảnh' button and a 'Video (tối đa 1 video MP4)' section with a '+ Tải video MP4 từ máy tính' button. 2. Basic information: 'Thông tin bất động sản' section with 'Nhu cầu của bạn' (Mua bán, Cho thuê), 'Tên bất động sản', 'Cấu trúc tiêu đề nên có' (Loại căn hộ + Diện tích + Số PN + Vị trí), 'Mô tả', 'Loại nhà đất', 'Diện tích (m²)', 'Giá bán', and 'Giá thuê'. 3. Address: 'Địa chỉ' section with 'Tỉnh/thành phố', 'Quận/huyện', 'Phường/xã', 'Đường/phố', and 'Địa chỉ cụ thể'. 4. Detailed property information: 'Chi tiết điểm thông tin' section with 'Hình ảnh và video' (0/4 đ), 'Thông tin BĐS' (0,2/2 đ), and 'Chi tiết BĐS' (0,1/1 đ). 5. Contact and legal documents: 'Liên hệ' section with 'Số điện thoại', 'Email', and 'Địa chỉ email'.

3.4.4. Màn hình quản lý lịch hẹn

Giao diện chia thành 2 tab: 'Lịch của tôi' (các lịch hẹn tôi đã đặt với tư cách khách) và 'Lịch của khách' (các yêu cầu xem nhà từ khách hàng). Mỗi thẻ lịch hẹn hiển thị: thông tin căn hộ, tên và SĐT người liên hệ, ngày giờ hẹn, trạng thái (badge màu) và các nút hành động phù hợp với trạng thái hiện tại.



Hình 48: Màn hình trang Quản lý lịch hẹn

3.4.5. Màn hình quản lý tin đăng – Danh sách tin đăng

Giao diện gồm thanh tìm kiếm theo từ khóa, bộ lọc loại tin và các tab trạng thái (Tất cả, Chờ duyệt, Tin đang đăng, Tin nháp, Từ chối, Tin bị khóa, Đã gỡ) kèm số lượng tin tương ứng. Danh sách tin đăng được hiển thị dưới dạng bảng gồm các thông tin: ID, ảnh đại diện, mã tin đăng, tiêu đề, ngày tạo, ngày đăng, địa chỉ, gói tin và trạng thái. Trạng thái được thể hiện bằng nhãn màu để người dùng dễ nhận biết. Mỗi tin đăng có menu thao tác (⋮) cho phép thực hiện các chức năng như xem chi tiết, chỉnh sửa, gia hạn hoặc gỡ tin tùy theo trạng thái.

ID	Ảnh	Mã tin đăng	Tin đăng	Ngày tạo	Ngày đăng	Địa chỉ	Giá	Gói tin	Trạng thái
44		LH-00000044	Mặt bằng tầng trệt đường Cách Mạng Tháng 8	30/6/2026	30/6/2026	Hẻm 207 Lê Văn Sỹ, Phường Nhiều Lộc, Thành phố Hồ Chí Minh	42.000.000	Diamond	Đang đăng
43		LH-00000043	Biệt thự nghỉ dưỡng ven sông Đồng Nai	30/6/2026	30/6/2026	Hẻm 22 Lũ Gia, Phường Phú Thọ, Thành phố Hồ Chí Minh	26.500.000	Diamond	Đang đăng
42		LH-00000042	Nhà hàng khách sạn mặt tiền biển Vũng Tàu	30/6/2026	30/6/2026	117 Hồng Hà, Hồng Hà, Phường Tân Sơn Hòa, Thành phố Hồ Chí Minh	68.000.000	Diamond	Đang đăng
41		LH-00000041	Biệt thự sân vườn khu Thảo Điền	30/6/2026	30/6/2026	Cộng Hòa, Phường Tân Bình, Thành phố Hồ Chí Minh	48.000.000	Diamond	Đang đăng
40		LH-00000040	Căn hộ studio full nội thất Quận 7	30/6/2026	30/6/2026	Hẻm 53/44, Phường Phú Thọ Hòa, Thành phố Hồ Chí Minh	8.500.000	Ruby	Đang đăng
39		LH-00000039	Mặt bằng kinh doanh đường Lê Văn Sỹ	30/6/2026	30/6/2026	Hẻm 6 Đỗ Sơn, Phường Tân Sơn Nhất, Thành phố Hồ Chí Minh	28.000.000	Diamond	Đang đăng

Hình 49: Màn hình trang Quản lý tin đăng – Danh sách tin đăng

3.4.6. Màn hình quản lý tin đăng – Danh sách xác thực tin đăng

Giao diện hiển thị danh sách các tin đăng cần xác thực dưới dạng bảng, hỗ trợ tìm kiếm theo từ khóa, lọc theo loại tin và trạng thái (Tất cả, Chờ duyệt, Tin đang đăng, Tin nháp, Từ chối, Tin bị khóa, Đã gỡ). Mỗi tin đăng hiển thị các thông tin gồm: ID, ảnh đại diện, mã tin đăng, tiêu đề, ngày tạo, ngày đăng, địa chỉ, trạng thái xác thực, gói tin và trạng thái đăng tin. Cột **Xác thực** hiển thị tình trạng xác thực của từng tin (Chưa xác thực, Đã xác thực...) và mỗi tin đăng có menu thao tác để thực hiện chức năng gửi hoặc xem thông tin xác thực.

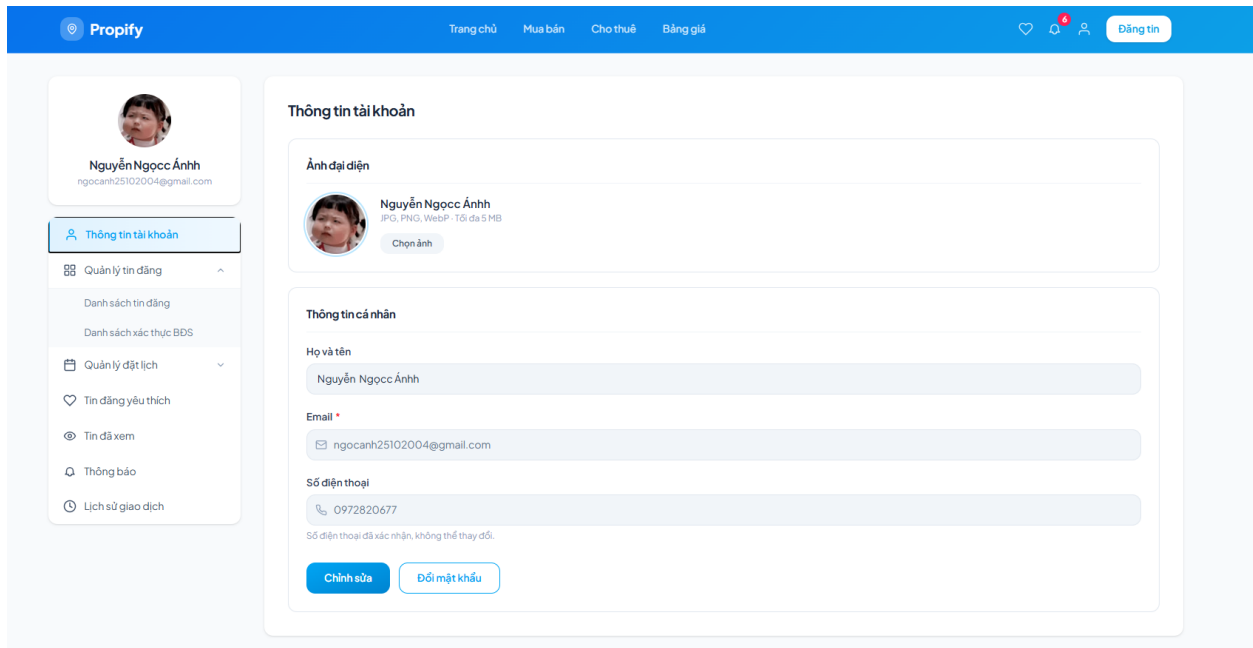
ID	Ảnh	Mã tin đăng	Tin đăng	Ngày tạo	Ngày đăng	Địa chỉ	Xác thực	Gói tin	Trạng thái
44		LH-00000044	Mặt bằng tầng trệt đường Cách Mạng Tháng 8	30/6/2026	30/6/2026	Hẻm 207 Lê Văn Sỹ, Phường Nhiều Lộc, Thành phố Hồ Chí Minh	Không cần thiết	Diamond	Đang đăng
43		LH-00000043	Biệt thự nghỉ dưỡng ven sông Đồng Nai	30/6/2026	30/6/2026	Hẻm 22 Lũ Gia, Phường Phú Thọ, Thành phố Hồ Chí Minh	Chưa xác thực	Diamond	Đang đăng
42		LH-00000042	Nhà hàng khách sạn mặt tiền biển Vũng Tàu	30/6/2026	30/6/2026	117 Hồng Hà, Hồng Hà, Phường Tân Sơn Hòa, Thành phố Hồ Chí Minh	Chưa xác thực	Diamond	Đang đăng
41		LH-00000041	Biệt thự sân vườn khu Thảo Điền	30/6/2026	30/6/2026	Cộng Hòa, Phường Tân Bình, Thành phố Hồ Chí Minh	Chưa xác thực	Diamond	Đang đăng
40		LH-00000040	Căn hộ studio full nội thất Quận 7	30/6/2026	30/6/2026	Hẻm 53/44, Phường Phú Thọ Hòa, Thành phố Hồ Chí Minh	Không cần thiết	Ruby	Đang đăng
39		LH-00000039	Mặt bằng kinh doanh đường Lê Văn Sỹ	30/6/2026	30/6/2026	Hẻm 6 Đỗ Sơn, Phường Tân Sơn Nhất, Thành phố Hồ Chí Minh	Không cần thiết	Diamond	Đang đăng

Hình 50: Màn hình trang Quản lý tin đăng – Danh sách xác thực tin đăng

3.4.7. Màn hình quản lý tài khoản

Giao diện hiển thị ảnh đại diện và thông tin cá nhân của người dùng, bao gồm họ

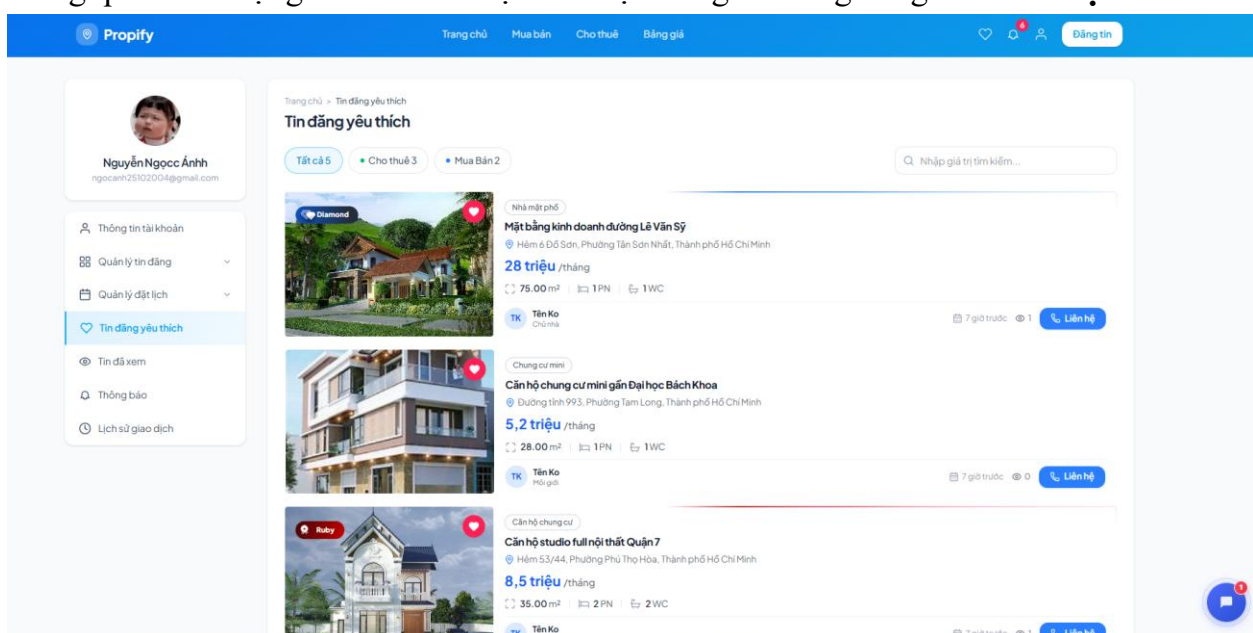
và tên, email và số điện thoại. Giao diện hỗ trợ cập nhật ảnh đại diện (JPG, PNG, WebP, tối đa 5 MB) và chỉnh sửa các thông tin được phép thay đổi. Email và số điện thoại đã xác thực được hiển thị dưới dạng chỉ đọc. Màn hình cung cấp các chức năng **Chỉnh sửa**, **Cập nhật mật khẩu** và **Lưu thay đổi** sau khi người dùng cập nhật thông tin.



Hình 51: Màn hình trang Quản lý tài khoản

3.4.8. Màn hình Tin đăng yêu thích

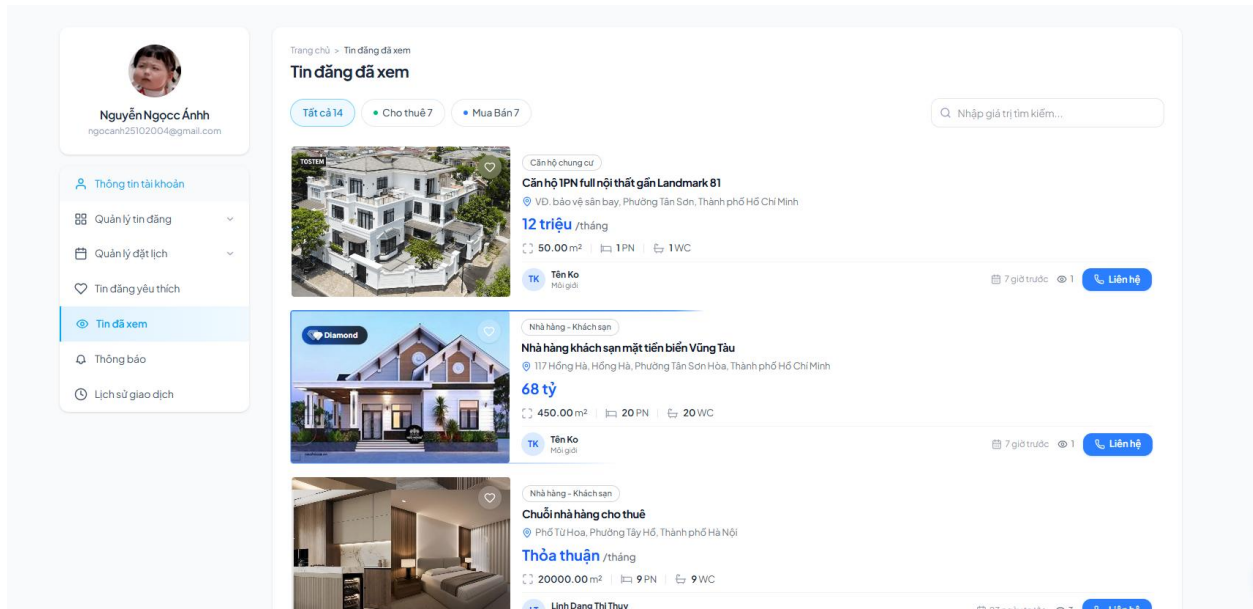
Giao diện hiển thị danh sách các tin đăng mà người dùng đã lưu, hỗ trợ tìm kiếm theo từ khóa và lọc theo loại tin (Tất cả, Mua bán, Cho thuê). Mỗi tin đăng hiển thị các thông tin gồm: hình ảnh, loại bất động sản, tiêu đề, địa chỉ, giá, diện tích, số phòng ngủ, số phòng vệ sinh, thông tin người đăng, ngày đăng và lượt xem. Người dùng có thể bỏ lưu tin thông qua biểu tượng **Yêu thích** hoặc liên hệ với người đăng bằng nút **Liên hệ**.



Hình 52: Màn hình trang Tin đăng yêu thích

3.4.9. Màn hình Tin đăng đã xem

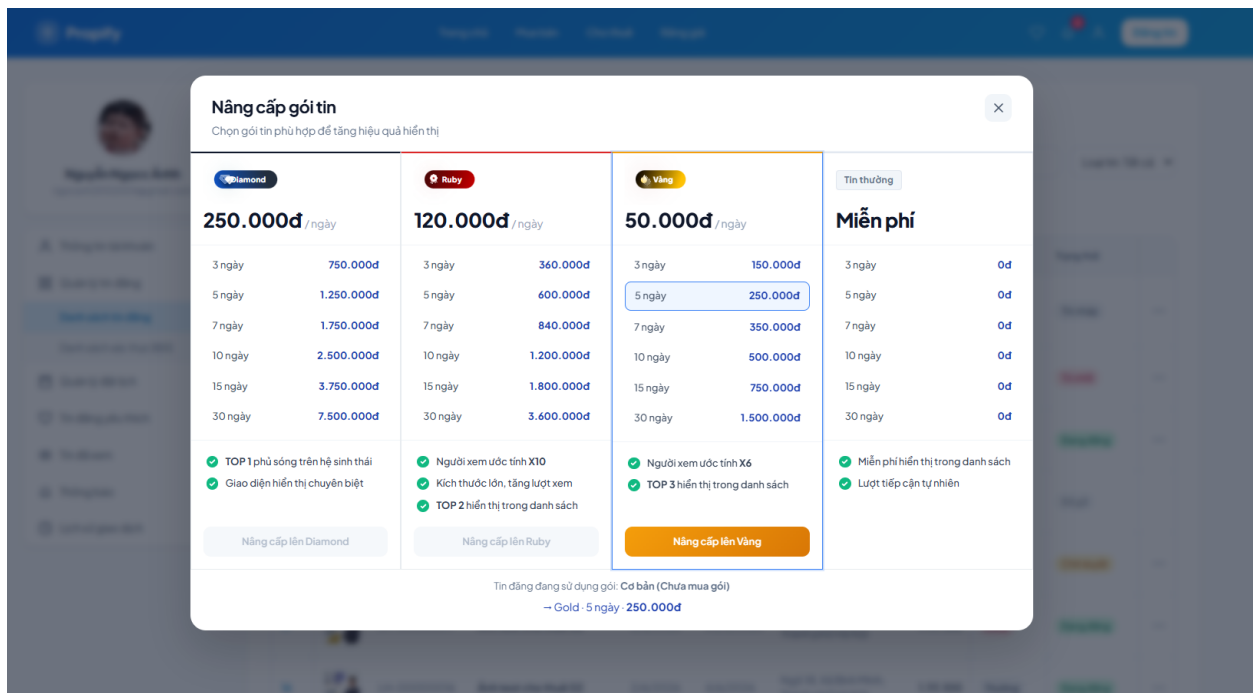
Giao diện hiển thị danh sách các tin đăng mà người dùng đã truy cập, hỗ trợ tìm kiếm theo từ khóa, lọc theo loại tin (Tất cả, Mua bán, Cho thuê) và phân trang. Mỗi tin đăng hiển thị các thông tin gồm: hình ảnh, loại bất động sản, tiêu đề, địa chỉ, giá, diện tích, số phòng ngủ, số phòng vệ sinh, thông tin người đăng, ngày đăng và lượt xem. Người dùng có thể lưu tin vào danh sách yêu thích thông qua biểu tượng **Yêu thích** hoặc liên hệ với người đăng bằng nút **Liên hệ**.



Hình 53: Màn hình trang Tin đã xem

3.4.10. Màn hình Nâng cấp gói tin

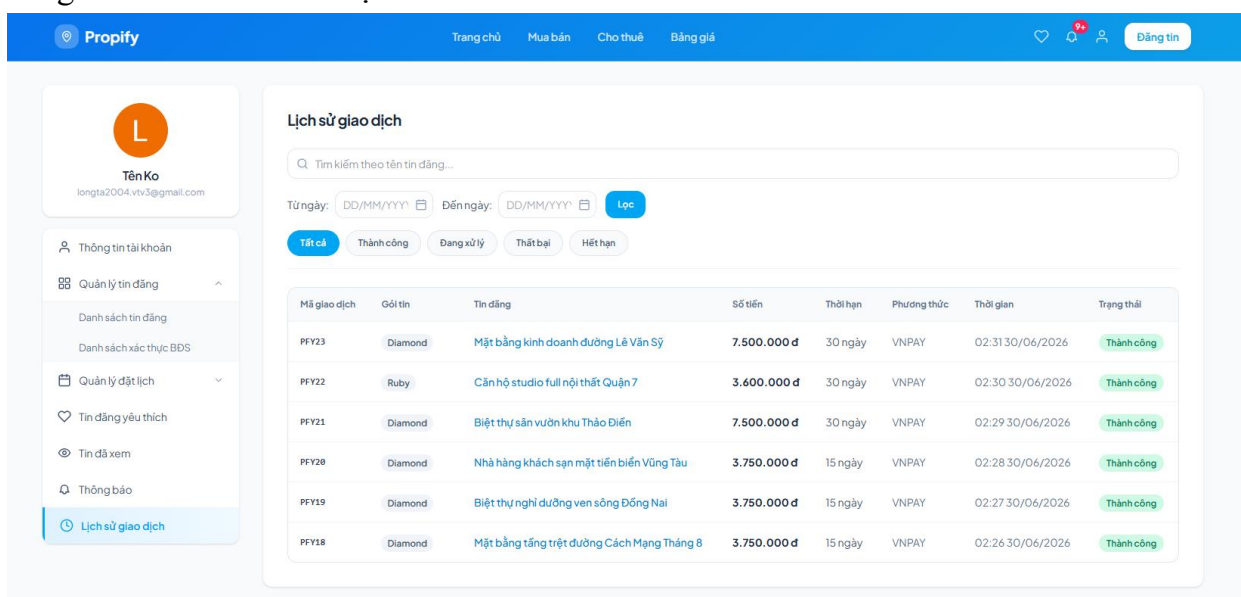
Giao diện hiển thị danh sách các gói đăng tin gồm **Diamond**, **Ruby**, **Vàng** và **Tin thường**, kèm thông tin về đơn giá, thời hạn, tổng chi phí và quyền lợi của từng gói. Người dùng có thể lựa chọn thời gian đăng tin (3, 5, 7, 10, 15 hoặc 30 ngày), hệ thống tự động cập nhật số tiền tương ứng. Sau khi chọn gói và thời hạn, người dùng nhấn **Nâng cấp** để chuyển sang bước thanh toán.



Hình 54: Màn hình trang Nâng cấp gói tin

3.4.11. Màn hình Lịch sử giao dịch

Giao diện hiển thị danh sách các giao dịch thanh toán của người dùng, hỗ trợ tìm kiếm theo tên tin đăng, lọc theo khoảng thời gian và trạng thái giao dịch (Tất cả, Thành công, Đang xử lý, Thất bại, Hết hạn). Danh sách giao dịch được hiển thị dưới dạng bảng gồm các thông tin: mã giao dịch, gói tin, tin đăng, số tiền, thời hạn, phương thức thanh toán, thời gian giao dịch và trạng thái. Trạng thái giao dịch được hiển thị bằng nhãn màu tương ứng như **Chờ thanh toán**, **Thành công**, **Thất bại** và **Hết hạn**, giúp người dùng dễ dàng theo dõi và tra cứu lịch sử thanh toán.



Hình 55: Màn hình trang Quản lý lịch sử giao dịch

CHƯƠNG 4. CÀI ĐẶT VÀ TRIỂN KHAI HỆ THỐNG

4.1. Môi trường triển khai

Thành phần	Thông tin
Hệ điều hành	Ubuntu 22.04 LTS / Windows 10+ / macOS 13+
Backend Runtime	PHP 8.2+, Composer 2.x
Frontend Runtime	Node.js 18+ LTS, npm 9+
Web Server	Nginx 1.24+ (production) / Laravel Sail (development)
Database chính	MySQL 8.0+
Cache & Session	Redis 7.0+
Search Engine	Elasticsearch 8.x
Container	Docker 24+ & Docker Compose 2.x (development)
Cloud Storage	Cloudinary (free tier hỗ trợ 25GB)
Xác thực bên thứ ba	Google OAuth 2.0 (Google Cloud Console)
Thanh toán	VnPay Sandbox (mock)
Realtime	Socket.IO + Laravel WebSockets
IDE khuyến nghị	VS Code với các extension PHP, Vue.js, ESLint

4.2. Cài đặt hệ thống

4.2.1. Yêu cầu hệ thống

Yêu cầu tối thiểu để chạy môi trường phát triển:

- CPU: 4 cores trở lên
- RAM: 8GB tối thiểu (16GB khuyến nghị)
- Ổ đĩa: 20GB trống
- Kết nối Internet: để tải các dependency và kết nối dịch vụ bên thứ ba
- Phần mềm: Docker Desktop (hoặc cài đặt trực tiếp PHP, Node.js, MySQL, Redis)

4.2.2. Các bước cài đặt chương trình

Bước 1: Clone mã nguồn từ repository

Clone repository: `git clone <repository-url> && cd propify`

Bước 2: Cài đặt Backend với Docker (khuyến nghị)

1. Chạy toàn bộ stack: `docker compose up -d --build`

2. Copy .env.example .env
3. Sửa file .env theo cấu hình

Bước 3: Cài đặt Frontend

- Cd PropifyFrontend/PropifyAdmin
- npm i
- npm run dev
- Mở Fe ở port 5173 và admin ở 5174

CHƯƠNG 5. KẾT QUẢ VÀ ĐÁNH GIÁ

5.1. Kết quả thử nghiệm hệ thống

Quá trình thử nghiệm hệ thống được tiến hành trên môi trường Staging với dữ liệu thực tế nhằm đánh giá toàn diện tính chính xác của các chức năng, khả năng chịu tải của hệ thống và tính toàn vẹn của luồng nghiệp vụ liên kết (End-to-End).

5.1.1. Kết quả kiểm thử theo module

STT	Nhóm chức năng / Module kiểm thử	Số ca kiểm thử	Kết quả đạt	Ghi chú thực tế theo nghiệp vụ
I	PHÂN HỆ NGƯỜI DÙNG (CLIENT)	198	196/198	
1	Quản lý tài khoản (Đăng ký, Đăng nhập, Profile, Đổi/Quên mật khẩu)	35	35/35 (100%)	Kiểm tra xác thực OTP, login Google, rate limit nhập sai mật khẩu và policy mật khẩu mới.
2	Tìm kiếm & Bộ lọc (Trang chủ, Danh sách tin, Map view, Tìm kiếm, Lọc nâng cao)	40	40/40 (100%)	Kiểm tra phân trang, sắp xếp, lọc min-max, hiển thị ghim vị trí và đồng bộ dữ liệu trên bản đồ.
3	Nghiệp vụ Tin đăng (Tạo tin, Sửa, Gỡ, Yêu thích, Xác thực chính chủ)	45	44/45 (97.7%)	1 ca lỗi upload tệp video căn hộ dung lượng lớn gặp Gateway

				<i>Timeout (Đã fix).</i>
4	Cảnh báo tin đăng (Báo cáo vi phạm)	12	12/12 (100%)	Kiểm tra popup lý do vi phạm, chặn gửi chuỗi rỗng và ghi nhận trạng thái chờ duyệt của Admin.
5	Giao tiếp trực tuyến (Chat với người đăng)	20	20/20 (100%)	Kiểm tra tự động tạo phòng chat từ trang chi tiết và truyền tải tin nhắn văn bản real-time qua Socket.
6	Nghệ vụ Lịch hẹn (Đặt lịch, Xem, Xác nhận, Từ chối, Hủy lịch)	26	25/26 (96.1%)	<i>1 ca lỗi edge case hủy lịch sát giờ vi phạm quy tắc cách 2 tiếng do lệch múi giờ (Đã fix).</i>
7	Tài chính cá nhân (Nâng cấp gói tin, Thanh toán, Lịch sử GD, Lọc GD, Chi tiết GD)	20	20/20 (100%)	Kiểm tra webhook công thanh toán, hiển thị số tiền âm/dương (- đ, + đ) và bộ lọc hóa đơn cá nhân.
II	PHÂN HỆ QUẢN TRỊ (ADMIN CMS)	154	154/154	
8	Quản lý tài khoản người dùng (Xem, Tìm kiếm, Lọc, Chi tiết, Khóa/Mở khóa)	30	30/30 (100%)	Kiểm tra BR thu hồi phiên đăng nhập (Token revoke) ngay lập tức khi Admin thực hiện khóa tài khoản.

9	Kiểm duyệt tin đăng (Xem danh sách, Chi tiết, Tìm, Lọc, Xác thực, Duyệt/Từ chối, Khóa/Mở khóa)	40	40/40 (100%)	Kiểm tra đổi trạng thái tin đăng, gửi thông báo lý do từ chối kiểm duyệt và ẩn tin khi bị khóa.
10	Quản lý Lịch sử giao dịch & Xuất báo cáo (Xem, Tìm, Lọc số tiền min-max, Xuất file CSV)	34	34/34 (100%)	Kiểm tra tính chính xác của 3 khối thống kê đầu trang và cấu trúc file .csv sau khi xuất báo cáo.
11	Cấu hình hệ thống (Chi tiết gói tin, Thêm/Sửa/Cài đặt hiển thị tiện ích)	25	25/25 (100%)	Thử nghiệm các tác vụ CRUD tiện ích (Gym, Bể bơi...) và toggle bật/tắt hiển thị danh mục.
12	Báo cáo chiến lược (Dashboard thống kê, Xem doanh thu, Xuất báo cáo doanh thu Excel/PDF)	25	25/25 (100%)	Kiểm tra các biểu đồ tăng trưởng, biểu đồ tin mới và chức năng trích xuất báo cáo doanh thu định kỳ.
	TỔNG CỘNG HỆ THỐNG	352	350/352 (99.4%)	Hệ thống vận hành ổn định, tỷ lệ Passed đạt tiêu chuẩn nghiệm thu.

5.1.2. Kịch bản kiểm thử tích hợp End-to-End

Để đảm bảo các module có thể phối hợp nhịp nhàng và không xảy ra xung đột dữ liệu trên một hành trình trải nghiệm thực tế, kịch bản tích hợp xuyên suốt sau đã được thực thi:

Kịch bản 1 – Luồng đăng tin, phê duyệt và đặt lịch hẹn hoàn chỉnh

- **Mục tiêu:** Kiểm tra tính liên tục từ khi tạo tài khoản, đăng tin, kiểm duyệt cho đến khi phát sinh tương tác đặt lịch giữa hai người dùng khác nhau.
- **Các bước thực hiện:**
 1. **Người dùng A (Chủ nhà):** Đăng ký tài khoản mới và xác thực thành công qua mã OTP gửi về Email.
 2. **Người dùng A:** Cập nhật thông tin hồ sơ cá nhân, tải lên ảnh đại diện để tăng độ uy tín.
 3. **Người dùng A:** Truy cập chức năng "Tạo tin đăng", nhập đầy đủ thông tin căn hộ, chọn các tiện ích (Wifi, Điều hòa, Bể bơi), tải lên 5 hình ảnh và nhấn hoàn tất. Trạng thái tin lúc này trong DB là Pending (Chờ duyệt).
 4. **Admin:** Đăng nhập vào hệ thống CMS, truy cập menu "Kiểm duyệt tin đăng", tìm thấy bài đăng của Người dùng A. Admin kiểm tra nội dung và nhấn "**Duyệt tin đăng**". Trạng thái tin chuyển sang Active (Đang đăng) và hiển thị ngoài trang chủ.
 5. **Người dùng B (Khách thuê):** Đăng nhập vào ứng dụng, sử dụng "Bộ lọc tin đăng" theo khoảng giá và tiện ích để tìm kiếm. Hệ thống trả về kết quả chứa bài đăng của Người dùng A.
 6. **Người dùng B:** Click xem chi tiết tin đăng, sau đó nhấn nút "**Đặt lịch hẹn**", chọn ngày 29/06/2026 và khung giờ 08:00 - 11:30 rồi nhấn gửi.
 7. **Người dùng A:** Nhận được thông báo đầy, truy cập tab "Lịch của khách" và nhấn "**Xác nhận lịch hẹn**".
- **Kết quả:** Luồng nghiệp vụ chạy thành công 100%. Trạng thái lịch hẹn chuyển sang "Đã xác nhận", hệ thống gửi thông báo xác nhận thành công cho cả hai bên theo thời gian thực.

Kịch bản 2 – Luồng giao dịch nâng cấp gói tin và đối soát báo cáo

- **Mục tiêu:** Kiểm tra tính chính xác của dòng tiền, sự thay đổi mức độ ưu tiên hiển thị của tin đăng và chức năng xuất dữ liệu của kế toán.
- **Các bước thực hiện:**
 1. **Người dùng A:** Chọn một tin đăng đang ở gói thường và nhấn nút "**Nâng cấp gói tin**". Người dùng chọn gói "Ruby (Thời hạn 7 ngày)" với số tiền 840.000 đ.
 2. **Người dùng A:** Chọn phương thức "Thanh toán qua VNPAY". Hệ thống điều hướng sang cổng thanh toán thanh toán, người dùng nhập thông tin thẻ và nhấn xác nhận thanh toán thành công.
 3. **Hệ thống:** Nhận kết quả từ Webhook cổng thanh toán, tự động trừ hạn mức/ghi nhận giao dịch trừ tiền trên tài khoản người dùng (-840.000 đ), đồng thời chuyển nhãn tin đăng của Người dùng A thành gói "Ruby" để đẩy lên vị trí ưu tiên ngoài Trang chủ.

4. **Admin (Kế toán):** Truy cập menu "Lịch sử giao dịch" trên CMS. Hệ thống hiển thị khối "Tổng doanh thu" tăng thêm 840.000 đ và xuất hiện 1 dòng bản ghi mới của Người dùng A với trạng thái "Thành công".
 5. **Admin (Kế toán):** Áp dụng bộ lọc trạng thái "Thành công" và loại gói tin "Ruby", sau đó nhấn nút "**Xuất báo cáo (CSV)**".
- **Kết quả:** File CSV tải xuống thiết bị hiển thị chính xác thông tin giao dịch của Người dùng A, khớp hoàn toàn với số liệu ghi nhận trong Cơ sở dữ liệu và số tiền hiển thị trên giao diện của Admin.

5.2. Đánh giá hiệu quả hệ thống

5.2.1. Đánh giá theo mục tiêu đề ra

Mục tiêu	Kết quả đạt được	Mức độ
Xác thực người dùng an toàn	Triển khai đầy đủ JWT, OTP, Google OAuth2	Đạt
Quản lý tin đăng toàn diện	CRUD đầy đủ, kiểm duyệt, chấm điểm, media upload	Đạt
Tìm kiếm và lọc hiệu quả	Tìm kiếm tương đối và tuyệt đối, lọc đa điều kiện	Đạt
Hệ thống lịch hẹn	4 luồng nghiệp vụ đầy đủ: đặt/xác nhận/từ chối/hủy	Đạt
Gói dịch vụ và thanh toán	Tích hợp VnPay sandbox, quản lý lịch sử giao dịch	Đạt
Chat thời gian thực	WebSocket/Socket.IO hoạt động ổn định	Đạt
Dashboard Admin	Thống kê tin đăng, người dùng, doanh thu	Đạt
Hiệu năng hệ thống	Phản hồi < 2s với tải thông thường	Đạt
Bảo mật dữ liệu	Không lộ thông tin nhạy cảm, kiểm soát quyền đầy đủ	Đạt

5.2.2. Điểm mạnh của hệ thống

- Kiến trúc 3-layer rõ ràng, code có thể mở rộng và bảo trì dễ dàng.
- Hệ thống xác thực và bảo mật đa lớp: JWT + OTP + rate limiting + bcrypt.
- Quy trình kiểm duyệt tin đăng chặt chẽ giúp đảm bảo chất lượng nội dung.
- Hệ thống lịch hẹn với logic xử lý time-out tự động và thông báo hai chiều.
- Tích hợp đầy đủ các dịch vụ bên thứ ba (Cloudinary, Google Auth, Elasticsearch).

5.2.3. Hạn chế và hướng phát triển

Hạn chế hiện tại:

- Thanh toán hiện ở chế độ mock/sandbox; xác minh số điện thoại chưa được triển khai do không có SMS gateway thật.

Hướng phát triển:

- Tích hợp thanh toán thực tế và xác minh SĐT qua SMS OTP (Twilio, ESMS).
- Bổ sung tính năng đề xuất BĐS dựa trên AI/ML từ lịch sử hành vi người dùng.
- Phát triển ứng dụng mobile (React Native/Flutter) cho trải nghiệm tốt hơn trên thiết bị di động.
- Xây dựng hệ thống thống kê và báo cáo nâng cao với biểu đồ phân tích xu hướng thị trường.